MICROSOFT POWER PLATFORM



Microsoft Power Automate for End Users

Two-day course book

Collect, report, remind, approve and brief: automate a training coordinator's routine with automated, instant and scheduled cloud flows, variables, parallel branches and Copilot.

Participant edition · 2026

Delivered by Asraf · Microsoft Certified Trainer

Microsoft Power Automate for End Users

Two-day course book · Participant edition, version 2.0

© 2026 Asraf Jaafar Sidik. All rights reserved.

Online resources: github.com/Asraf-JS/Microsoft-Power-Automate

Microsoft, Microsoft 365, Power Automate, Power Platform, Copilot, Excel, Outlook, Teams, Forms, OneDrive and SharePoint are trademarks of the Microsoft group of companies. This book is independent training material and is not published or endorsed by Microsoft.

Screenshots show the product at the time of writing. Microsoft updates its apps often, so labels and layouts in your tenant may differ slightly.

About the author

Asraf is a Microsoft Certified Trainer and HRD Corp-certified IT trainer based in Malaysia. He has been teaching working professionals since 2008, helping teams at banks, pharmaceutical companies, construction firms, and retailers get real work done with Microsoft 365, Power Platform, Excel, Google Workspace, and, more recently, AI tools like Copilot.

He holds a Bachelor of Information Technology (Multimedia) from the University of Malaya, along with Microsoft certifications in Power BI (PL-300) and AI for business (AB-730). Outside the classroom he writes his own automation tools in Python and Node.js, mostly to save himself from doing the same task twice.

He runs his training practice solo, which means he has personally felt every inefficiency this book tries to fix.

Contents

Before you begin 10

1 Understand Power Automate 13

What You Will Learn 13
Your Business Scenario 13
1.1 What Is Automation? 14
1.2 Recognise the Parts of a Flow 14
1.3 Choose How the Flow Starts 14
1.4 Connectors and Connections 15
1.5 Explore Power Automate 15
1.6 Plan Before You Build 18
Independent Practice: Plan a Weekly Reminder 19
Tips and Troubleshooting 19
Lesson Summary 20
Hands-on exercises 21

2 Prepare Your Training Workspace 22

What You Will Learn 22
Apps You'll Use 23
2.1 Create the Training Folders in OneDrive 23
2.2 Create the Excel Workbook 24
2.3 Prepare a Test Email 27
2.4 Confirm Forms Readiness 28
2.5 Confirm the Teams Training Destination 28
2.6 Confirm the Power Automate Environment 29
Independent Practice 29
Lesson Summary 29
Hands-on exercises 30

3 Save Email Attachments Automatically 32

What You Will Learn 32
Before You Begin 32
How to Insert Dynamic Content 33
How to Add an Action and Check Your Flow 34
3.1 Create the Automated Cloud Flow 34
3.2 Configure the Trigger 35
3.3 Add Apply to Each 37
3.4 Add the PDF Condition 39
3.5 Create the OneDrive File 41
3.6 Test a Positive Case 43
3.7 Test a Negative Case 43

Independent Practice 45
Tips and Troubleshooting 45
Lesson Summary 46
Hands-on exercises 47

4 Create and Distribute an Excel Summary 48
What You Will Learn 48
How the Flow Works 48
Before You Begin 49
4.1 Confirm the Source Table 49
4.2 Create an Instant Cloud Flow 50
4.3 Read the Excel Records 51
4.4 Choose the Report Columns 53
4.5 Build One HTML Table 54
4.6 Send the Coordinator Summary 56
4.7 Your First Expression: Count the Rows 57
4.8 Test and Inspect 59
4.9 Optional Teams Update 60
Independent Practice 60
Troubleshooting 60
Lesson Summary 61
Hands-on exercises 62

5 Send Weekly Training Reminders 63
What You Will Learn 63
How the Flow Works 63
Before You Begin 64
5.1 Create the Weekly Schedule 64
5.2 Produce a Readable Local Timestamp 66
5.3 Read the Excel Rows 68
5.4 Process One Row at a Time 70
5.5 Prepare the Reminder Email 71
5.6 Make the Date Readable with an Expression 73
Trainer-Controlled Test 74
Independent Practice 74
Troubleshooting 75
Lesson Summary 75
Hands-on exercises 76

6 Process a Training Approval 77
What You Will Learn 77
How the Flow Works 77
Before You Begin 77
6.1 Prepare the Request Form 78

6.2 Create the Automated Flow 82
6.3 Select the Form 83
6.4 Retrieve the Answers 84
6.5 Request the Decision 86
6.6 Branch on the Outcome 87
6.7 Prepare the Approved Message 89
6.8 Prepare the Rejected Message 90
6.9 Save and Check the Flow 91
Trainer-Controlled Test 92
Independent Practice 92
Troubleshooting 92
Lesson Summary 92
Hands-on exercises 93

7 **Refine the Weekly Reminders** 95
What You Will Learn 95
How the Flow Works 95
Before You Begin 96
7.1 Make a Safe Copy 96
7.2 Remove Completed Rows with Filter Array 97
7.3 Work Out the End of the Window 98
7.4 Start the Reminder Counter 99
7.5 Loop Through the Remaining Rows 101
7.6 Check the Date Window 102
7.7 Count Each Reminder 104
7.8 Decide Whether a Summary Is Needed 105
7.9 Prepare the Coordinator Summary 107
7.10 Handle a Week with No Matching Rows 109
7.11 Save and Check the Flow 110
Trainer-Controlled Test 111
Independent Practice 111
Troubleshooting 112
Lesson Summary 112
Hands-on exercises 113

8 **My Morning Briefing: Variables and Parallel Branches** 115
What You Will Learn 115
How the Flow Works 115
8.1 Create the Scheduled Flow 116
8.2 Create the Variables 117
8.3 Branch 1: The Weather 119
8.4 Branch 2: Your Meetings 120
8.5 Branch 3: Important Unread Email 123
8.6 Bring the Branches Back Together 125

8.7 Change the Subject on a Busy Day 126
8.8 Send the Briefing 127
8.9 Test It 130
Independent Practice 130
Troubleshooting 131
Lesson Summary 131
Hands-on exercises 132

9 Plan an Automation with Copilot 134
What You Will Learn 134
The Business Problem 134
Before You Begin 134
9.1 Open the Copilot Planning Experience 135
9.2 Describe the Complete Business Outcome 135
9.3 Review the First Suggestion 136
9.4 Clarify the Filter and Count 137
9.5 Identify an Unnecessary Action 138
9.6 Request a Streamlined Design 138
9.7 Validate the Final Suggestion 138
9.8 Exit Without Creating the Flow 139
9.9 Plan Your Own Automation 139
Independent Practice 141
Troubleshooting 142
Lesson Summary 142
Hands-on exercises 143

10 Move Automations with Environments and Solutions 146
What You Will Learn 146
The Migration Path 146
Before You Begin 147
Environments in Plain Language 147
10.1 Confirm the Current Environment 147
10.2 Create the Solution 147
10.3 Add the Chapter 7 Flow 148
10.4 Verify the Solution Components 150
10.5 Review the Solution Before Export 150
10.6 Publish and Export 150
10.7 Switch Environment and Start the Import 151
10.8 Repair Connections and Settings 151
10.9 Verify and Test Safely 151
Independent Practice 152
Troubleshooting 152
Lesson Summary 152
Hands-on exercises 153

11 Look Up Currency Rates with a Desktop Flow 154

What You Will Learn 154

How the Flow Works 154

Before You Begin 155

11.1 Create the Desktop Flow 156

11.2 Open Google in Edge 157

11.3 Open the Workbook in Excel 157

11.4 Find the End of the Table 158

11.5 Loop Through the Rows 159

11.6 Read the Two Currency Codes 161

11.7 Type the Search into Google 162

11.8 Click and Press Enter 163

11.9 Wait for the Results 165

11.10 Read the Converted Amount 166

11.11 Write the Rate into Excel 167

11.12 Check Your Finished Flow 168

11.13 Run and Check 170

Independent Practice 171

Troubleshooting 171

Lesson Summary 172

Hands-on exercises 173

12 Read PDF Invoices into Excel 175

What You Will Learn 175

How the Flow Works 175

Before You Begin 176

12.1 Create the Desktop Flow 177

12.2 List the PDF Files 177

12.3 Open the Log in Excel 177

12.4 Repeat for Each File 177

12.5 Extract the Text 178

12.6 Find the Three Values 178

12.7 Find the Next Empty Row 180

12.8 Write the Four Cells 180

12.9 Save and Close Excel 181

12.10 Check Your Finished Flow 181

12.11 Run and Check 182

Independent Practice 182

Troubleshooting 182

Lesson Summary 183

Hands-on exercises 184

13 Extra Practice: Personal Automations 186

Which One Should I Try? 186
A. Flagged Email to To Do 187
B. Send Me a Note 189
C. Save an Email to OneDrive 191
D. News Digest 193
Lesson Summary 197
Hands-on exercises 197

INTRODUCTION

Before you begin

How to use this book

This book collects the notes and copy-paste values from the two-day Microsoft Power Automate for End Users course in one place, so you can follow along offline and keep it at your desk afterwards.

Each chapter matches one chapter of the course and has two parts:

- **Notes** explain the concepts and walk you through each build step by step, mostly point and click, with screenshots, checkpoints and troubleshooting.
- **Hands-on exercises** give you every flow name and text value you type, ready to copy.

You'll write just a few short expressions on the main path, one each in Chapters 4, 5 and 7, each with a step-by-step explanation. More are waiting in optional *Going further* boxes for when you're ready.

Look out for the coloured boxes. Blue boxes are notes, teal boxes are tips, purple boxes are things to try, and orange boxes are key points and warnings.

Prefer to copy values rather than retype them? Everything in this book is also online at **github.com/Asraf-JS/Microsoft-Power-Automate**, where each grey box has a copy button. Copying from there is safer than copying from a PDF, which can add line breaks or change quotation marks.

Course scenario

Throughout the course you automate the work of one training coordinator.

> **You coordinate training for a small organisation.** Course materials arrive by email, session details live in an Excel register, requests need approval, and participants need reminders.

Each chapter turns one of those manual tasks into a flow:

Stage	What happens
Plan	Learn how flows work and write a plan before building anything
Prepare	Set up the OneDrive folders, Excel register, test email, Forms and Teams destinations
Collect	Save PDF attachments from training emails to OneDrive automatically
Report	Send the coordinator a one-click summary table from the Excel register
Remind	Send weekly reminders to every participant on a schedule
Approve	Route a training request form through an approval and post the result to Teams
Refine	Filter reminders to upcoming incomplete sessions, count them and summarise the week

Brief	Build a personal morning briefing with variables and parallel branches
Design	Use Copilot to plan a flow, then review and correct its suggestion
Release	Watch the trainer package the finished flow in a solution for another environment

Ground rules

> **Important:** Use only fictional data. Keep flows turned off until your trainer asks you to test them, and only send test emails to addresses your trainer has approved. Testing a flow can create files, send messages, start approvals or change shared data.

Sample files

Four practice files come with the course. All of them are fictional.

File	Used in	What it is
TrainingRegister.xlsx	Chapters 2 to 7	A ready-made register with the tblTraining table and six records. Use it only if your trainer says so
CourseOutline.pdf	Chapters 2 and 3	The attachment the Chapter 3 flow should save
TrainerPhoto.jpg	Chapters 2 and 3	The attachment the Chapter 3 flow should skip
SessionNotes.docx	Chapter 3	Used in the independent practice

Download them from the course repository: **github.com/Asraf-JS/Microsoft-Power-Automate**.

Your Power Automate training journey

One coordinator's routine, start to finish: turn each repeated manual task into a cloud flow.

FOUNDATIONS

01
Understand Power Automate
Triggers, actions, flow types and planning

02
Prepare your workspace
OneDrive folders, the Excel register and test data

THE PROJECT, APP BY APP



03 COLLECT
Outlook + OneDrive
Save PDF attachments from training emails

OUTPUT
Automated flow

04 REPORT
Excel
Email one summary table on demand

OUTPUT
Instant flow

05 REMIND
Recurrence
Send weekly reminders for each row

OUTPUT
Scheduled flow

06 APPROVE
Forms + Approvals
Route a request and post the result to Teams

OUTPUT
Approval flow

07 REFINE
Refine
Filter, count and summarise the week

OUTPUT
Refined reminders

08 BRIEF
Briefing
Variables and parallel branches in a daily email

OUTPUT
Morning briefing

09 DESIGN
Copilot
Plan a flow and correct the suggestion

OUTPUT
Reviewed plan

10 RELEASE
Solutions
Trainer demo: package a flow for another environment

OUTPUT
Solution package

Collect → Report → Remind → Approve → Refine → Brief → Design → Release

01

CHAPTER 1

Understand Power Automate

Before you build anything, learn the vocabulary of a flow, the three ways a cloud flow can start, and where the main pages live in Power Automate.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 45 minutes

Your result: A simple automation plan and familiarity with the Power Automate Home and Create pages.

Files for this lab: None. You only explore Power Automate in this chapter.

What You Will Learn

- Identify a repetitive business task that could benefit from automation
- Explain the trigger, actions, data, condition and result of a flow
- Choose between an automated, instant and scheduled cloud flow
- Open Power Automate and find Home, Create and My flows
- Write a short plan before building an automation

You need a Windows computer with Microsoft Edge, an internet connection and your training work or school account. You don't need the Excel workbook, a form or any test files yet. You prepare those in Chapter 2.

Your Business Scenario

You coordinate training for a small organisation. Course materials arrive by email, session details are stored in Excel, and participants need reminders. You keep downloading attachments, copying information into messages and checking whether requests were approved.

Each of those tasks has a recognisable starting point and repeatable steps. Over the two days you use Power Automate to do some of those steps for you.

1.1 What Is Automation?

Automation means setting up a repeatable process so software performs the steps when a defined event happens, or when you ask it to run.

Power Automate calls an automation a **flow**. This course builds **cloud flows**, which work with online services such as Outlook and OneDrive for Business.

A flow applies your rule every time, including a wrong one. If you pick the wrong folder or recipient, the flow repeats that mistake. So every flow needs a clear rule and a test.

> **Tip:** Start with a small task whose result you can check easily. A good first flow saves one test attachment to the correct folder.

1.2 Recognise the Parts of a Flow

Take this requirement: *When an email with the subject marker [PA TRAINING] arrives, examine each attachment. If the attachment is a PDF, save it to the training attachments folder.*

Part	Plain-language meaning	Example
Trigger	What starts the flow	A matching email arrives
Action	A step the flow carries out	Create a file in OneDrive for Business
Data	Information a step receives or produces	The attachment's name and contents
Condition	A question that selects a path	Is this attachment a PDF?
Loop	Steps repeated for each item	Check each attachment in the email
Result	The business outcome you verify	Matching files appear in the intended folder

The basic pattern is: something starts the flow, the flow follows its steps, you check the result. A flow only needs a condition or a loop when the process calls for one.

> **Try it:** Imagine a training email with two attachments, `CourseOutline.pdf` and `TrainerPhoto.jpg`. Walk the rule through each one. The PDF is saved, the image is skipped, and the folder should end up with `CourseOutline.pdf` only. If an email has no PDFs at all, the flow still runs successfully and creates nothing.

1.3 Choose How the Flow Starts

Cloud-flow type	When it starts	Training example
Automated	When a specified event occurs	Save attachments when a training email arrives
Instant	When you deliberately start it	Create an Excel summary when you're ready to share it

Scheduled	At a time and frequency you configure	Send training reminders every Monday morning

Pick the type by asking *what should make this process begin?* Sending an email doesn't tell you the type. All three can send email. Only their starting points differ.

Quick check. Pick a type for each request, then check the answers.

1. Whenever a new training request is submitted, ask someone to approve it.
2. At 9:00 AM every Friday, prepare a weekly reminder.
3. Let me generate a summary whenever I need one.

Answers: 1 is Automated, 2 is Scheduled, 3 is Instant.

1.4 Connectors and Connections

A **connector** gives Power Automate a set of triggers and actions for one service. Office 365 Outlook provides email operations. OneDrive for Business provides file operations.

A **connection** links those operations to an account. Whenever a later lesson asks you to sign in to a connector, check that you're using the training account, and that it has permission for the mailbox, folder or resource involved.

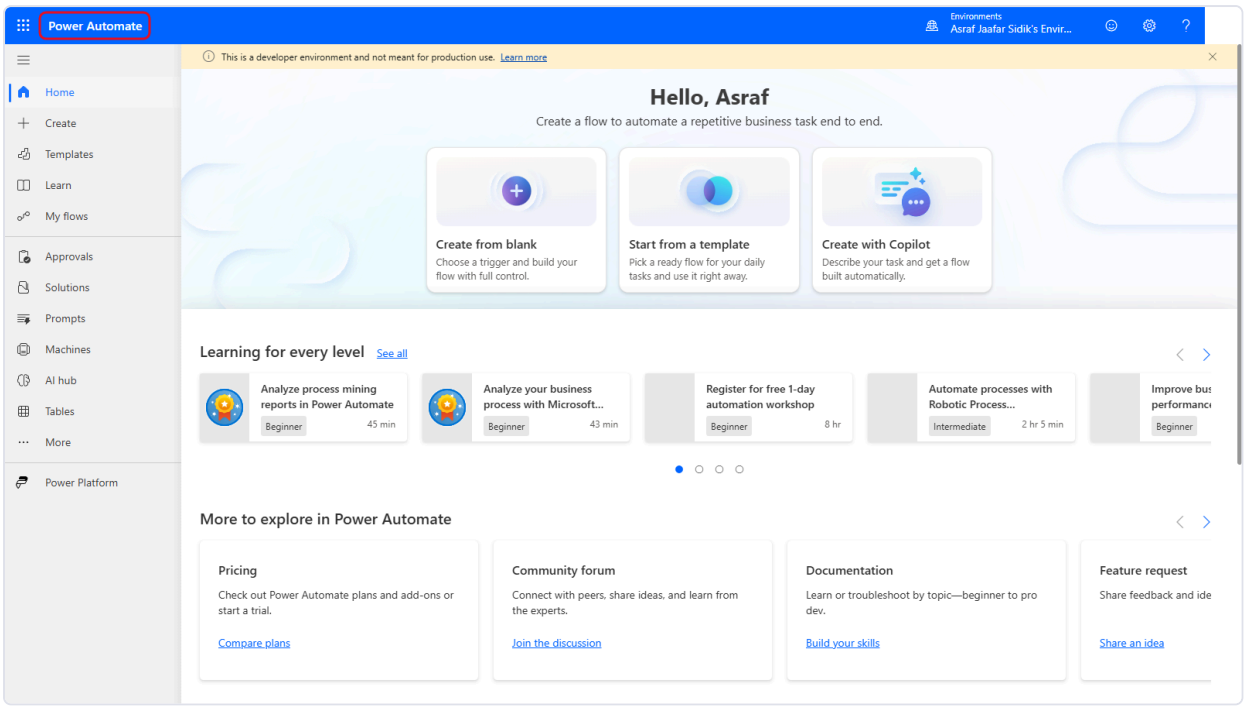
The core exercises use standard connectors only. Power Automate Free can create and run cloud flows with standard connectors but can't share flows, so you build and own your own flows. See Power Automate license types.

> **Tip:** Office 365 Outlook and Outlook.com are different connectors. Go by the connector name in the lesson, not the icon.

1.5 Explore Power Automate

Open the training workspace

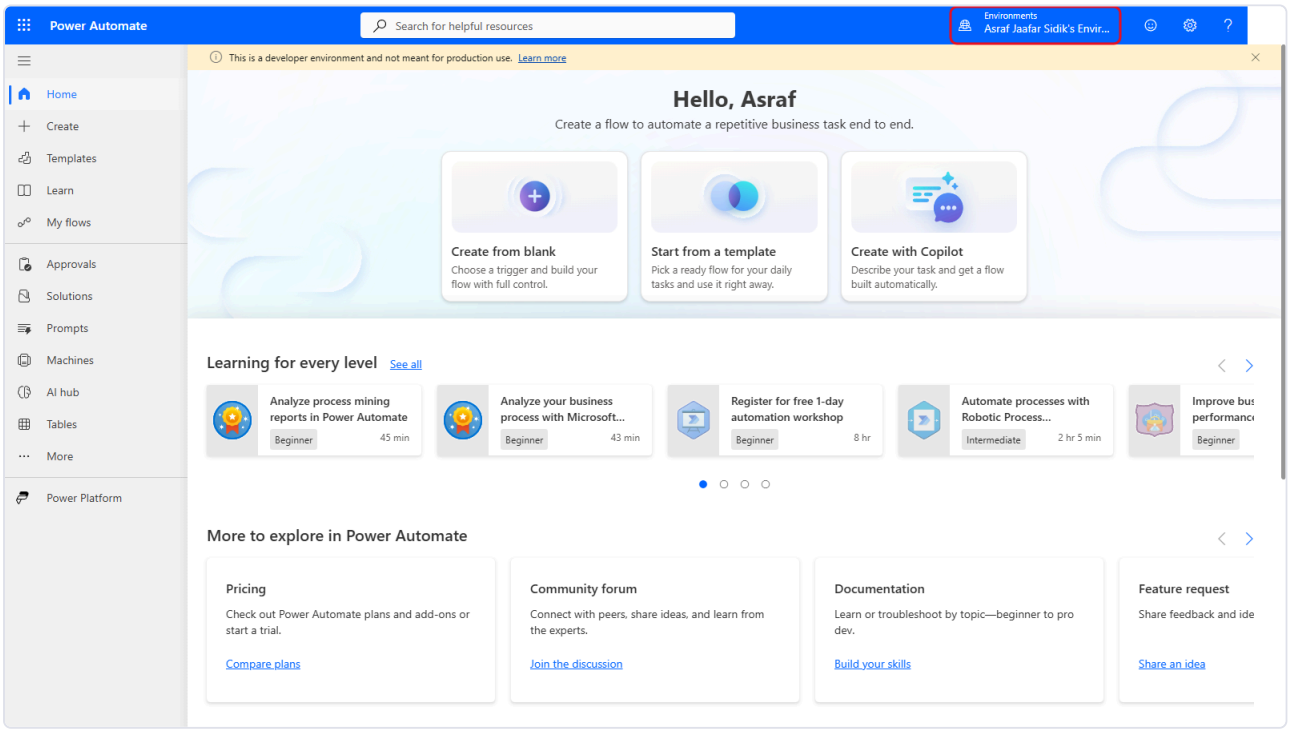
1. In Edge, go to make.powerautomate.com and sign in with your training account.
2. Check that the page says **Power Automate** at the upper left.



The Power Automate home page.

3. Look at the top right of the blue bar. The small word **Environments** sits above your environment's name. An environment is a workspace that holds Power Platform resources, including flows.

4. Compare it with the environment your trainer gave you. If they differ, ask before you build anything.

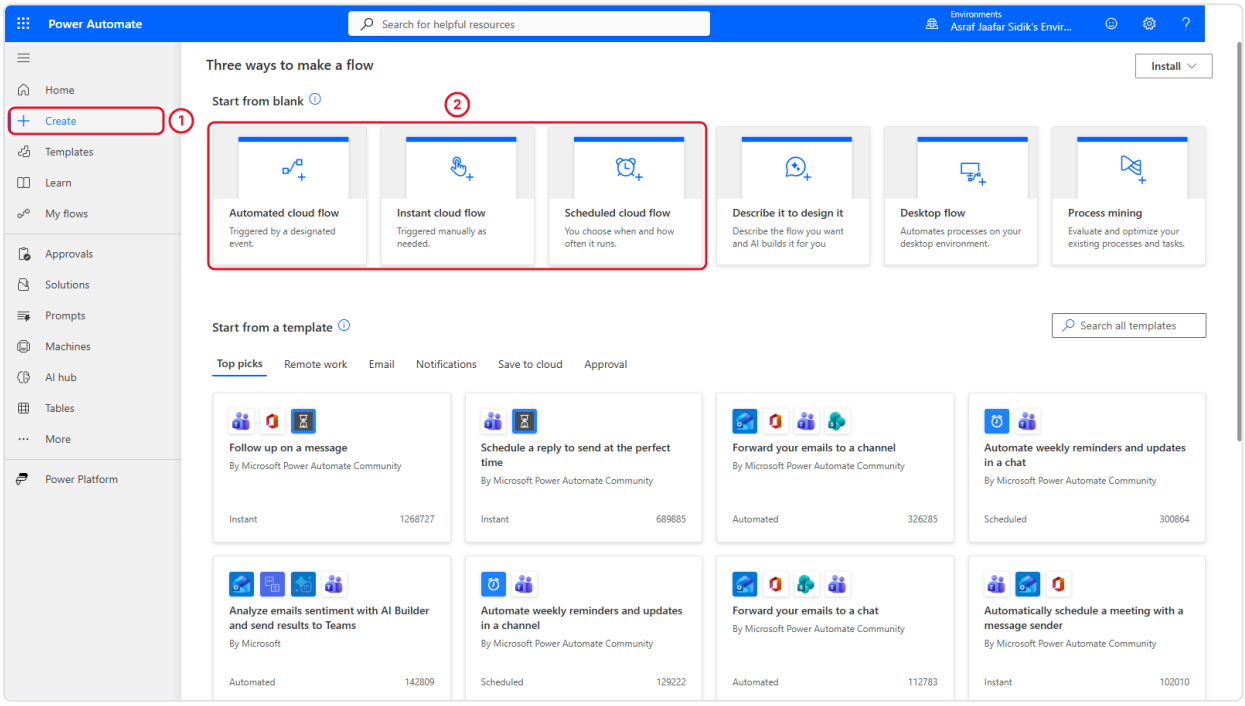


The environment selector shows which workspace you're in. Yours will have a different name.

Find the three cloud-flow choices

1. If you see only icons on the left, select the three-line menu icon at the top left (above **Home**) to show the labels. Then select **Create** in the left navigation.
2. Under **Start from blank** you see six tiles. Three of them create cloud flows from scratch, and those are what this course uses:

Tile	Description on the tile	Used in this course
Automated cloud flow	Triggered by a designated event	Yes
Instant cloud flow	Triggered manually as needed	Yes
Scheduled cloud flow	You choose when and how often it runs	Yes
Describe it to design it	Describe the flow you want and AI builds it for you	Chapter 9
Desktop flow	Automates processes on your desktop environment	Chapters 11 and 12 (bonus)
Process mining	Evaluate and optimize your existing processes and tasks	No



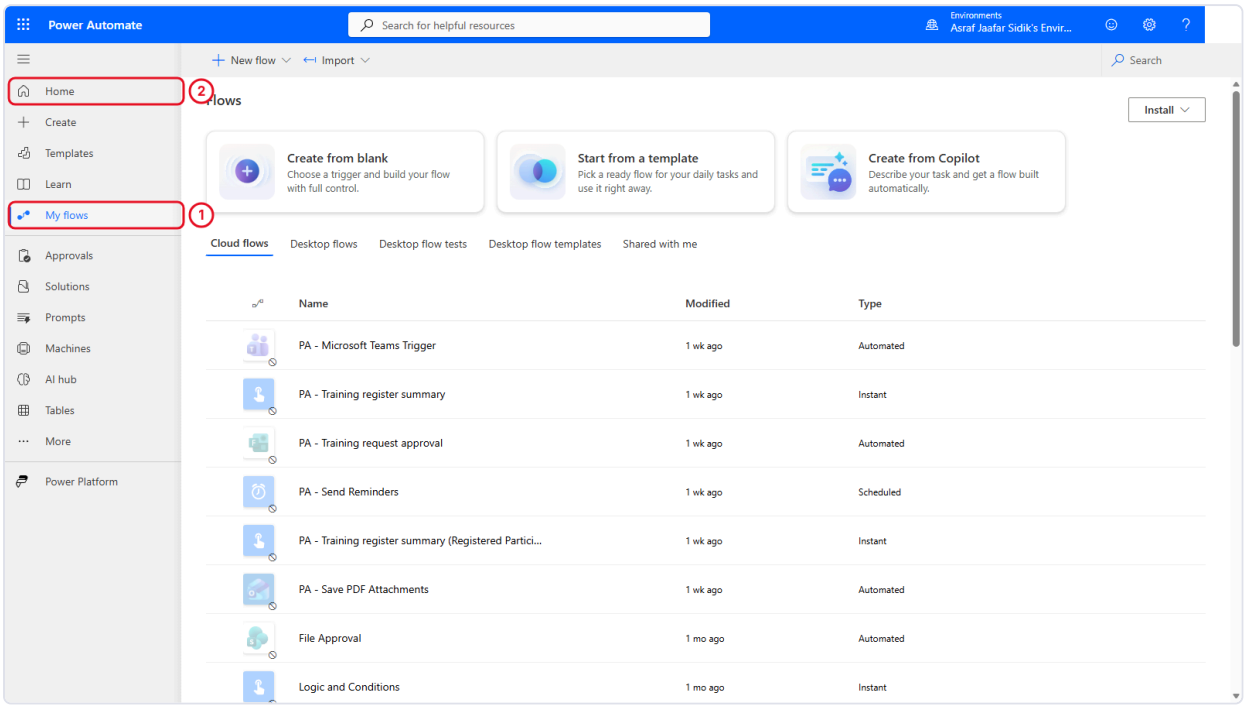
The three cloud-flow tiles come first under Start from blank. Describe it to design it is the Copilot option you'll try in Chapter 9.

> **Note:** The heading **Three ways to make a flow** refers to the three sections of the page: Start from blank, Start from a template and Start from a connector. Those aren't the three cloud-flow types. The types are three of the six tiles inside the first section.

3. Decide which tile you'd use for a flow that starts when someone submits a form. The answer is **Automated cloud flow**: a form submission is an event that happens without you starting it. Instant means *you* start the flow.

Locate your flows

1. Find **My flows** in the left navigation. You'll come back here to find every flow you build.



My flows lists every flow you've built. Your list is probably empty for now. The flows you build in Chapters 3 to 8 appear here. This screenshot shows the trainer's flows, so the names may differ slightly from the ones you build (for example, Save PDF Attachments instead of Save training PDF attachments).

2. Select **Home** at the top of the left navigation to return to the start page.

1.6 Plan Before You Build

A short plan makes the designer much easier to use. Here's the plan for the attachment scenario:

Planning question	Example answer
What task repeats?	Saving PDF training attachments from email
What starts it?	An email with the training subject marker arrives
What information is needed?	Attachment name and contents
What rule applies?	Save PDF attachments; skip other file types
Where should the output go?	The training attachments folder in OneDrive for Business
How will I test it?	Use an email with a PDF and an image, then check the folder

Copy this template into your notes and fill in each line before you build a flow. You use the same questions for the independent practice below.

```
What task repeats?
What starts it?
What information is needed?
What rule applies?
Where should the output go?
How will I test it?
```

Chapter 2 prepares the resources this plan refers to. Chapter 3 turns it into a working flow.

Independent Practice: Plan a Weekly Reminder

Allow five minutes. Every Monday morning, a training coordinator wants to remind people about upcoming sessions. Session details are in Excel. People whose training is marked Completed should not get a reminder.

1. Choose a cloud-flow type.
2. State the start time and time zone.
3. List three pieces of information the flow needs from Excel.
4. Write one rule that decides whether a reminder is sent.
5. Describe a test with one person who should get a reminder and one who shouldn't.

A good plan uses a scheduled flow with a local time zone, needs values like email address, session date, course title and status, sends only for upcoming incomplete training, and tests both an included and an excluded record.

Tips and Troubleshooting

What you notice	What to do
A different Microsoft account opens automatically	Check the account shown on the page. Use the Edge profile meant for training
You can't open Power Automate	Check the address and connection, then ask the trainer to verify access. Note the error message
The environment name is unfamiliar	Compare it with the class instructions before creating anything
Create isn't visible	Select the three-line menu icon at the top left to show the navigation labels. If it's still missing, ask the trainer to check access
The page looks different from the screenshot	Look for the named command. Layouts vary
A trial or upgrade prompt appears	Check what you selected. The core lessons use the Free baseline
You can't decide which flow type to use	Finish the sentence "This process should begin when..." and match it to a type

Lesson Summary

A trigger starts a flow, actions perform the steps, and data moves between them. Conditions choose a path and loops repeat work for each item. You can now tell automated, instant and scheduled flows apart, find the main Power Automate pages, and describe an automation before you build it.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner.

Power Automate address (section 1.5)

```
https://make.powerautomate.com
```

Automation planning template (section 1.6)

Paste this into your notes and fill in each line before you build a flow. You use the same questions for the independent practice at the end of the chapter.

```
What task repeats?
What starts it?
What information is needed?
What rule applies?
Where should the output go?
How will I test it?
```

02

CHAPTER 2

Prepare Your Training Workspace

Every flow in this course reads from or writes to something: a folder, a table, a mailbox, a form, a Teams chat. Set those up first and the later build steps become predictable.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 30 minutes

Your result: A training folder, an Excel workbook with a table, and safe test data for the first two build exercises.

> **Download:** the files for this lab
>
> - test-files.zip: the three test attachments for 2.3 (CourseOutline.pdf, TrainerPhoto.jpg, SessionNotes.docx). Download it and choose **Extract All**.
> - TrainingRegister.xlsx: a ready-made training register, only if your trainer tells you to upload it instead of building it in 2.2. Update its dates as in 2.2 step 4.

What You Will Learn

- Create a dedicated training folder structure in OneDrive for Business
- Build an Excel workbook and format its data as a named table
- Prepare a safe test email with sample attachments
- Confirm that Forms, Teams and your Power Automate environment are ready

> **Note:** Use your own training account and the environment your trainer names. Never put real participant information in the practice resources. Use fictional names and test addresses only.

Forms and Teams preparation in this chapter are trainer-led readiness checks, because those destinations depend on your organisation's policy.

Apps You'll Use

Everything in this course runs in the browser. Start from the Microsoft 365 home page and you can reach every app from it.

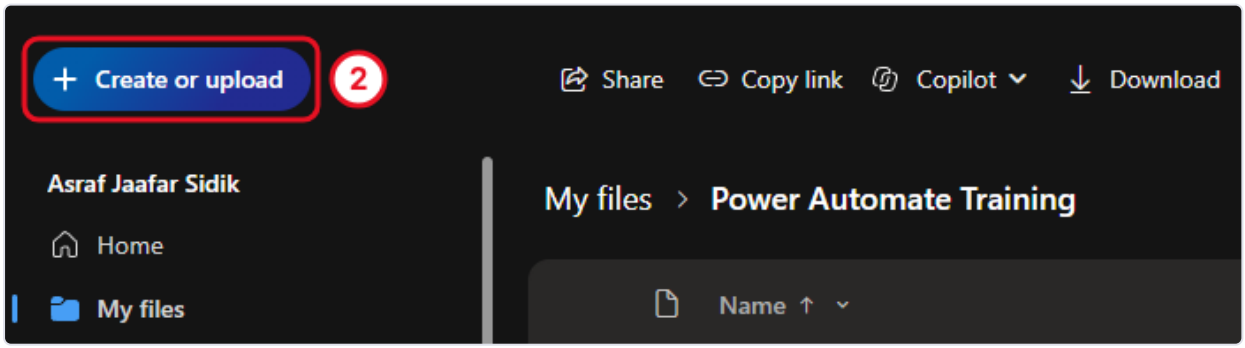
App	Web address	What you use it for
Microsoft 365 home	m365.cloud.microsoft	Your starting point. The app launcher (nine dots, top left) opens every app below
OneDrive	Open from the app launcher	The training folder and the Excel register
Outlook	outlook.office.com	Test emails and the emails your flows send
Microsoft Forms	forms.office.com	The training request form (Chapter 6)
Teams	teams.microsoft.com	Flow bot messages (Chapters 6 and 11)
Power Automate	make.powerautomate.com	Building your flows

Sign in to each one with your **work or school account**, the same one every time.

> **Note:** Used to OneDrive as a folder in File Explorer? That folder and OneDrive on the web are the same storage, so files appear in both. During class, use the web version so your screen matches the screenshots. Power Automate can only see files inside OneDrive: a workbook saved to your Desktop or Documents outside the OneDrive folder is invisible to it.

2.1 Create the Training Folders in OneDrive

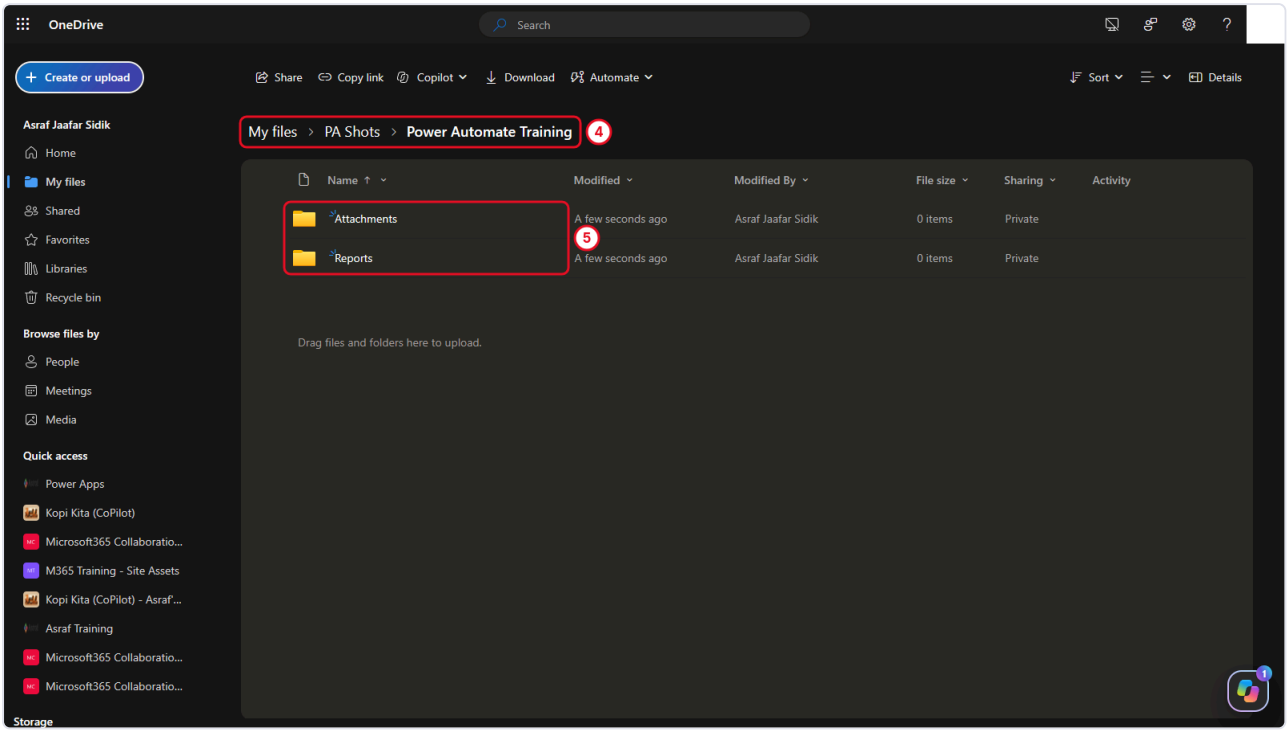
1. Open m365.cloud.microsoft, select the app launcher (nine dots, top left), then **OneDrive**. Select **My files** on the left.
2. Select the blue **Create or upload** button at the top left, then **Folder**.



Create or upload makes folders, uploads files and creates new Office documents. The picture was taken inside Power Automate Training, but the button is in the same place in My files.

3. Name the folder `Power Automate Training` and select **Create**.

4. Select the new folder's name to open it. The path at the top now reads **My files > Power Automate Training**.
5. Inside it, create two subfolders the same way (**Create or upload** > **Folder**): `Attachments` and `Reports`.



Both subfolders inside Power Automate Training. The breadcrumb in the picture includes an extra PA Shots folder used for the screenshots. Yours reads My files > Power Automate Training.

Checkpoint: You can see `Attachments` and `Reports` inside `Power Automate Training`. The Chapter 3 flow writes to Attachments. Summary outputs can go to Reports.

2.2 Create the Excel Workbook

1. Inside `Power Automate Training`, select **Create or upload**, then **Excel workbook**. Check the breadcrumb first: the workbook is created in whichever folder you're viewing.
2. When Excel for the web opens, select the file name in the title bar and rename it `TrainingRegister`. Excel adds `.xlsx` for you.
3. Select cell **A1**, then paste the headers and sample rows. Copy them from the box below. Each header lands in its own column (TrainingID, ParticipantName, Email, CourseTitle, SessionDate, Status), and the six fictional rows fill rows 2 to 7.

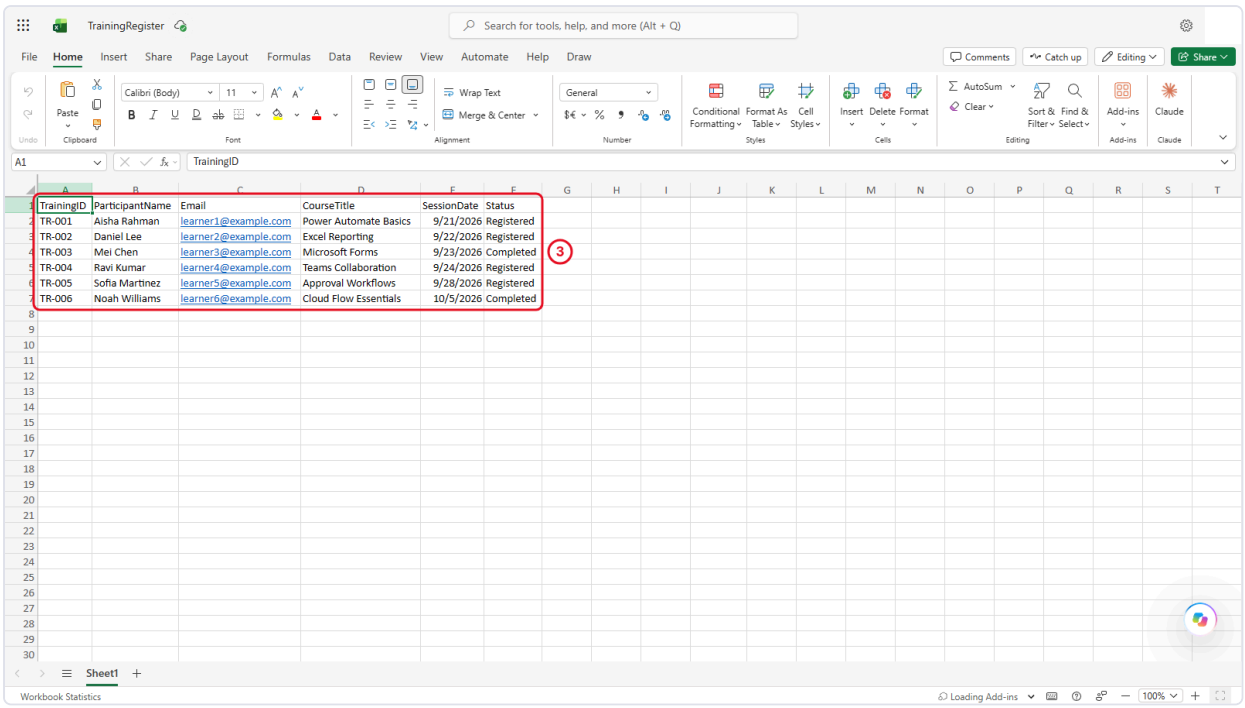
```
TrainingID    ParticipantName    Email    CourseTitle    SessionDate    Status
TR-001    Aisha Rahman    learner1@example.com    Power Automate Basics    9/21/2026
Registered
TR-002    Daniel Lee    learner2@example.com    Excel Reporting    9/22/2026    Registered
TR-003    Mei Chen    learner3@example.com    Microsoft Forms    9/23/2026    Completed
TR-004    Ravi Kumar    learner4@example.com    Teams Collaboration    9/24/2026
Registered
TR-005    Sofia Martinez    learner5@example.com    Approval Workflows    9/28/2026
Registered
TR-006    Noah Williams    learner6@example.com    Cloud Flow Essentials    10/5/2026
Completed
```

4. If your trainer gave you approved test addresses, type them over the addresses in the **Email** column. Then change the **SessionDate** values so they fit your class week, because the sample dates (21 September to 5 October 2026) may already be in the past. Count from today and make sure you have:
   - at least two **Registered** rows with a date within the next 14 days
   - one **Completed** row
   - one row with a date more than 14 days away

   Chapter 7 needs this mix to show which rows the 14-day window keeps and which it leaves out.

> **Tip:** If the dates paste left-aligned, Excel has stored them as text. Retype them in your local date format (for example, `21/09/2026` instead of `9/21/2026`) and they line up on the right.

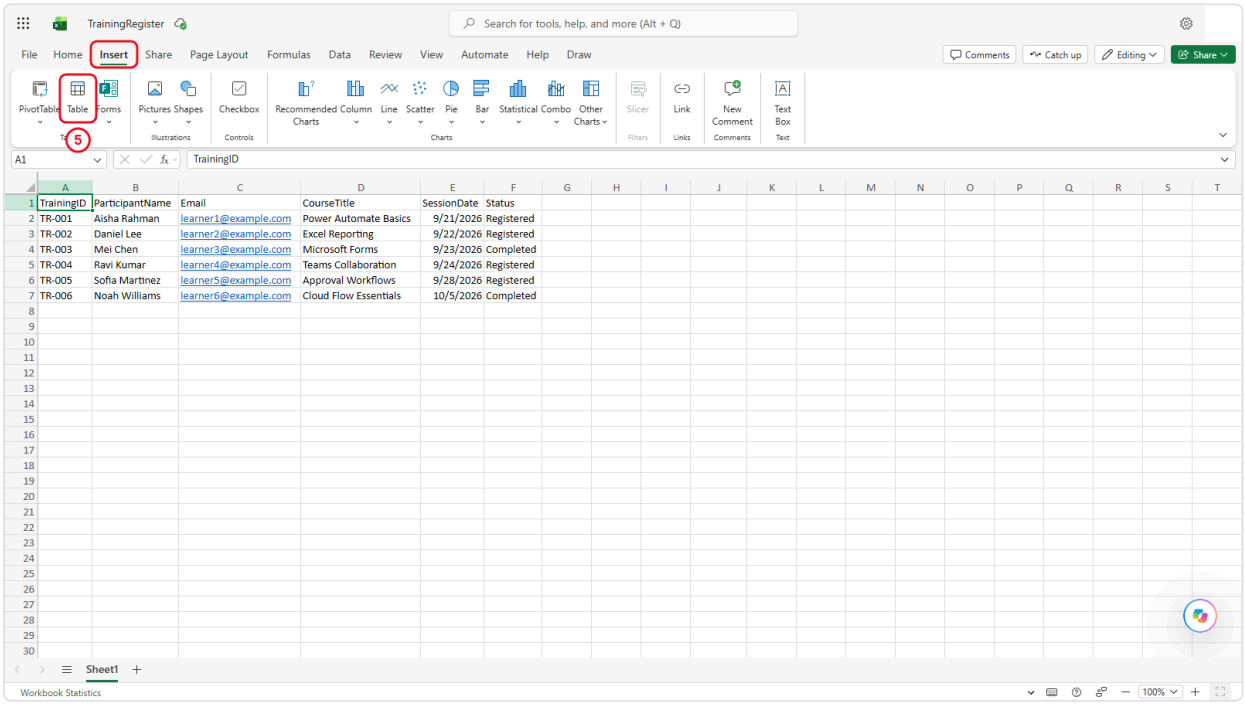
> **Tip:** Short on time? Your trainer may tell you to upload the ready-made TrainingRegister.xlsx to `Power Automate Training` instead (**Create or upload** > **Files upload**). It already has the `tblTraining` table and six fictional records, so you can skip steps 5 to 7. You still need to update its SessionDate values as described in this step.



TrainingID	ParticipantName	Email	CourseTitle	SessionDate	Status
TR-001	Aisha Rahman	learner1@example.com	Power Automate Basics	9/21/2026	Registered
TR-002	Daniel Lee	learner2@example.com	Excel Reporting	9/22/2026	Registered
TR-003	Mei Chen	learner3@example.com	Microsoft Forms	9/23/2026	Completed
TR-004	Ravi Kumar	learner4@example.com	Teams Collaboration	9/24/2026	Registered
TR-005	Sofia Martinez	learner5@example.com	Approval Workflows	9/28/2026	Registered
TR-006	Noah Williams	learner6@example.com	Cloud Flow Essentials	10/5/2026	Completed

Fictional rows. The example.com addresses can't receive mail. The cells look plain until you format them as a table in the next step.

5. Select any cell with data in it, then select the **Insert** tab and **Table**. In the box that opens, make sure **My table has headers** is ticked, and select **OK**. The rows turn into a striped table.

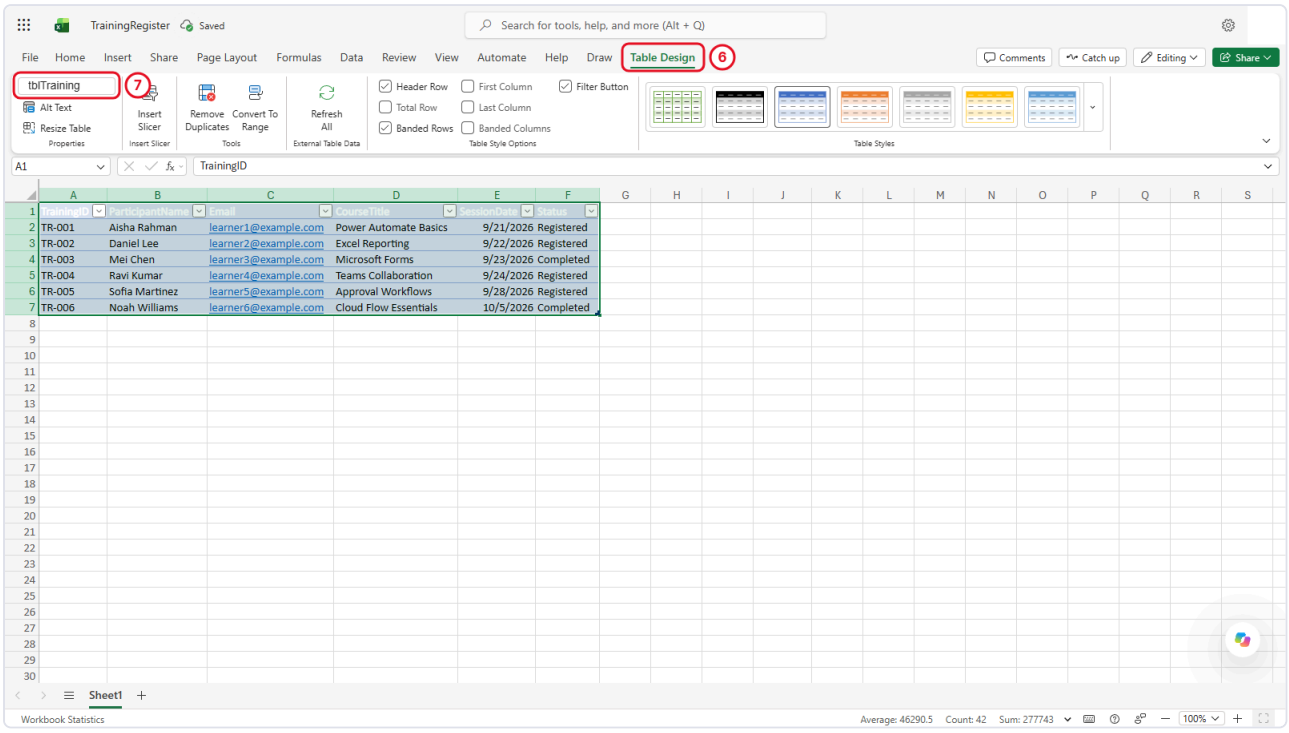


TrainingID	ParticipantName	Email	CourseTitle	SessionDate	Status
TR-001	Aisha Rahman	learner1@example.com	Power Automate Basics	9/21/2026	Registered
TR-002	Daniel Lee	learner2@example.com	Excel Reporting	9/22/2026	Registered
TR-003	Mei Chen	learner3@example.com	Microsoft Forms	9/23/2026	Completed
TR-004	Ravi Kumar	learner4@example.com	Teams Collaboration	9/24/2026	Registered
TR-005	Sofia Martinez	learner5@example.com	Approval Workflows	9/28/2026	Registered
TR-006	Noah Williams	learner6@example.com	Cloud Flow Essentials	10/5/2026	Completed

Insert is the third tab, after File and Home. Table is near the left end of the Insert tab.

6. With a cell in the table still selected, select the **Table Design** tab that has appeared at the end of the ribbon.

7. At the far left of that tab, select the name box (it reads something like **Table1**), type `tblTraining` and press Enter.



Set the table name on the Table Design tab.

Checkpoint: The workbook is named `TrainingRegister.xlsx`, the data is a formatted table named `tblTraining`, and the table has six fictional records.

If Power Automate can't find the table later, check three things: the workbook is in OneDrive for Business, the range is formatted as a table, and the table name is exactly `tblTraining`.

2.3 Prepare a Test Email

> **Download:** **test-files.zip** (one click, all three files).

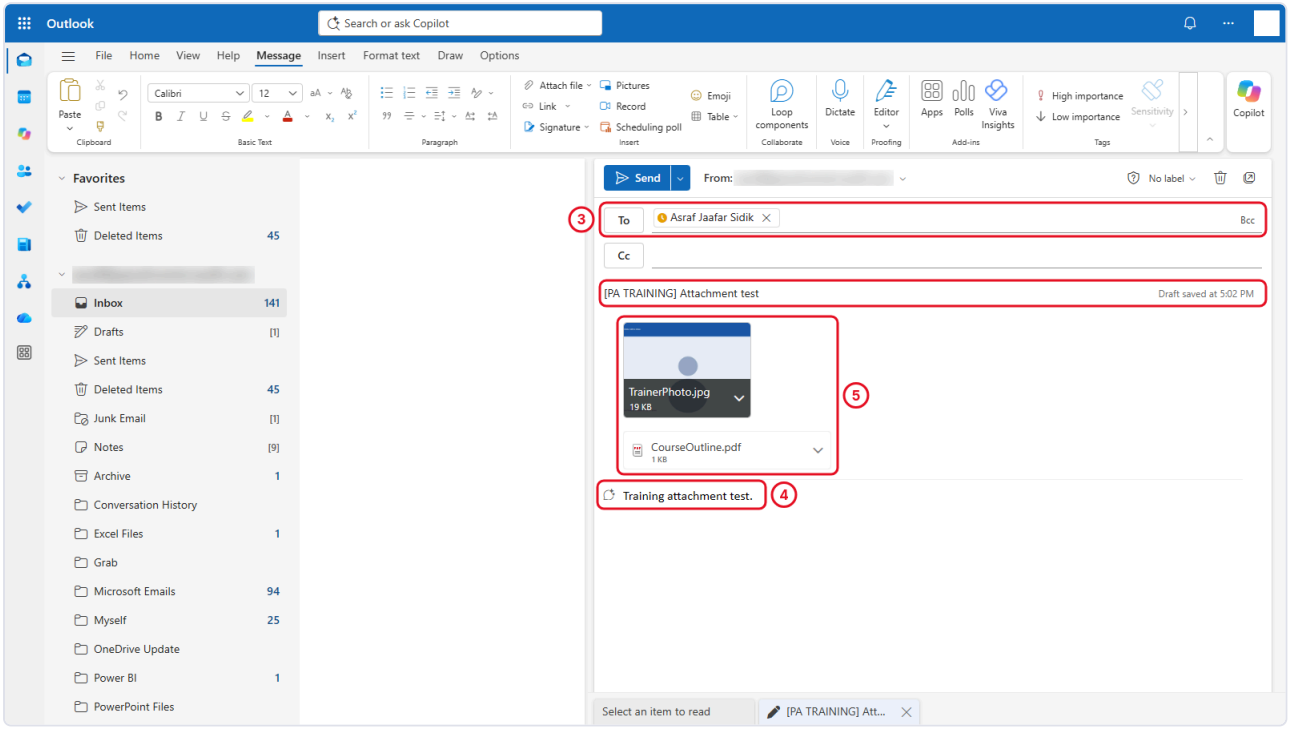
1. Open your Downloads folder, right-click `test-files.zip` and choose **Extract All**.
2. Open Outlook on the web (outlook.office.com) with the training account and select **New mail** (it may just say **New**) at the top left.
3. In **To**, type your own email address, the mailbox you connect to Power Automate in Chapter 3. In the subject line, enter the subject below.

```
[PA TRAINING] Attachment test
```

4. Select the large empty area under the subject line (the message body) and type the text below.

```
Training attachment test.
```

5. On the **Message** tab of the ribbon, select **Attach file** > **Browse this computer**, go to the extracted test files and pick `CourseOutline.pdf`. Repeat for `TrainerPhoto.jpg`. If Outlook asks whether to attach the picture or insert it inline, choose to attach it.
6. **Don't send it.** Outlook saves the message automatically (you see **Draft saved** at the right of the subject line). Select **Inbox** in the folder list to leave it. The message waits in **Drafts**.



The picture shows as a thumbnail and the PDF as a smaller chip. Both are attachments. The test email waits in Drafts until Chapter 3.

> **Important:** Don't attach real course materials or any file whose name contains a person's name. The filename shows up in run history and in your OneDrive folder.

The Chapter 3 flow only reacts to new messages, so you send this draft during the test in section 3.6.

2.4 Confirm Forms Readiness

Open Microsoft Forms and confirm the training account can create a form. You build the **Training Request** form in Chapter 6, right before the flow that uses it. It will ask for Requester name, Course, Preferred date and Business reason, with fictional answers only.

2.5 Confirm the Teams Training Destination

Ask the trainer for the team and channel, or the training chat, and write its name down. The trainer confirms posting access before the Teams exercises: the optional extension in Chapter 4 and the approval messages in Chapter 6.

The Teams connector is standard, but your organisation may restrict posting or require the Workflows app. If you can't post a test message, tell the trainer before building the summary flow.

2.6 Confirm the Power Automate Environment

1. Return to Power Automate.
2. Look at the top right of the blue bar. The small word **Environments** sits above your environment's name.
3. Confirm it matches the environment your trainer gave you.

> **Key point:** OneDrive files, Excel workbooks, Forms and Teams destinations aren't packaged automatically when a solution is moved. Keep a note of where each resource lives. Your trainer demonstrates moving flows in Chapter 10.

Independent Practice

Find the rows in `tblTraining` whose Status is Completed (there are two: TR-003 and TR-006). Which later reminder test should exclude them, and why? Then write a second test email without the `[PA TRAINING]` marker, save it as a draft, and predict whether the attachment flow should process it once it's sent.

Lesson Summary

You created a dedicated OneDrive training area with its folders and Excel table, prepared safe inputs for the attachment and summary exercises, confirmed the environment, and noted where the Forms and Teams work will happen. Next you build the flow that saves qualifying email attachments.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Anything marked **text** is typed or pasted straight into a field.

Folder names (text, section 2.1)

Main folder:

```
Power Automate Training
```

Subfolders:

```
Attachments
```

```
Reports
```

Workbook name (text, section 2.2)

```
TrainingRegister
```

Excel headers and sample rows (section 2.2)

Click cell A1 in Sheet1, then paste. Each value lands in its own column: the headers fill row 1 and the six fictional rows fill rows 2 to 7.

The sample dates may already be in the past on your class day. After pasting, change the SessionDate values as described in step 4 of section 2.2 (at least two Registered rows within the next 14 days, one Completed row, and one row more than 14 days away). If the dates paste left-aligned, retype them in your local date format.

```
TrainingID      ParticipantName Email   CourseTitle     SessionDate     Status
TR-001  Aisha Rahman    learner1@example.com    Power Automate Basics   9/21/2026
Registered
TR-002  Daniel Lee      learner2@example.com    Excel Reporting 9/22/2026       Registered
TR-003  Mei Chen        learner3@example.com    Microsoft Forms 9/23/2026       Completed
TR-004  Ravi Kumar      learner4@example.com    Teams Collaboration     9/24/2026
Registered
TR-005  Sofia Martinez  learner5@example.com    Approval Workflows      9/28/2026
Registered
TR-006  Noah Williams   learner6@example.com    Cloud Flow Essentials   10/5/2026
Completed
```

Table name (text, section 2.2)

```
tblTraining
```

Test email subject (text, section 2.3)

```
[PA TRAINING] Attachment test
```

Test email body (text, section 2.3)

```
Training attachment test.
```

03

CHAPTER 3

Save Email Attachments Automatically

Your first build: an automated flow that watches for training emails and saves only the PDF attachments to OneDrive. Everything in this chapter is point and click.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 50 minutes

Your result: An automated cloud flow that examines new training emails and saves only PDF attachments to OneDrive for Business.

> **Download:** the files for this lab
>
> - test-files.zip: the test attachments for the email you send in 3.6 (you downloaded these in Chapter 2). SessionNotes.docx is for the Independent Practice.

Connectors: Office 365 Outlook and OneDrive for Business, both standard.

What You Will Learn

- Create an automated cloud flow from a business requirement
- Configure an Outlook trigger with a subject filter
- Process several attachments with **Apply to each**
- Use a condition to select PDF files
- Insert values with the **dynamic content** picker
- Test a matching and a non-matching case, then inspect run history

Before You Begin

Complete Chapter 2. Check that the `Attachments` folder exists and the test email from section 2.3 is saved in Drafts. That draft uses the subject `[PA TRAINING] Attachment test` and has two of the three files from the Chapter 2 test files attached:

File	Purpose

CourseOutline.pdf	The attachment that should be saved
TrainerPhoto.jpg	The attachment that should be skipped
SessionNotes.docx	Used only in the independent practice

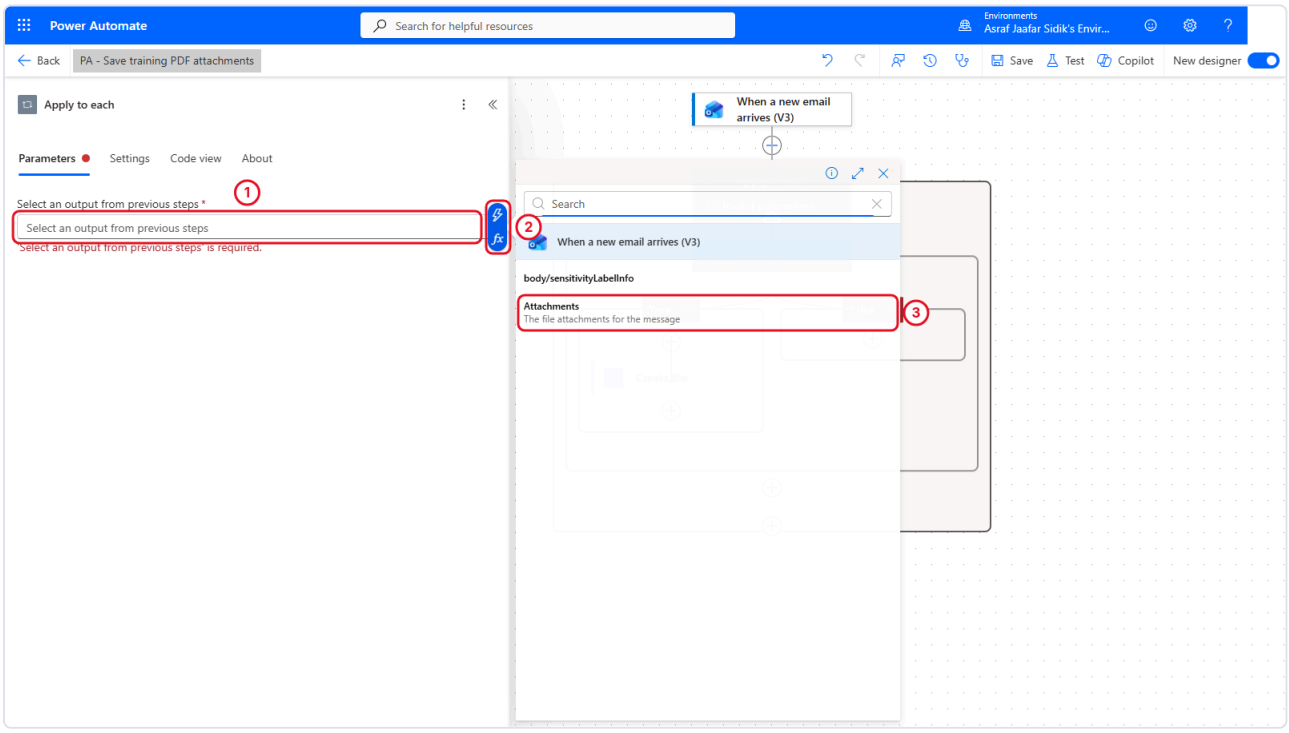
> **Note:** OneDrive never silently overwrites a file. If you run the test twice with the same attachment, Create file either fails with a duplicate-name error or OneDrive saves `CourseOutline (1).pdf`. Both are correct behaviour. Delete the file from Attachments before a repeat test.

The rule: when a new email arrives with `[PA TRAINING]` in the subject, examine each attachment. If its name ends in `.pdf`, create a file in the Attachments folder.

How to Insert Dynamic Content

You'll do this in every chapter, so learn it once. **Dynamic content** is a value produced by an earlier step, such as an email's subject or an attachment's name.

1. Click into the field you want to fill.
2. Select the **lightning bolt** icon that appears beside the field (or type `/` and choose **Insert dynamic content**).
3. Pick the value from the list. Values are grouped under the step that produces them. Use the search box if the list is long.



The lightning bolt opens the dynamic content picker. Values are grouped by the step that produced them.

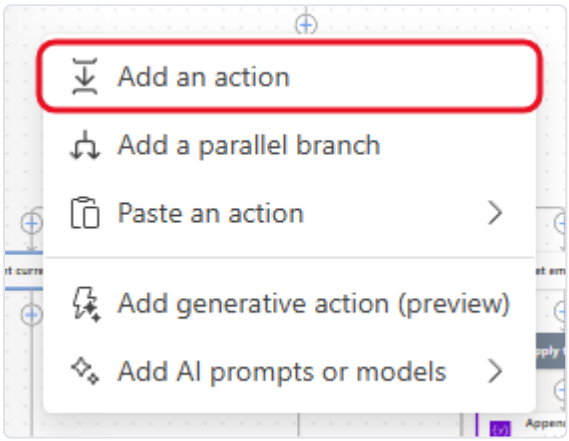
The value appears in the field as a coloured token. To remove it, select the **x** on the token.

> **Note:** After you save and come back, some tokens show their technical name instead of the friendly one: **Attachments Name** shows as `name`, **Attachments Content** as `contentBytes`, **value** as `body/value`, and Forms answers as codes like `body/r96281c9…`. That's normal, and the screenshots in this course show them that way too. They still point at the right value.

How to Add an Action and Check Your Flow

You'll also do these in every chapter.

- **Add an action:** select the **+** on the line where the new step should go (below the last step, or between two steps), then **Add an action**. A panel opens on the left with a search box. Type the action's name; results are grouped by connector, such as **Office 365 Outlook** or **Control**. Select the action you want.
- **Save:** the **Save** button is at the top right of the designer.
- **Flow checker:** the small stethoscope icon just left of **Save**. It opens a panel listing errors and warnings. Close it with the **X**.
- **Test:** the **Test** button next to **Save**. A panel opens on the right; choose **Manually**, then select **Test**.



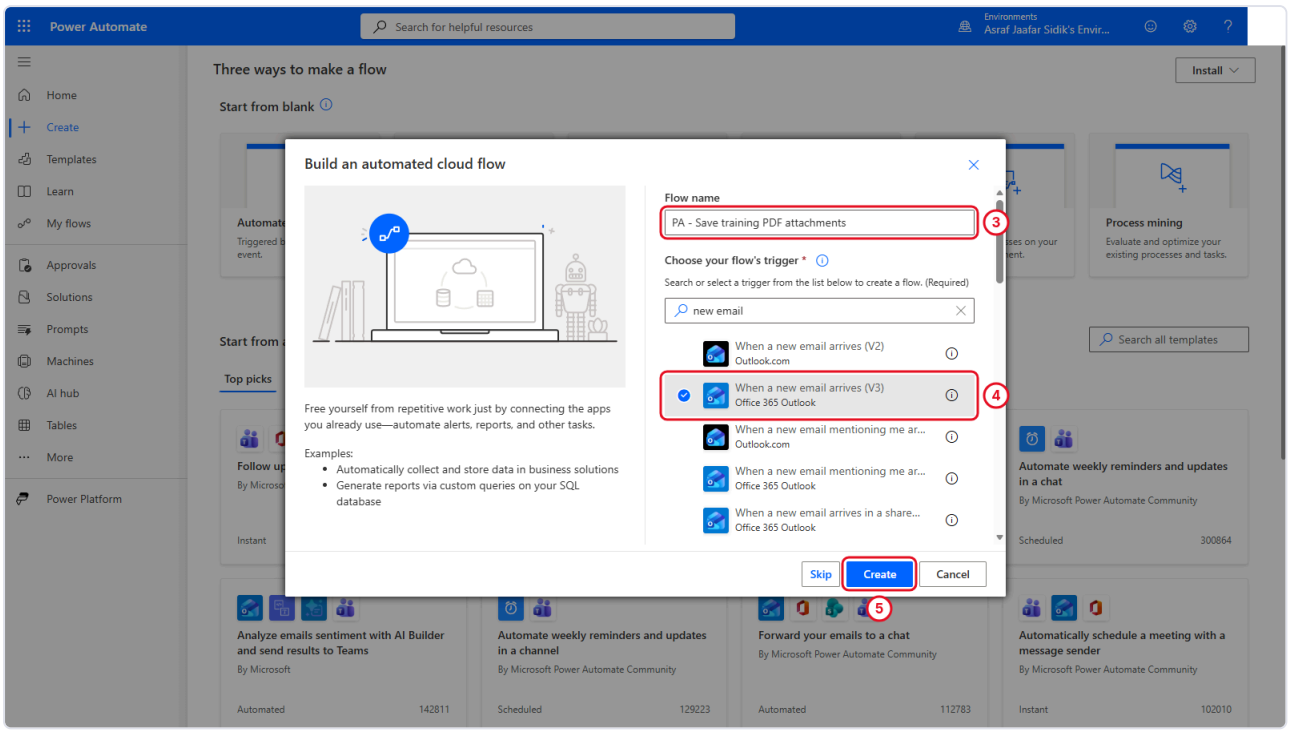
Select a + on the canvas and this menu opens. Add an action is at the top.



Top right of the designer: Flow checker (the stethoscope), Save and Test.

3.1 Create the Automated Cloud Flow

1. Open Power Automate and confirm the correct environment.
2. Select **Create** on the left, then the **Automated cloud flow** tile (shown in Chapter 1, section 1.5).
3. In **Flow name**, enter `PA - Save training PDF attachments`.
4. In the trigger search box, type `new email`, then select **When a new email arrives (V3)** under **Office 365 Outlook**.
5. Select **Create**. The designer opens with only the trigger on the canvas.



Name the flow and pick the Office 365 Outlook trigger before selecting Create.

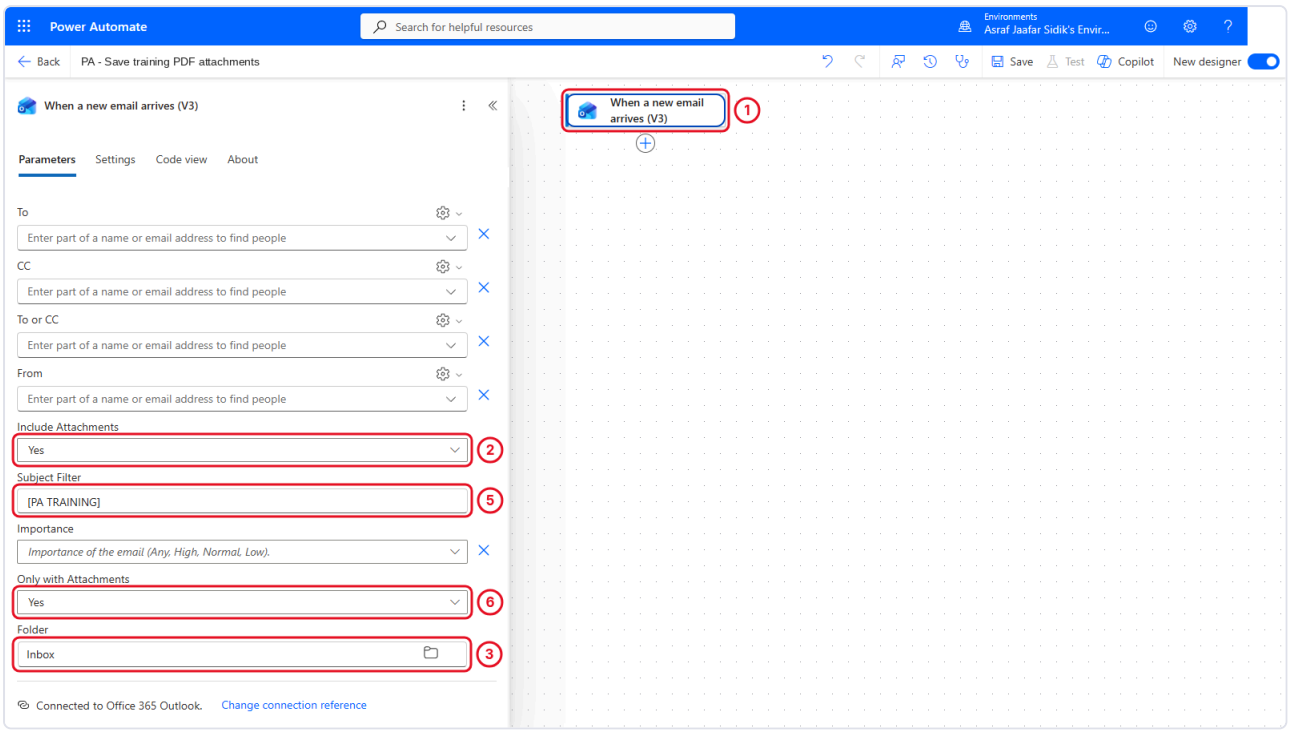
3.2 Configure the Trigger

1. Select the trigger (**When a new email arrives (V3)**) on the canvas. Its settings open in a panel on the left.

   The fields below may sit in a different order on your screen. Find each one by its name.

2. Set **Include Attachments** to **Yes**.
3. Set **Folder** to **Inbox**.
4. Select **Show all** under **Advanced parameters** (if the advanced fields are already showing, skip this).
5. In **Subject Filter**, enter `[PA TRAINING]`.
6. Set **Only with Attachments** to **Yes**.

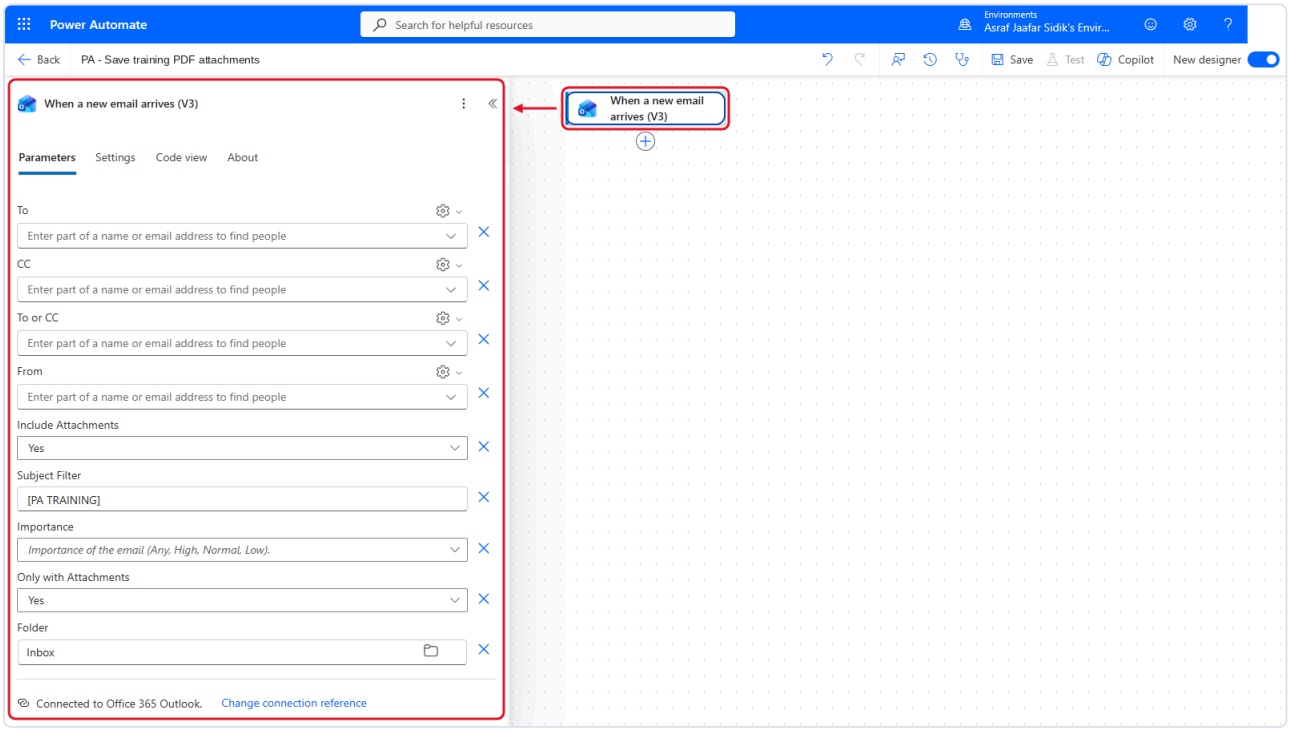
Don't save yet. Power Automate won't save a flow that has only a trigger, so you'll save at the end of section 3.3, once the first action is in place.



All four trigger fields set. Use Show all if a field is hidden.

Checkpoint: No field shows a red validation message. The flow only starts for *new* matching messages. It doesn't go back and process mail already in the inbox.

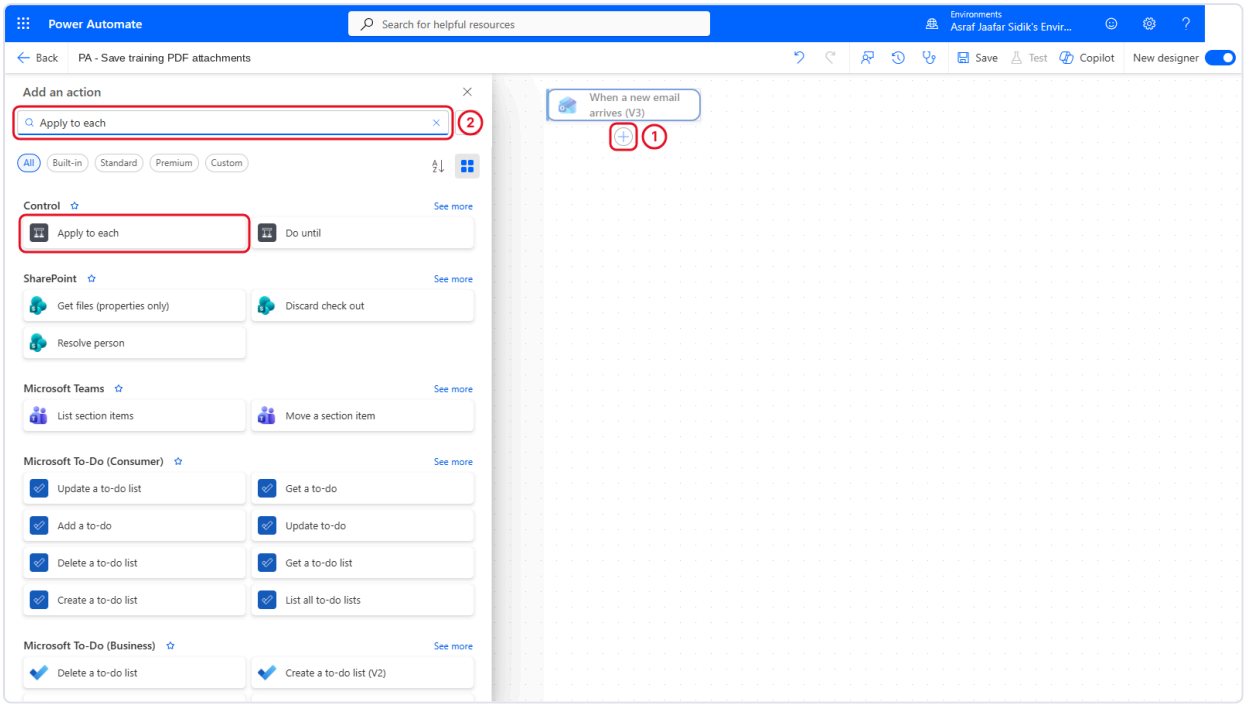
Check your flow so far. Your screen should look like this.



Only the trigger so far. Its settings are open on the left.

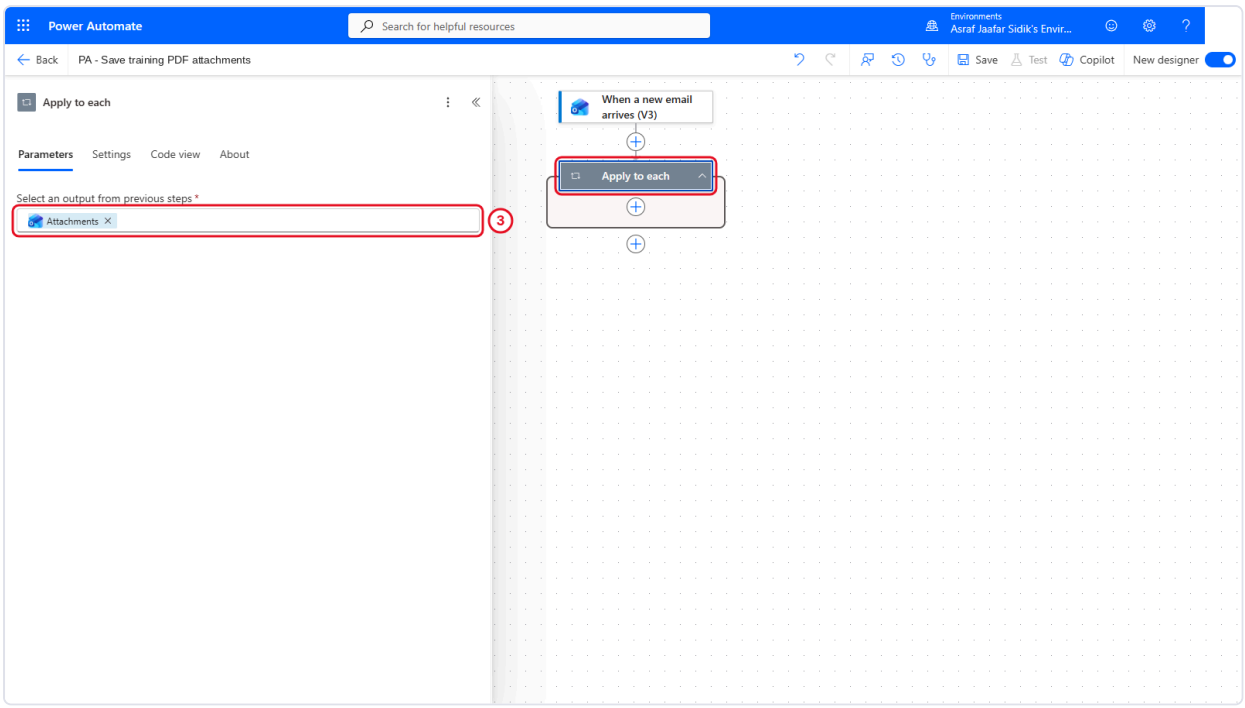
3.3 Add Apply to Each

1. Select the **+** below the trigger, then **Add an action**.
2. Search for `Apply to each` and select it (it's under **Control**).



Search by name, then pick the result under Control.

3. The loop's settings open on the left. Click into **Select an output from previous steps**, open the dynamic content picker (the lightning bolt), and select **Attachments** under *When a new email arrives (V3)*.

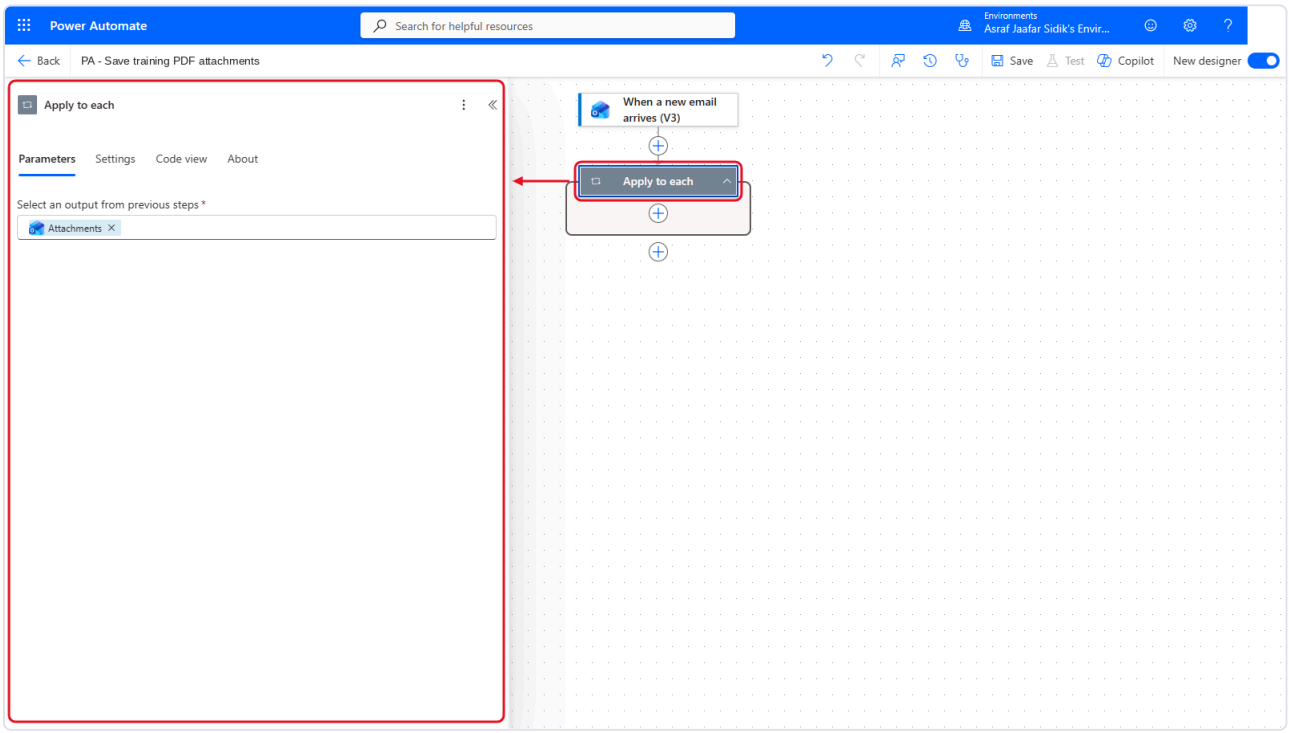


The loop runs once for every attachment in the email.

If **Attachments** isn't in the list, go back to the trigger and confirm Include Attachments is set to Yes.

4. Select **Save** (top right). Now that there's an action, the flow can be saved.

Check your flow so far. Your screen should look like this.

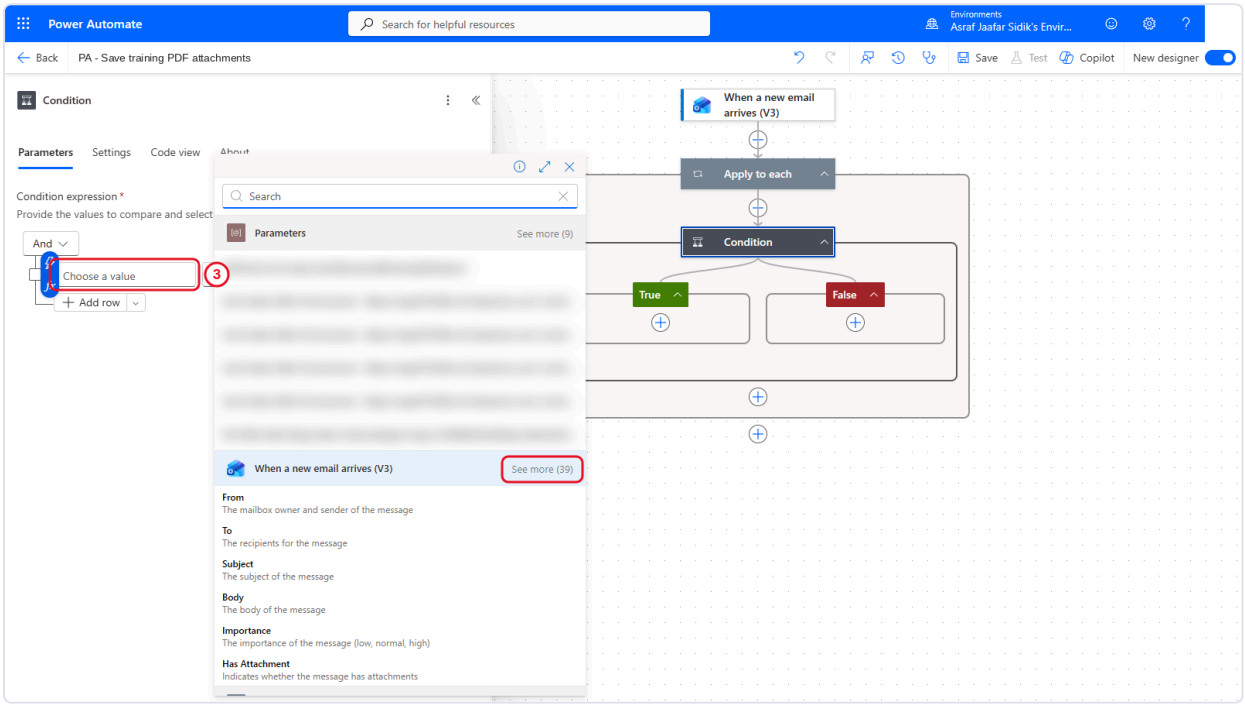


The trigger, then an empty Apply to each.

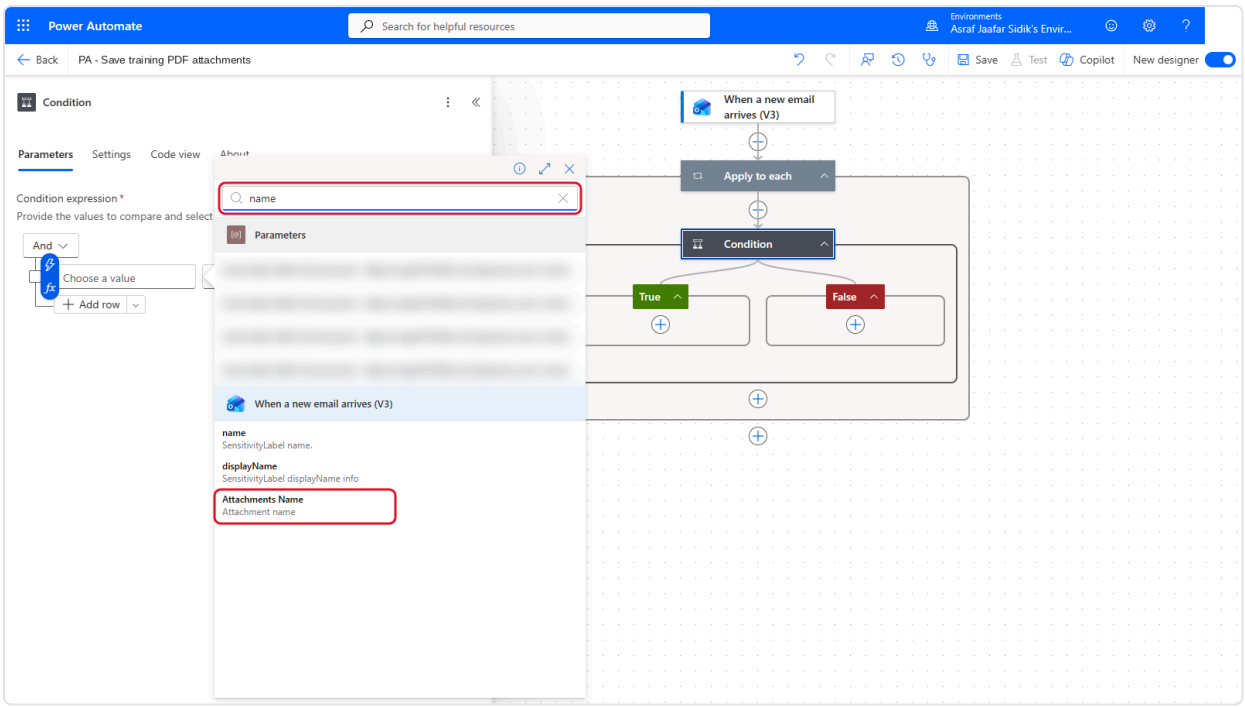
3.4 Add the PDF Condition

1. Select the **+** inside the **Apply to each** box (not the one below it), then **Add an action**.
2. Search for `Condition` and select it (under **Control**).
3. Click into the left **Choose a value** box. Open the dynamic content picker and select **Attachments Name**. (After saving it shows as `name`.)

> **Can't see Attachments Name?** The picker only shows a few values per step at first. Select **See more** beside *When a new email arrives (V3)*, or type `name` in the picker's search box.



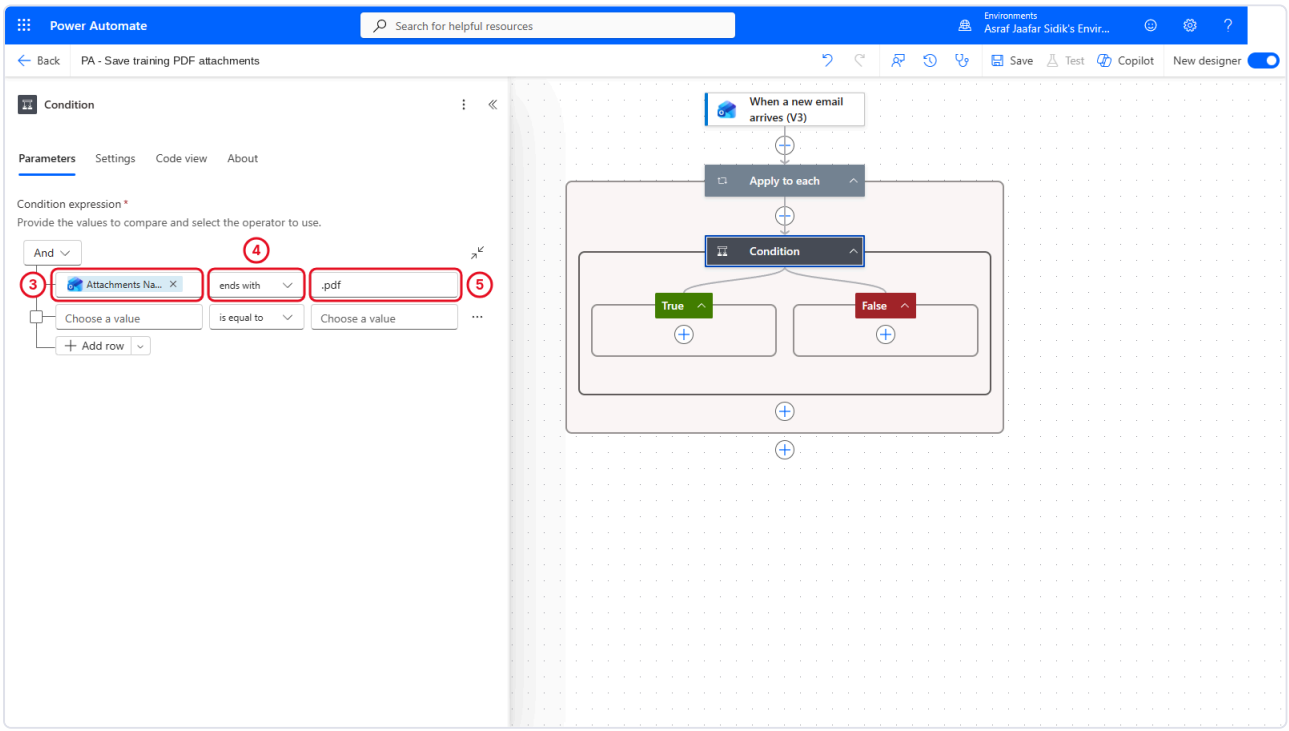
See more lists every value from the trigger.



Or search: typing name brings Attachments Name straight up.

4. Set the middle box to **ends with**.
5. In the right box, type `.pdf`.

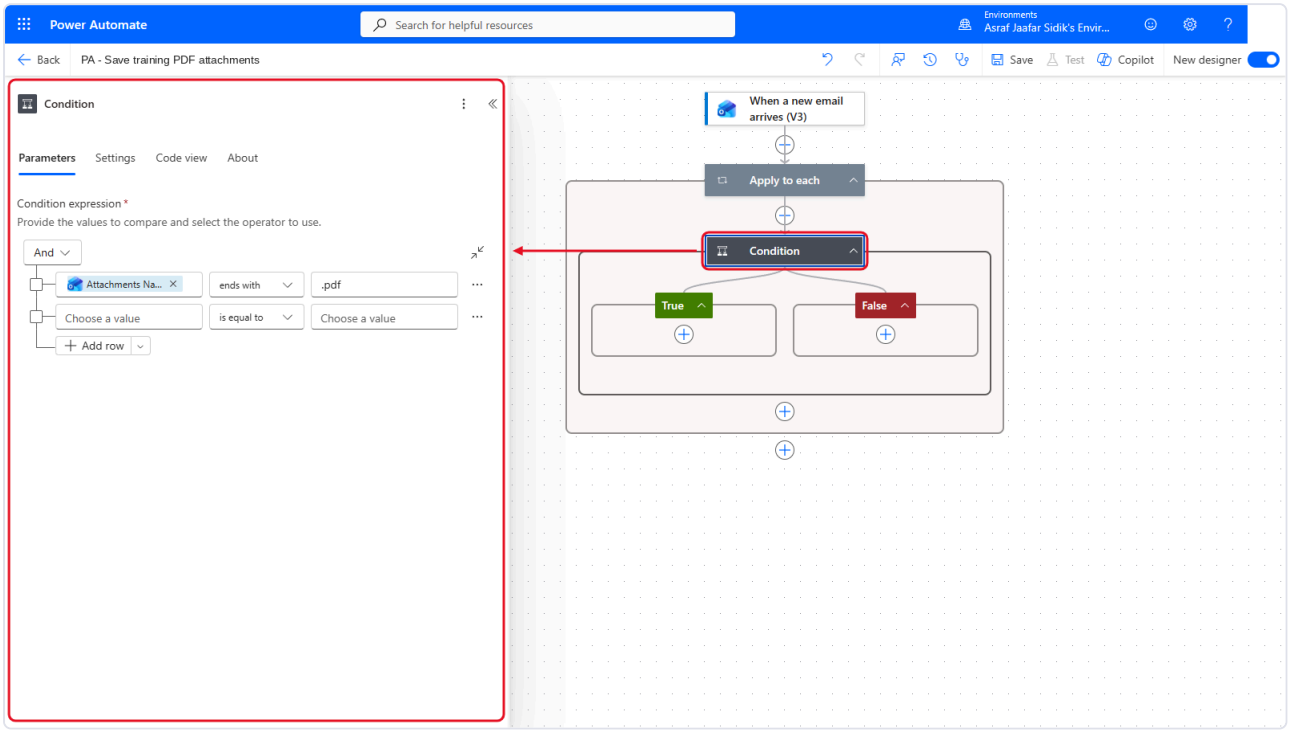
   If a second, empty row appears below yours, select its **...** and then **Delete**.



The condition asks one question about the current attachment: does its name end in .pdf? The screenshot still shows the empty row; delete it on yours.

> **Tip:** Make sure you picked **Attachments Name** (the current attachment's name), not **Subject** or another text value. Inside a loop, the "Attachments ..." values always refer to the item the loop is currently on.

Check your flow so far. Your screen should look like this.



The Condition sits inside Apply to each, with empty True and False boxes.

3.5 Create the OneDrive File

1. Select the **+** inside the green **True** box, then **Add an action**.
2. Search for `OneDrive for Business`, then select **Create file** from that connector's group.

   > **Tip:** Search by connector name, not action name. Several connectors have an action called Create file, and it's easy to grab the wrong one.

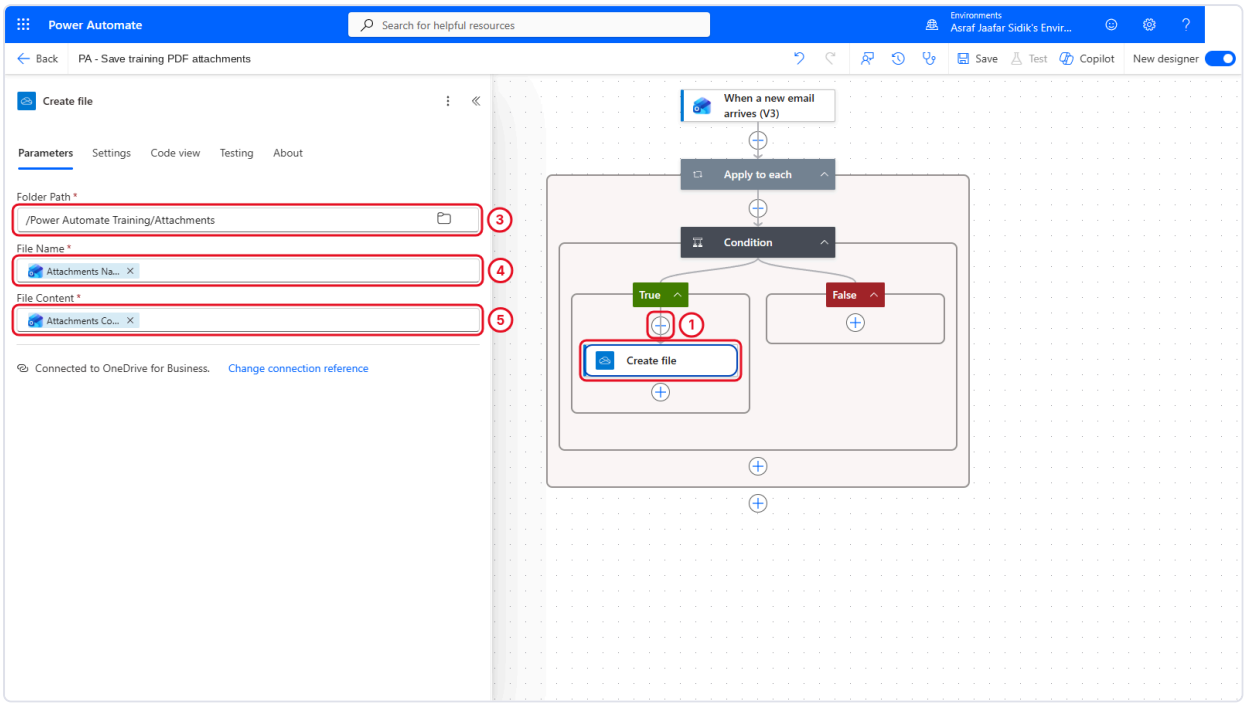
3. In **Folder Path**, select the folder icon at the right end of the box. A list of your OneDrive folders opens: select the **>** arrow beside `Power Automate Training` to open it, then select `Attachments`.



The folder icon opens your OneDrive folders.

4. In **File Name**, insert **Attachments Name** from the dynamic content picker (use **See more** or search `name` if it isn't listed).

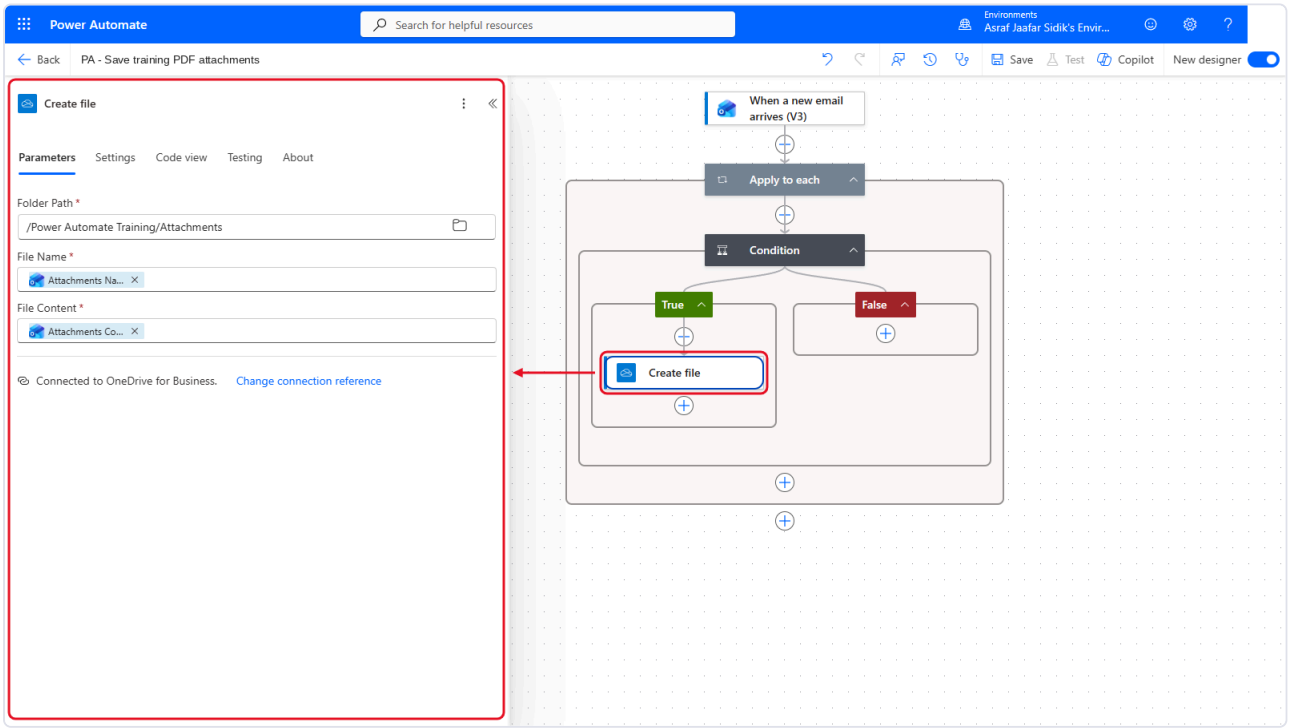
5. In **File Content**, insert **Attachments Content** (search `content` if it isn't listed).
6. Leave the **False** branch empty. That's the deliberate decision to skip non-PDF files.



Create file sits inside True. False stays empty.

7. Select **Save** (top right), then the **Flow checker** icon (the stethoscope just left of **Save**). Confirm **Errors (0)** and **Warnings (0)**, then close the panel.

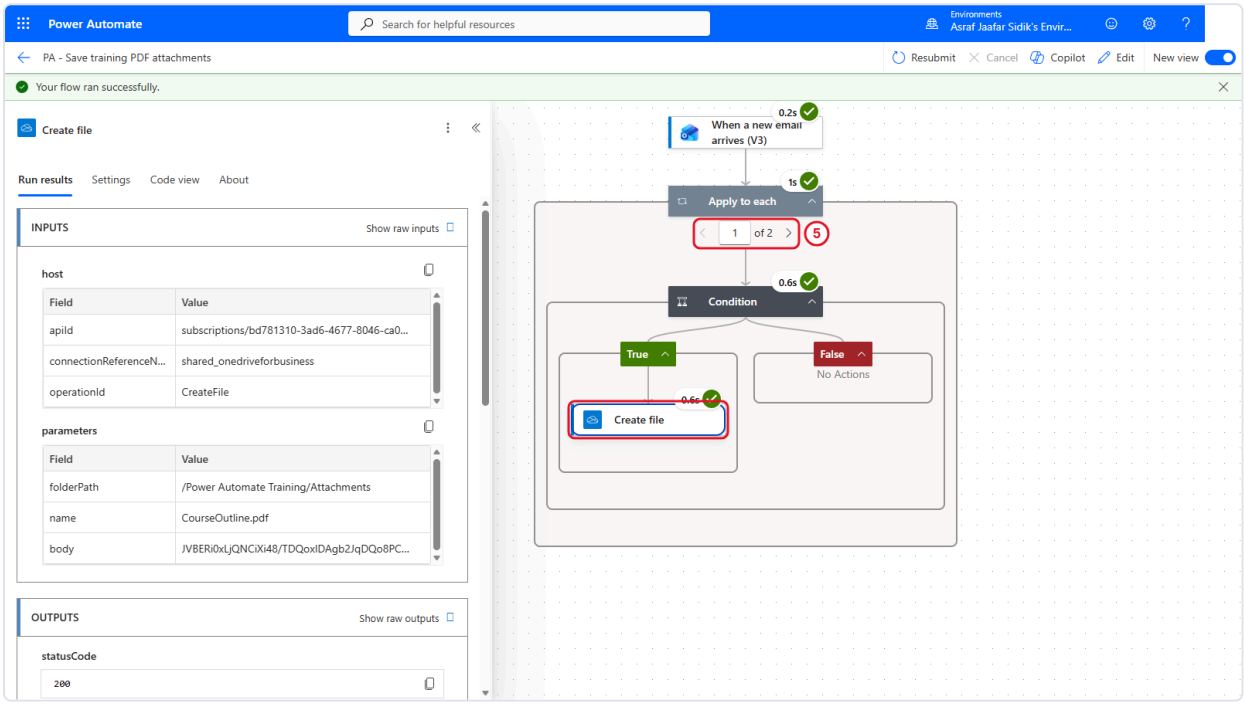
Check your flow so far. Your screen should look like this.



Your finished flow: trigger, Apply to each, Condition, and Create file in True.

3.6 Test a Positive Case

1. Select **Test** (top right). In the panel that opens, choose **Manually**, then select **Test**.
2. Wait for the banner asking you to send a new email. The flow is now listening.
3. Open the draft from section 2.3 in Outlook and send it.
4. Back in Power Automate, wait for the run to finish. Every step shows a green tick.
5. Select **Apply to each**. The counter shows which attachment you're looking at, out of 2. Use the arrows to find the iteration where **Create file** has a green tick, then select **Create file** to see that it succeeded.



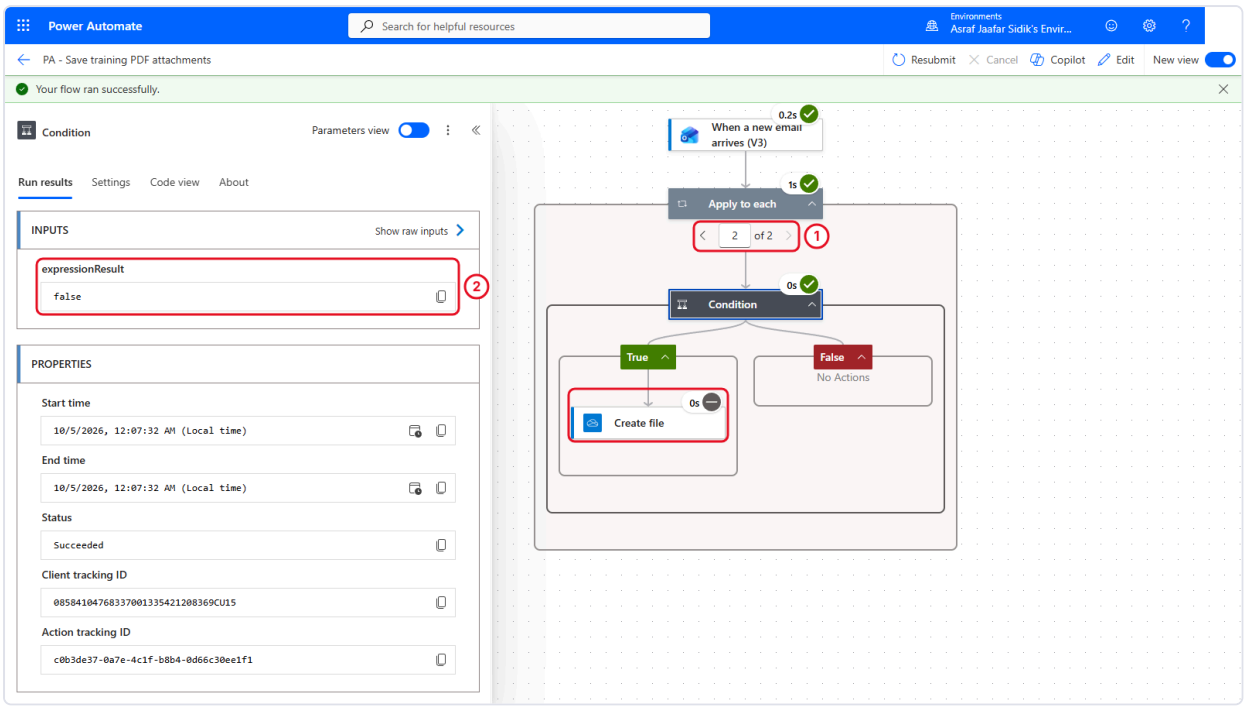
The run processed both attachments.

6. Open the Attachments folder in OneDrive. The PDF is there. No image file was created.

3.7 Test a Negative Case

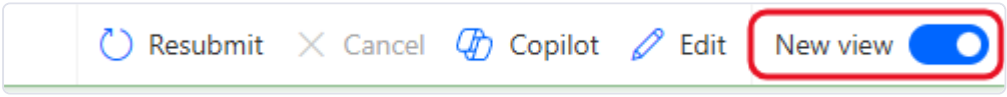
You don't need another email. The mixed test already produced both outcomes.

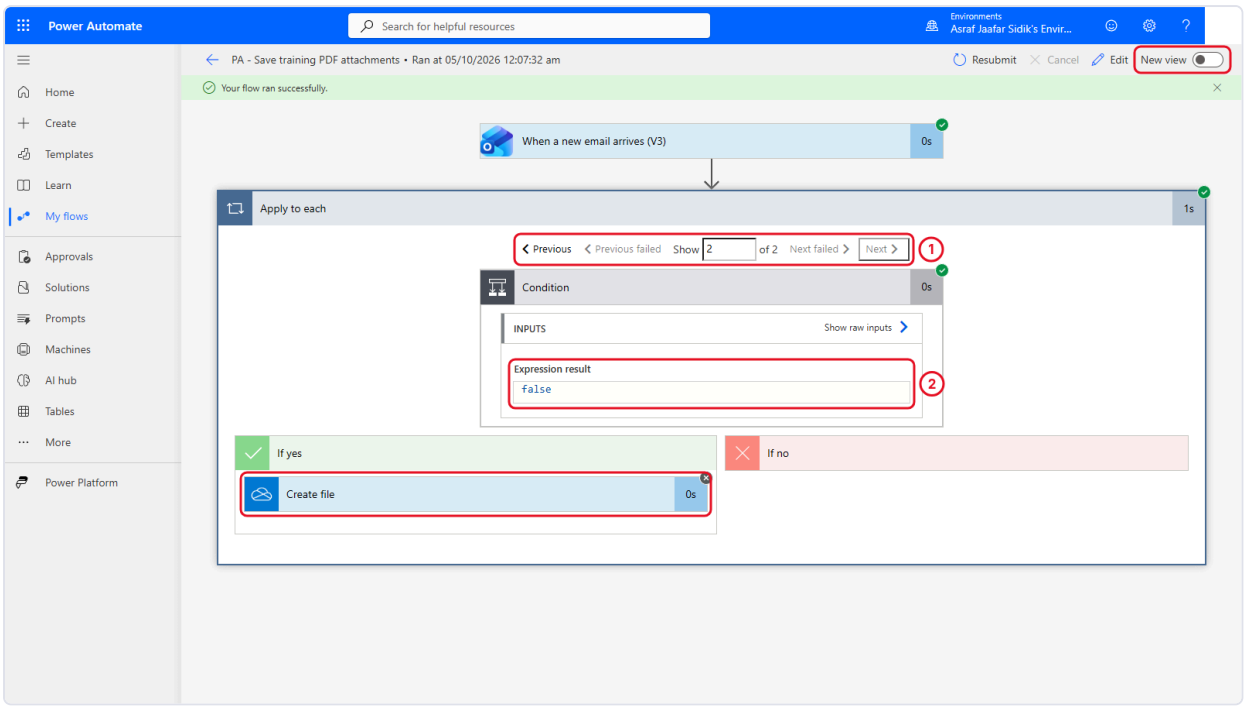
1. In the same run, use the arrows on **Apply to each** to switch to the other iteration.
2. Select **Condition**. The result is **false**, Create file shows as skipped, and the False branch has nothing to run.



The image attachment took the False branch, so nothing was created.

> **The panel still shows the old result?** The run view sometimes doesn't refresh the details panel when you change the iteration. Select a different step (such as **Apply to each**), then select **Condition** again. You can also trust the canvas: on this iteration **Create file** shows a grey skip icon instead of a green tick. If it still won't update, switch **New view** off at the top right of the run page to use the classic run view, where each iteration opens inline.





The classic view: True and False show as If yes and If no, and the skipped Create file has a grey cross.

> **Key point:** A run can succeed while deliberately producing nothing. *Succeeded* means the process completed, not that every branch created an output.

Independent Practice

Change the condition to save `.docx` attachments instead of PDFs. Select **Save**, then **Test** > **Manually** > **Test**. Predict which file will appear, then send a new email to yourself with the subject `[PA TRAINING] Docx test` and both `SessionNotes.docx` and `CourseOutline.pdf` attached. Check the run and the Attachments folder, then change the condition back to `.pdf` and save.

> **Going further: your first look at an expression.** Send the same PDF twice and OneDrive numbers the copy or refuses it. To give every file a unique name, replace the **File Name** token with an expression that adds the date and time in front: select the **fx** button beside the box (or type `/` and choose **Insert expression**) and paste the timestamp expression. Copy it from the box below. It produces names like `20260913-094512-CourseOutline.pdf` . You'll meet expressions properly in Chapters 4 and 5. Optional.
>
> ```
> concat(formatDateTime(utcNow(),'yyyyMMdd-HHmmss'), '-', item()?['name'])
> ```

Tips and Troubleshooting

Problem	Check

The flow doesn't run	The message must be new, contain `[PA TRAINING]` in the subject, and reach the mailbox used by the Outlook connection
Attachments isn't in the dynamic content list	Turn on Include Attachments in the trigger, save (once the flow has an action), and reopen the loop
The loop runs but creates nothing	Check the condition: left is **Attachments Name**, middle is *ends with*, right is `.pdf`
Create file fails	OneDrive connection, folder path, file name and file content
Duplicate file error	Delete the old file from Attachments, or try the Going further box
The JPG is saved	Create file is outside the condition. Drag it into True

Lesson Summary

This flow combines an event trigger, a loop over attachments, a condition for the business rule and a OneDrive action for the result, all built by clicking and picking dynamic content. You tested both a file-producing path and a successful no-output path. Next, an instant flow reads the Excel table and sends a summary.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Everything in the main steps is **text**, typed or pasted straight into a field.

Flow name (text, section 3.1)

```
PA - Save training PDF attachments
```

Trigger subject filter (text, section 3.2)

```
[PA TRAINING]
```

Condition right value (text, section 3.4)

```
.pdf
```

Going further (optional)

Unique file name with a timestamp

Paste into the expression editor (click File Name, then select the **fx** button beside it, or type `/` and choose **Insert expression**).

```
concat(formatDateTime(utcNow(),'yyyyMMdd-HHmmss'), '-', item()?['name'])
```

`item()` means "the attachment the loop is on right now", the same thing the **Attachments Name** token points to.

04

CHAPTER 4

Create and Distribute an Excel Summary

The coordinator wants a readable list of training sessions on demand, without copying cells into an email by hand. An instant flow reads the register and sends one summary table. Along the way you write your first expression.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 60 minutes

Your result: An instant cloud flow that reads the training register, picks useful columns, counts the rows and sends one summary email. A Teams update is an optional extension.

> **Download:** the files for this lab
>
> - TrainingRegister.xlsx: only if you don't have the workbook from Chapter 2 yet. Upload it to your `Power Automate Training` folder in OneDrive.

What You Will Learn

- Start a flow on demand with an instant trigger
- Read rows from an Excel table with **List rows present in a table**
- Choose report columns with the **Select** action
- Turn the rows into one HTML table and send it in a single email
- Write your first expression to count the rows
- Inspect run history to confirm what each action produced

How the Flow Works

Manually trigger a flow > List rows present in a table > Select > Create HTML table > Send an email (V2)

The Excel action returns a list of rows. Select keeps only the columns readers need. Create HTML table turns the whole list into one table. The email action runs once, after the table is complete.

Before You Begin

You need `Power Automate Training/TrainingRegister.xlsx` in OneDrive for Business with a formatted table named `tblTraining`:

Column	Contents	Used in this chapter for
TrainingID	TR-001 to TR-006	The independent practice
ParticipantName	Fictional names	A report column
Email	Placeholder addresses	Read but deliberately left out of the report
CourseTitle	Fictional course names	A report column
SessionDate	Real Excel dates	A report column
Status	Registered or Completed	A report column

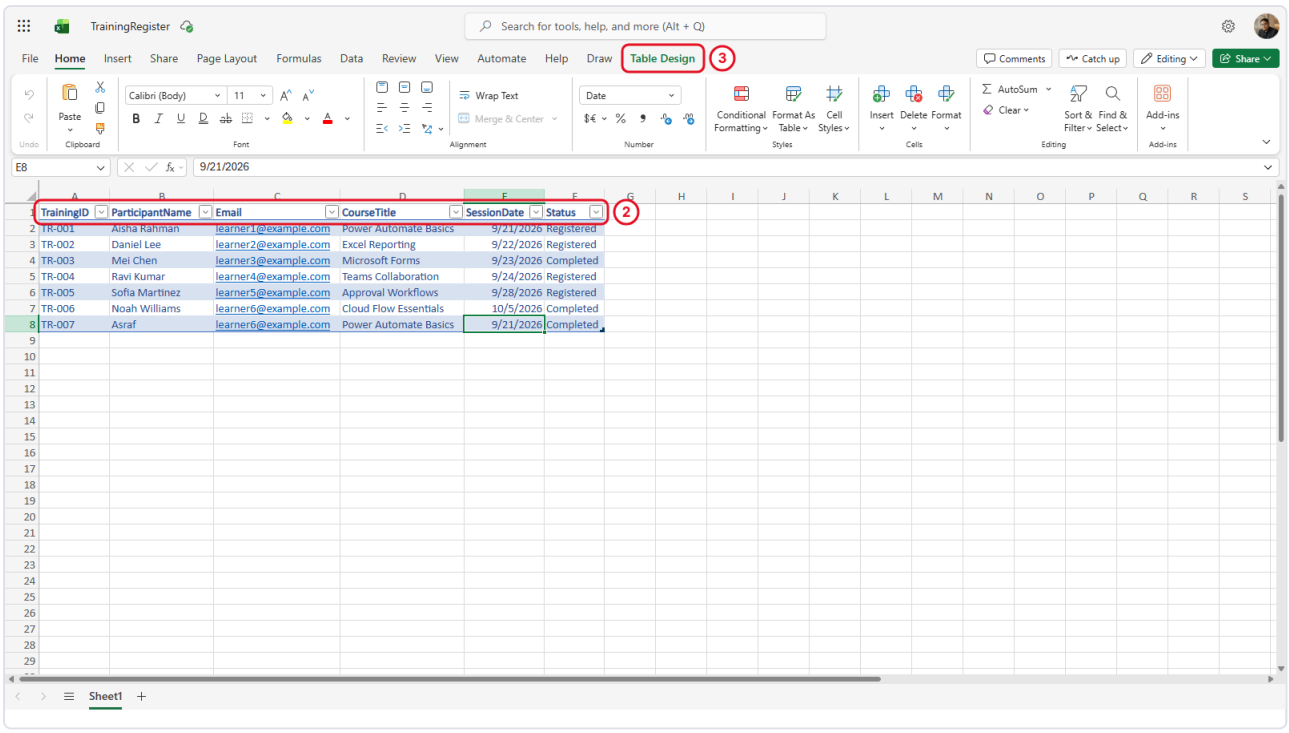
Note: Addresses like `learner1@example.com` are just data in this chapter. `example.com` can't receive mail. The summary goes to your own training mailbox, not to the people in the table.

Tip: Before each run, finish any cell edit and let Excel save. The connector reads the saved cloud file, so an open edit gives you a stale report.

Important: Keep the workbook in Excel for the web during class. If it's open in the Excel desktop app while a flow runs, the flow can read old data or fail because the file is locked. Close the desktop app before testing.

4.1 Confirm the Source Table

1. Open `TrainingRegister.xlsx` from the training folder. You built it in section 2.2, so you're only checking it here.
2. Check the header row. All six columns must be present and spelled exactly as above.
3. Select a cell in the data, open **Table Design**, and confirm the table name is exactly `tblTraining`.

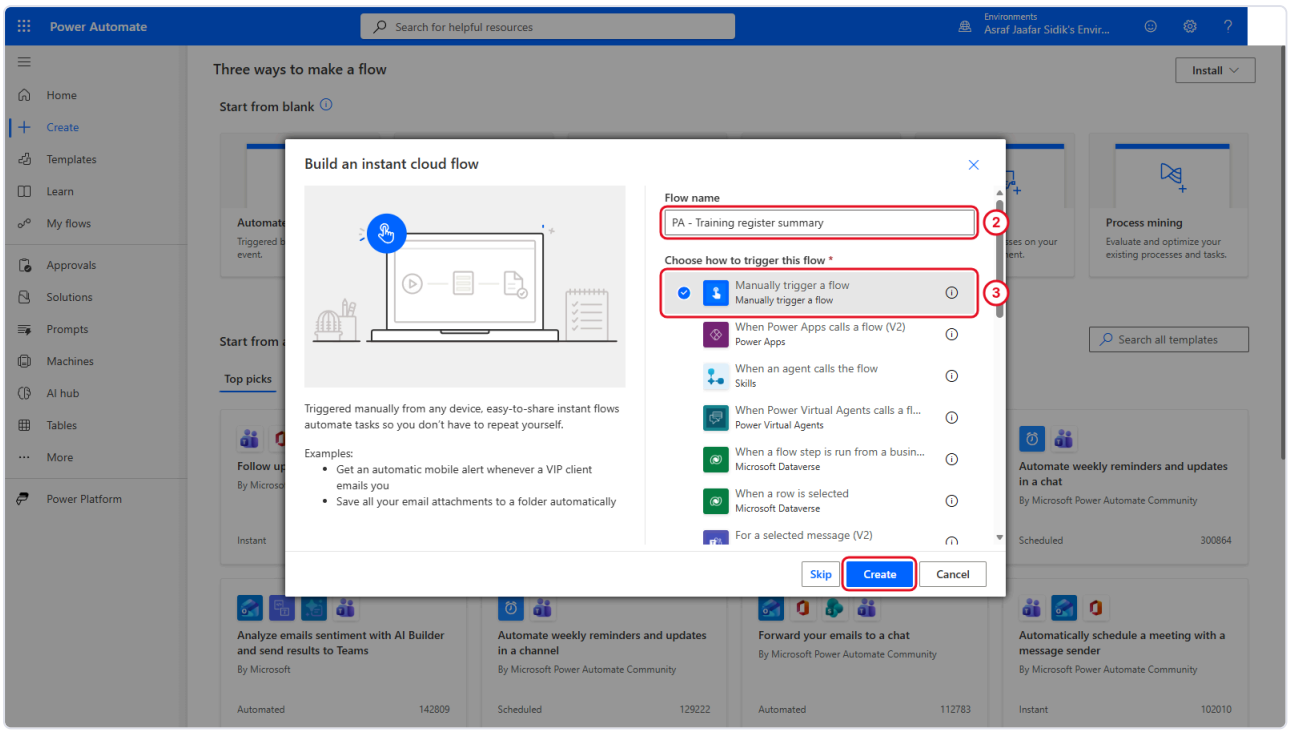


Select any cell in the table, then open the Table Design tab to see its name. The table name, not the sheet name, is what Power Automate looks for. The screenshot has an extra test row, TR-007; yours has six.

> **Key point:** A worksheet name and a table name are different things. If the Table dropdown is empty later, a missing or misnamed table is almost always why.

4.2 Create an Instant Cloud Flow

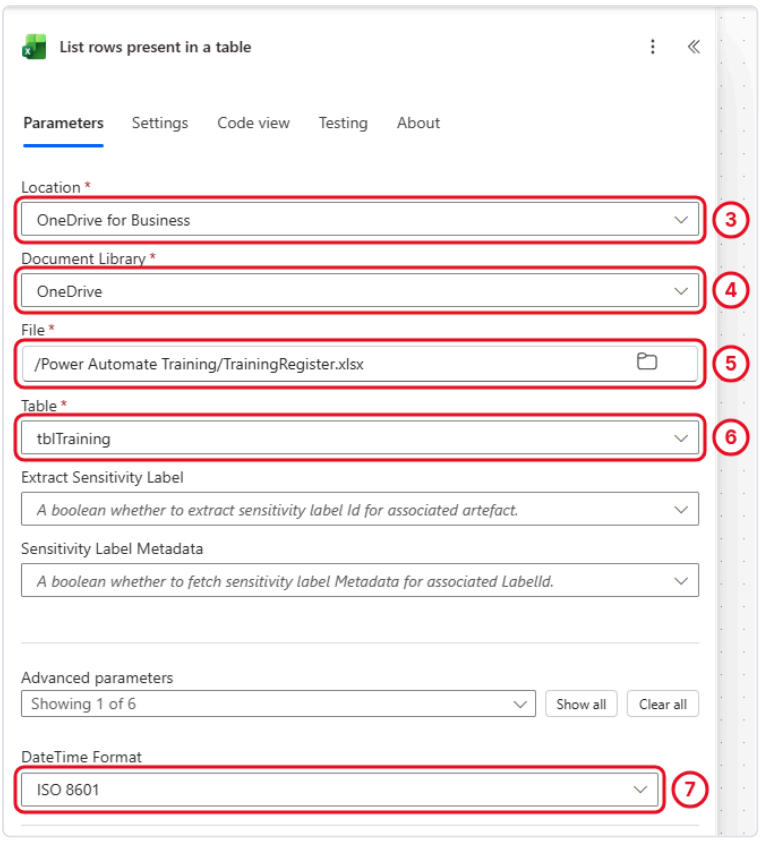
1. Confirm the training environment, select **Create** on the left, then the **Instant cloud flow** tile.
2. In **Flow name**, enter `PA - Training register summary`.
3. Select **Manually trigger a flow**, then **Create**.



This trigger starts only when you run it.

4.3 Read the Excel Records

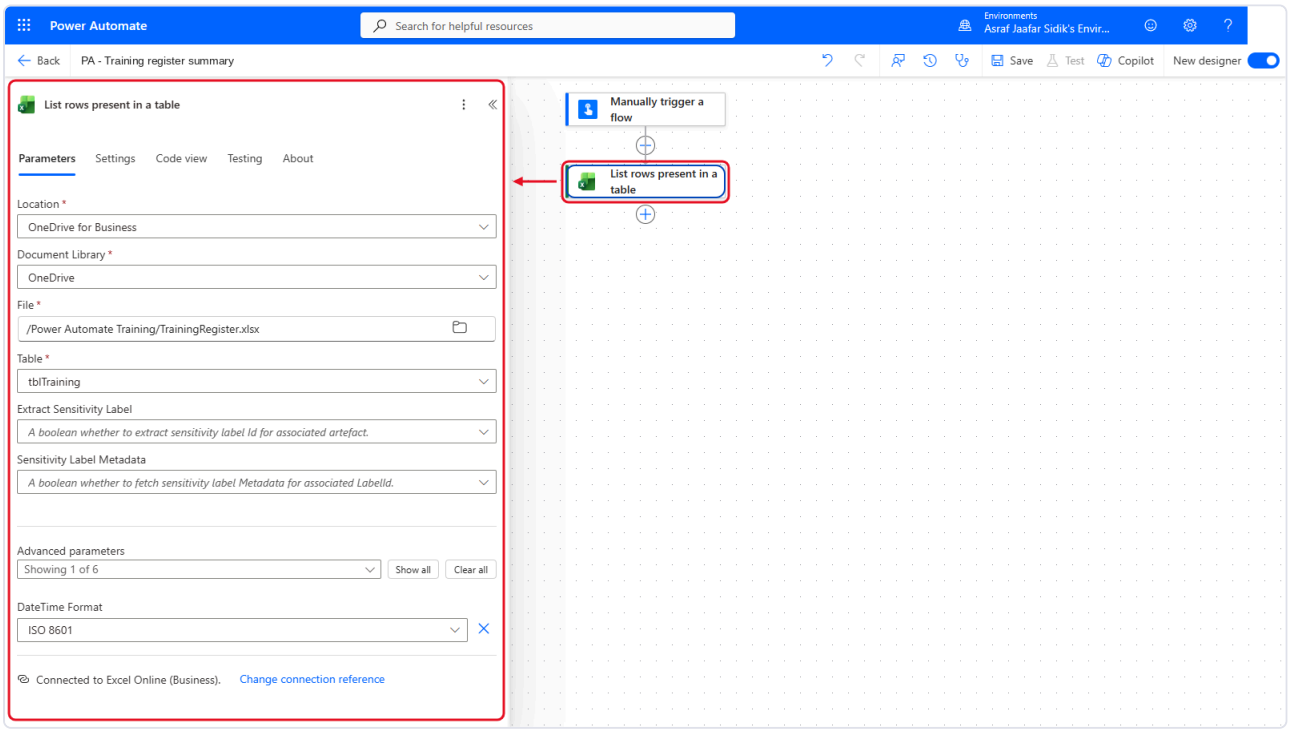
1. On the canvas, select the **+** below **Manually trigger a flow**, then **Add an action**.
2. In the search box at the top of the panel, type `List rows present in a table`. Under the **Excel Online (Business)** heading, select **List rows present in a table**. Its settings open on the left.
3. Open the **Location** dropdown and select **OneDrive for Business**.
4. Open the **Document Library** dropdown and select **OneDrive**.
5. In **File**, select the folder icon at the right end of the box. Select the **>** beside `Power Automate Training`, then select `TrainingRegister.xlsx`.
6. Open the **Table** dropdown and select `tblTraining`.
7. Under **Advanced parameters**, open the dropdown (it reads **Showing 0 of 6**) and tick **DateTime Format**. A new **DateTime Format** box appears: choose **ISO 8601**. This returns dates as dates instead of serial numbers like `46286`.



Pick the file by browsing, then the table from the dropdown. Once saved, Location, Document Library and Table show internal IDs like `me` or `{4672…}` instead of the names you picked. That's normal.

If **Table** shows *No items*, stop. The workbook has no formatted table. Go back to section 2.2.

Check your flow so far. Your screen should look like this.

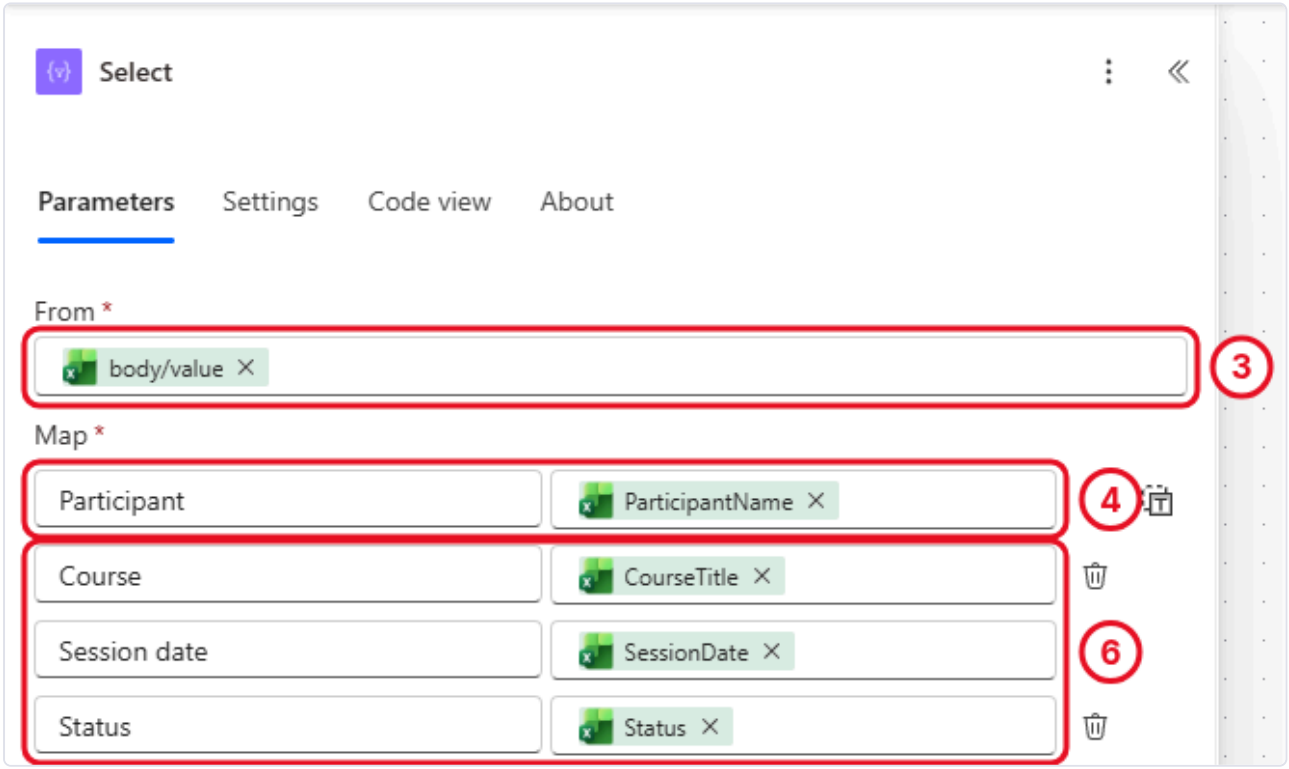


The trigger and List rows present in a table.

4.4 Choose the Report Columns

1. On the canvas, select the **+** below **List rows present in a table**, then **Add an action**.
2. Search for `Select`. Under the **Data Operation** heading, select **Select**.
3. Click into the **From** box. Select the lightning bolt icon that appears beside it to open the dynamic content picker. Under *List rows present in a table*, select **body/value**. This is the list of all rows. (It may be labelled **value**. If you can't see it, select **See more** or type `value` in the picker's search box.)
4. In the **Map** area, click the first **Enter key** box (left) and type `Participant`.
5. Click the **Enter value** box beside it (right), open the lightning bolt, and select **ParticipantName** under *List rows present in a table*. Type `Participant` in the picker's search box if it isn't listed. Don't type the value yourself: it must be the green token from the picker.
6. A new empty row appears underneath. Fill three more rows the same way, typing the key and picking the value. Leave the last empty row blank:

Enter key	Enter value (dynamic content)
Participant	ParticipantName
Course	CourseTitle
Session date	SessionDate
Status	Status

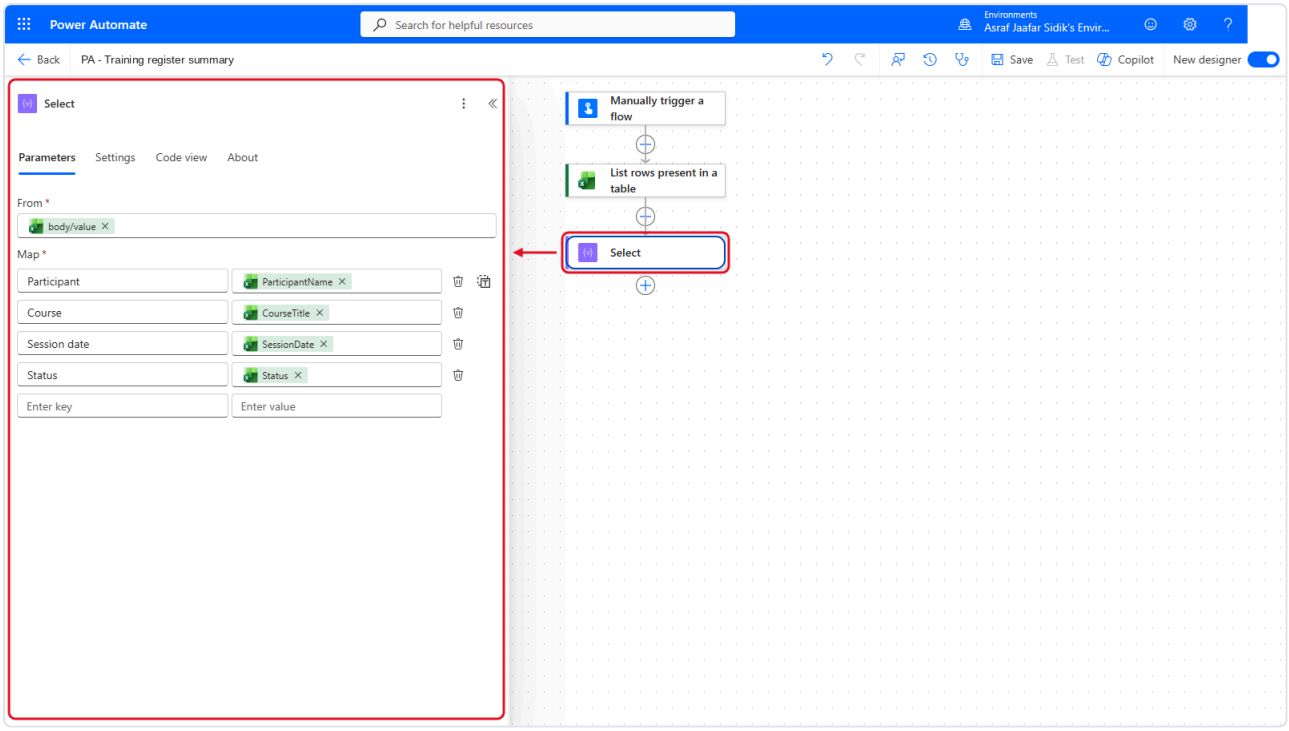


Each key becomes a column heading. Every value is a green Excel token you picked, not typed. Email and the internal row data are left out.

The Session date comes through as `2026-09-21T00:00:00.000Z`. That's fine for now: the Going further box at the end of the chapter shows how to tidy it.

Select handles every row for you, so you don't need an Apply to each here.

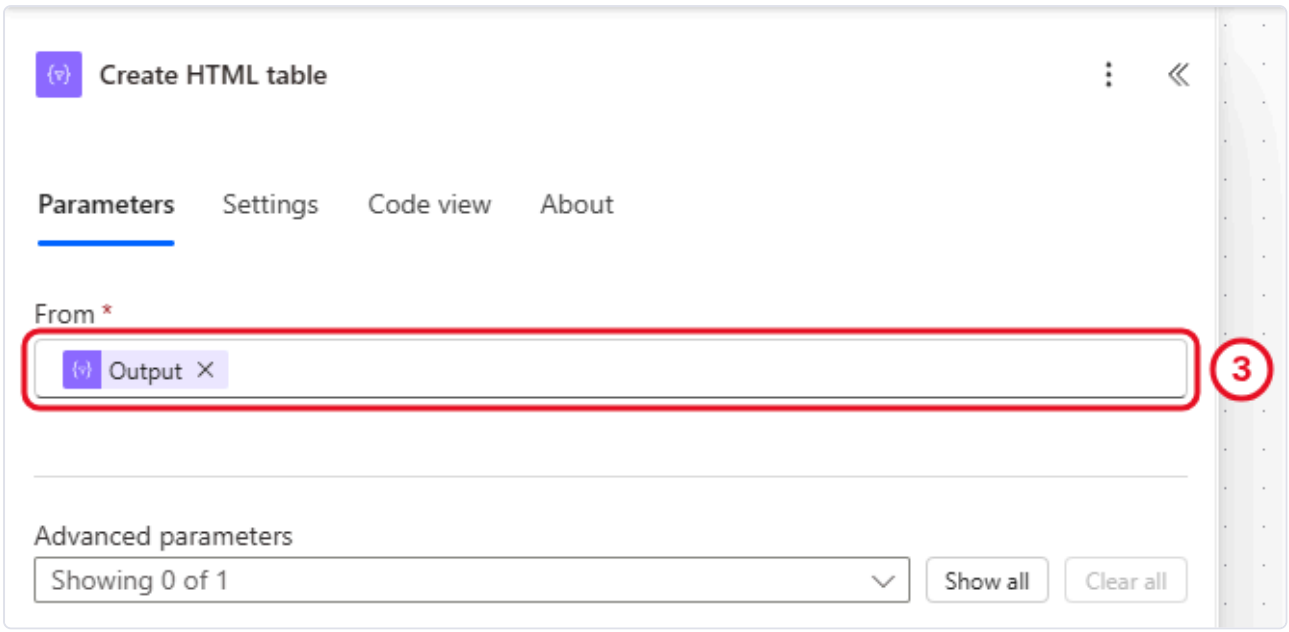
Check your flow so far. Your screen should look like this.



Select comes after List rows present in a table.

4.5 Build One HTML Table

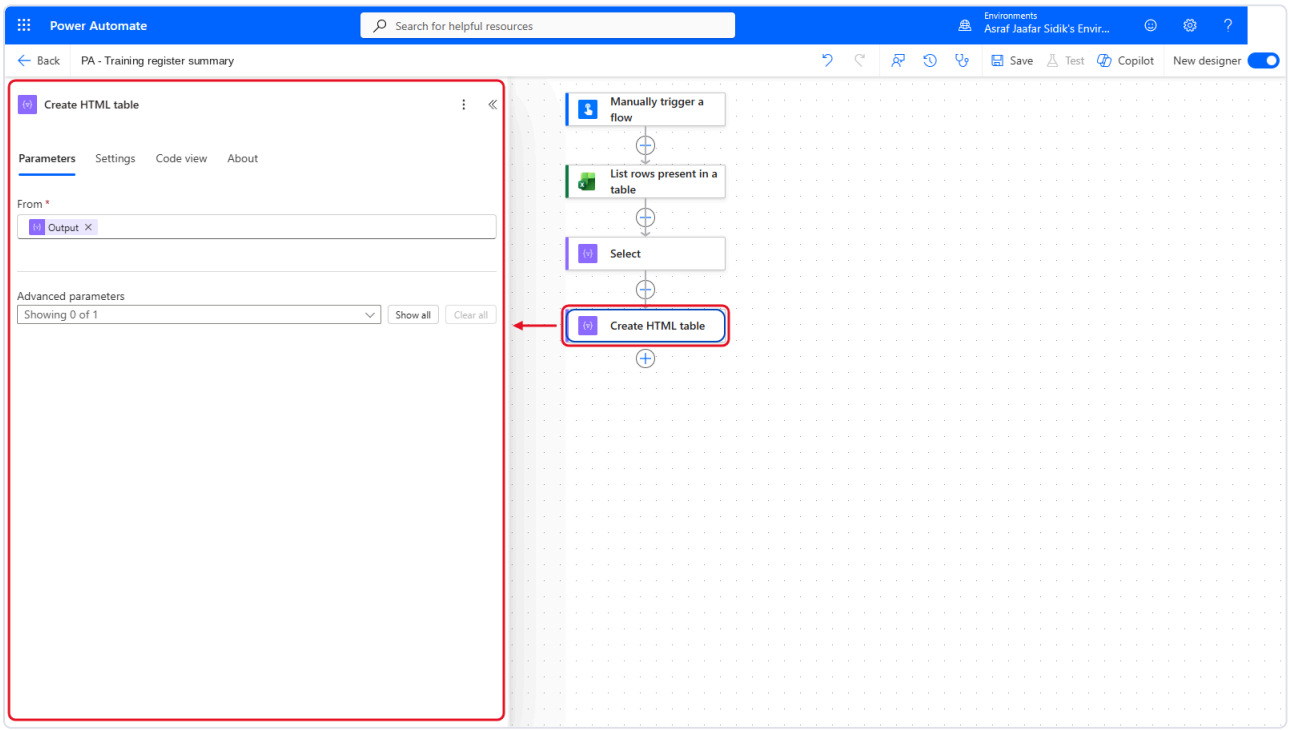
1. On the canvas, select the **+** below **Select**, then **Add an action**.
2. Search for `Create HTML table` and select it under **Data Operation**.
3. Click into **From**, open the lightning bolt, and select **Output** under *Select*.
4. Leave everything else as it is. Select already defined the headings.



The whole list goes in, one table comes out.

Checkpoint: If an Apply to each suddenly wraps this action, you picked a single column instead of the whole Output. Delete the loop and pick **Output** again.

Check your flow so far. Your screen should look like this.



Create HTML table comes after Select.

4.6 Send the Coordinator Summary

1. On the canvas, select the **+** below **Create HTML table**, then **Add an action**. Search for `Send an email`, then select **Send an email (V2)** under **Office 365 Outlook**.
2. In **To**, start typing your name or email address, then select yourself from the list that appears.
3. In **Subject**, enter the subject. Copy it from the box below.

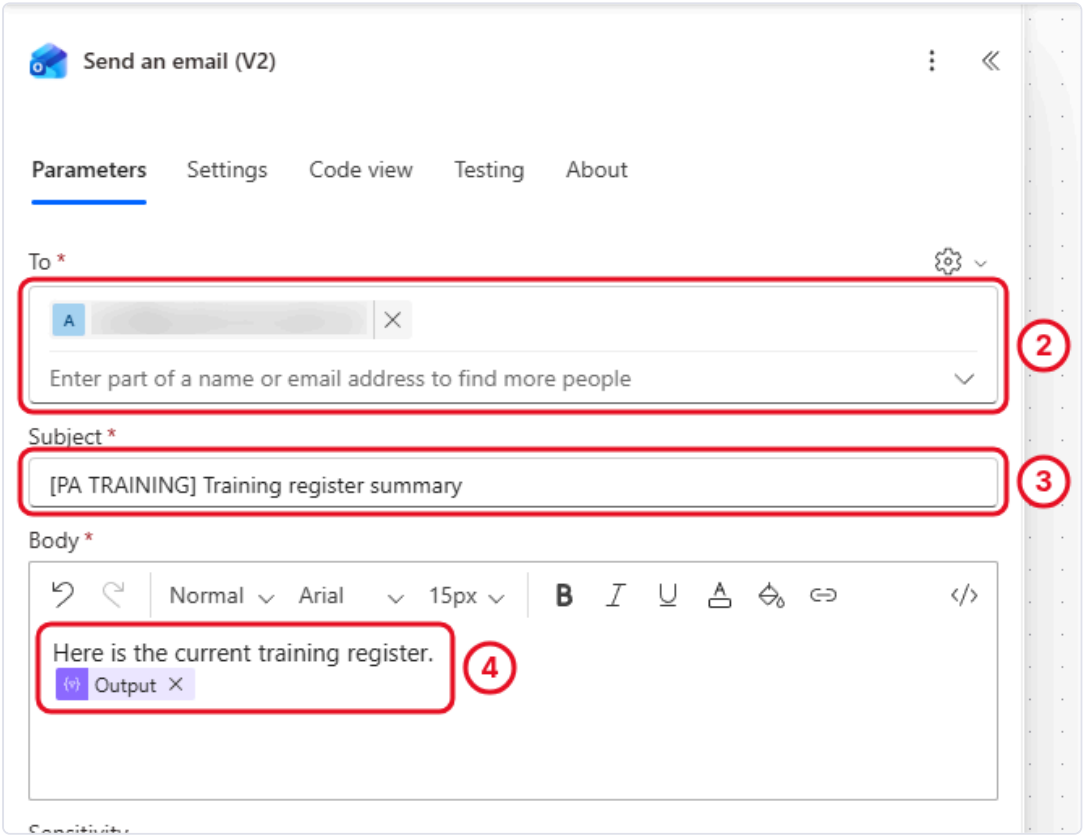
   ```
   [PA TRAINING] Training register summary
   ```

4. Click into **Body**, type the introduction below, then press Enter.

   ```
   Here is the current training register.
   ```

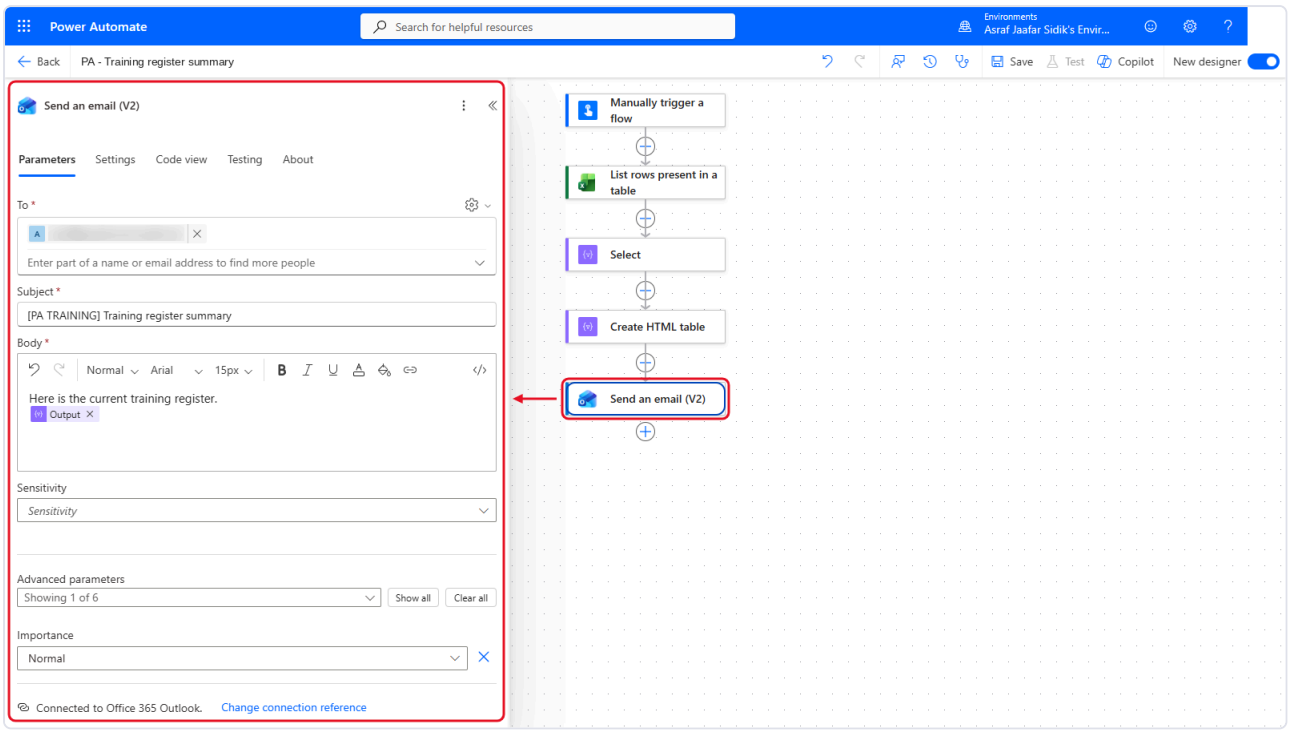
5. On the new line, type `/` and choose **Insert dynamic content** (or select the lightning bolt). Select **Output** under the *Create HTML table* heading. The picker lists two values called **Output**: pick the one under *Create HTML table*, not the one under *Select*.

   Leave the body as these two lines for now. You add the count sentence in 4.7.



The table token goes in the body below a short introduction.

Check your flow so far. Your screen should look like this.

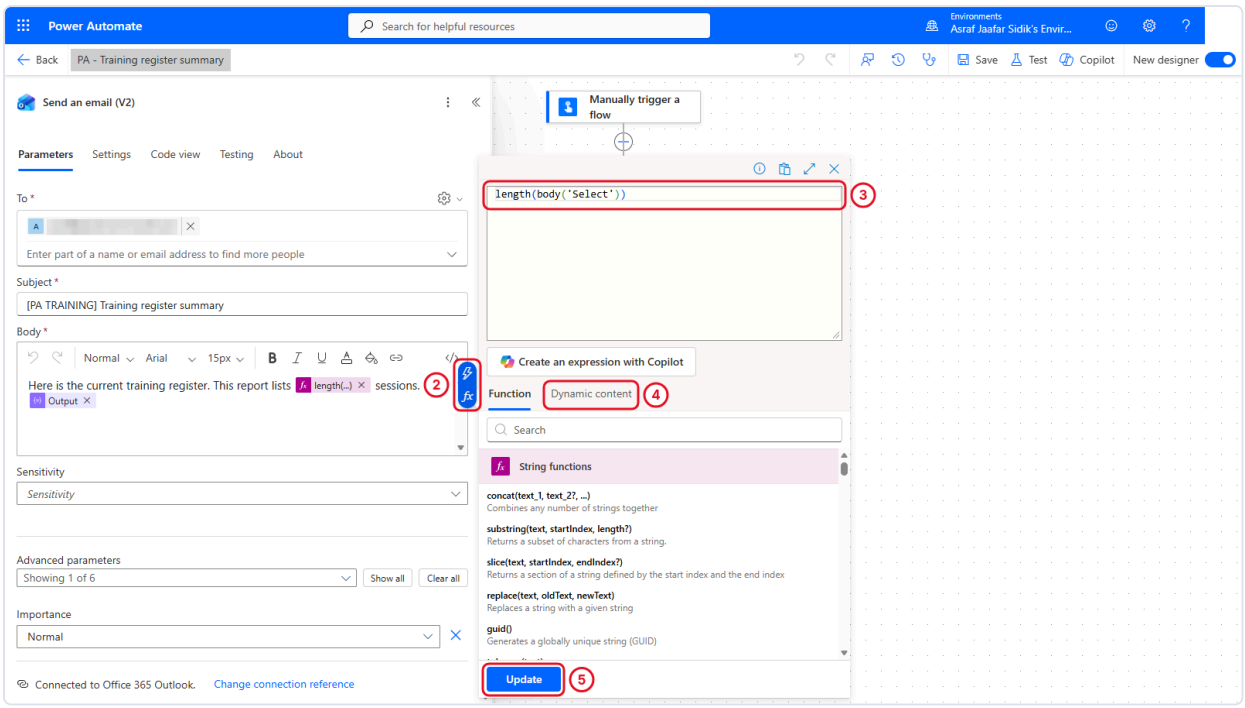


Five steps, with Send an email (V2) last. No count sentence yet.

4.7 Your First Expression: Count the Rows

The coordinator wants a line like *"This report lists 6 sessions."* None of the dynamic content values is a count. Dynamic content gives you values that already exist. When you need a value *calculated*, you write an **expression**.

1. On the canvas, select **Send an email (V2)** to open it again. In **Body**, click right after `register.` at the end of the first line, then type a space, then `This report lists`, then another space.
2. Select the **fx** button that appears at the right edge of the Body box, just below the lightning bolt. The expression editor opens. (Typing `/` and choosing **Insert expression** works on some screens too, but not all.)
3. In the expression box, type `length(`
4. At the top of the same panel, switch from the **Function** tab to the **Dynamic content** tab and select **Output** under *Select*. It drops into the expression.
5. Type `)` to close the bracket, then select **Add** (it reads **Update** if you're editing an expression that's already there).
6. After the new token, type `sessions.`



length() counts the items in a list. Here it counts the rows Select produced.

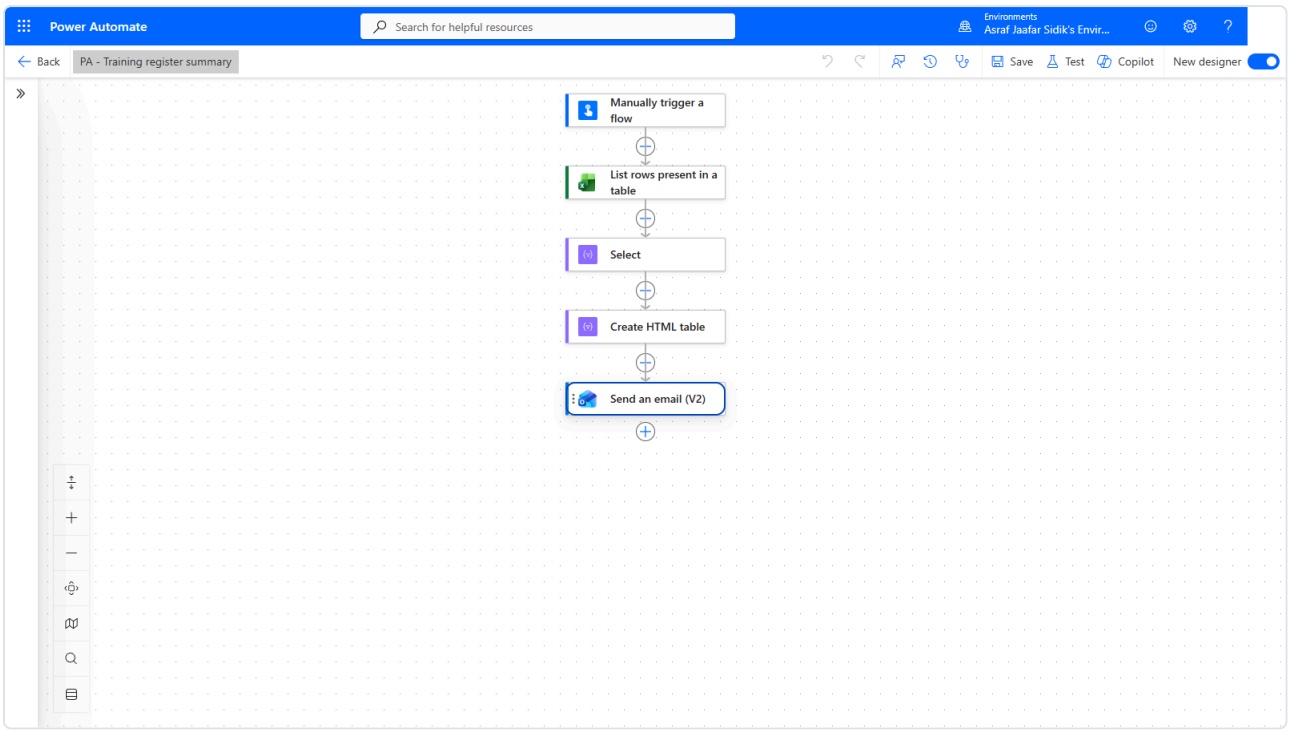
The expression reads as below. You can also paste it into the expression box instead: copy it from the box below.

```
length(body('Select'))
```

`length()` is a **function**: you give it something inside the brackets and it gives back an answer.

> **Tip:** If you renamed the Select action, the name inside the quotes changes too. Picking Output from the Dynamic content tab handles that for you, which is why step 4 picks it instead of typing it.

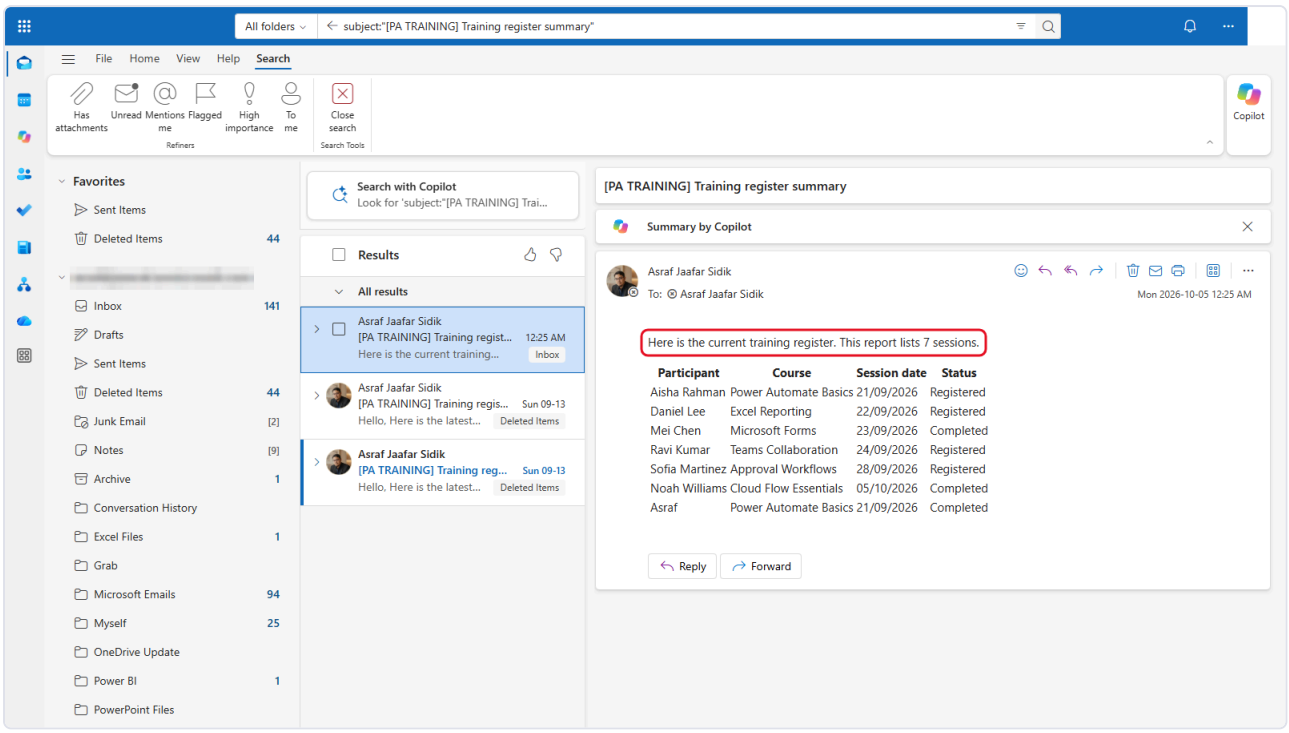
7. Select **Save**, then **Flow checker**, and fix anything it reports.



Five steps. One email.

4.8 Test and Inspect

1. Select **Test** > **Manually** > **Test**. A **Run flow** panel opens; select **Continue** if it asks about connections, then **Run flow**, then **Done**. This sends a real email to you, so follow your trainer's instruction first.
2. Open the completed run. Every action has a green tick, and Send an email (V2) ran once.
3. Select the **Select** step on the canvas. In the panel on the left, scroll to **Outputs** and select **Show raw outputs**. You see six records with real names and courses.
4. Open the email. You should see the sentence *This report lists 6 sessions.*, four headings, and six rows. The number always matches the rows in your table, so if you added a row it goes up too.



One email, one table, counted by your first expression. This workbook had a seventh test row, so it reads 7. Its dates were already formatted with the Going further tip; yours show values like 2026-09-21T00:00:00.000Z.

Each manual run sends one current report. Earlier emails stay as historical snapshots.

4.9 Optional Teams Update

Trainer-led only, once a permitted destination is confirmed. After the email, add **Microsoft Teams** > **Post message in a chat or channel**, choose **Flow bot** and **Chat with Flow bot**, pick yourself as the recipient, and post a short message such as *Training register summary sent*. Keep it short. Teams doesn't show the email's HTML table the same way.

Independent Practice

Add Training ID to the report without adding Email. Predict where the change belongs, make it, and check the received message. Then explain why a six-record table still produces only one email.

> **Going further: format the date.** The Session date column shows values like `2026-09-21T00:00:00.000Z`. In Chapter 5 you'll learn the `formatDateTime()` expression that turns it into `21 Sep 2026`. Come back afterwards and use it in Select's **Session date** value. Optional.

Troubleshooting

Symptom	What to check
No Excel table is listed	Workbook location, saved table, table name and connector account

One email arrives for every row	The email ended up inside a loop. Use **Output** from Create HTML table
The report includes internal columns	Use Select to define headings and values
The email shows HTML tags	Check you inserted **Output** from Create HTML table, not from Select
A date appears as a number	Set DateTime Format to ISO 8601 in List rows present in a table
The expression shows an error	Brackets must match: `length(` … `)`. Pick Output from the Dynamic content tab instead of typing it
The report is stale	Finish the Excel edit, wait for save, and run again

Lesson Summary

An instant trigger lets the coordinator decide when to run. Excel supplies the records, Select defines the report columns, Create HTML table produces one report, and your first expression counted the rows. Next, a recurrence trigger starts a reminder process on a schedule.

For reference, see the Excel Online (Business) connector and data operations in Power Automate.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Anything marked **text** is typed or pasted straight into a field.

Flow name (text, section 4.2)

```
PA - Training register summary
```

Select keys (text, section 4.4)

Type each key, then insert the matching column from dynamic content as its value.

```
Participant
```

```
Course
```

```
Session date
```

```
Status
```

Email subject (text, section 4.6)

```
[PA TRAINING] Training register summary
```

Email introduction (text, section 4.6)

```
Here is the current training register.
```

Your first expression: count the rows (section 4.7)

In the Body, select the **fx** button (or type `/` and choose **Insert expression**), paste, then select **Add**. Works if the Select action still has its default name.

```
length(body('Select'))
```

05

CHAPTER 5

Send Weekly Training Reminders

Every week the coordinator opens the register and writes the same reminder for each participant. A scheduled flow can do that at a predictable local time.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 40 minutes

Your result: A scheduled cloud flow that runs every Monday at 9:00 AM Malaysia time, reads the training register, and prepares one personalised reminder email for each row.

> **Download:** the files for this lab
>
> - TrainingRegister.xlsx: only if you don't have the workbook from Chapter 2 yet. Upload it to your `Power Automate Training` folder in OneDrive and update the dates as in 2.2 step 4.

What You Will Learn

- Schedule a flow with the **Recurrence** trigger and a local time zone
- Convert the current time into readable local time
- Loop through Excel rows with **Apply to each**
- Build a personalised email by typing text and inserting dynamic content
- Use a second expression, `formatDateTime()`, to make a date readable

How the Flow Works

Recurrence > Current time > Convert time zone > List rows present in a table > Apply to each > Send an email (V2)

Recurrence decides when the flow starts. Current time and Convert time zone produce a readable local timestamp for the message. The Excel action returns the rows, and Apply to each handles them one at a time.

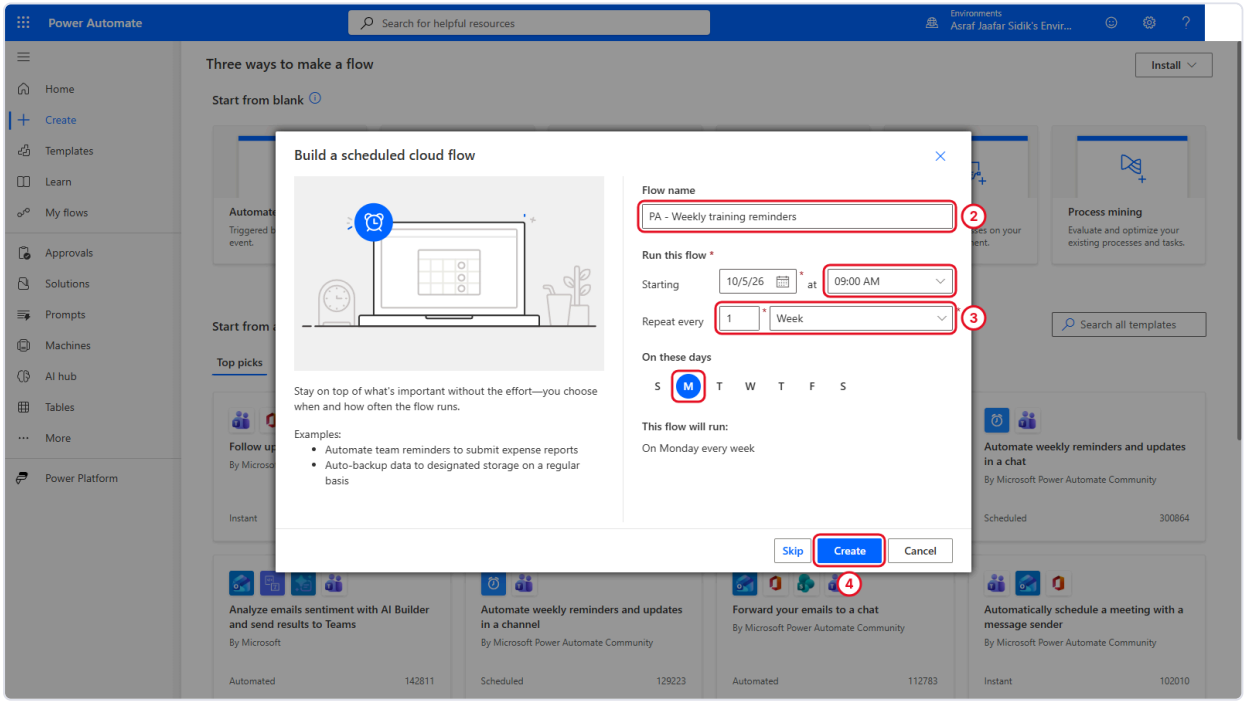
Before You Begin

You need `Power Automate Training/TrainingRegister.xlsx` with the `tblTraining` table. This chapter uses every column: ParticipantName for the greeting, Email as the recipient, CourseTitle in the subject and body, SessionDate and Status in the body.

> **Warning: the email addresses in the workbook are not real.** The Email column holds made-up addresses like `learner1@example.com`. If you test the flow with them, Outlook can't deliver the reminders: you get an undeliverable message or an address that won't resolve. To test, first type your own email address into the Email column of the rows you want to try, and only when the trainer says so.

5.1 Create the Weekly Schedule

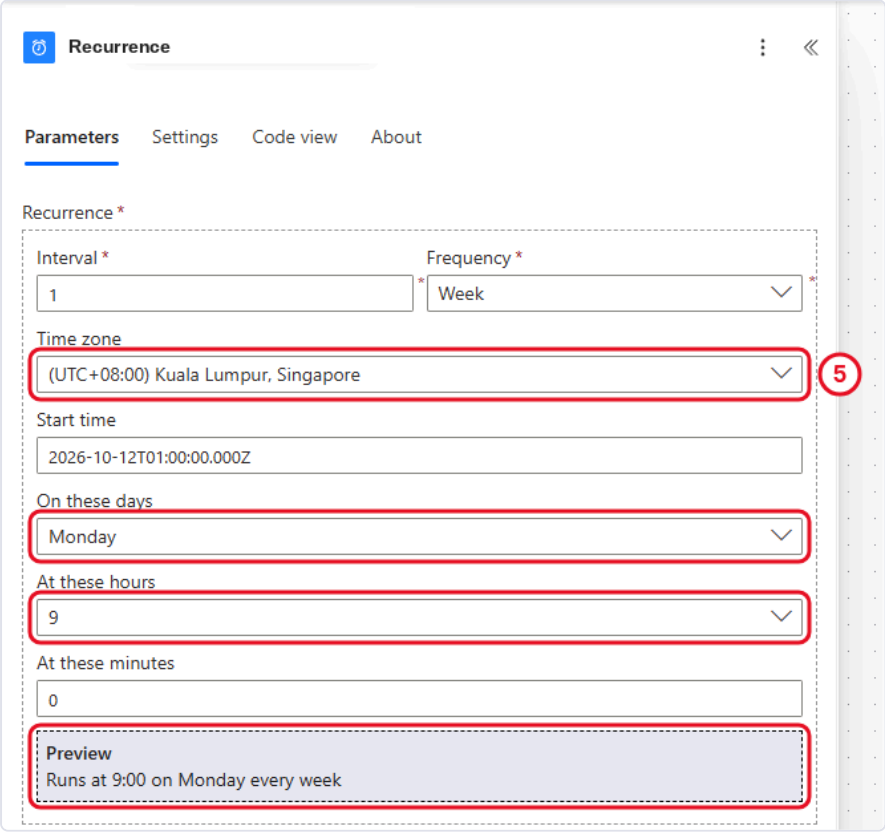
1. Select **Create** on the left, then the **Scheduled cloud flow** tile.
2. In **Flow name**, enter `PA - Weekly training reminders`.
3. Set **Repeat every** to **1 Week**, tick **M** (Monday), and set the start time to **9:00 AM**. Your starting date will differ.
4. Select **Create**.



The dialog creates the Recurrence trigger for you.

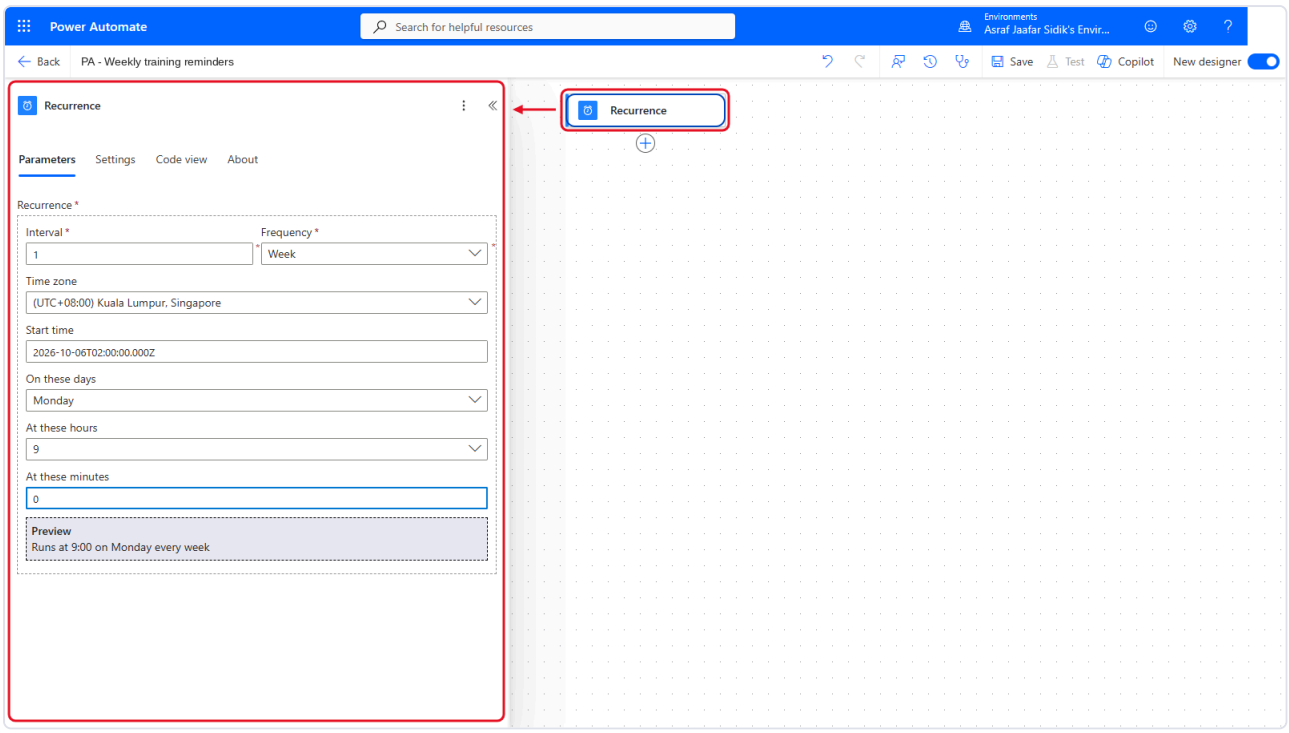
5. The designer opens. On the canvas, select the **Recurrence** card to open its settings on the left, and check these values. If you can't see the time zone fields, open **Advanced parameters** and select **Show all**.

Setting	Value
Interval	1
Frequency	Week
Time zone	(UTC+08:00) Kuala Lumpur, Singapore
Start time	Filled in by the dialog. Leave it.
On these days	Monday
At these hours	9
At these minutes	0



Read the preview line at the bottom. It should say 9:00 on Monday every week.

Check your flow so far. Your screen should look like this.

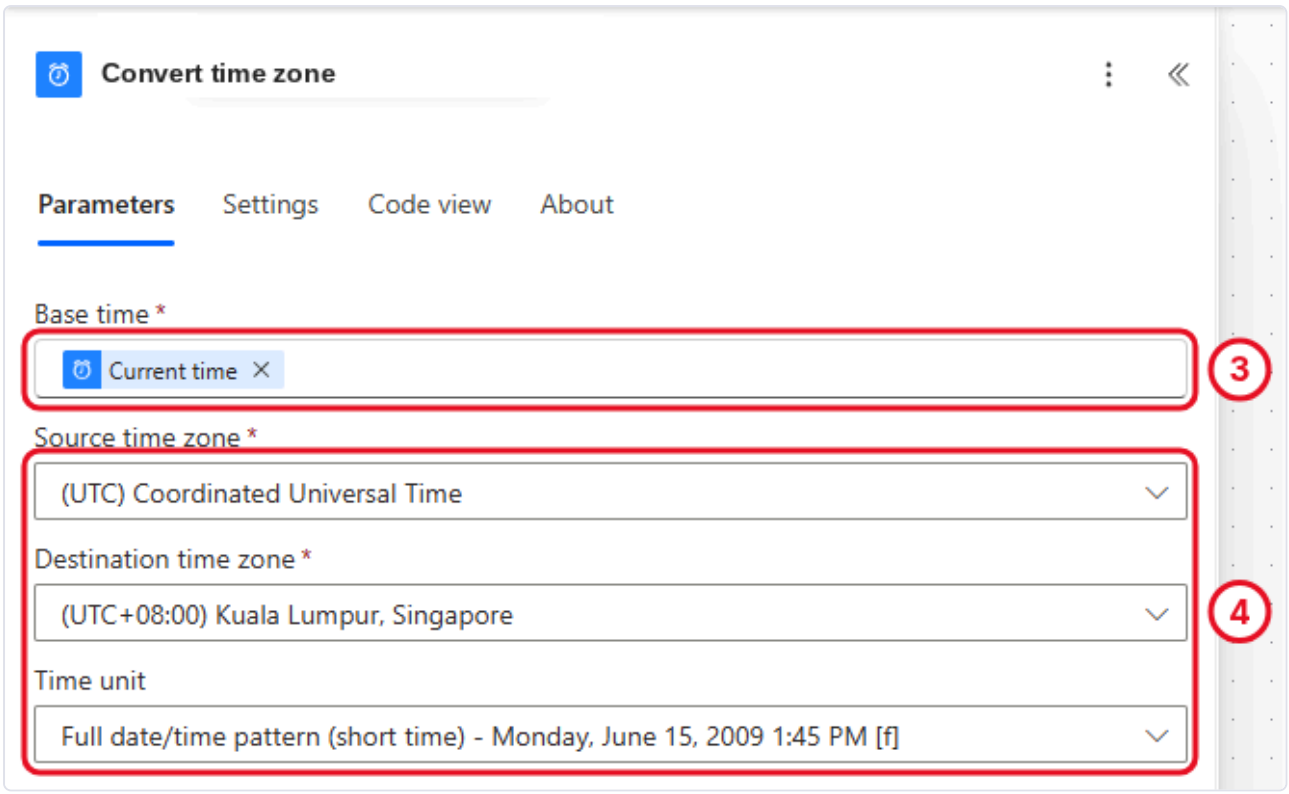


Only Recurrence so far.

5.2 Produce a Readable Local Timestamp

1. On the canvas, select the **+** below **Recurrence**, then **Add an action**. In the search box, type `Current time`. Under the **Date Time** heading, select **Current time**. It needs no settings.
2. Select the **+** below **Current time**, then **Add an action**. Search for `Convert time zone` and select it under **Date Time**.
3. Click into **Base time**, select the lightning bolt icon beside it, and pick **Current time** under *Current time*.
4. Set the other boxes from their dropdowns:

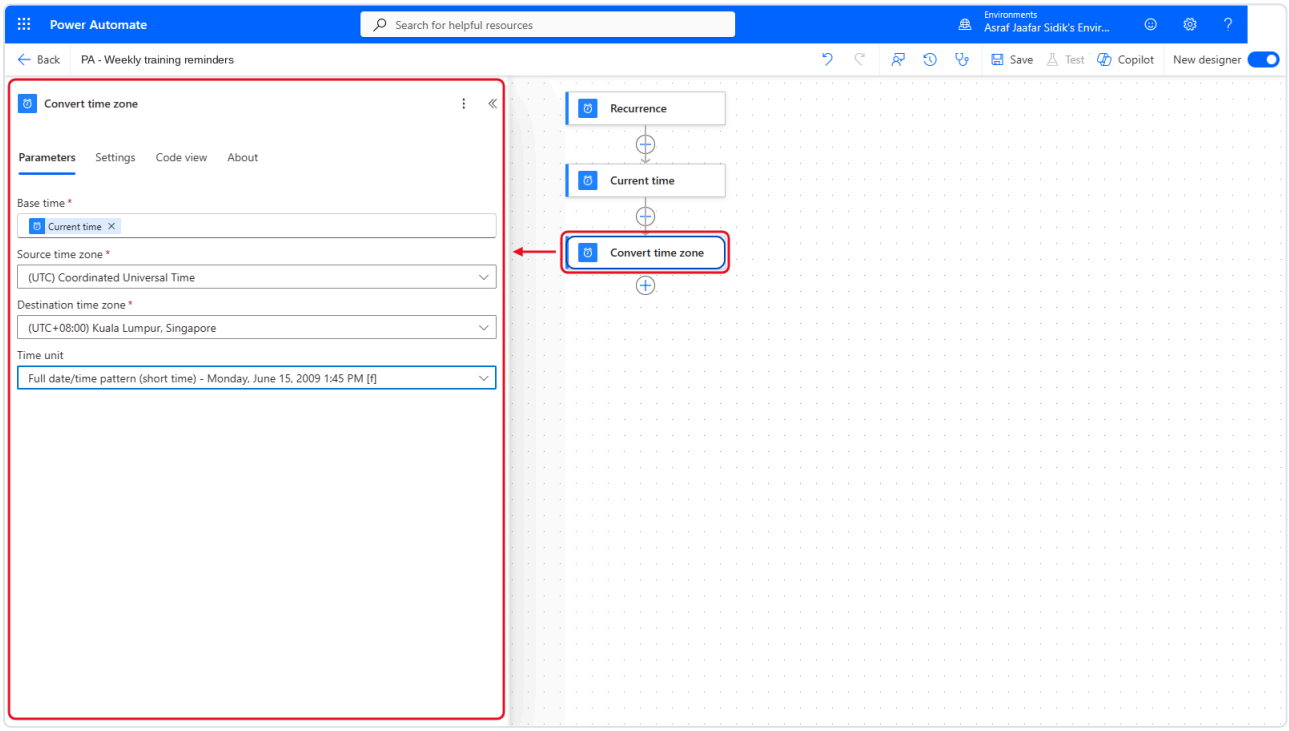
Setting	Value
Base time	**Current time** (the token from step 3)
Source time zone	(UTC) Coordinated Universal Time
Destination time zone	(UTC+08:00) Kuala Lumpur, Singapore
Format string (may be labelled **Time unit**)	Full date/time pattern (short time)



Pick the format from the dropdown. No expression needed.

> **Key point:** The two time zone settings do different jobs. Recurrence decides *when the flow starts*. Convert time zone only changes a timestamp shown in the message.

Check your flow so far. Your screen should look like this.



Recurrence, Current time and Convert time zone.

5.3 Read the Excel Rows

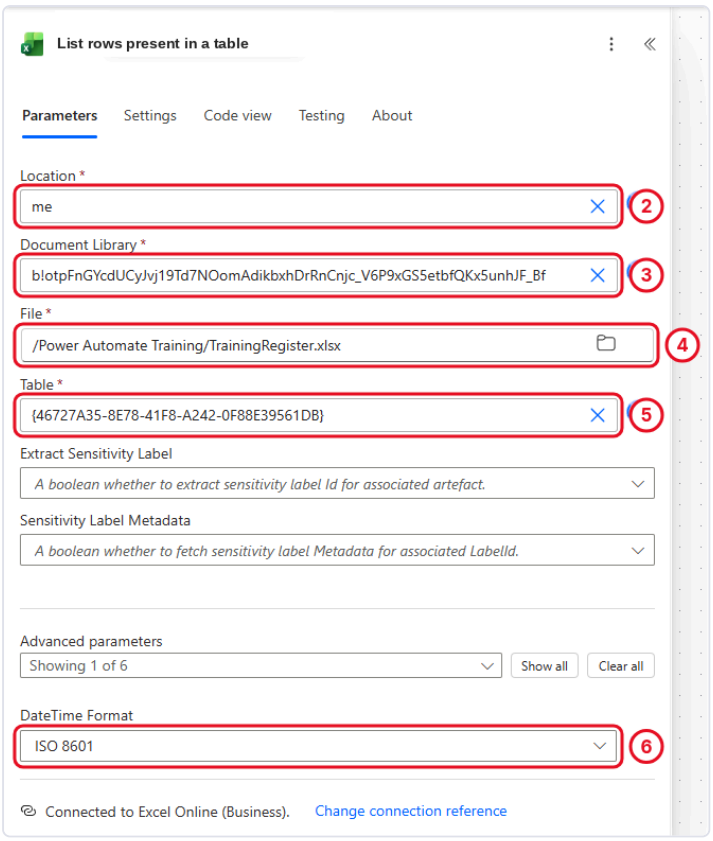
The settings are the same as Chapter 4, section 4.3.

1. On the canvas, select the **+** below **Convert time zone**, then **Add an action**. Search for `List rows present in a table` and select it under **Excel Online (Business)**.
2. Open the **Location** dropdown and select **OneDrive for Business**.
3. Open the **Document Library** dropdown and select **OneDrive**.
4. In **File**, select the folder icon at the right end of the box. Select the **>** beside `Power Automate Training`, then select `TrainingRegister.xlsx`.

   > **Tip:** Clicking a folder's name selects it but doesn't open it. Use the **>** arrow on the right to go inside.

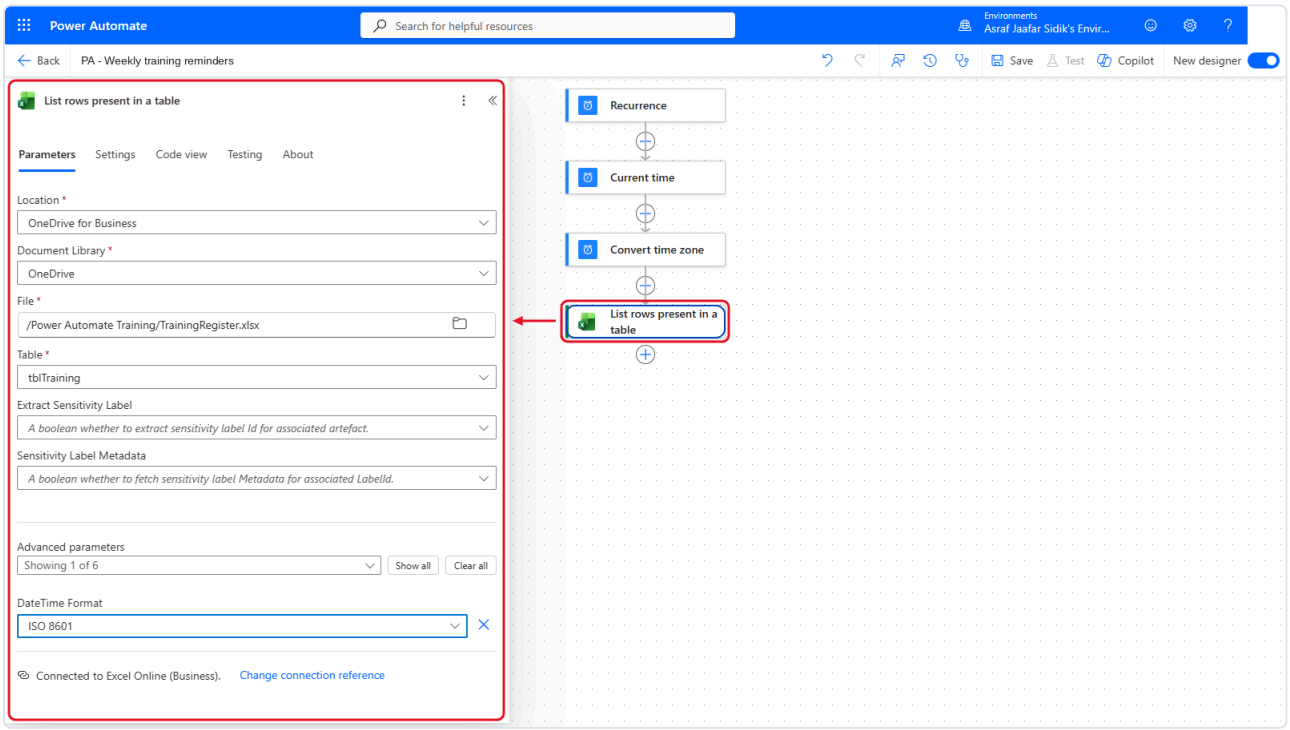
5. Open the **Table** dropdown and select `tblTraining`.
6. Under **Advanced parameters**, open the dropdown (it reads **Showing 0 of 6**) and tick **DateTime Format**. In the new **DateTime Format** box, choose **ISO 8601**.

> **Note:** After you pick them, some boxes may switch to an internal value: **Location** to `me`, **Document Library** to a long code, and **Table** to an ID in braces like `{46727A35-…}`. It doesn't happen every time. Names or codes, both are fine.



Same settings as Chapter 4. If your boxes show codes instead of names, they stand for the same choices.

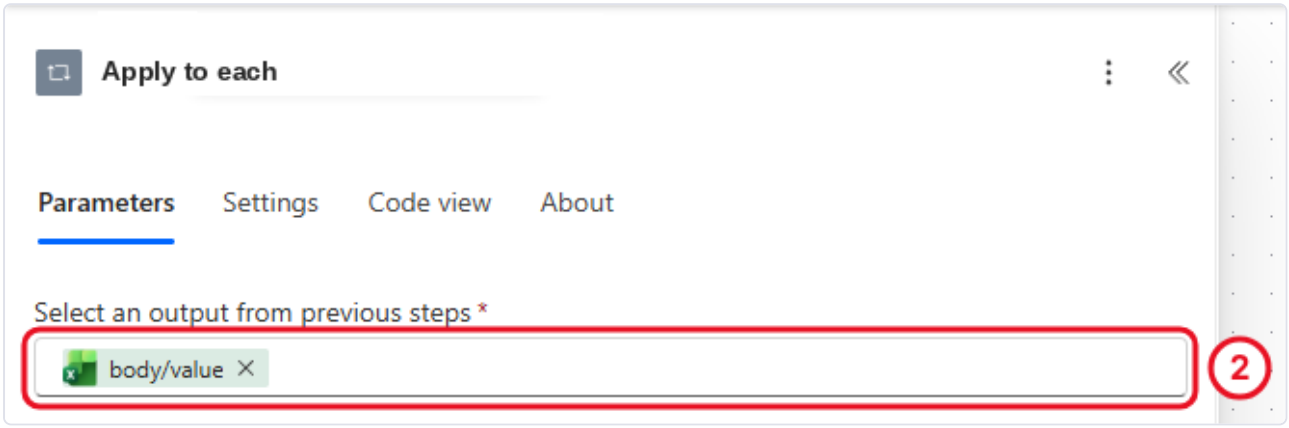
Check your flow so far. Your screen should look like this.



List rows present in a table comes next.

5.4 Process One Row at a Time

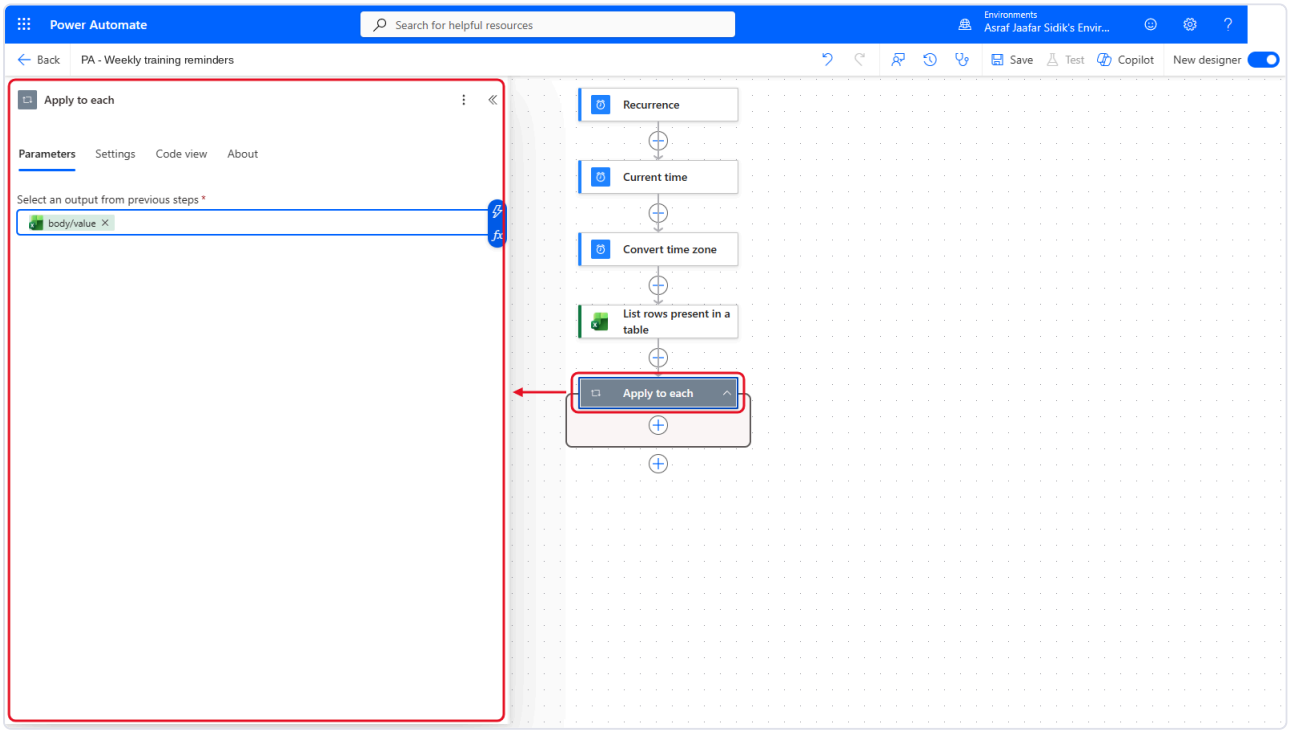
1. On the canvas, select the **+** below **List rows present in a table**, then **Add an action**. Search for `Apply to each` and select it under **Control**.
2. Click into **Select an output from previous steps**, select the lightning bolt icon beside it, and under *List rows present in a table* pick **body/value** (it may be labelled **value**). If you can't see it, select **See more** or type `value` in the picker's search box.



The loop runs once per row in the table.

Checkpoint: The input must be the whole list (**value**). A single column such as Email isn't a list of rows.

Check your flow so far. Your screen should look like this.



An empty Apply to each at the bottom.

5.5 Prepare the Reminder Email

On the canvas, select the **+** inside the **Apply to each** box (below its header), then **Add an action**. Search for `Send an email`, then select **Send an email (V2)** under **Office 365 Outlook**.

1. **To:** this box opens a people picker, so it needs switching first. Select the small **settings** (gear) icon above the right end of the **To** box, on the same line as the **To** label. Choose **Use dynamic content**. Then click into the box, select the lightning bolt icon, and pick **Email** under *List rows present in a table* (select **See more** or type `Email` in the search box if it isn't listed).



The gear switches To from picking people to accepting dynamic content (shown here after Email was inserted).

2. **Subject:** click into the box and paste the subject start (it ends with a space). Copy it from the box below.

```
[PA TRAINING] Reminder:
```

   Select the lightning bolt icon and pick **CourseTitle** under *List rows present in a table*. If the trailing space disappears after typing or pasting, type a space before inserting CourseTitle.

3. **Body:** click into the box and type the message below. Where a value belongs, put the cursor there, type `/` and choose **Insert dynamic content** (or select the lightning bolt), then pick the value.

Type this, inserting the tokens where shown:

> Hello **[ParticipantName]**,
>
> This is your weekly training reminder.
>
> Course: **[CourseTitle]**
> Session date: **[SessionDate]**
> Status: **[Status]**
> Reminder generated: **[Converted time]**
>
> Please contact the training coordinator if your plans change.

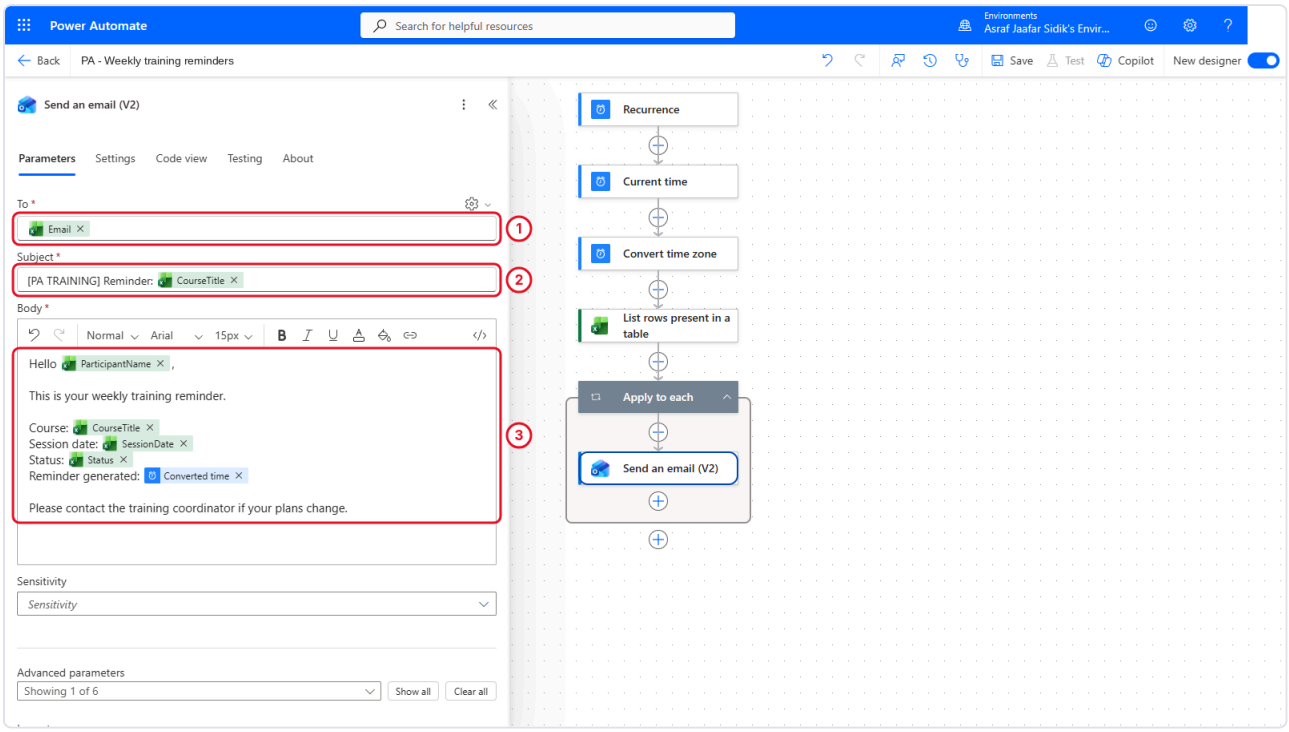
Each **[bold name]** is a dynamic content token. **Converted time** comes from *Convert time zone*. The rest come from *List rows present in a table*. If a value isn't listed, select **See more** or type its name in the picker's search box.

The two plain sentences contain no tokens, so you can copy them from the boxes below:

```
This is your weekly training reminder.
```

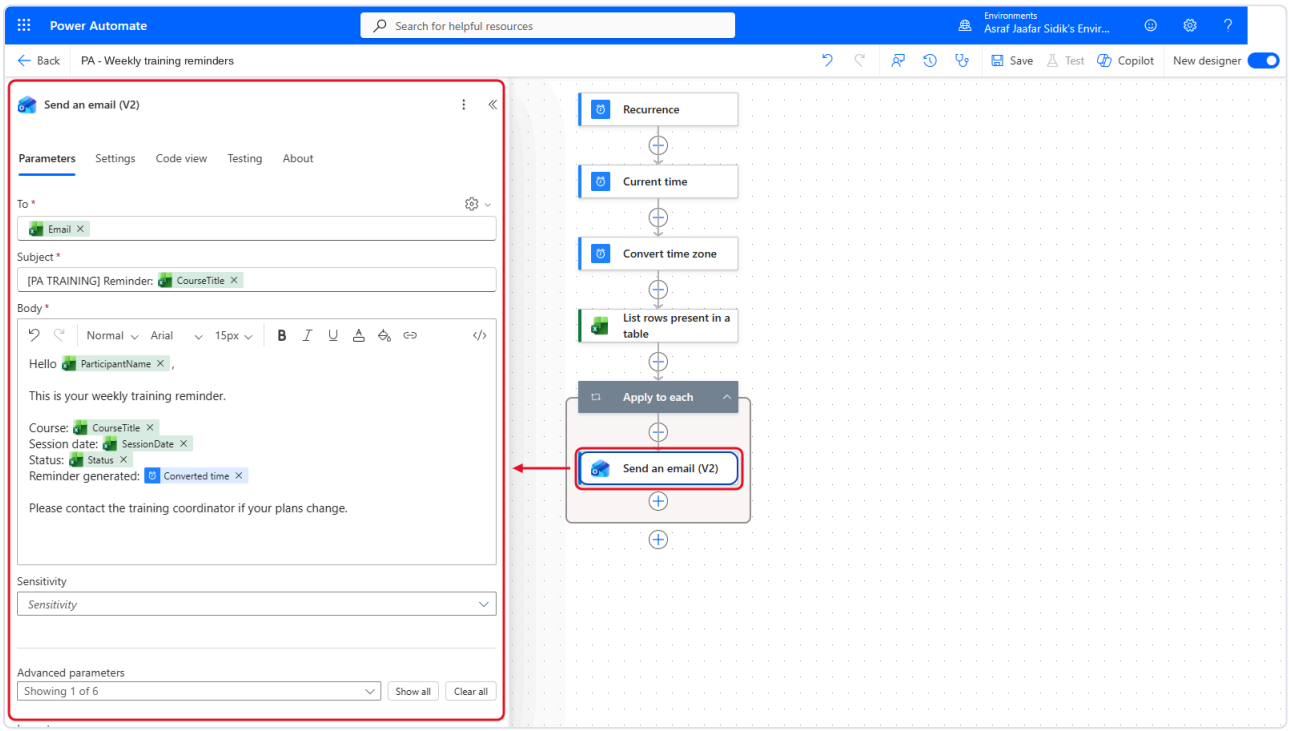
```
Please contact the training coordinator if your plans change.
```

The copy-paste page also has the whole body with `[Name]` placeholders, if you'd rather paste it and replace each placeholder with its token.



Type the words, insert the tokens. The email reads like a normal message.

Check your flow so far. Your screen should look like this.

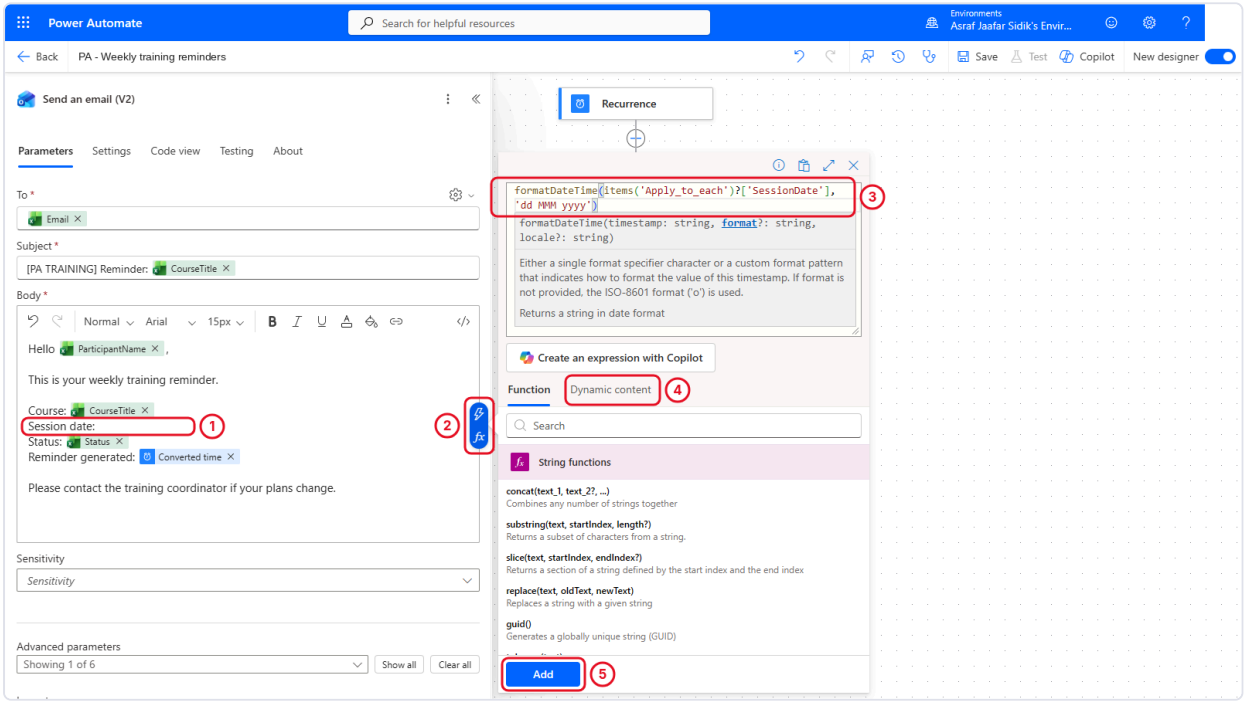


Send an email (V2) sits inside Apply to each.

5.6 Make the Date Readable with an Expression

Run the flow now and the session date reads `2026-09-21T00:00:00.000Z`. That's correct, but nobody wants to read it. There's no dynamic content value for "the date, written nicely", so this is a job for an expression.

1. In the **Body** of **Send an email (V2)**, select the **x** on the **SessionDate** token to remove it. Make sure the cursor stays right after `Session date:` on that line (click there if it moved).
2. Select the **fx** button that appears at the right edge of the Body box, just below the lightning bolt. The expression editor opens. (Typing `/` and choosing **Insert expression** works on some screens too, but not all.)
3. In the expression box, type `formatDateTime(`
4. At the top of the same panel, switch from the **Function** tab to the **Dynamic content** tab and select **SessionDate** (type `Session` in its search box if it isn't listed). It drops into the expression.
5. Type `, 'dd MMM yyyy')` and select **Add** (it reads **Update** if you're editing an expression that's already there).



formatDateTime() takes a date and a pattern, and gives back the date written in that pattern.

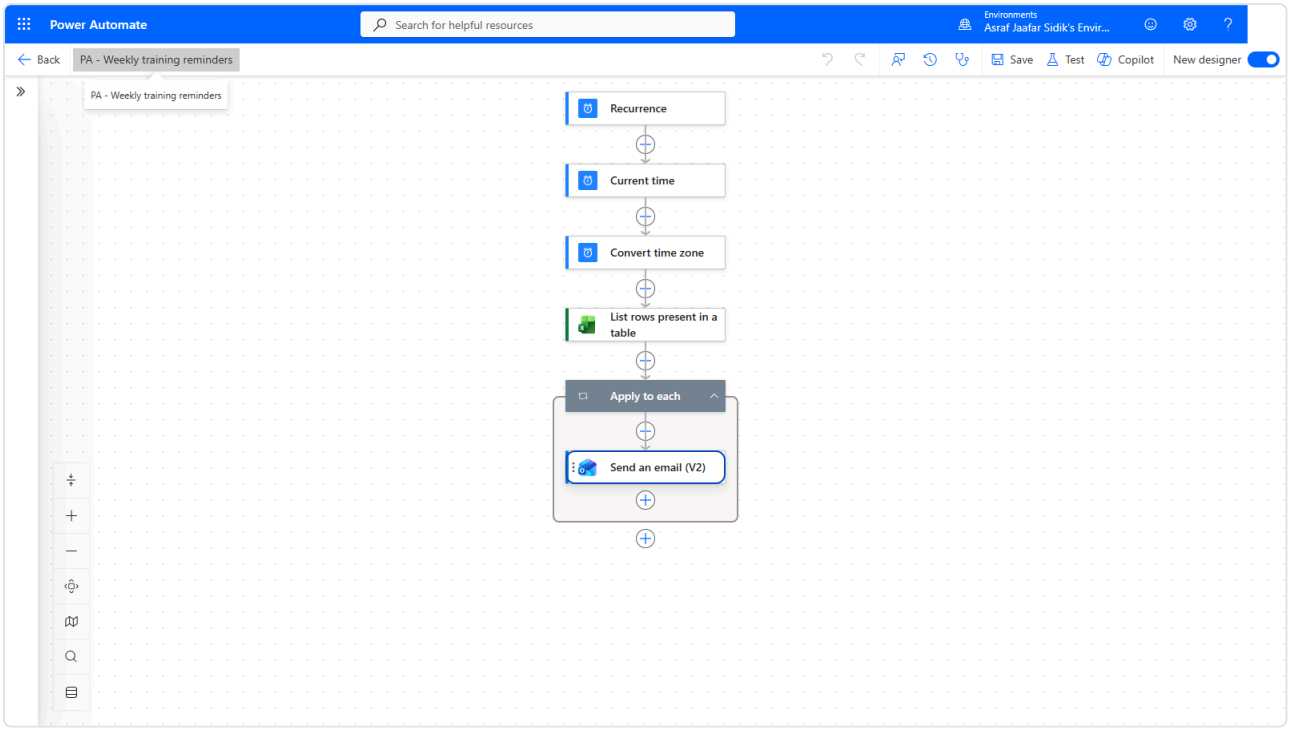
If you'd rather paste the finished expression, copy it from the box below. It works if your loop still has its default name, **Apply to each**.

```
formatDateTime(items('Apply_to_each')?['SessionDate'], 'dd MMM yyyy')
```

The pattern `dd MMM yyyy` means two-digit day, short month name, four-digit year, so `21 Sep 2026`.

> **Tip:** Compare this with Chapter 4. `length()` counted a list, `formatDateTime()` reshapes a date. Both follow the same shape: a function name, then what you give it in brackets.

6. On the toolbar at the top, select **Save**, then **Flow checker**, and fix any error it reports.



Recurrence, Current time, Convert time zone, List rows, and the loop with its email.

Trainer-Controlled Test

> **Important:** Don't select Test while the workbook still has `example.com` placeholders. A test sends one email per row, so six messages with the sample data. The trainer confirms the recipients and authorises the run immediately before you select Run flow.

After an authorised test, check the run history:

- Every step before the loop succeeded
- Apply to each shows six iterations
- Each email used that row's Email, CourseTitle and Status, and the date reads like `21 Sep 2026`
- The reminder timestamp is in Malaysia time

In this version every row gets a reminder, including Completed ones. Chapter 7 fixes that.

Independent Practice

Change the scheduled hour to a value your trainer gives you, read the Recurrence preview, then set it back to 9:00. Explain why changing Convert time zone wouldn't change when the flow starts.

> **Going further: try other date patterns.** Change `'dd MMM yyyy'` to `'dddd, d MMMM'` and the date reads *Monday, 21 September*. You can also go back to Chapter 4 and use the same expression in Select's **Session date** value. Optional.

Troubleshooting

Symptom	What to check
The flow starts at the wrong local time	Recurrence time zone, day, hour, minute and preview
The timestamp is eight hours early	Convert time zone source and destination
No Excel rows are returned	OneDrive location, workbook saved, formatted table and table name
Apply to each is empty	Use the Excel action's **value** list
A recipient or course is blank	Header spelling in the workbook. Re-pick the token
The date shows as a number	Set DateTime Format to ISO 8601 in List rows present in a table
The date expression shows an error	Brackets and quotes: `formatDateTime(` ... `, 'dd MMM yyyy')`
Completed records get reminders	Expected here. Chapter 7 adds the filter

Lesson Summary

A scheduled trigger starts the flow at a predictable local time. Date Time actions give a readable timestamp, Excel supplies the rows, Apply to each hands the current row to the email, and a second expression turned a raw date into something a person wants to read. Next you add a human approval decision.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Anything marked **text** is typed or pasted straight into a field.

Flow name (text, section 5.1)

```
PA - Weekly training reminders
```

Email subject start (text, section 5.5)

Paste, then insert **CourseTitle** after it.

```
[PA TRAINING] Reminder:
```

Email body (text, section 5.5)

Paste, then replace each `[Name]` with the matching dynamic content token.

```
Hello [ParticipantName],

This is your weekly training reminder.

Course: [CourseTitle]
Session date: [SessionDate]
Status: [Status]
Reminder generated: [Converted time]

Please contact the training coordinator if your plans change.
```

Readable session date (section 5.6)

With the cursor right after `Session date:` (where the SessionDate token was), select the **fx** button at the right edge of the Body box (or type `/` and choose **Insert expression**), paste, then select **Add**.

```
formatDateTime(items('Apply_to_each')?['SessionDate'], 'dd MMM yyyy')
```

> **Note:** If your loop has a different name (for example `Apply_to_each_1`), build the expression by picking **SessionDate** from the Dynamic content tab as in section 5.6 instead of pasting.

06

CHAPTER 6

Process a Training Approval

A training request needs a person to say yes or no before anything is confirmed. This flow collects the request with Microsoft Forms, asks an approver to decide, and posts a different Teams message for each outcome.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 75 minutes

Your result: An automated cloud flow that reads a Forms response, asks a person to approve or reject it, and prepares a different Teams message for each outcome.

Files for this lab: None. You build the Microsoft Form yourself in 6.1.

What You Will Learn

- Start a flow when a Microsoft Forms response is submitted
- Retrieve the answers with **Get response details**
- Request a decision with **Start and wait for an approval**
- Branch on the approval outcome and post a different Teams message for each result

How the Flow Works

When a new response is submitted > Get response details > Start and wait for an approval > Condition > Approved or rejected Teams message

The Forms trigger only supplies a response ID. Get response details uses that ID to fetch the answers. The approval action pauses the flow until someone decides. The Condition reads the approval **Outcome** and sends the flow down True or False.

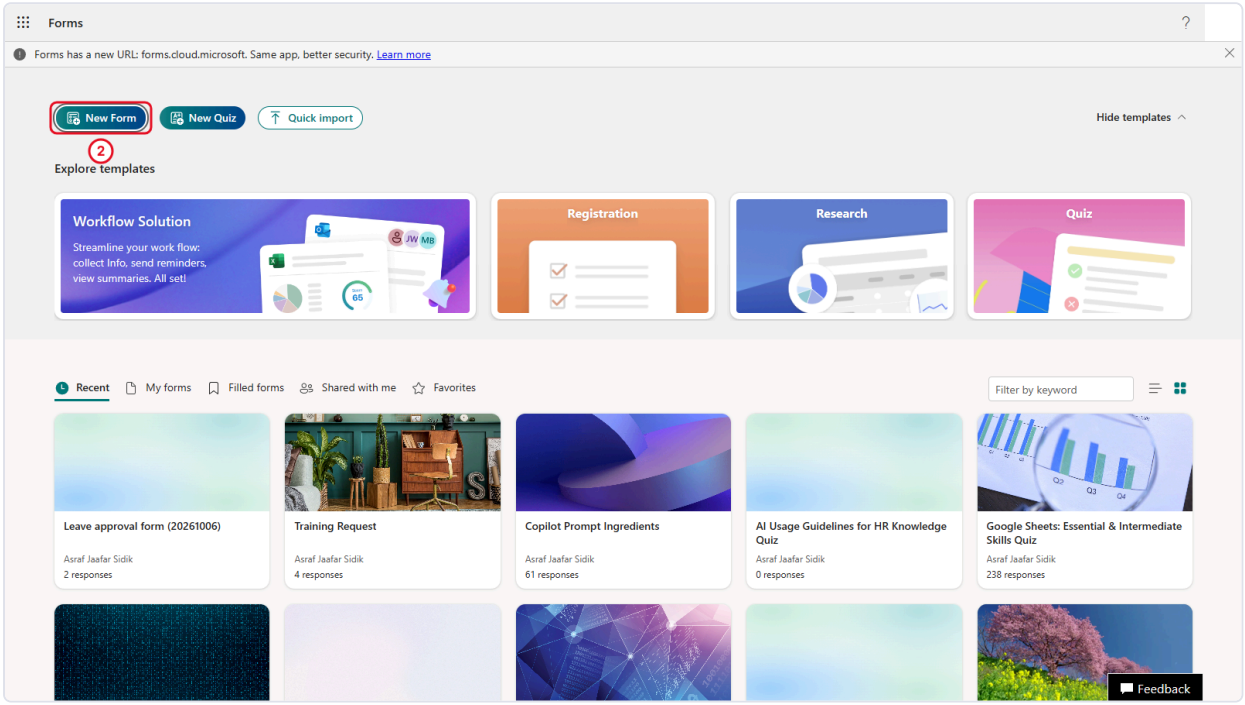
Before You Begin

You need Microsoft Forms, Approvals and Microsoft Teams through your work or school account. Your trainer names an approved test approver and Teams destination before any live test.

> **Important:** The screenshots use `approver@example.com`, a documentation placeholder. Replace it with a trainer-approved account before testing. Don't submit the form or run the flow while the placeholder is still there.

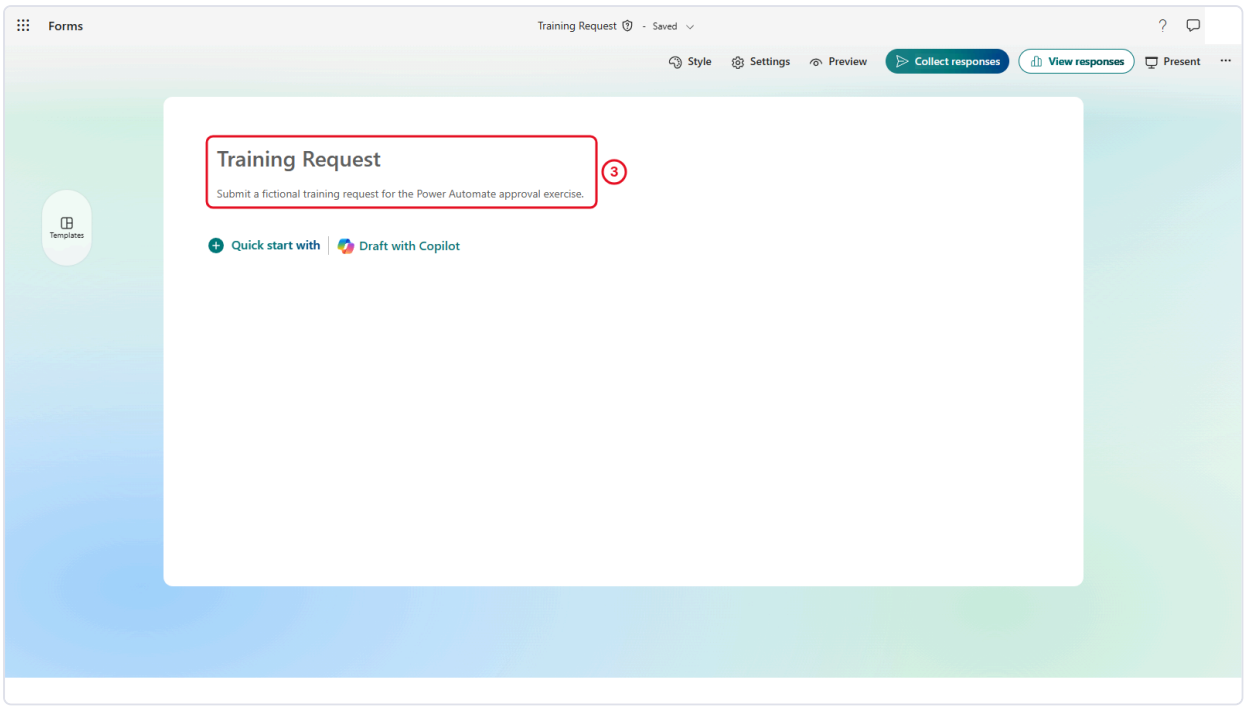
6.1 Prepare the Request Form

1. Go to forms.office.com and sign in with your training account. It may open as forms.cloud.microsoft, which is the same app.
2. Select **New Form**. If a **Draft with Copilot** pane opens, close it.

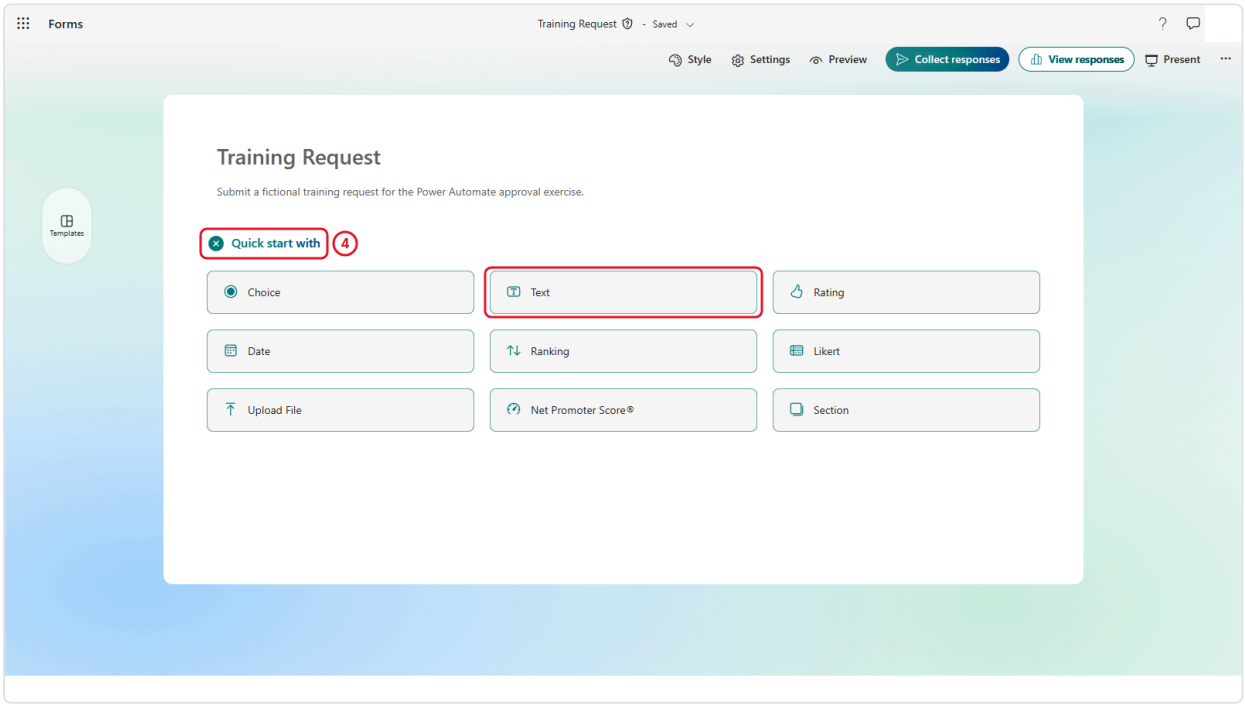


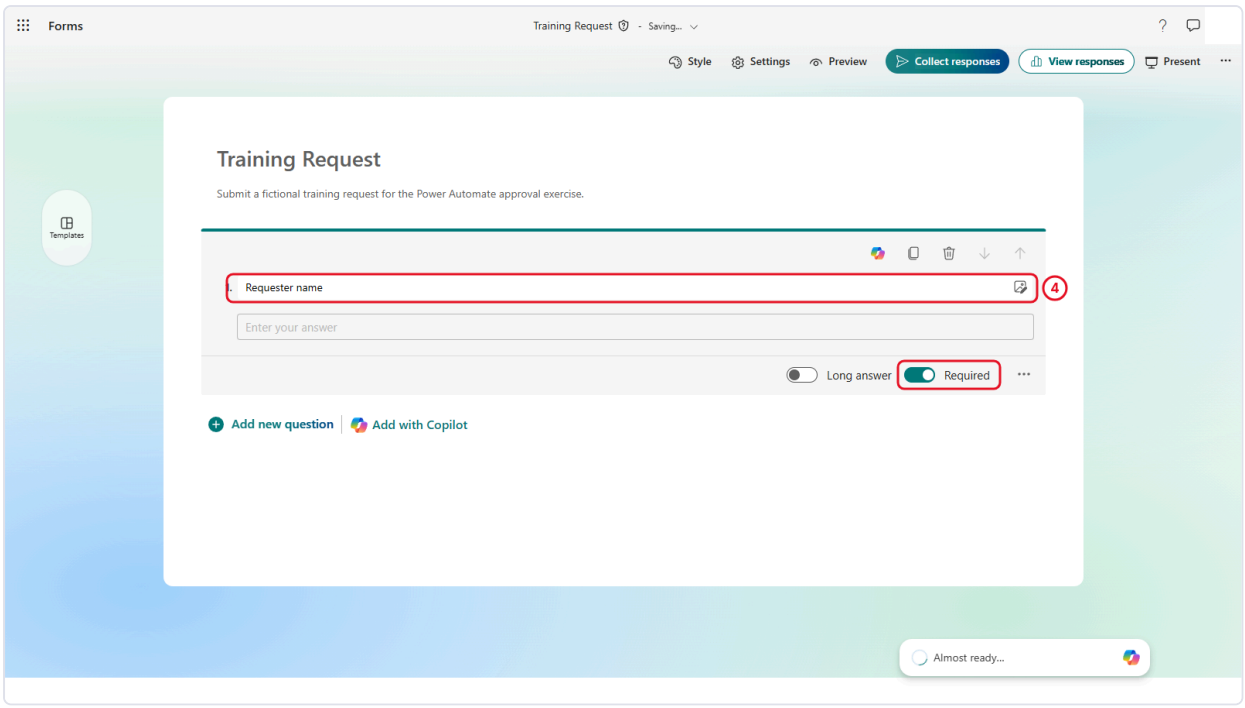
3. Select **Untitled form** at the top and type `Training Request`. In the description box under it, paste the description. Copy it from the box below.

```
Submit a fictional training request for the Power Automate approval exercise.
```

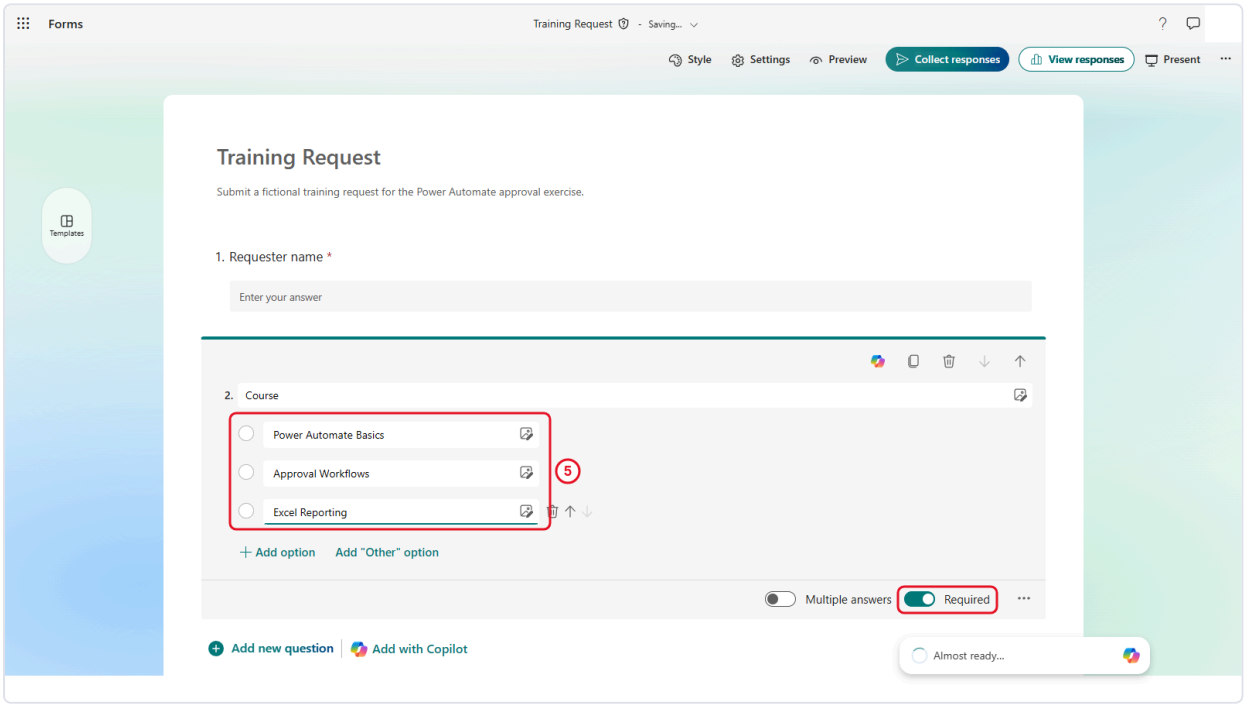


4. Select **+ Add new question** (on a brand-new form it reads **Quick start with**), then **Text**. Type `Requester name` as the question. Switch on **Required**: it's a switch along the bottom edge of the question you're editing.

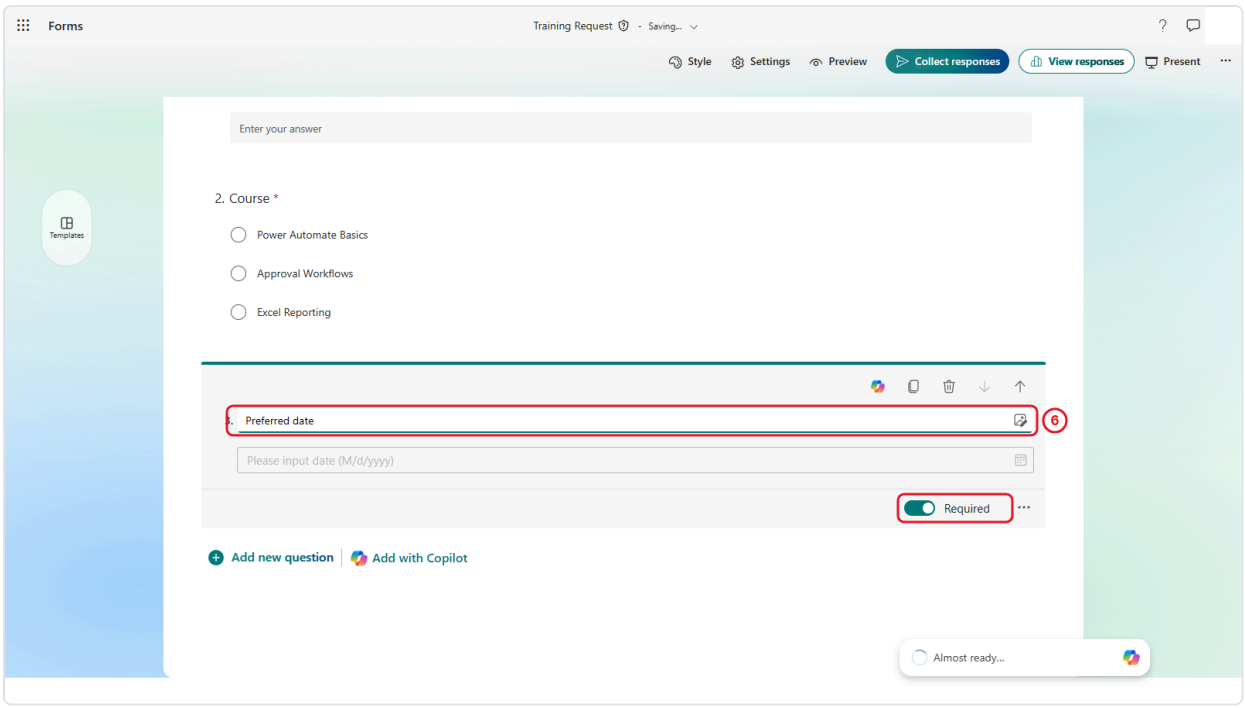




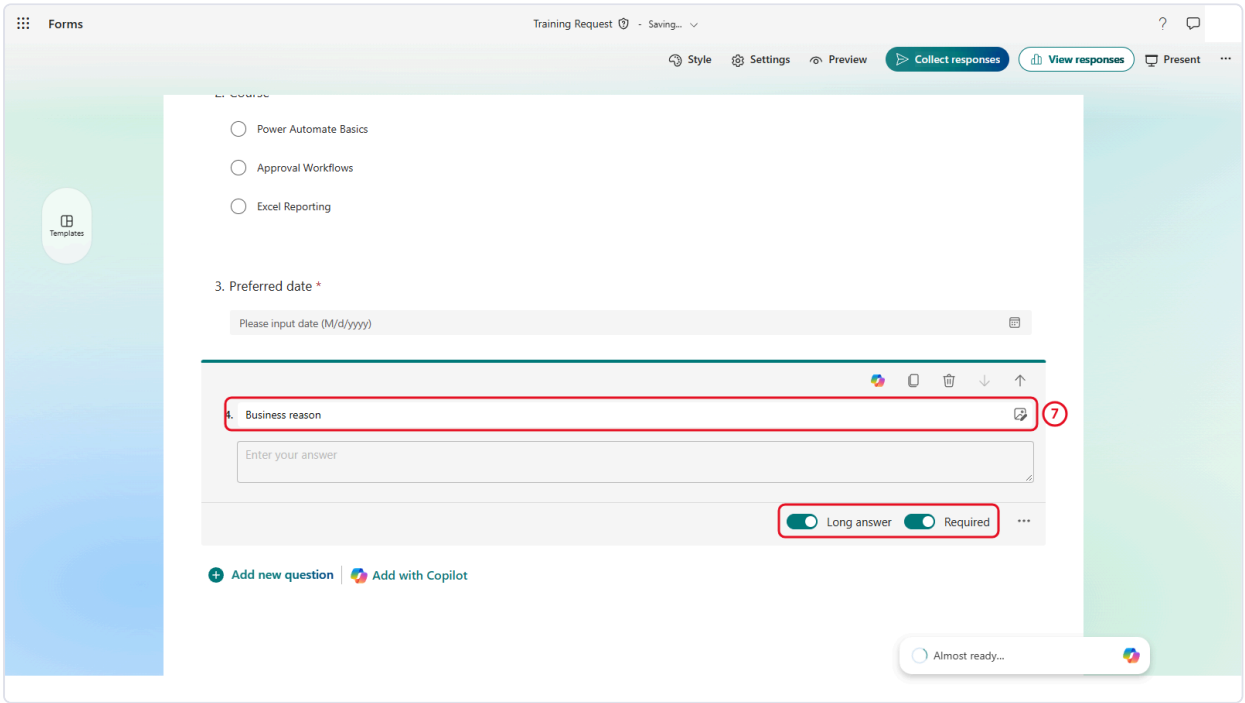
5. Scroll to the bottom of the form and select **+ Add new question** > **Choice**. Type `Course` as the question, then type the three options, one per option box: `Power Automate Basics`, `Approval Workflows`, `Excel Reporting`. Select **Add option** if you need a third box. Switch on **Required**.



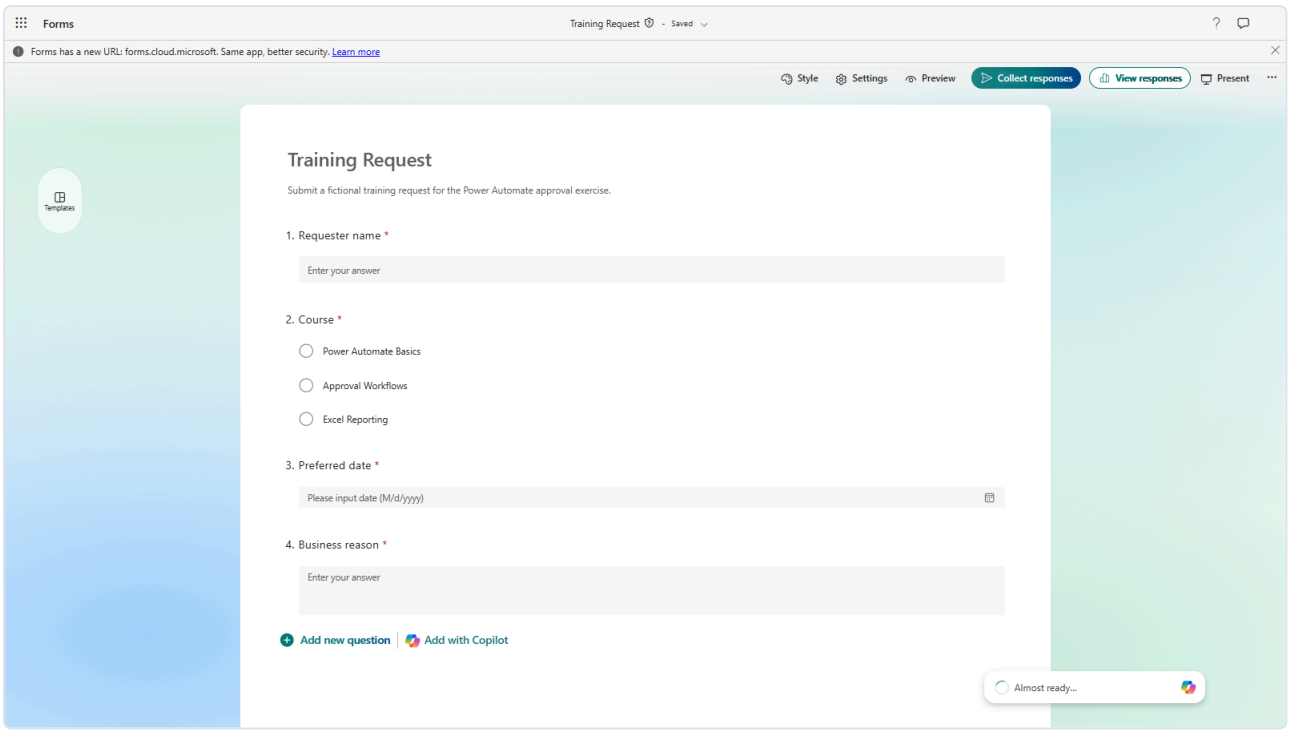
6. Select **+ Add new question** > **Date**. Type `Preferred date` and switch on **Required**.



7. Select **+ Add new question** > **Text**. Type `Business reason`. Switch on **Long answer** and **Required**, both along the bottom edge of the question.



Forms saves as you go. There's no Save button.

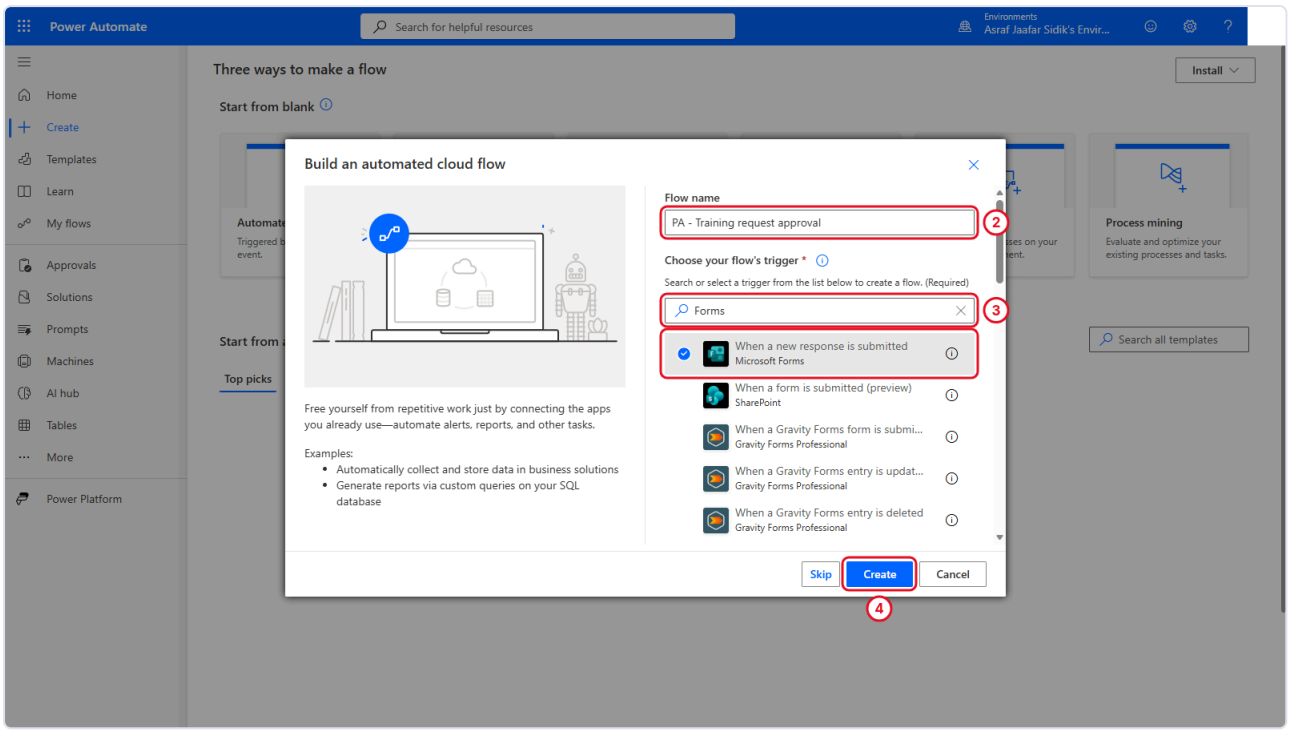


Four required questions. The red asterisk marks each one as required.

Checkpoint: All four questions show the red required marker. A missing answer gives the approver an incomplete card.

6.2 Create the Automated Flow

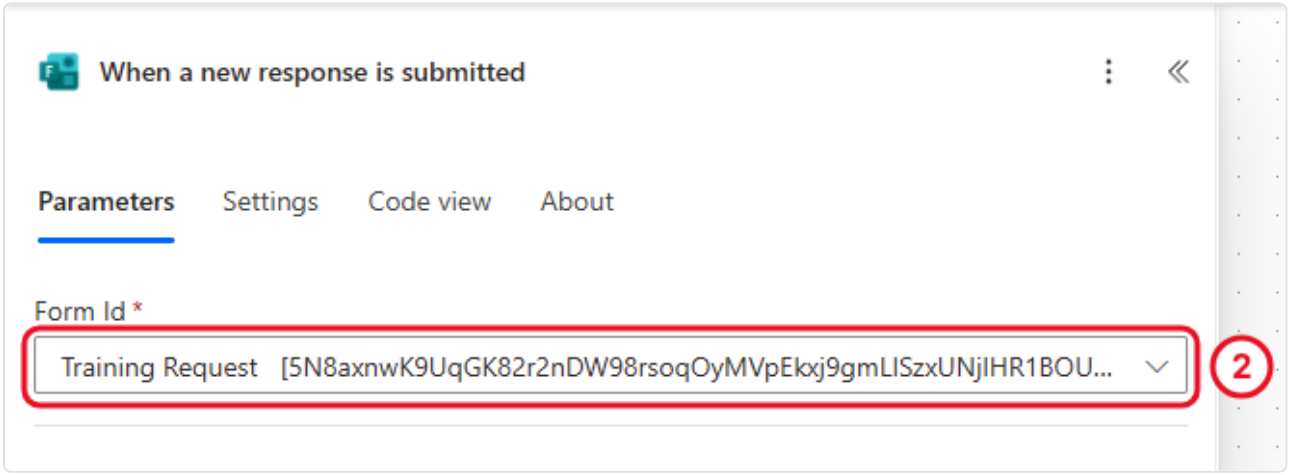
1. In Power Automate, select **Create** on the left, then the **Automated cloud flow** tile.
2. In **Flow name**, enter `PA - Training request approval`.
3. In the trigger search box, type `Forms` and select **When a new response is submitted** (Microsoft Forms).
4. Select **Create**.



The Forms trigger starts the flow whenever the form gets a response.

6.3 Select the Form

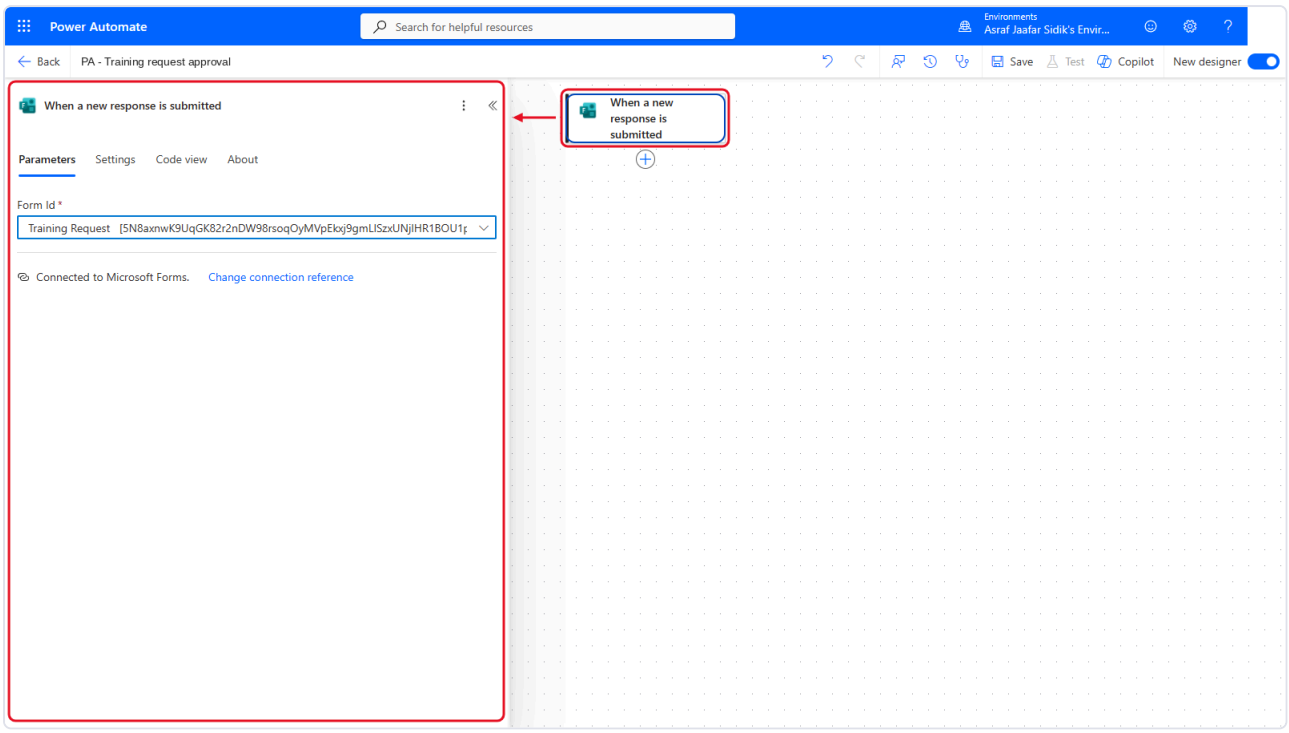
1. On the canvas, select the **When a new response is submitted** card. Its settings open on the left.
2. Open the **Form Id** dropdown and select **Training Request**.



After saving, Form Id shows the form's long internal ID instead of its name. That's normal.

If the form isn't listed, check that you own or can access it, refresh the list, and confirm the Forms connection uses the right account.

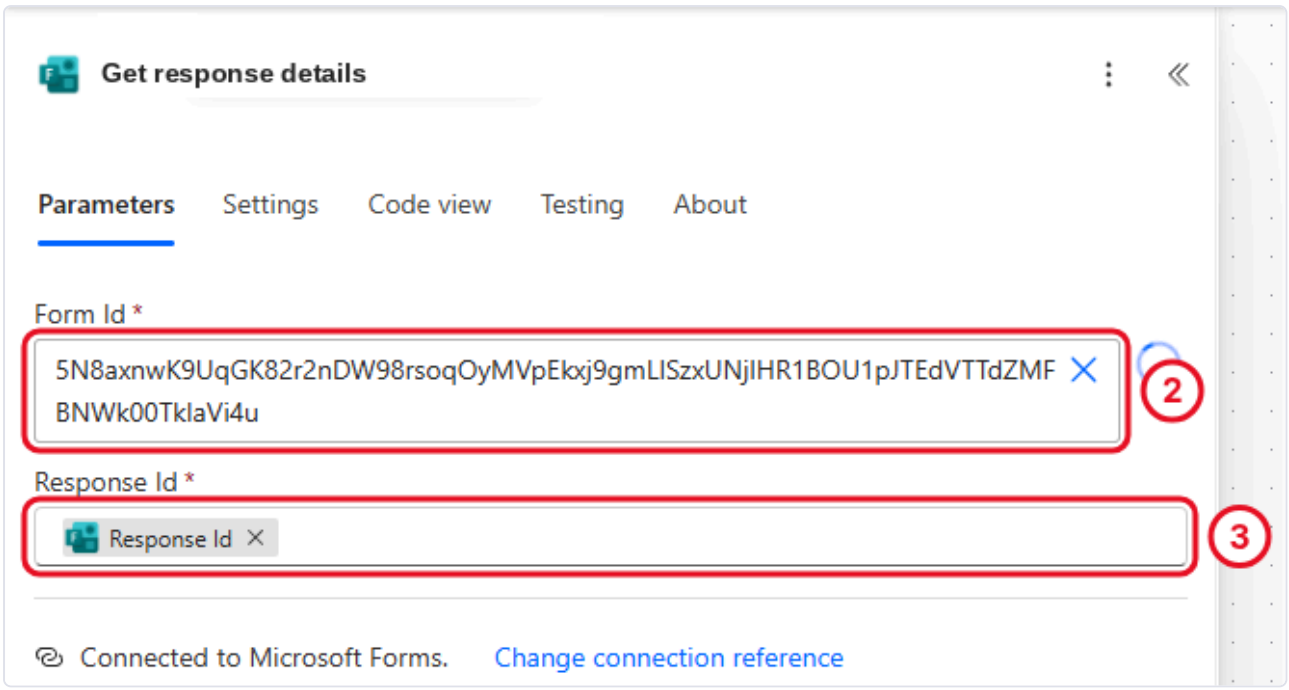
Check your flow so far. Your screen should look like this.



Only the trigger so far.

6.4 Retrieve the Answers

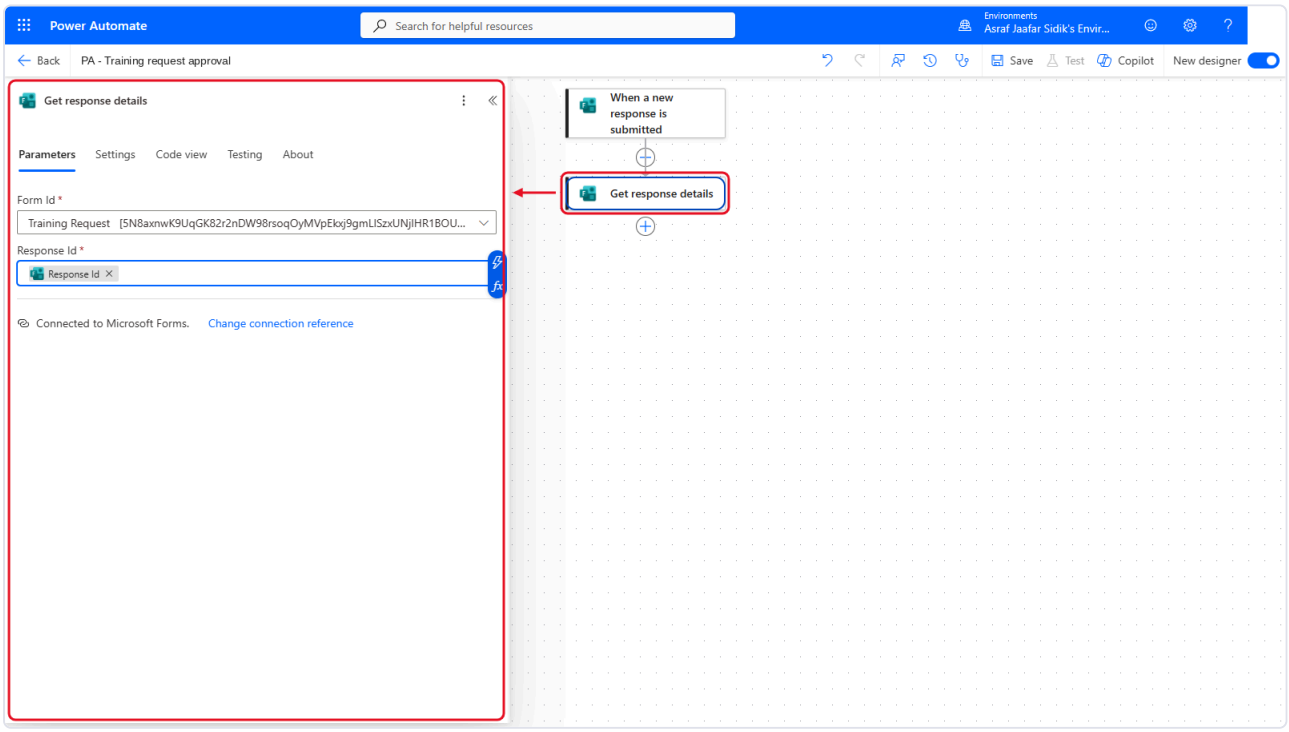
1. On the canvas, select the **+** below **When a new response is submitted**, then **Add an action**. Search for `Get response details` and select it under **Microsoft Forms**.
2. Open the **Form Id** dropdown and select **Training Request** again.
3. Click into **Response Id**, select the lightning bolt icon beside it, and pick **Response Id** under *When a new response is submitted*.



The Response Id token links this step to the exact submission that started the flow. As in 6.3, Form Id shows the form's internal ID after saving.

The trigger only says a response exists. Get response details returns the individual answers so later actions can use them.

Check your flow so far. Your screen should look like this.



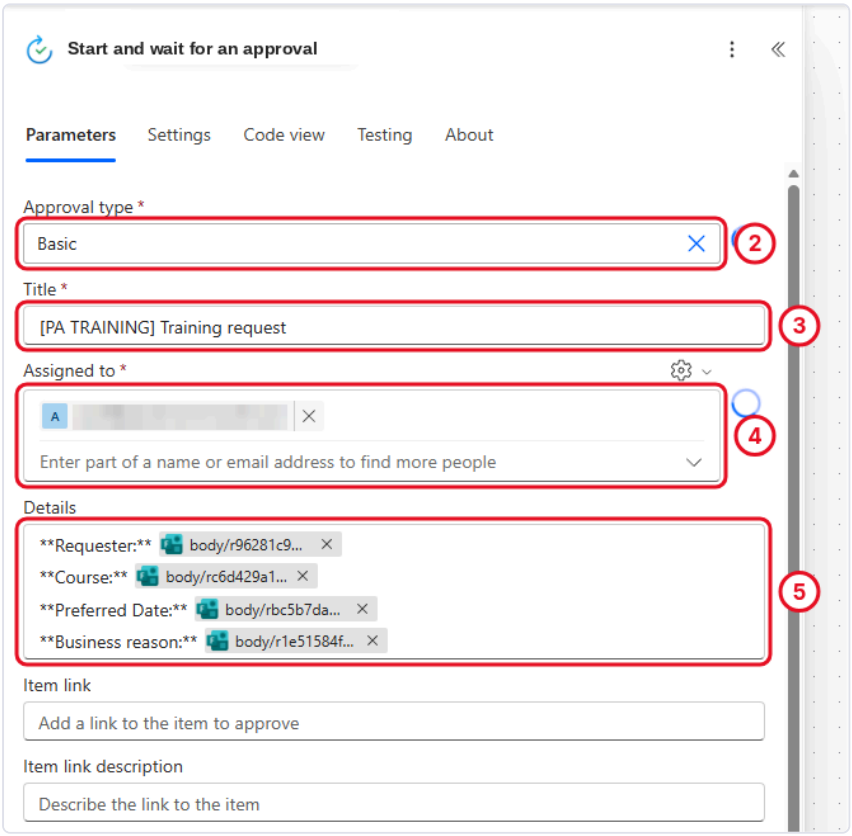
Get response details sits under the trigger.

6.5 Request the Decision

1. On the canvas, select the **+** below **Get response details**, then **Add an action**. Search for `Start and wait for an approval` and select it under **Approvals**.
2. Set **Approval type** to **Approve/Reject - First to respond**. If your list doesn't have it, choose **Basic**.
3. In **Title**, enter `[PA TRAINING] Training request`.
4. In **Assigned to**, type the trainer-approved approver's name or email and select them from the list.
5. In **Details**, paste the details template. Copy it from the box below.

```
**Requester name:**
**Course:**
**Preferred date:**
**Business reason:**
```

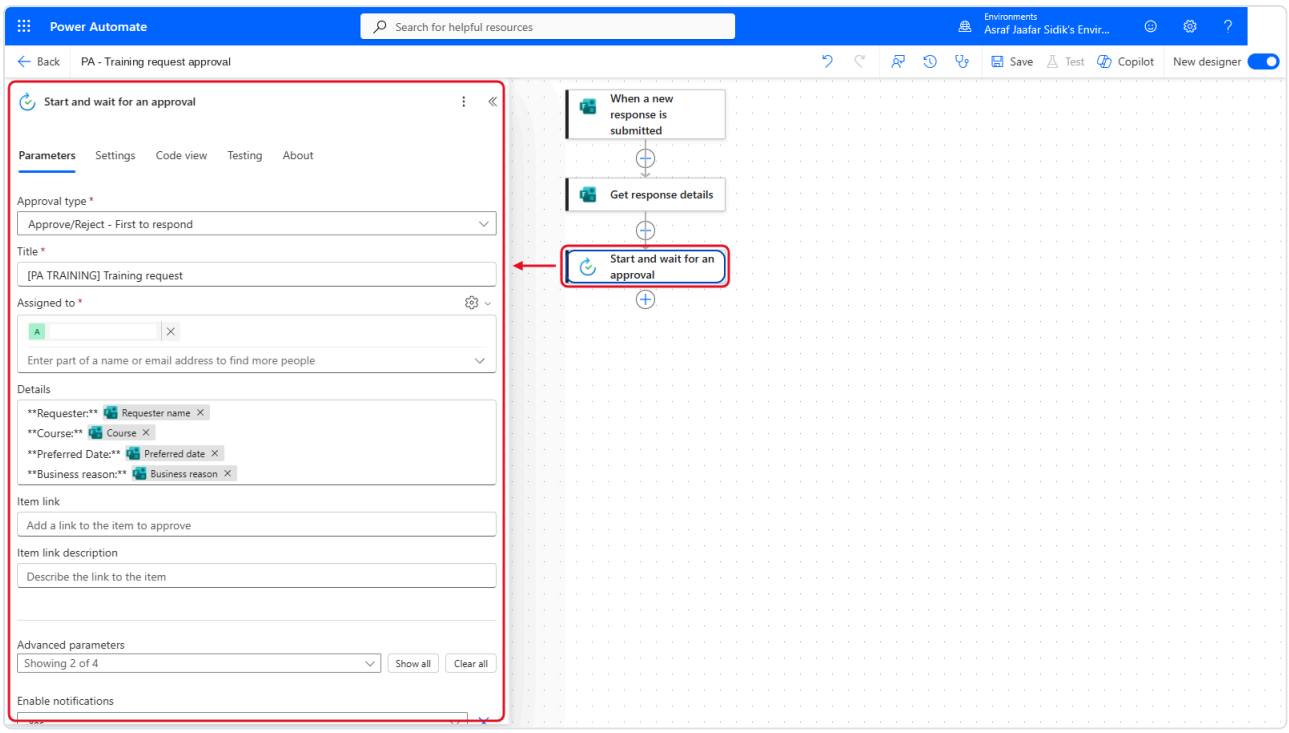
Put the cursor after each label, select the lightning bolt icon, and under *Get response details* pick the matching answer: **Requester name**, **Course**, **Preferred date**, **Business reason**. If the list is cut short, select **See more** or type the question name in the picker's search box.



Each label in Details is followed by its answer. In the picker the answers show their question names; after saving they show as codes like `body/r96281c9…`. The screenshot uses Basic.

Checkpoint: Use the values from **Get response details**, not similarly named values from another action.

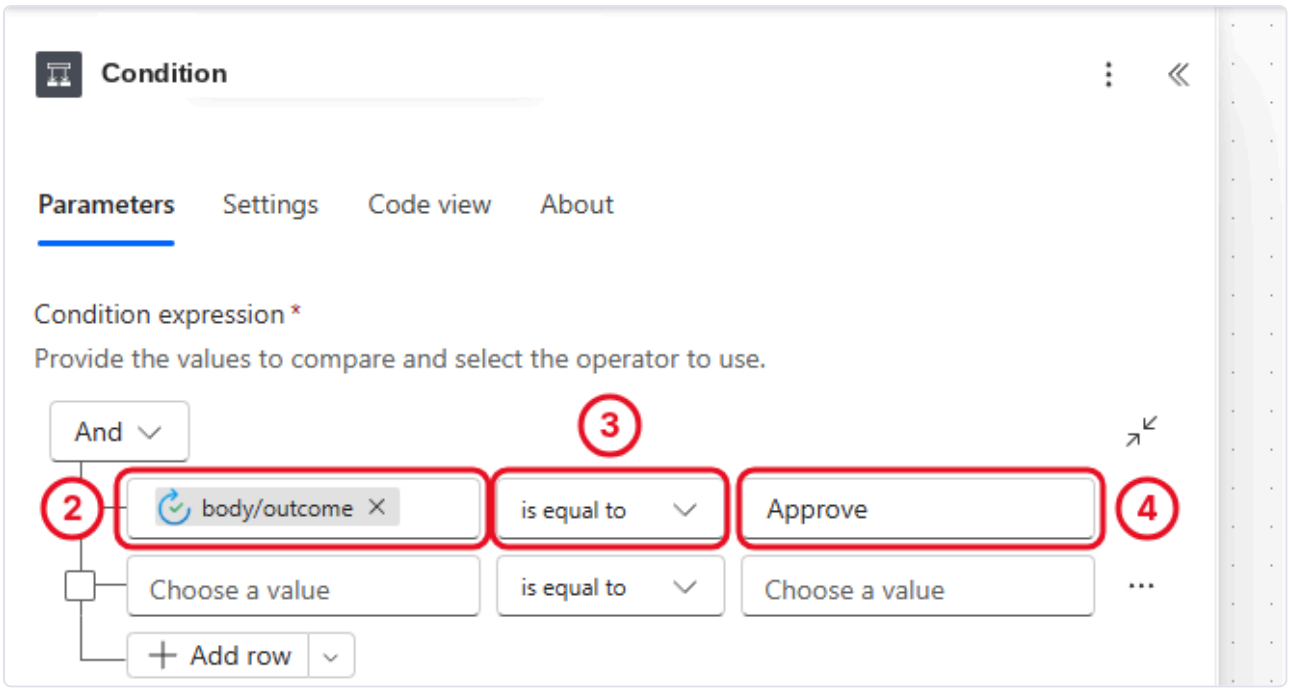
Check your flow so far. Your screen should look like this.



Start and wait for an approval comes next.

6.6 Branch on the Outcome

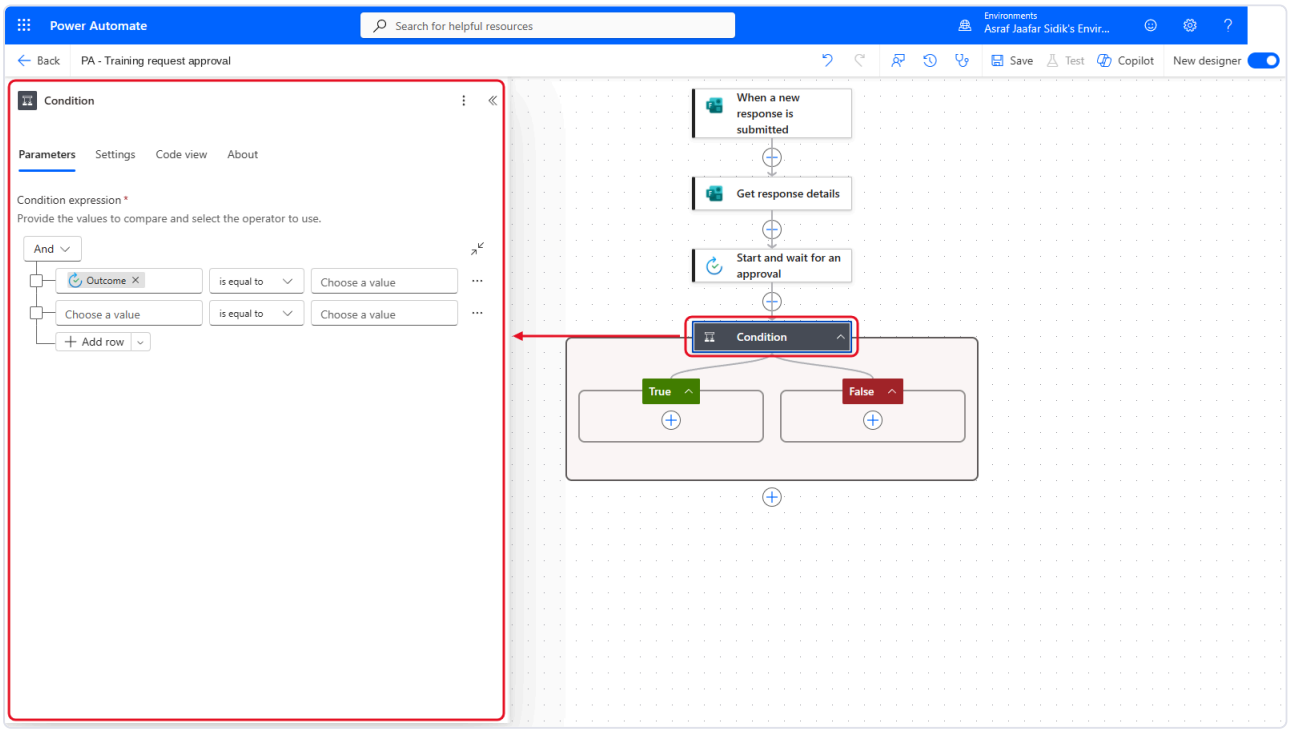
1. On the canvas, select the **+** below **Start and wait for an approval**, then **Add an action**. Search for `Condition` and select it under **Control**.
2. Click into the left **Choose a value** box, select the lightning bolt icon, and pick **Outcome** under *Start and wait for an approval*. (After saving it shows as `body/outcome`.)
3. Leave the middle box on **is equal to**.
4. In the right box, type `Approve`.
5. If an empty second row appears, select its **...** and then **Delete**.



An Approve outcome goes to True. Anything else goes to False. The screenshot still shows the empty row; delete it on yours.

> **Key point:** The text must be exactly `Approve`. A typo or a trailing space sends every approval down the False branch.

Check your flow so far. Your screen should look like this.



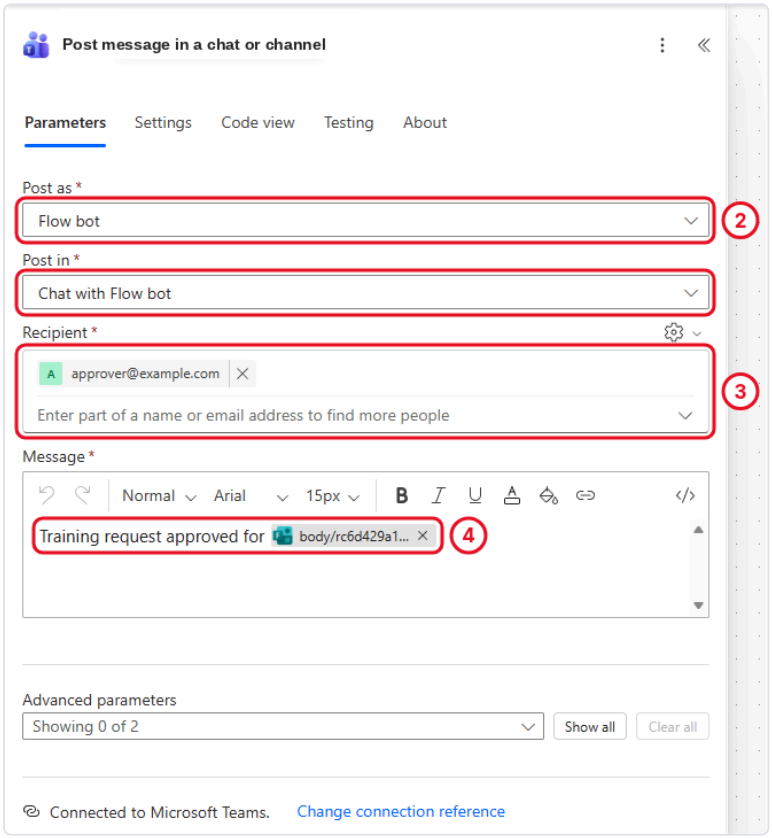
The Condition, with True and False still empty.

6.7 Prepare the Approved Message

1. On the canvas, select the **+** inside the green **True** box, then **Add an action**. Search for `Post message in a chat or channel` and select it under **Microsoft Teams**.
2. Open the **Post as** dropdown and select **Flow bot**. Open **Post in** and select **Chat with Flow bot**.
3. In **Recipient**, type the trainer-approved Teams account and select it from the list.
4. Click into **Message** and type the text below, with the space at the end. Copy it from the box below.

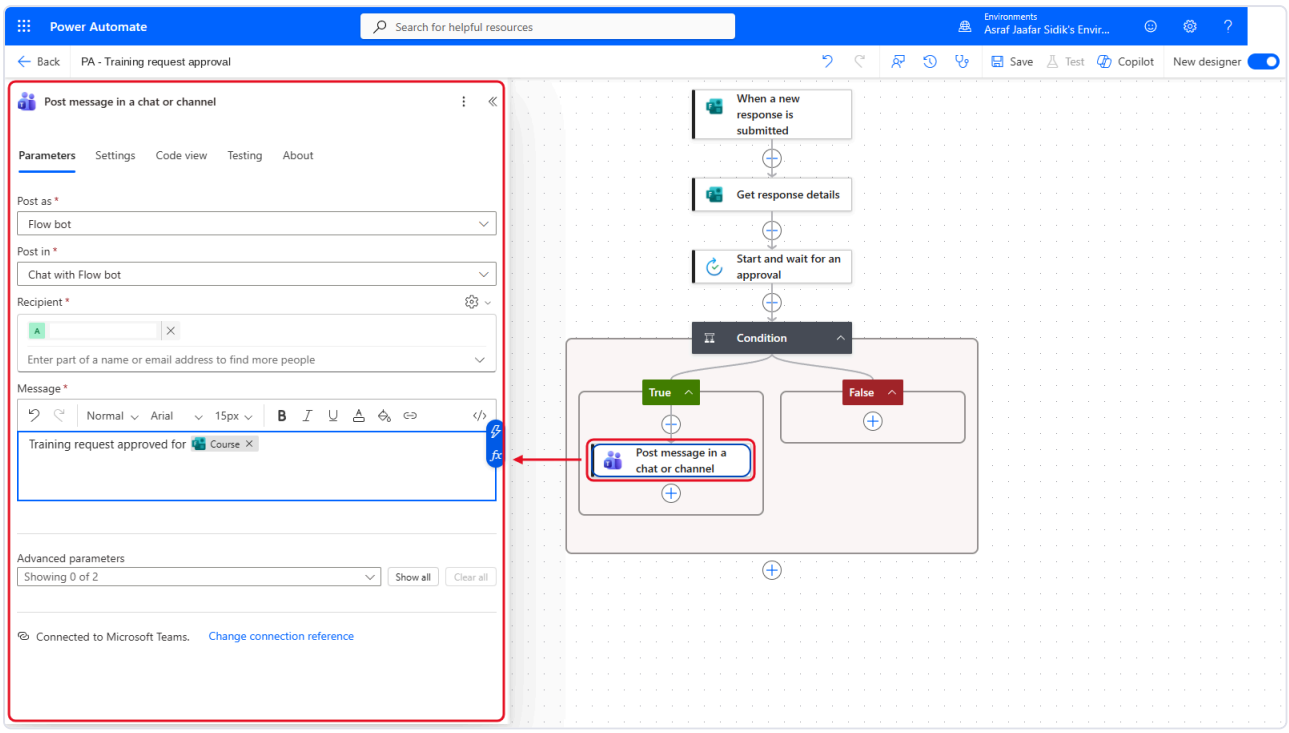
> `Training request approved for`

Select the lightning bolt icon and pick **Course** under *Get response details*.



The True branch posts the approved message. After saving, the Course token shows as a code like `body/rc6d429a1…`. That's normal. Replace `approver@example.com` with your trainer-approved account.

Check your flow so far. Your screen should look like this.



A Teams message in True. False is still empty.

6.8 Prepare the Rejected Message

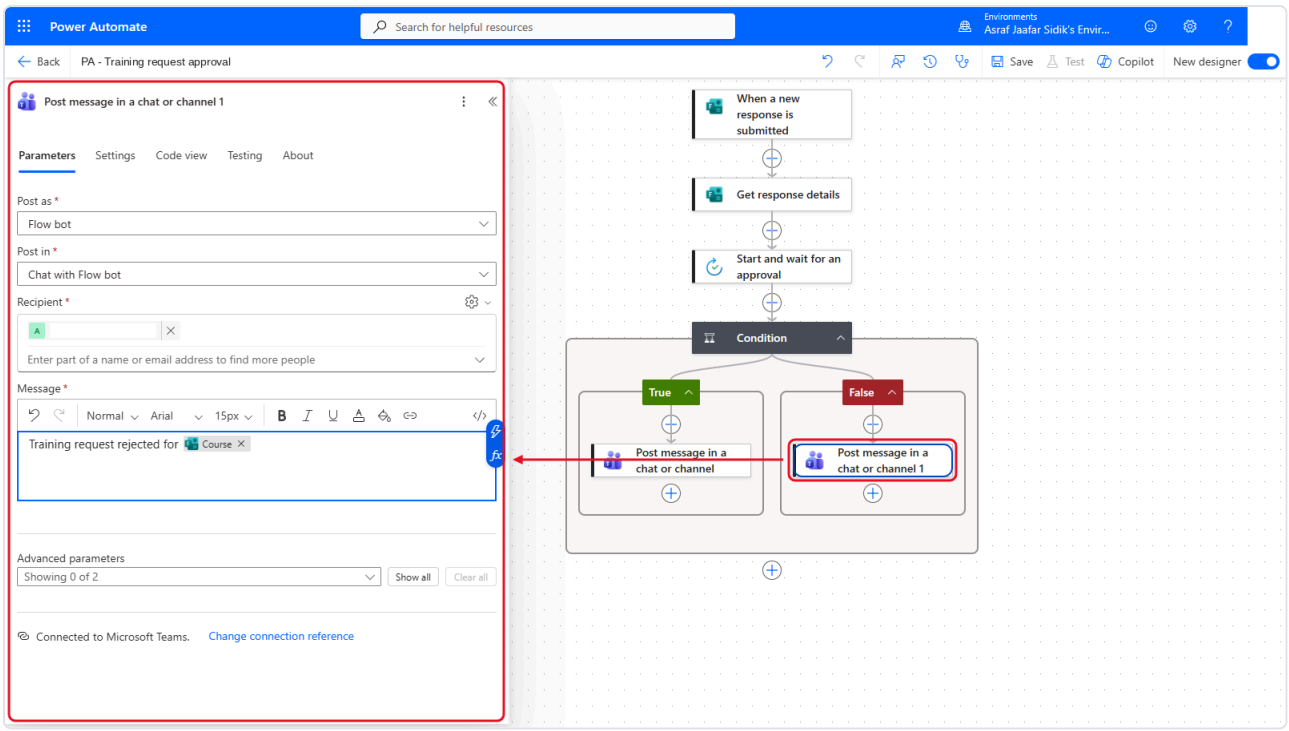
1. On the canvas, select the **+** inside the red **False** box, then **Add an action**, and add a second **Post message in a chat or channel** (Microsoft Teams).
2. Use the same **Post as**, **Post in** and **Recipient** as section 6.7.
3. In **Message**, type the text below, with the space at the end. Copy it from the box below.

   ```
   Training request rejected for
   ```

   Then insert **Course** from *Get response details* with the lightning bolt, as in 6.7.

A production process would also handle outcomes like cancelled or timed-out approvals. This exercise sticks to the two standard Approve and Reject buttons.

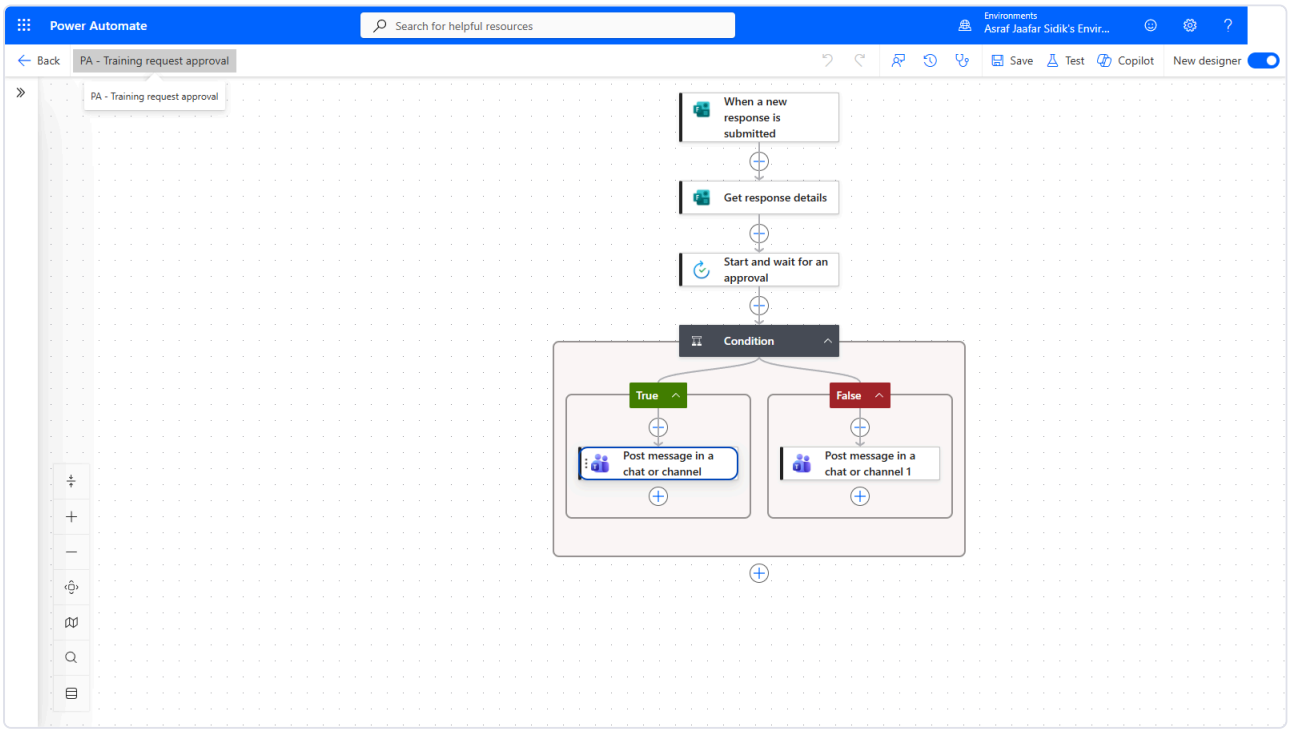
Check your flow so far. Your screen should look like this.



A Teams message in each branch.

6.9 Save and Check the Flow

1. On the toolbar at the top, select **Save**, then **Flow checker**. Fix every error before testing.
2. Confirm the canvas: three steps before the Condition, and one Teams action in each branch.



Three steps, then one Teams message per branch.

Trainer-Controlled Test

Testing creates an approval request and may post to Teams. Replace all three `approver@example.com` placeholders first (**Assigned to** in 6.5 and the **Recipient** in each Teams action in 6.7 and 6.8) and confirm the recipient is ready. Then, with the trainer's go-ahead:

1. Submit one fictional Training Request response.
2. Approve it, and check the True branch and the approved Teams message.
3. Submit a second fictional response.
4. Reject it, and check the False branch and the rejected Teams message.

After each test, open run history and confirm the right branch succeeded while the other was skipped.

Independent Practice

Without running the flow, change the approval title so it also includes the **Course** value. Which action supplies that value, and why can't the trigger supply it on its own?

Troubleshooting

Symptom	What to check
Training Request isn't in Form Id	Form ownership or access, the Forms connection account, and a refreshed list
The approval has no request details	Get response details Form Id, the Response Id token and the Details tokens
The flow waits forever	The assigned approver, the notification, and whether a decision was made
An approval goes down False	The Outcome token, *is equal to*, and the exact text `Approve`
Teams can't resolve the recipient	Replace the placeholder with a trainer-approved account
No Teams message appears	The branch taken, Teams connection, Post in setting, recipient and the action result
Flow checker is clean but you can't test	Flow checker checks structure only, not placeholder recipients or policy

Lesson Summary

A Forms trigger starts the flow with a response ID, Get response details turns it into usable answers, Start and wait for an approval records a human decision, and a Condition routes the request to the matching branch. The flow is ready for a trainer-controlled test once approved recipients replace the placeholders.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Everything on this page is **text**, typed or pasted straight into a field.

Form name (text, section 6.1)

```
Training Request
```

Form description (text, section 6.1)

```
Submit a fictional training request for the Power Automate approval exercise.
```

Course choices (text, section 6.1)

```
Power Automate Basics
Approval Workflows
Excel Reporting
```

Flow name (text, section 6.2)

```
PA - Training request approval
```

Approval title (text, section 6.5)

```
[PA TRAINING] Training request
```

Approval details template (text, section 6.5)

Paste into **Details**. Put the cursor after each label, select the lightning bolt icon, and pick the matching answer under *Get response details*.

```
**Requester name:**
**Course:**
**Preferred date:**
**Business reason:**
```

The `**` marks make the labels bold on the approval card.

Condition right value (text, section 6.6)

```
Approve
```

Teams messages (text, sections 6.7 and 6.8)

Type the text, then insert **Course** from Get response details after it.

```
Training request approved for
```

```
Training request rejected for
```

07

CHAPTER 7

Refine the Weekly Reminders

Chapter 5 sent a reminder to every row, including people who had already finished. This chapter turns that into a process a coordinator would actually use: skip completed training, limit the window to the next 14 days, count what was sent, and handle the week when nothing matches.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 65 minutes

Your result: A copy of the weekly reminder flow that keeps only upcoming incomplete records, counts the reminders, and follows a clear summary or no-records branch.

> **Download:** the files for this lab
>
> - TrainingRegister.xlsx: only if you don't have the workbook from Chapter 2 yet. Upload it to your `Power Automate Training` folder in OneDrive and update the dates so at least two Registered sessions fall in the next 14 days.

What You Will Learn

- Filter a list before a loop with **Filter array**
- Work out a future date with **Get future time**
- Use a condition with two rules that must both be true
- Count processed items with an integer variable
- Branch after a loop, including a week with no matching records

How the Flow Works

Recurrence > Current time > Convert time zone > List rows > Filter array > Get future time > Initialize variable > Apply to each (Condition > Send reminder > Increment variable) > Condition > Summary or no-records message

Filter array removes completed rows before the loop starts. Inside the loop, a condition checks that the session falls in the next 14 days. A counter goes up once per reminder. After the loop, a second condition checks the counter and picks what to tell the coordinator.

Before You Begin

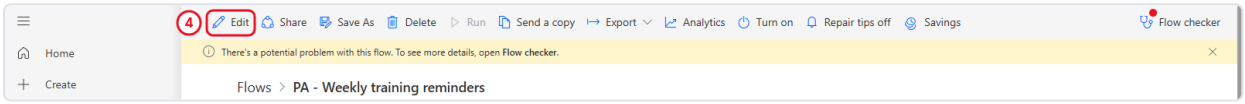
Complete Chapter 5.

> **Important:** The sample register uses `example.com` addresses, which can't receive mail. Keep this flow turned off and don't test it until the trainer has replaced every recipient with an approved destination.

> **Note:** The sample sessions run from 21 September to 5 October 2026. The SessionDate values in your workbook must include at least two **Registered** rows within the next 14 days of today. Otherwise the capstone finds nothing and goes to **False**. To update the dates, see Chapter 2, section 2.2, step 4, or ask your trainer.

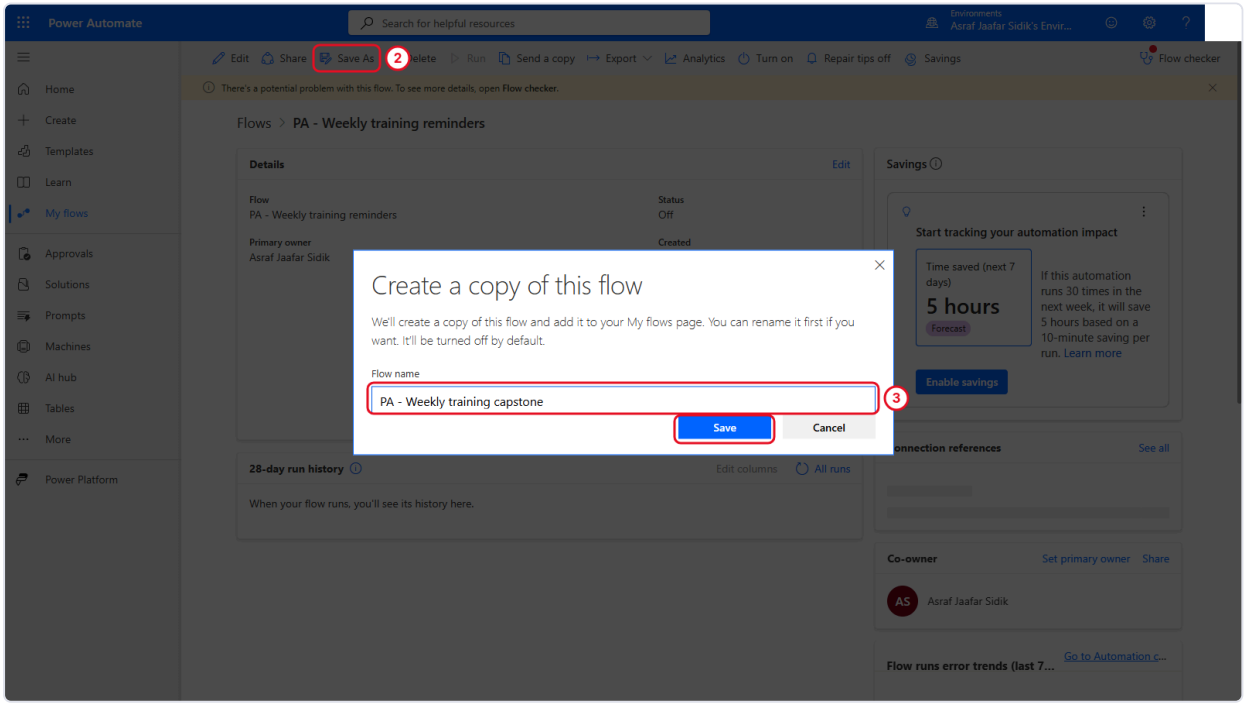
7.1 Make a Safe Copy

1. In the left navigation, select **My flows**. Select the flow's name `PA - Weekly training reminders` (not the pencil) to open its details page.



The original flow's details page. Use Save As here. Don't select Edit on this page: you edit the copy in step 4.

2. In the toolbar at the top, select **Save As** (in the **...** menu if it isn't visible).
3. In **Flow name**, enter `PA - Weekly training capstone` and select **Save**. The copy starts turned off.



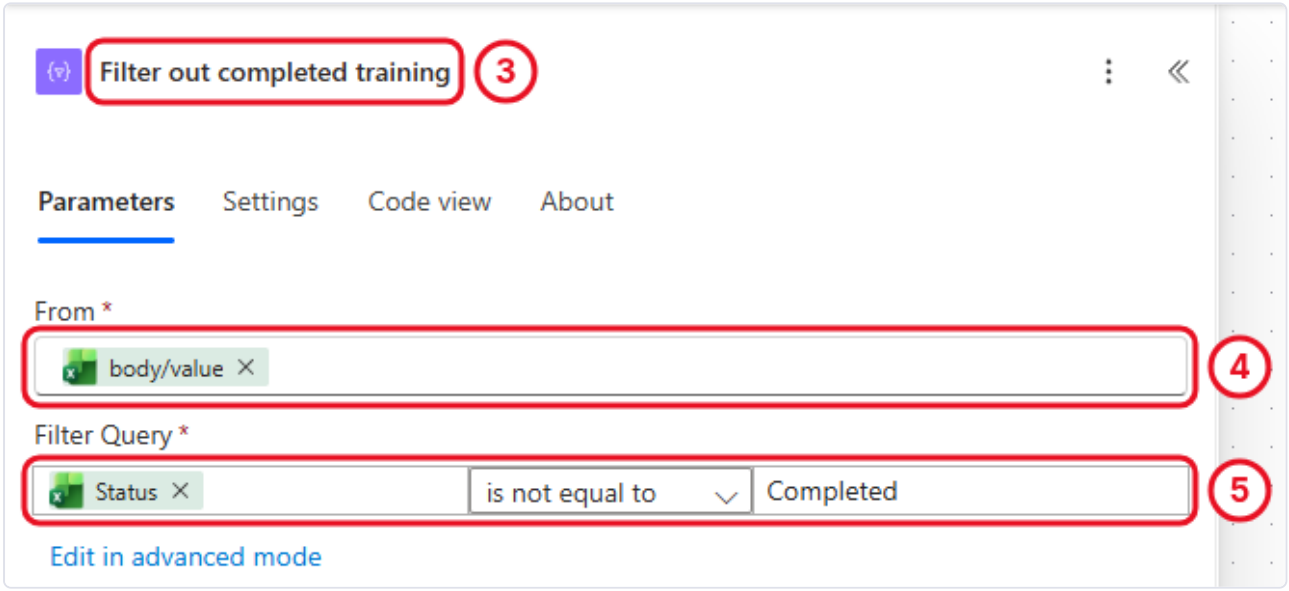
Working on a copy keeps your Chapter 5 flow safe.

4. Go back to **My flows** and select the copy's name (not the pencil). Its details page opens. Select **Edit** at the left of the toolbar to open it in the designer.

Checkpoint: The name at the top of the designer reads `PA - Weekly training capstone` before you change anything.

7.2 Remove Completed Rows with Filter Array

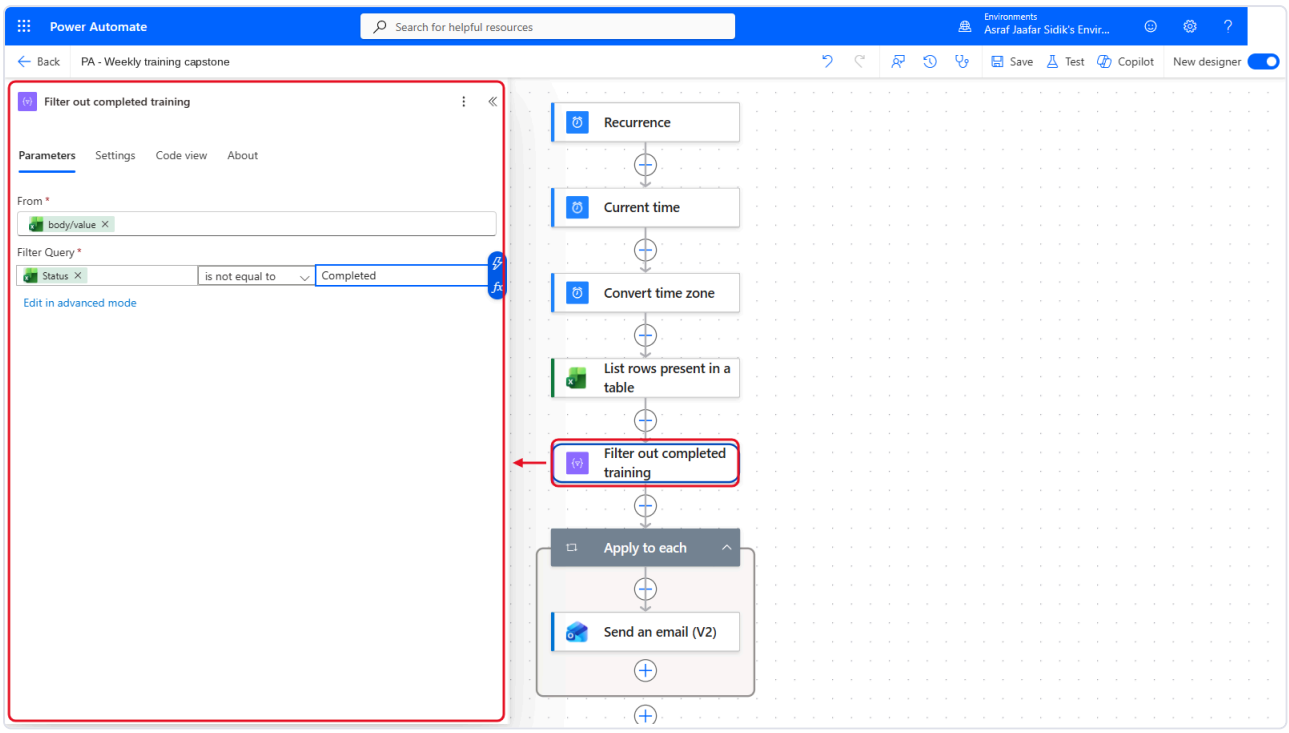
1. On the canvas, select the **+** between **List rows present in a table** and **Apply to each**, then **Add an action**.
2. In the search box at the top of the panel, type `Filter array`. Under the **Data Operation** heading, select **Filter array**. Its settings open on the left.
3. Rename it: select the action's title at the top of the panel and type `Filter out completed training`.
4. Click into the **From** box. Select the lightning bolt icon beside it (or type `/` and choose **Insert dynamic content**), then select **body/value** under *List rows present in a table*. (It may be labelled **value**. If you can't see it, select **See more** or type `value` in the picker's search box.)
5. Fill the row of three boxes under **Filter Query**, left to right:
   - Left box: open the lightning bolt and select **Status** under *List rows present in a table*. Type `Status` in the picker's search box if it isn't listed.
   - Middle dropdown: select **is not equal to**.
   - Right box: type `Completed`.



Filter array keeps only the rows where the rule is true. Completed rows are dropped before the loop.

> **Tip:** The comparison is exact, including capital letters. `Completed` and `completed` count as different. The sample data uses `Completed`.

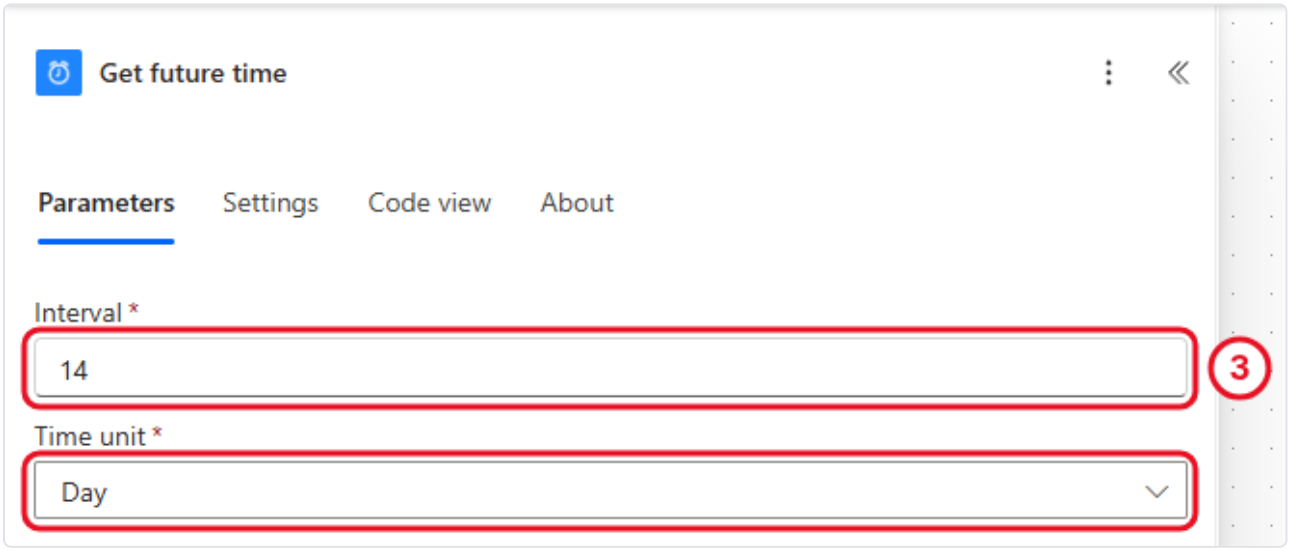
Check your flow so far. Your screen should look like this.



Filter out completed training sits between List rows and Apply to each.

7.3 Work Out the End of the Window

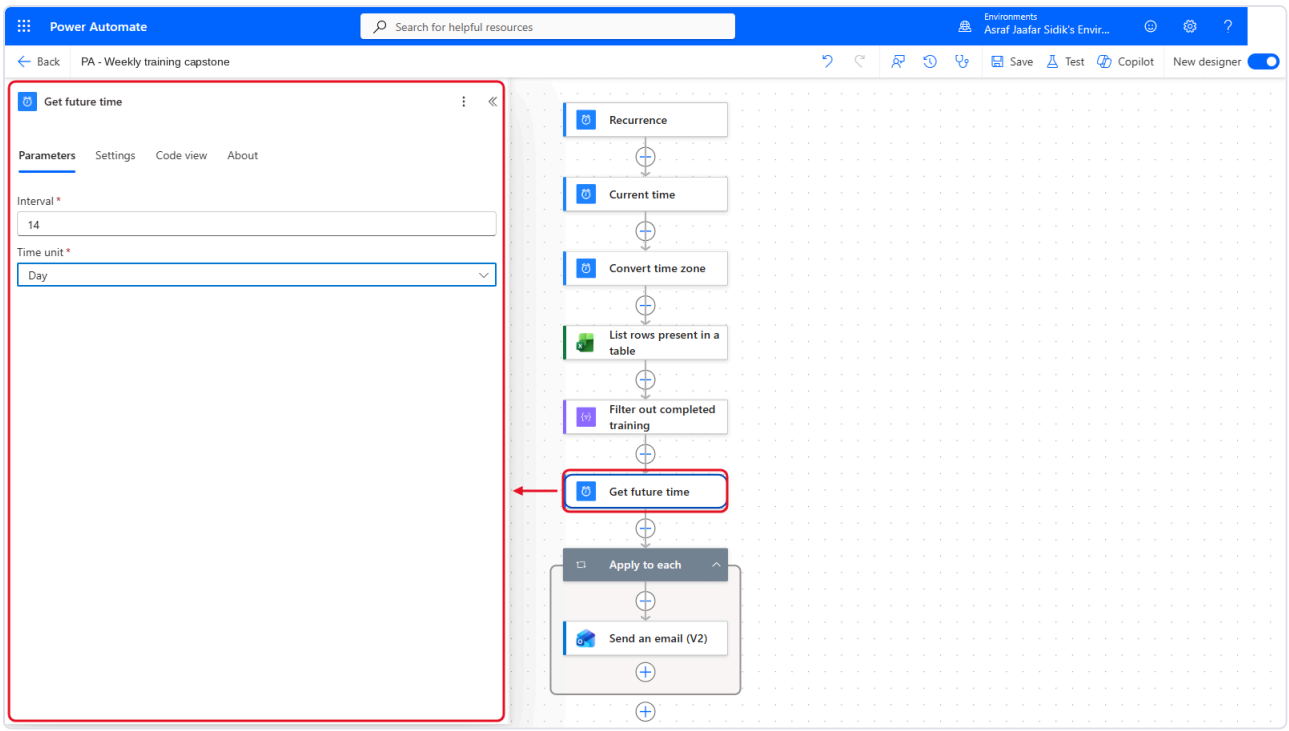
1. On the canvas, select the **+** below **Filter out completed training**, then **Add an action**.
2. Search for `Get future time` and select it under **Date Time**.
3. In **Interval**, enter `14`. Open the **Time unit** dropdown and select **Day**.



This gives you "14 days from now" without any expression.

You now have both ends of the window: **Current time** (already in the flow from Chapter 5) and **Future time** from this step.

Check your flow so far. Your screen should look like this.

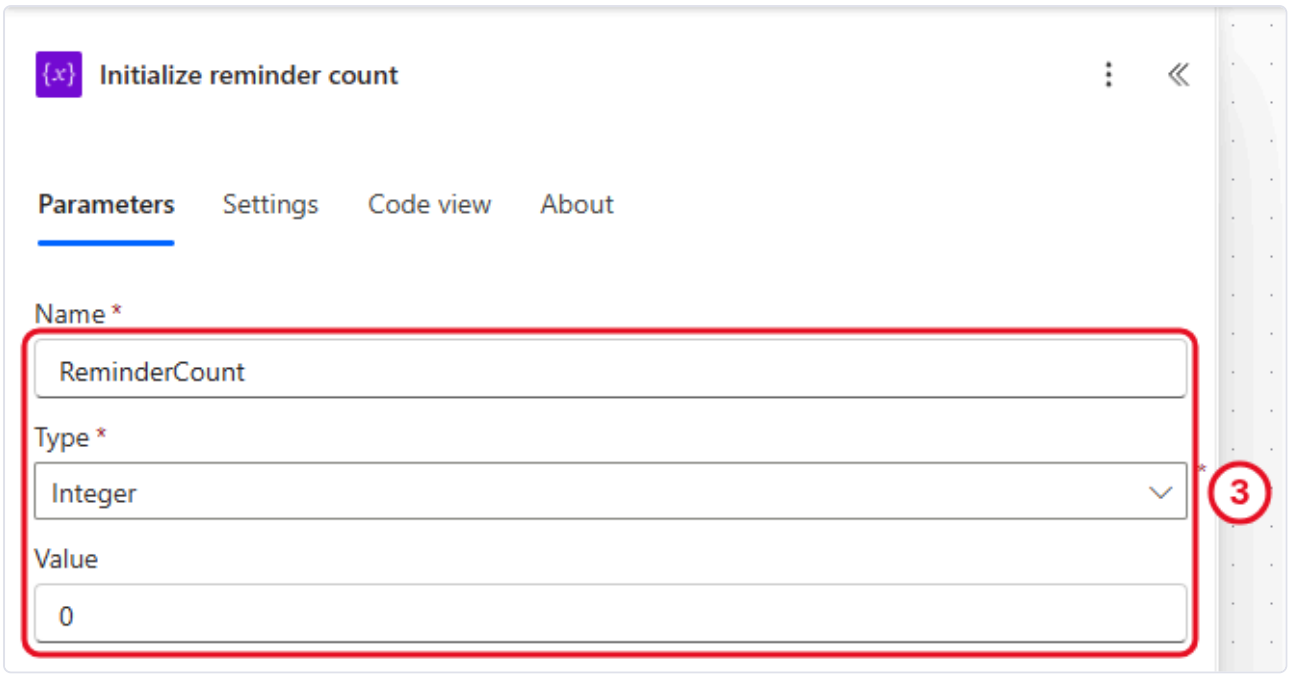


Get future time comes after the filter.

7.4 Start the Reminder Counter

1. On the canvas, select the **+** below **Get future time**, then **Add an action**. Search for `Initialize variable` and select it under **Variables**.
2. Rename it: select the title at the top of the panel and type `Initialize reminder count`.
3. Fill in the boxes below. For **Type**, open the dropdown and select **Integer**.

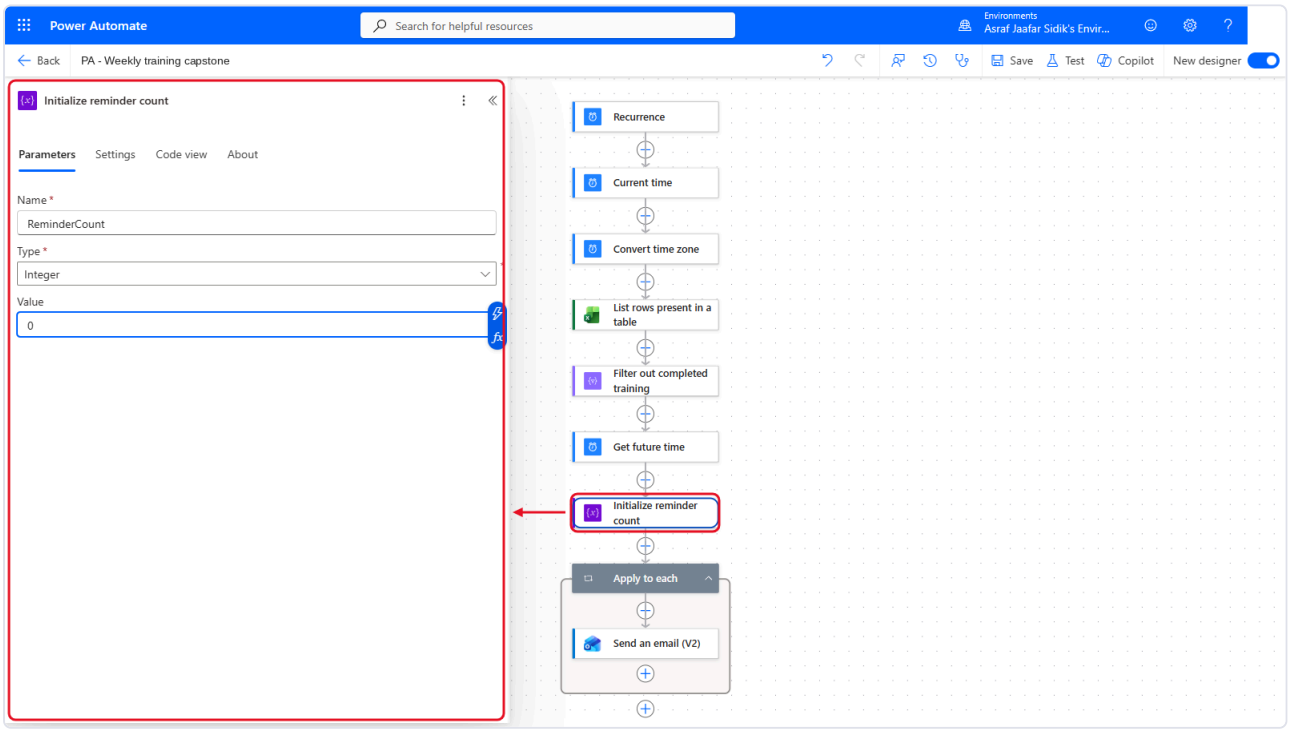
Field	Value
Name	`ReminderCount`
Type	Integer
Value	`0`



A variable is a named box that can change while the flow runs. This one starts at zero.

> **Key point:** Variables must be created at the top level of the flow, never inside a loop or a condition. That's why this step comes before Apply to each.

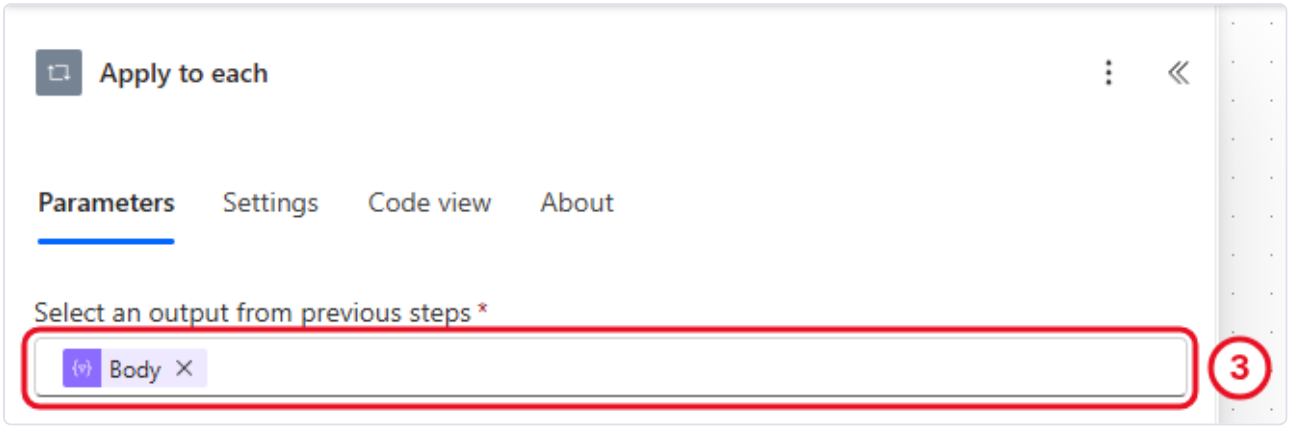
Check your flow so far. Your screen should look like this.



Initialize reminder count sits just above the loop.

7.5 Loop Through the Remaining Rows

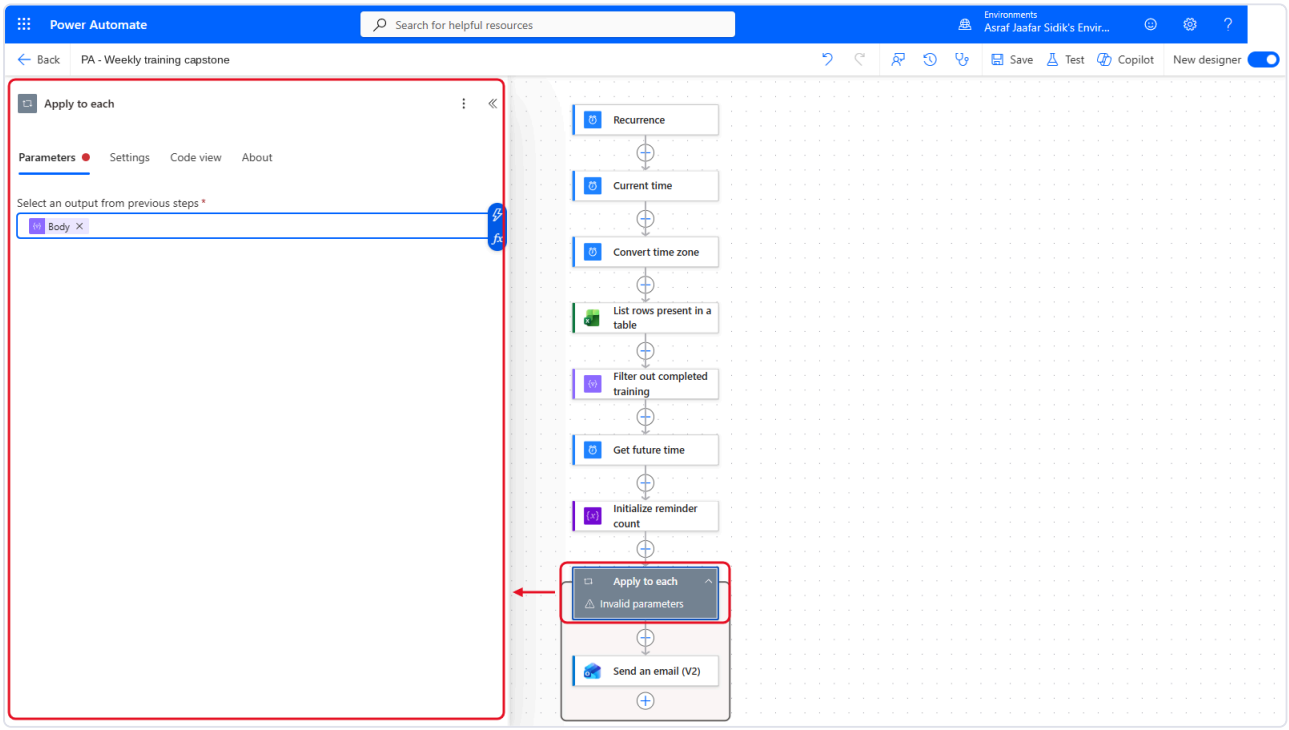
1. On the canvas, select the **Apply to each** card to open its panel.
2. In **Select an output from previous steps**, select the **x** on the current token to remove it.
3. Click into the empty box, open the lightning bolt, and select **Body** under *Filter out completed training*.



The loop now only sees rows that aren't completed.

> **Key point:** Don't leave the original Excel value here. That brings back the Chapter 5 behaviour and includes completed rows.

Check your flow so far. Your screen should look like this.



Apply to each now loops over the filtered rows. If the loop shows Invalid parameters straight after the change, select another card and then the loop again. Flow checker in 7.11 confirms there are no errors.

7.6 Check the Date Window

1. On the canvas, inside the **Apply to each** box, select the **+** just above **Send an email (V2)**, then **Add an action**. Search for `Condition` and select it under **Control**.

   > **Note:** The new condition appears *above* the email, and the email stays where it was, underneath the condition rather than inside it. You move it into **True** in step 7.

2. Rename it: select the title at the top of the panel and type `Is the session in the next 14 days`.

3. First row: click the left **Choose a value** box and select the **fx** button that appears beside it (or type `/` and choose **Insert expression**). Paste the expression (copy it from the box below) and select **Add**.

   ```
   items('Apply_to_each')?['SessionDate']
   ```

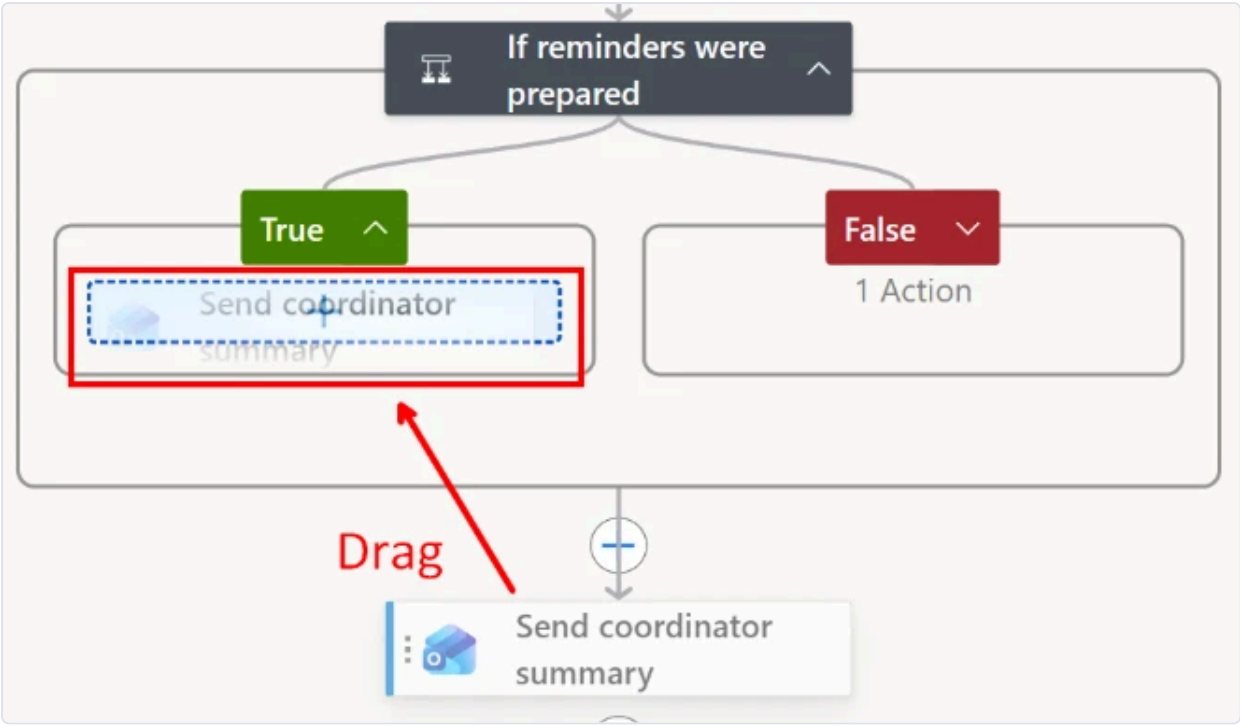
   (See the key point below for why.) If your loop has a different name (for example Apply_to_each_1), change the name in the expression to match. The box now shows a pink expression token. It may read **SessionDate** or **items(...)**: both are correct. Open the middle dropdown and select **is greater or equal to**. Click the right box, open the lightning bolt, and select **Current time** under *Current time*.

4. Select **Add row** below the rows. Second row: insert the same expression on the left, select **is less or equal to** in the middle, and on the right pick **Future time** under *Get future time*.

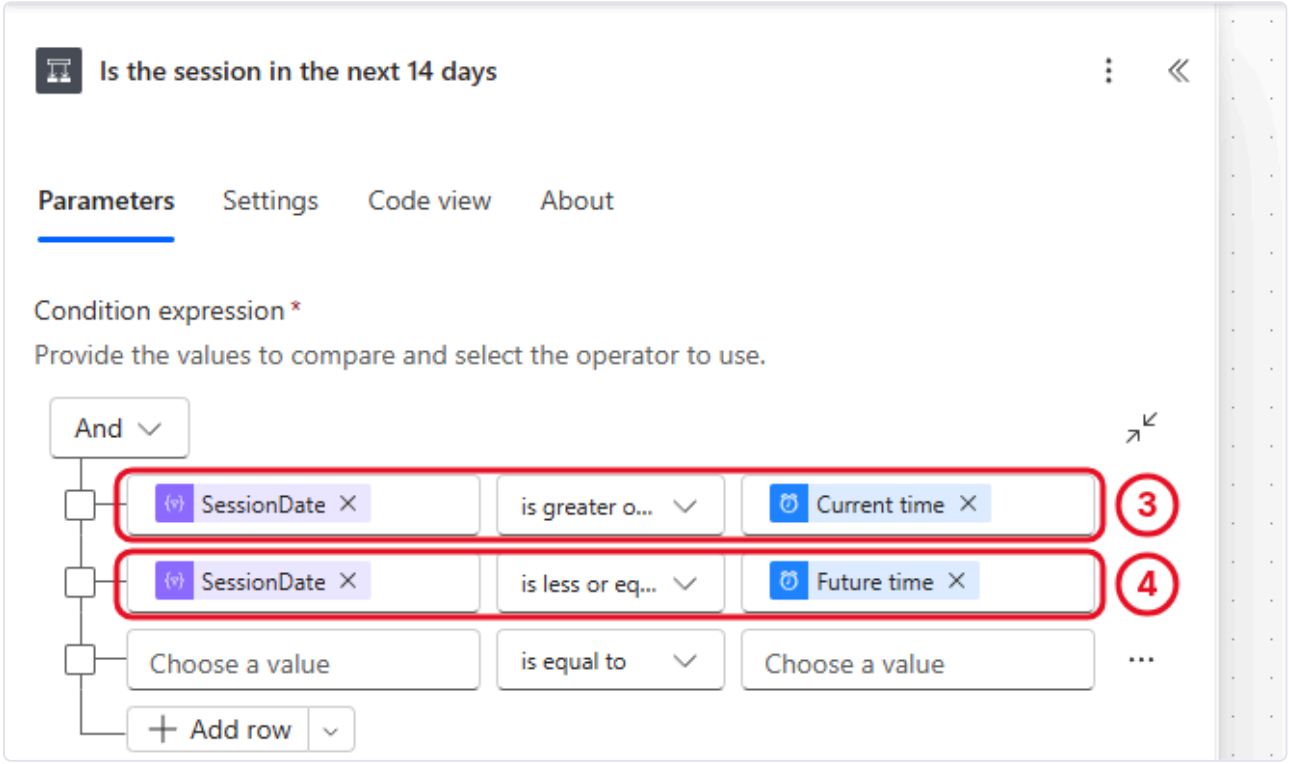
5. If an empty extra row appears below your two rows, select its **...** and then **Delete**.

6. Check the dropdown above the rows reads **And**, so both rules must be true.

7. Move the email into **True**: press and hold on the **Send an email (V2)** card below the condition, drag it up into the green **True** box, and let go when a dashed blue drop zone appears there. On the canvas, the email now sits inside **True**, and nothing is left below the condition inside the loop. (If dragging won't work, delete the email and add it again inside **True** with the same settings.)



The drag shown here is from 7.9, a later step; the move is the same. Hold the card (the dotted handle at its left edge is easiest to grab), drag it over True, and let go on the dashed blue drop zone.

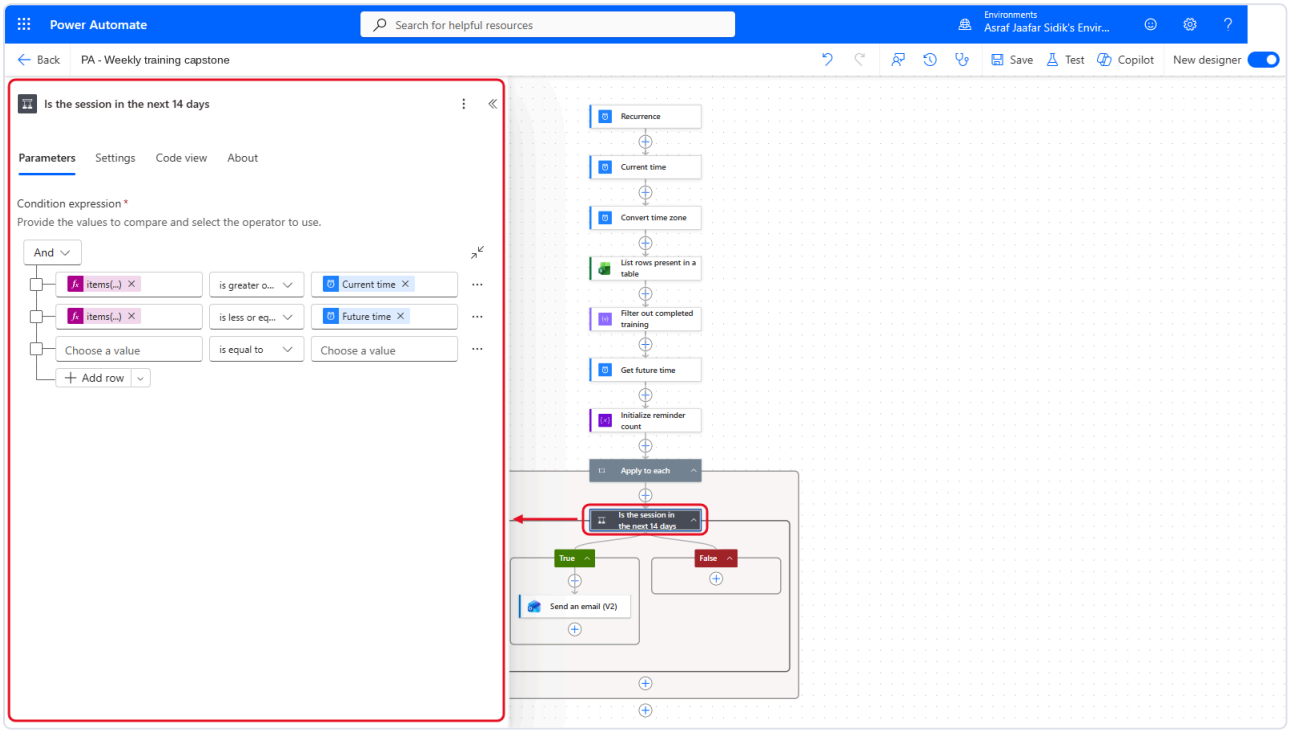


Two rules joined by AND. Your boxes show pink fx tokens from the expression; the picture shows the same values. The screenshot still shows the empty row; delete it on yours.

> **Key point: why an expression here?** Once the loop runs over Filter array's **Body**, the dynamic content picker no longer lists the Excel columns. If you pick **SessionDate** from *List rows present in a table* anyway, the designer wraps the condition in a second, unwanted loop. The expression `items('Apply_to_each')?['SessionDate']` simply means "the SessionDate of the row this loop is on right now". The email tokens you added in Chapter 5 keep working, because they already point at the loop's current row.

This works because the Excel dates are in ISO 8601 format (set in Chapter 5), the same format as Current time and Future time. Same format means they compare correctly.

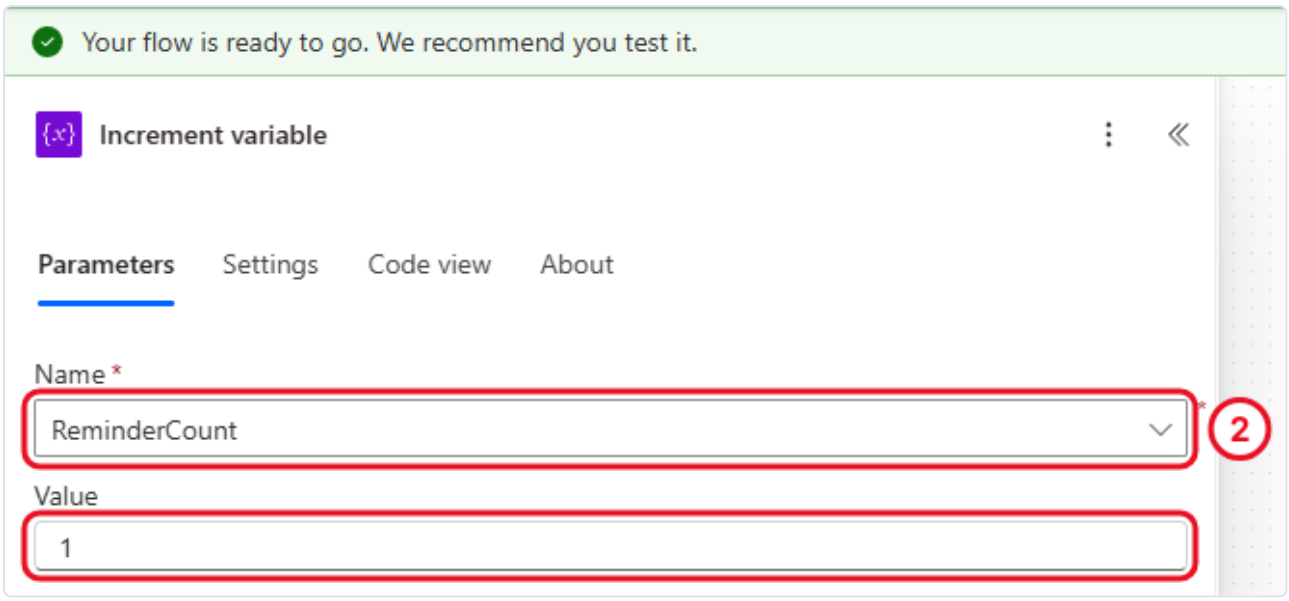
Check your flow so far. Your screen should look like this.



The date condition sits inside the loop, with the email in True.

7.7 Count Each Reminder

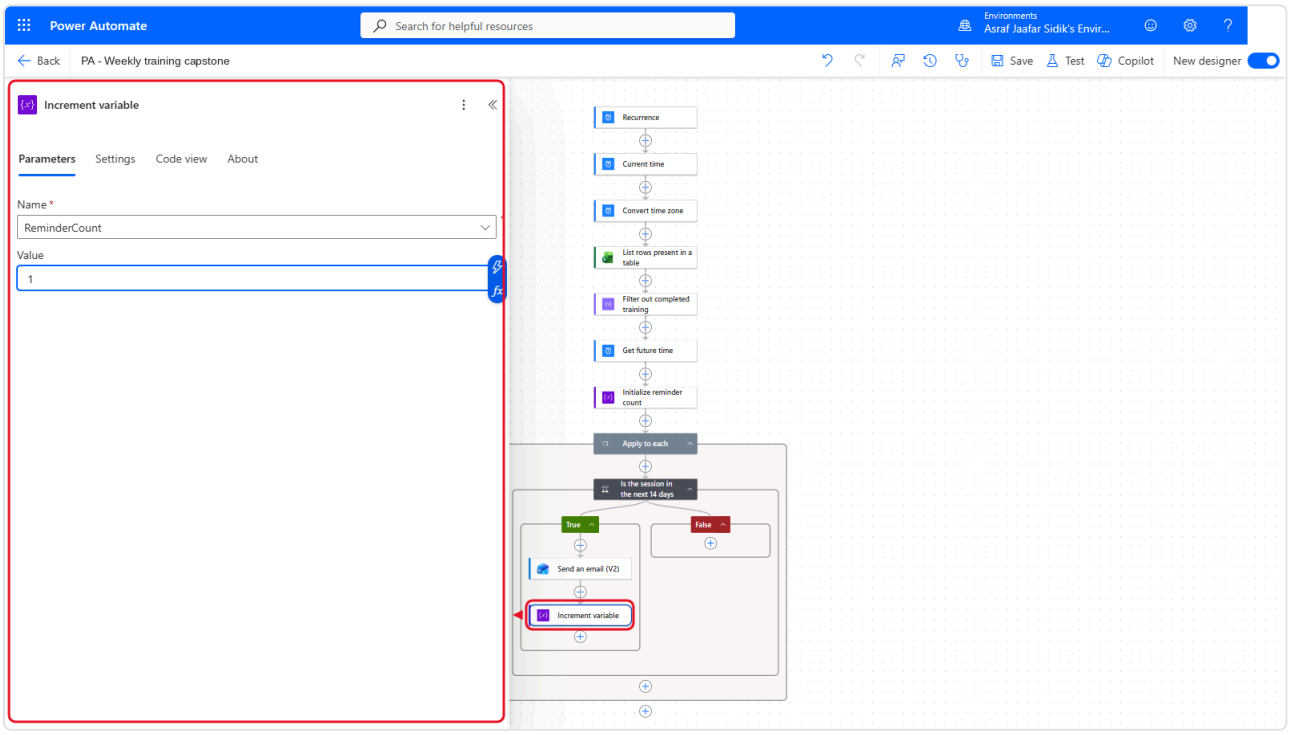
1. On the canvas, inside the green **True** box, select the **+** below **Send an email (V2)**, then **Add an action**. Search for `Increment variable` and select it under **Variables**.
2. Open the **Name** dropdown and select **ReminderCount**. In **Value**, enter `1`.



Each reminder adds one to the counter.

Keep the increment *after* the email, so the count only includes reminders that reached the email step.

Check your flow so far. Your screen should look like this.



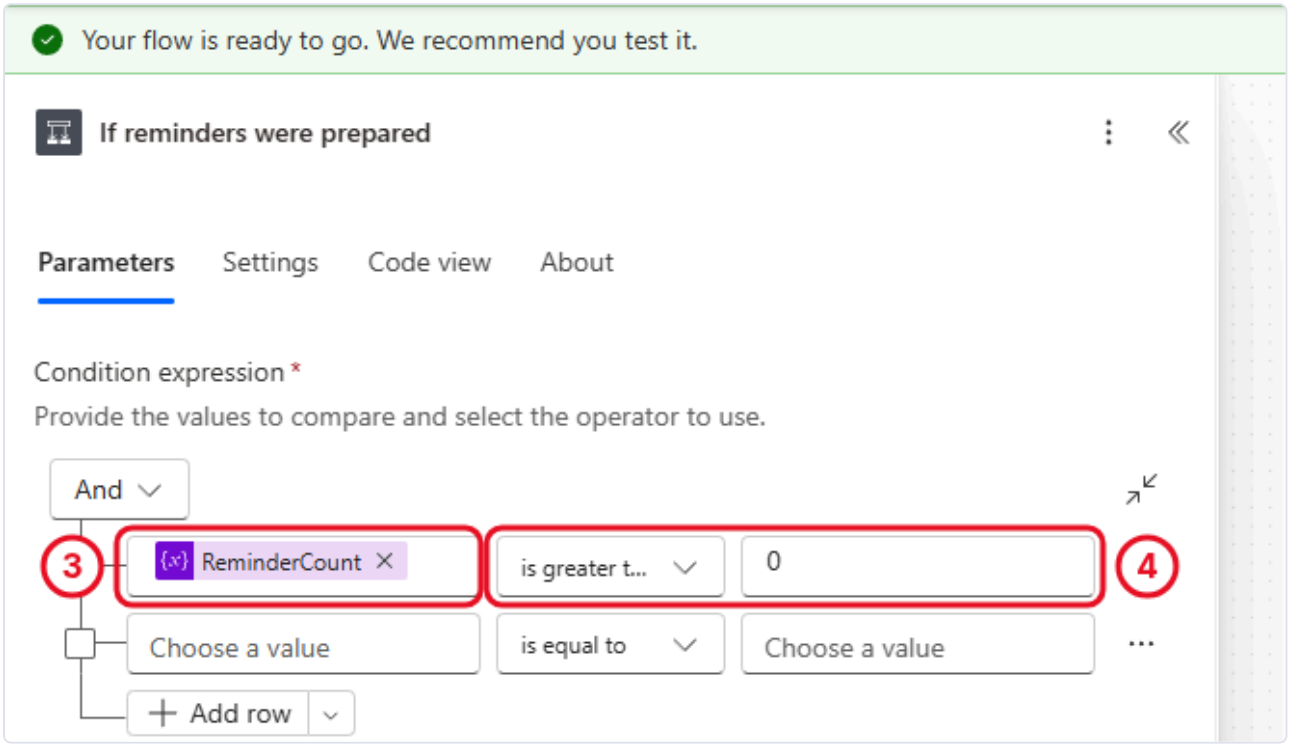
Increment variable sits under the email in True.

7.8 Decide Whether a Summary Is Needed

1. On the canvas, select the **+** below the **Apply to each** box, then **Add an action**. Search for `Condition` and select it under **Control**.

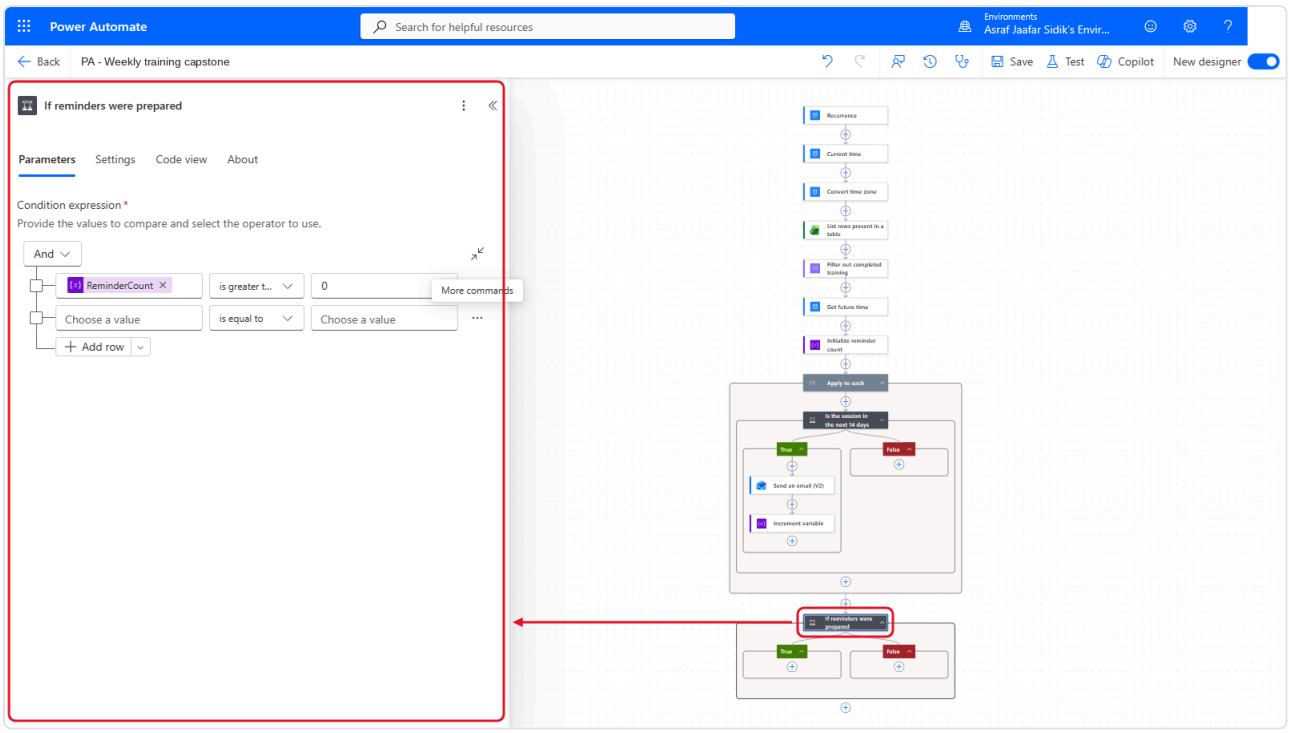
> **Warning: use the + outside the loop.** The loop's grey box has its own **+** at the bottom, just inside its border. Use the one *below* the grey border, the last **+** on the canvas. If the new condition appears inside the grey box, it would run once per row: delete it and add it again from the right **+**.

2. Rename it `If reminders were prepared`.
3. Click the left **Choose a value** box, select the lightning bolt (or type `/` and choose **Insert dynamic content**), and pick **ReminderCount** under *Variables*.
4. Open the middle dropdown and select **is greater than**. Click the right box and type `0`.
5. If an empty extra row appears, select its **...** and then **Delete**.



A positive count goes to True. Zero goes to False. The screenshot still shows the empty row; delete it on yours.

Check your flow so far. Your screen should look like this.



The summary condition sits below the loop.

7.9 Prepare the Coordinator Summary

On the canvas, inside the green **True** box under **If reminders were prepared**, select the **+**, then **Add an action**. (If the email lands below the condition instead, drag it into **True** the same way as in 7.6.) Search for `Send an email`, select **Send an email (V2)** under **Office 365 Outlook**, and rename it `Send coordinator summary`:

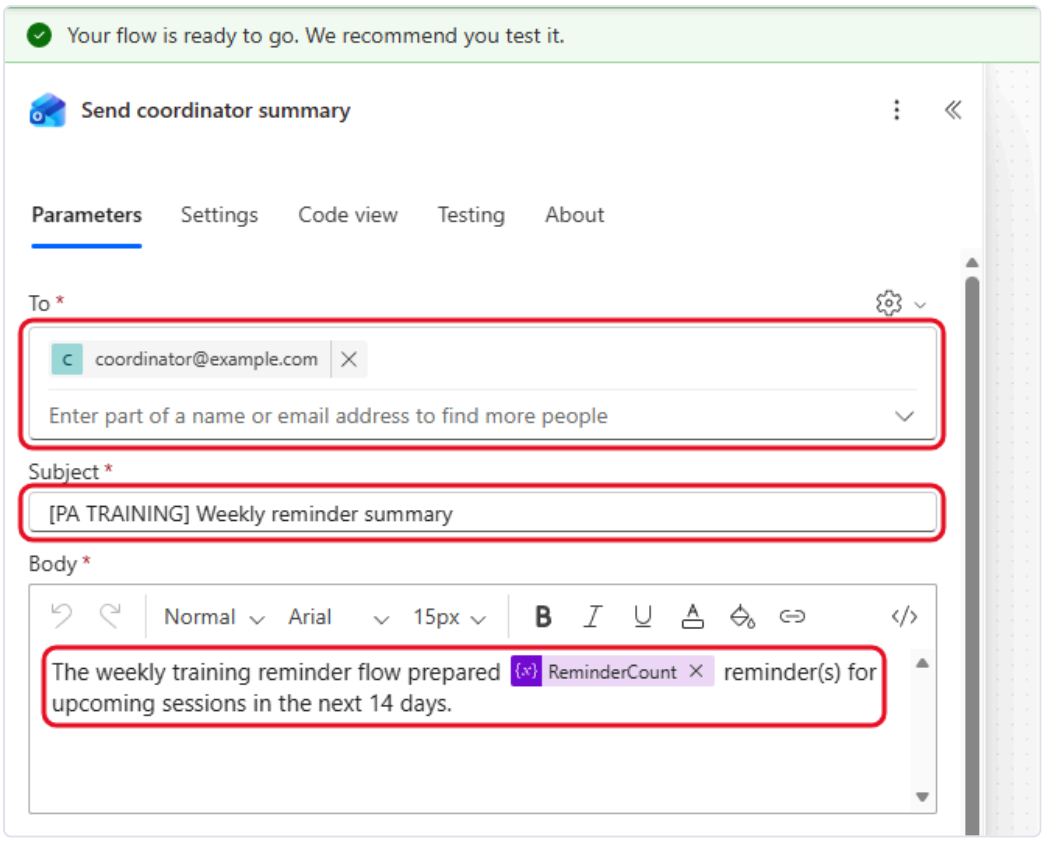
Field	Value
To	Type your own email address, then select it from the list. (The screenshot shows `coordinator@example.com`, a placeholder: see the warning below.)
Subject	`[PA TRAINING] Weekly reminder summary`
Body	Type `The weekly training reminder flow prepared`, then select the lightning bolt (or type `/` and choose **Insert dynamic content**) and pick **ReminderCount** under *Variables*. Then type `reminder(s) for upcoming sessions in the next 14 days.` (with the space at the start). Both text pieces are in the boxes below the table.

Body, first part (paste before the **ReminderCount** token, keep the space at the end):

```
The weekly training reminder flow prepared
```

Body, second part (paste after the token, keep the space at the start):

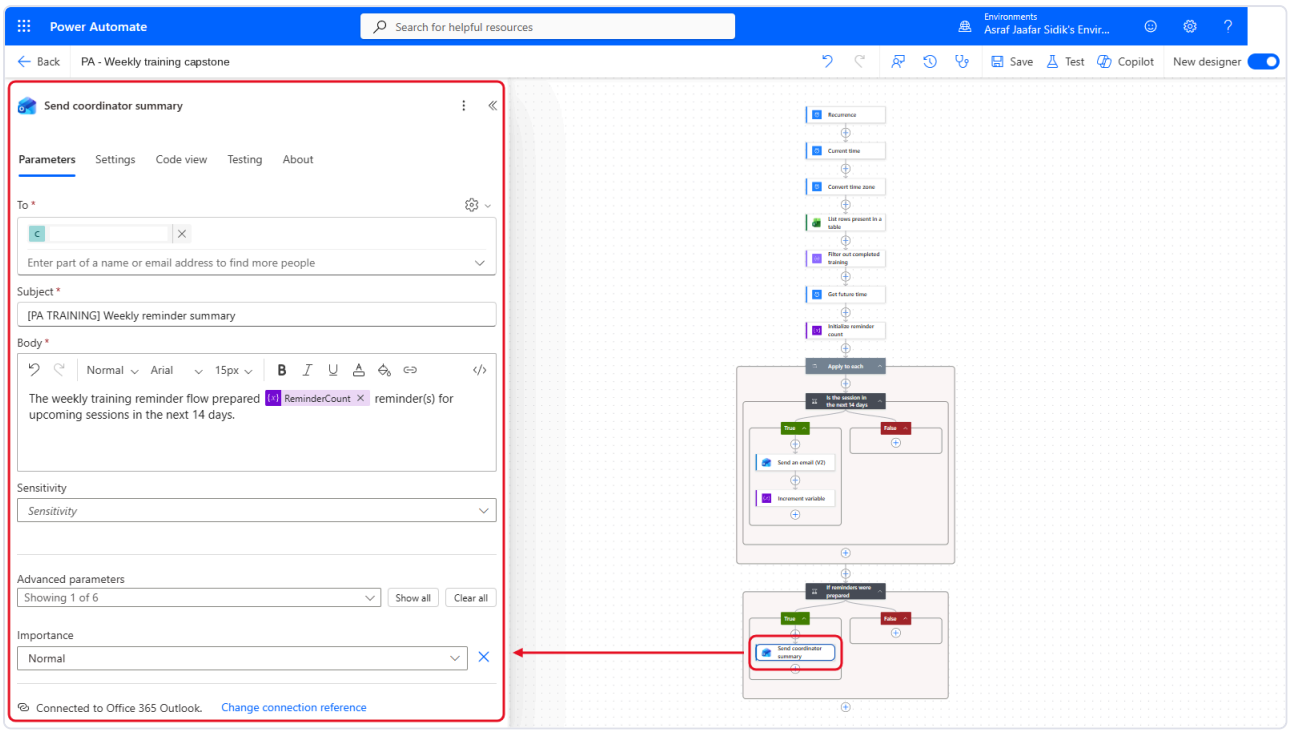
```
 reminder(s) for upcoming sessions in the next 14 days.
```



The variable drops into the email like any other dynamic content.

> **Warning:** `coordinator@example.com` **is not a real mailbox.** It stands in for the training coordinator. Mail sent to it bounces, so the summary would never arrive. Put your own email address in **To**, so the summary comes to you when you test. The learner reminders inside the loop go to the Email column in the workbook, so put your own address in the rows you test with too (Chapter 5 has the same warning).

Check your flow so far. Your screen should look like this.



Send coordinator summary sits in True.

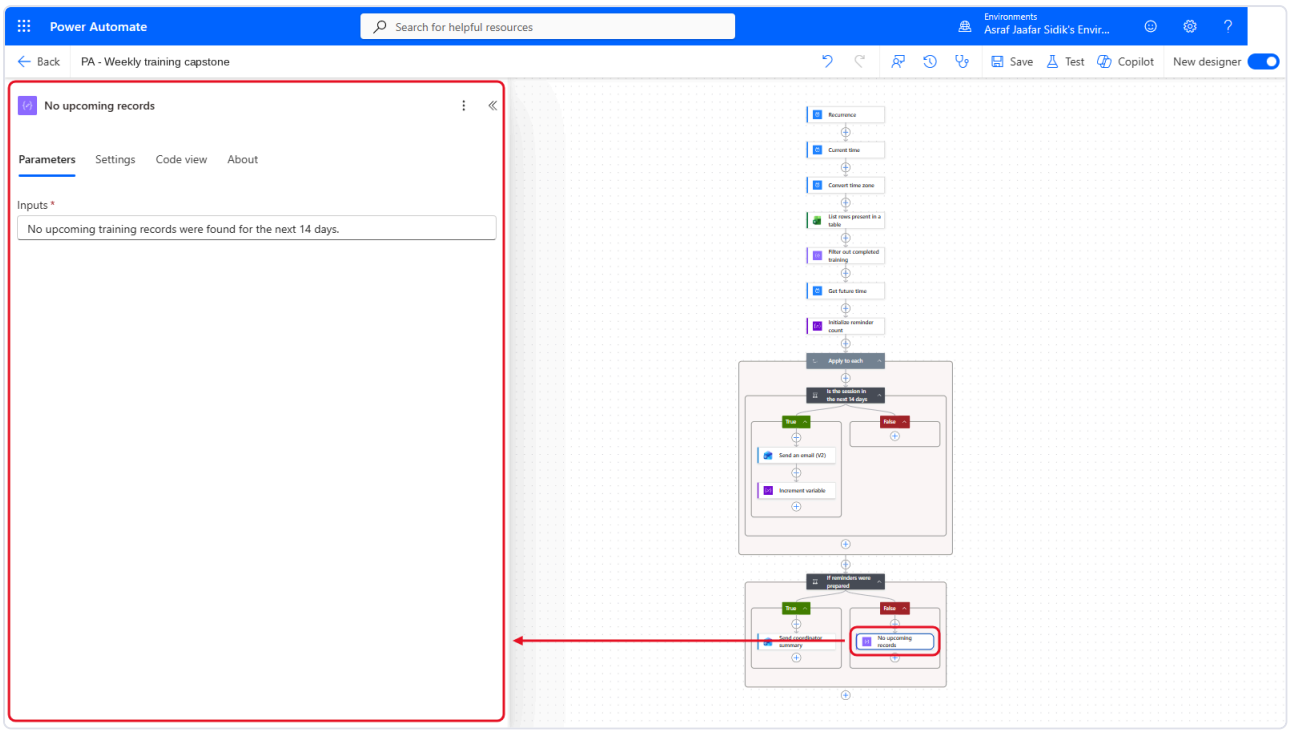
7.10 Handle a Week with No Matching Rows

On the canvas, inside the red **False** box, select the **+**, then **Add an action**. Search for `Compose` and select it under **Data Operation**. Rename it `No upcoming records`, and paste the text below in **Inputs**. Copy it from the box below.

```
No upcoming training records were found for the next 14 days.
```

That records a clear outcome in run history without sending an unnecessary email.

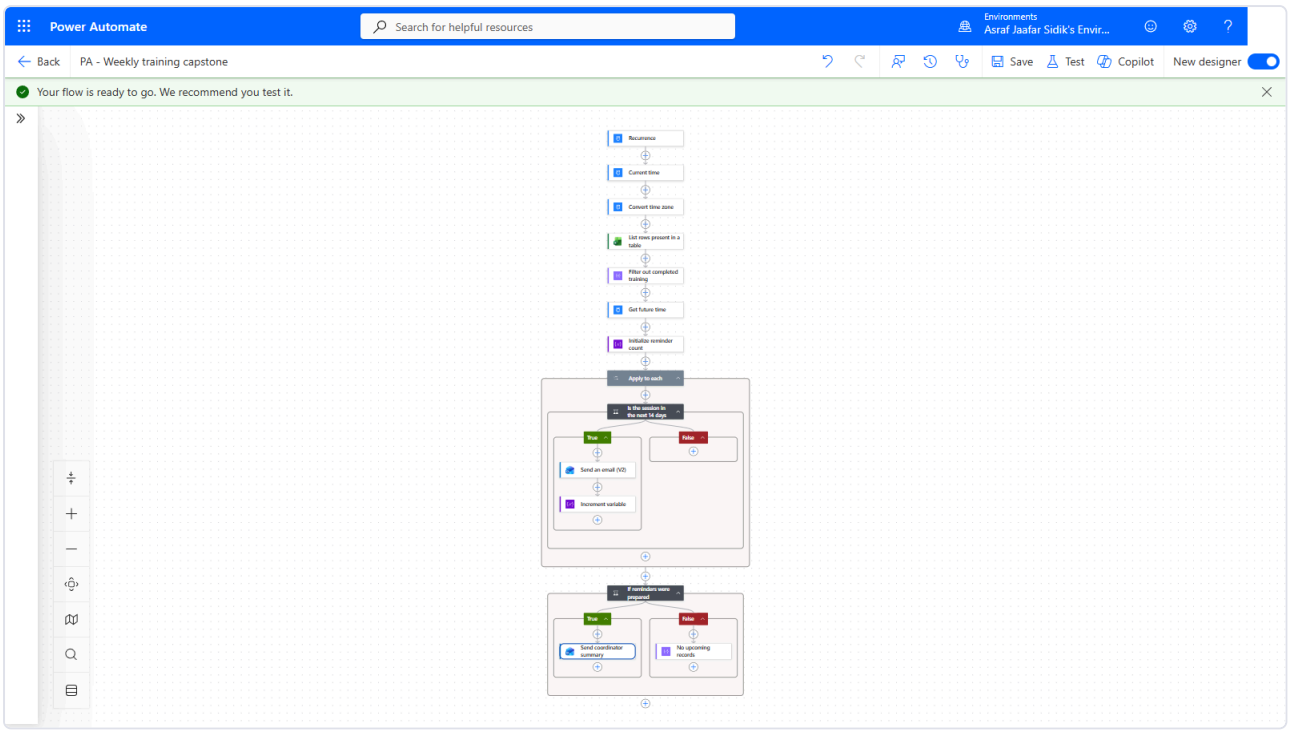
Check your flow so far. Your screen should look like this.



No upcoming records sits in False. That is the whole flow.

7.11 Save and Check the Flow

1. Select **Save** in the toolbar at the top right. Then select **Flow checker** in the same toolbar and confirm zero errors and zero warnings.
2. Close Flow checker. In the column of buttons at the bottom left of the canvas, select **Zoom view to fit**. Check the shape: filter before the loop, date check and count inside the loop, decision after the loop.



Filter before, check and count inside, decide after.

Trainer-Controlled Test

A test can send one learner reminder per matching row plus, when the count is positive, a coordinator summary. With dates set as in Chapter 2 (section 2.2, step 4), at least two rows match. Before testing, replace every `example.com` address with your own, both in the workbook's Email column and in **Send coordinator summary**, save the workbook, and wait for the trainer's go-ahead.

Afterwards, check the run history:

- Filter out completed training returned only the rows that aren't Completed (four with the sample data)
- Apply to each ran once for each of those rows, and the date condition was true for the ones dated within the next 14 days
- ReminderCount matches the number of true results
- The final condition took True and prepared the coordinator summary

Independent Practice

Without running the flow, change the window from 14 days to 7 by editing only **Get future time**. Which of your rows would match today? Then change it back to 14.

> **Going further: one filter that does it all.** Experienced makers often replace the filter and the date condition with a single Filter array in **advanced mode**, using an expression that also ignores capital letters and stray spaces. You'll build flows like that in the advanced course. Optional. To try it, in Filter array choose **Edit in advanced mode** and paste the expression below.

```
@and(
not(equals(toLower(trim(item()?['Status'])), 'completed')),
greaterOrEquals(ticks(item()?['SessionDate']), ticks(startOfDay(utcNow()))),
lessOrEquals(ticks(item()?['SessionDate']), ticks(addDays(startOfDay(utcNow()), 14)))
)
```

Troubleshooting

Symptom	What to check
A completed row still gets a reminder	Filter array right value is exactly `Completed`, and the loop uses Filter's **Body**
Every date fails the condition	DateTime Format is ISO 8601 in List rows present in a table
A second Apply to each appears inside the loop	You picked SessionDate from the Excel action. Delete the extra loop and use the expression from section 7.6
No date passes the condition	The sample dates may be in the past. Ask the trainer to update SessionDate
ReminderCount stays at zero	Increment variable is inside the True branch of the date condition
Initialize variable can't be added	You're inside the loop. Variables are created at the top level
The summary runs before reminders finish	Put the final Condition below Apply to each, outside its box
Flow checker is clean but a test is unsafe	Flow checker checks structure. It doesn't approve recipients

Lesson Summary

The refined flow filters out completed rows before looping, checks a date window with a two-rule condition, keeps a running count in a variable, and branches on the final count. That turns a broad weekly reminder into a controlled process with a clear result for the coordinator. Next, you'll use variables again, alongside parallel branches, in a flow that's all about you.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Everything in the main steps is **text**, typed or pasted straight into a field.

Flow name for the copy (text, section 7.1)

```
PA - Weekly training capstone
```

Filter array (text, section 7.2)

Action name:

```
Filter out completed training
```

Right value:

```
Completed
```

Variable (text, section 7.4)

Action name:

```
Initialize reminder count
```

Variable name:

```
ReminderCount
```

Date condition name (text, section 7.6)

```
Is the session in the next 14 days
```

Current row's session date (section 7.6)

Click the left **Choose a value** box of each condition row and select the **fx** button beside it (or type `/` and choose **Insert expression**). Paste, then select **Add**. The box shows a pink token that may read **SessionDate** or **items(...)**: both are correct. If your loop has a different name (for example Apply_to_each_1), change the name in the expression to match.

```
items('Apply_to_each')?['SessionDate']
```

Summary condition name (text, section 7.8)

```
If reminders were prepared
```

Coordinator summary (text, section 7.9)

Action name:

```
Send coordinator summary
```

Subject:

```
[PA TRAINING] Weekly reminder summary
```

Body: paste the first part. Select the lightning bolt (or type `/` and choose **Insert dynamic content**) and pick **ReminderCount** under *Variables*. Then paste the second part.

```
The weekly training reminder flow prepared
```

```
 reminder(s) for upcoming sessions in the next 14 days.
```

No-records Compose (text, section 7.10)

Action name:

```
No upcoming records
```

Inputs:

```
No upcoming training records were found for the next 14 days.
```

Going further (optional)

One filter that does it all

Replaces both Filter array and the date condition. In Filter array, choose **Edit in advanced mode** and paste. It ignores capital letters and stray spaces in Status, and compares dates from the start of today.

```
@and(
not(equals(toLower(trim(item()?['Status'])), 'completed')),
greaterOrEquals(ticks(item()?['SessionDate']), ticks(startOfDay(utcNow()))),
lessOrEquals(ticks(item()?['SessionDate']), ticks(addDays(startOfDay(utcNow()), 14)))
)
```

08

CHAPTER 8

My Morning Briefing: Variables and Parallel Branches

Every morning you check the weather, your calendar and your inbox before the day starts. This flow does all three at once and sends you one email. It only reads your own data and only emails you, so it's safe to build in a live environment.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 70 minutes

Your result: A scheduled flow that runs on weekday mornings, looks up the weather, your meetings and your important unread email in three parallel branches, and sends you one briefing email.

Files for this lab: None. This flow uses your own calendar, inbox and the weather.

What You Will Learn

- Create variables with **Initialize variable** and explain why they live at the top of a flow
- Change variables with **Increment variable**, **Append to string variable** and **Set variable**
- Run independent steps side by side with **parallel branches**
- Bring parallel branches back together before a final step
- Use a variable's value in a condition and an email

How the Flow Works

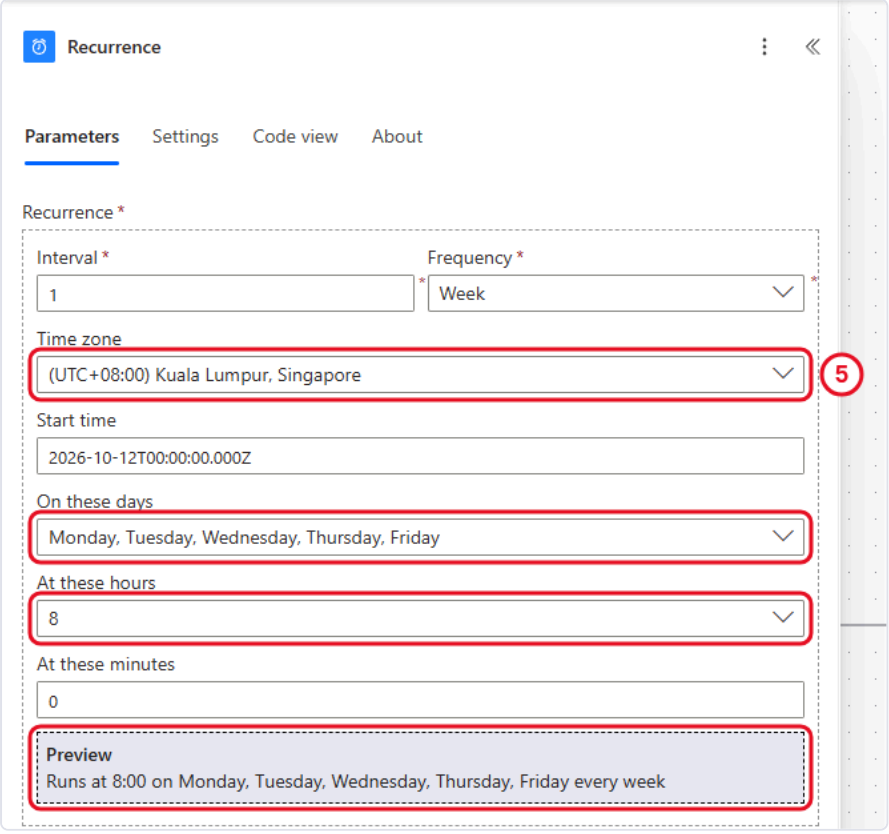
```
Recurrence
  > Initialize four variables
  > Three parallel branches:
      Weather:  Get current weather
      Meetings: Current time > Get future time > Get calendar view > loop (count + list)
      Email:    Get emails > loop (list)
  > Condition: more than 4 meetings? > Set the subject
  > Send me the briefing
```

The three lookups don't depend on each other. The weather doesn't need your calendar and your calendar doesn't need your inbox. When steps are independent, they can run **in parallel**, side by side, instead of waiting in a queue. The flow only moves on to the email once all three branches have finished.

> **Note:** Your trainer has confirmed that MSN Weather, Office 365 Outlook and the Date Time actions are allowed in your environment. Nothing in this flow changes your calendar or mailbox.

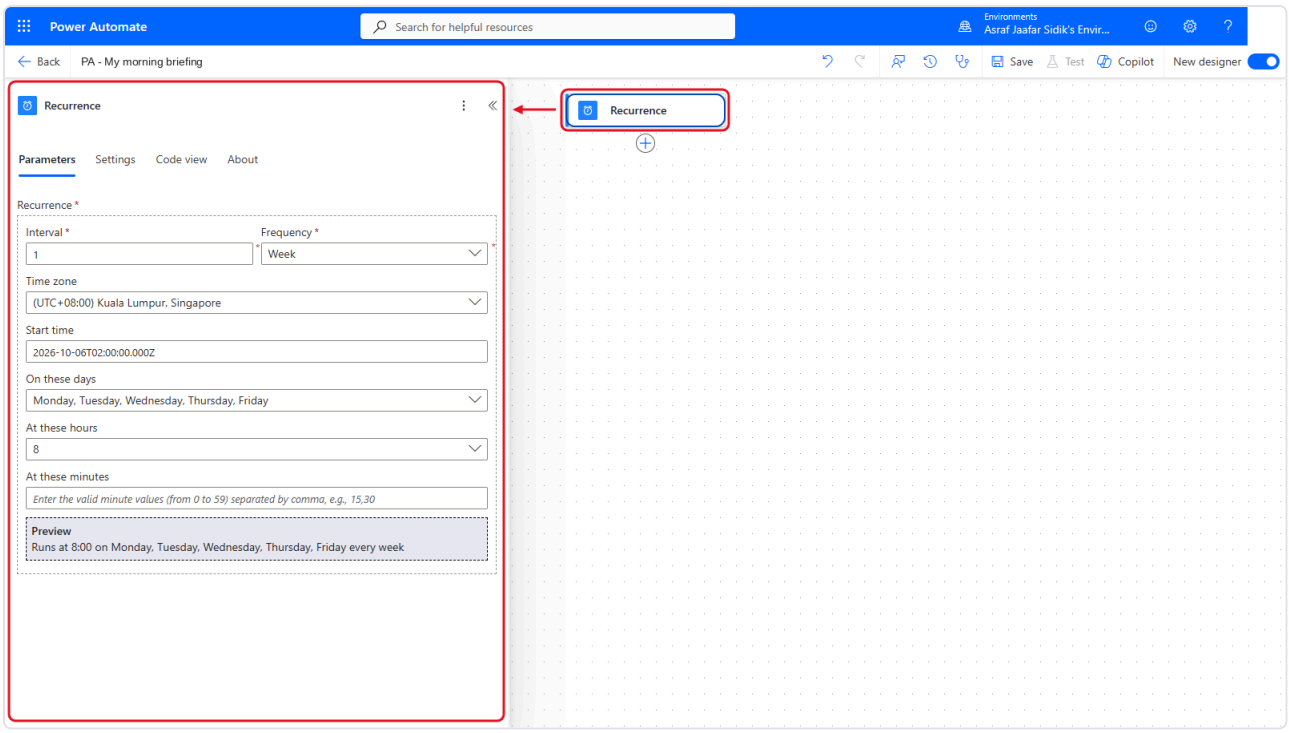
8.1 Create the Scheduled Flow

1. In the left navigation, select **Create**, then **Scheduled cloud flow**.
2. In **Flow name**, enter `PA - My morning briefing`.
3. Set **Repeat every** to **1 Week**. Make sure only **M T W T F** are selected: if **S** and **S** (the weekend) are already highlighted, select them to turn them off. Set the time to **8:00 AM**.
4. Select **Create**. The designer opens with a **Recurrence** card on the canvas.
5. Select the **Recurrence** card on the canvas to open its panel. If **Time zone** is blank, open the dropdown, type `Kuala` and pick **(UTC+08:00) Kuala Lumpur, Singapore**. If **At these hours** is blank, open it and select `8`. Leave **At these minutes** as it is (blank or `0` both work). Check the **Preview** at the bottom reads 8:00 on Monday to Friday. **Start time** is filled in for you; leave it.



Weekday mornings at 8:00, Malaysia time.

Check your flow so far. Your screen should look like this.



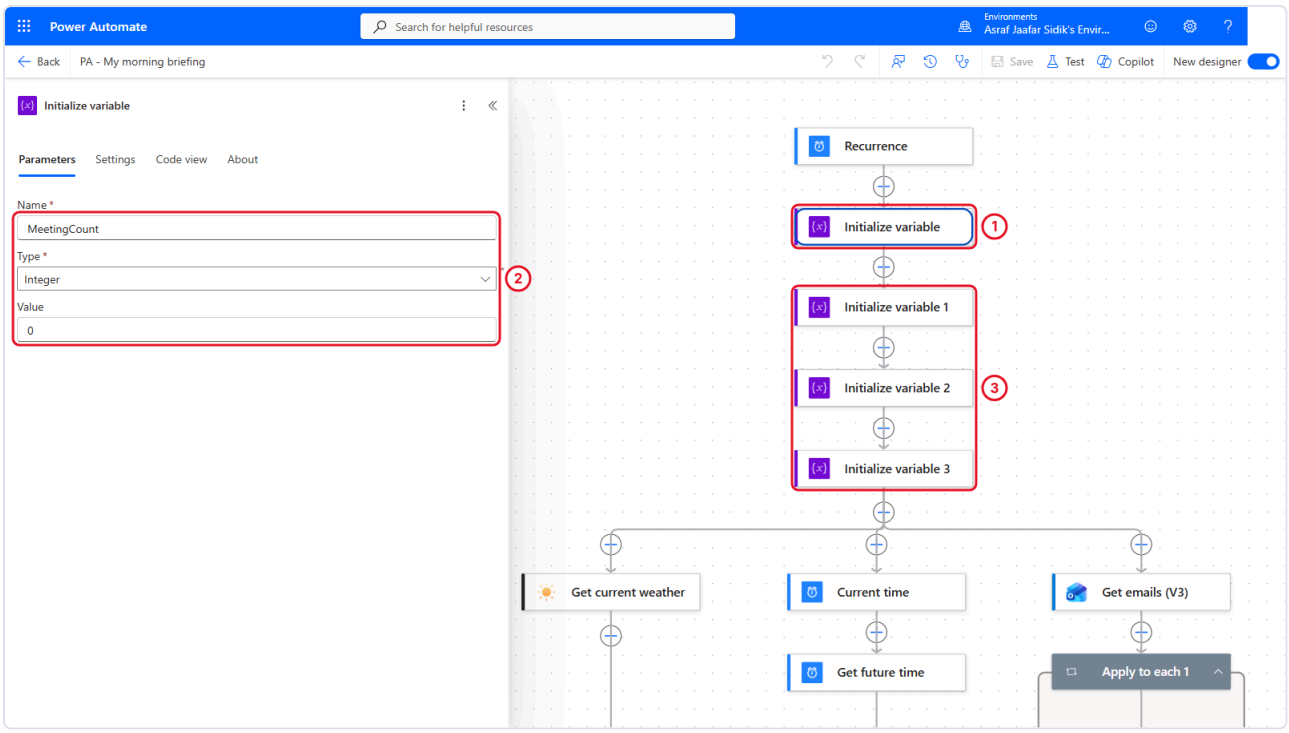
Only Recurrence so far.

8.2 Create the Variables

A **variable** is a named box that holds a value and can change while the flow runs. You've used one already: ReminderCount in Chapter 7. This flow uses four, of two types.

Name	Type	Starting value	What it's for
`MeetingCount`	Integer	`0`	Counts today's meetings
`MeetingList`	String	*(leave empty)*	Collects the meeting names
`EmailList`	String	*(leave empty)*	Collects the important email subjects
`BriefingSubject`	String	`Your day ahead`	The email subject. May change later

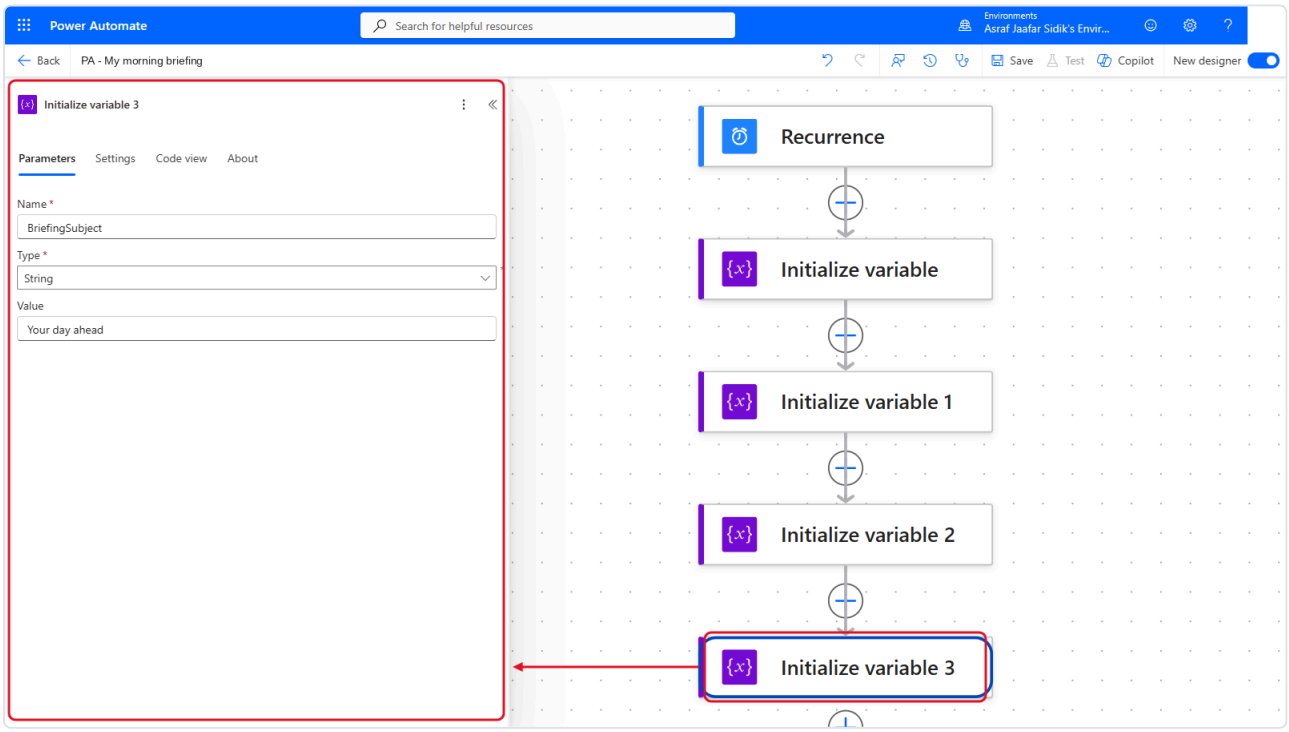
1. On the canvas, select the **+** below **Recurrence**, then **Add an action**. Search for `Initialize variable` and select it under **Variables**.
2. In **Name**, enter `MeetingCount`. Open the **Type** dropdown and select **Integer**. In **Value**, enter `0`.
3. Select the **+** below the variable you just made, then **Add an action** and search for `Initialize variable` again. Repeat until all four rows of the table are done. Leave **Value** empty for **MeetingList** and **EmailList**. For **BriefingSubject**, type `Your day ahead`.



All four variables are created before anything else happens. The canvas in the picture already shows the branches you add later; at this point you only have Recurrence and the four variables (see the checkpoint below).

> **Key point:** Initialize variable only works at the top level of a flow, not inside a loop, condition or branch. So every variable is created first, even ones you won't use until later.

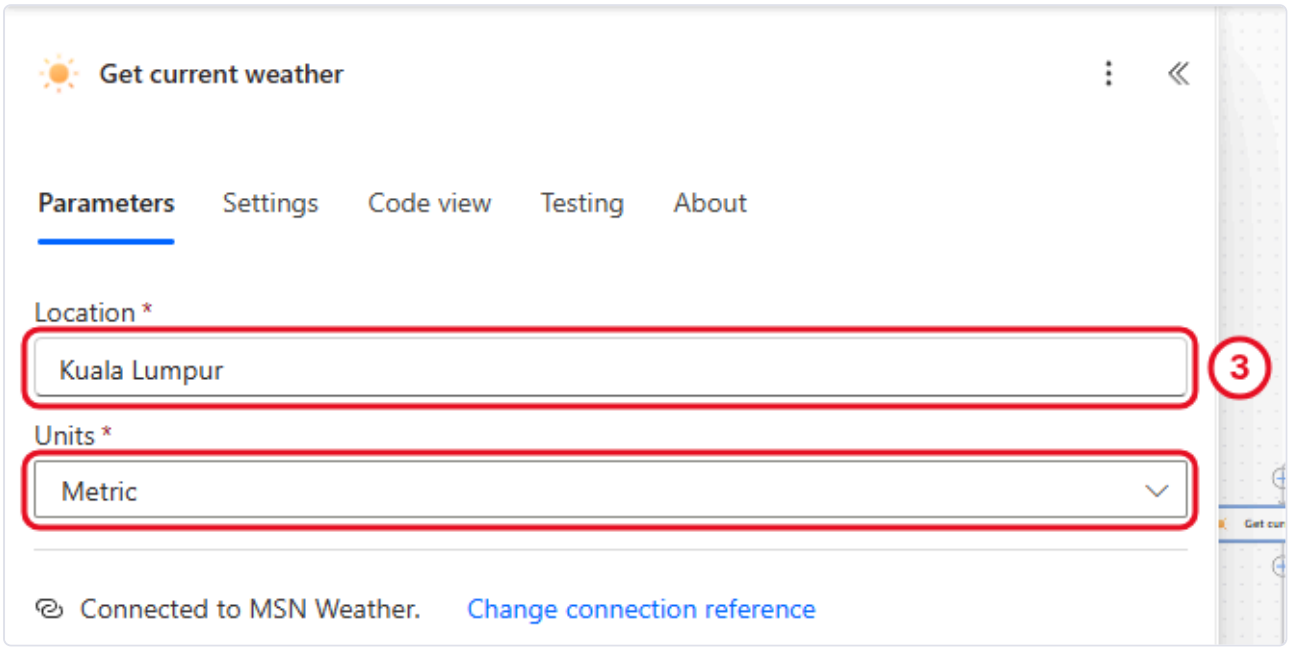
Check your flow so far. Your screen should look like this.



Recurrence and four Initialize variable actions.

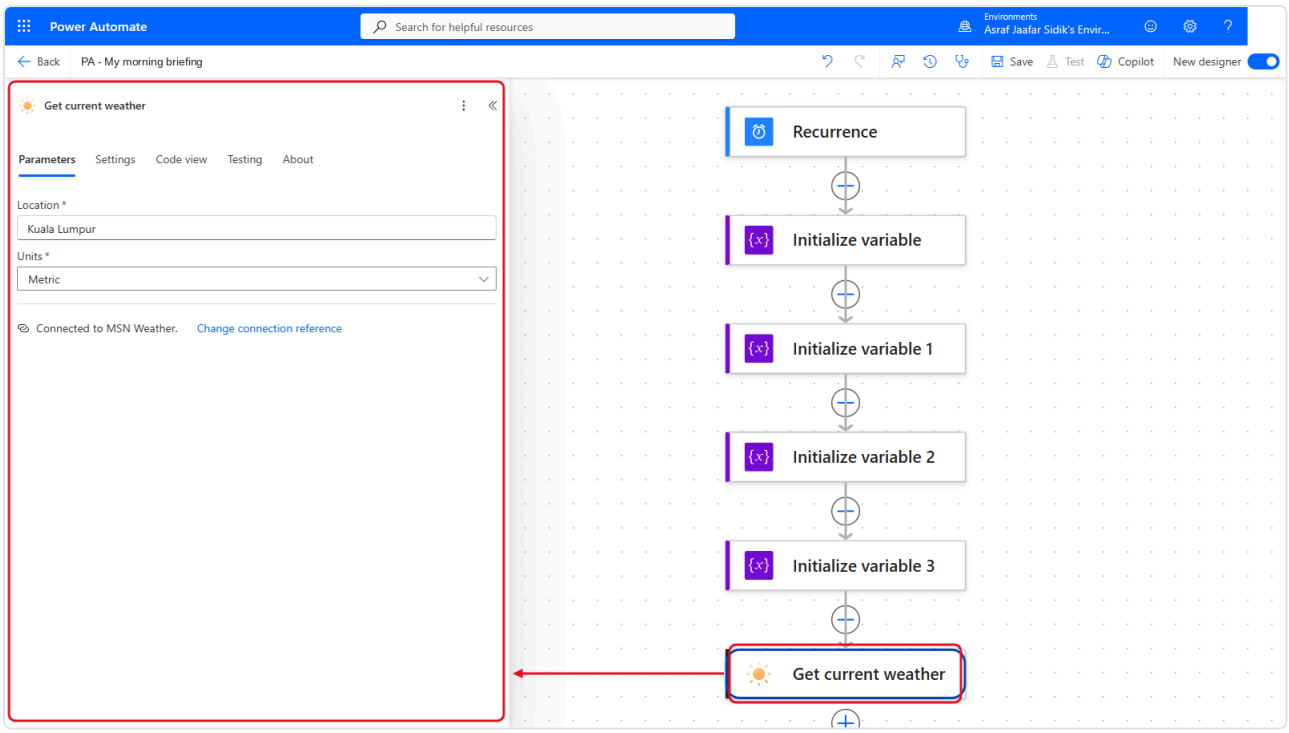
8.3 Branch 1: The Weather

1. On the canvas, select the **+** below the last **Initialize variable** (named **Initialize variable 3**), then **Add an action**.
2. Search for `Get current weather`. Under the **MSN Weather** heading, select **Get current weather**.
3. In **Location**, type `Kuala Lumpur`. Open the **Units** dropdown and select **Metric**.



One action. Its values (conditions, temperature) are used in the email later.

Check your flow so far. Your screen should look like this.

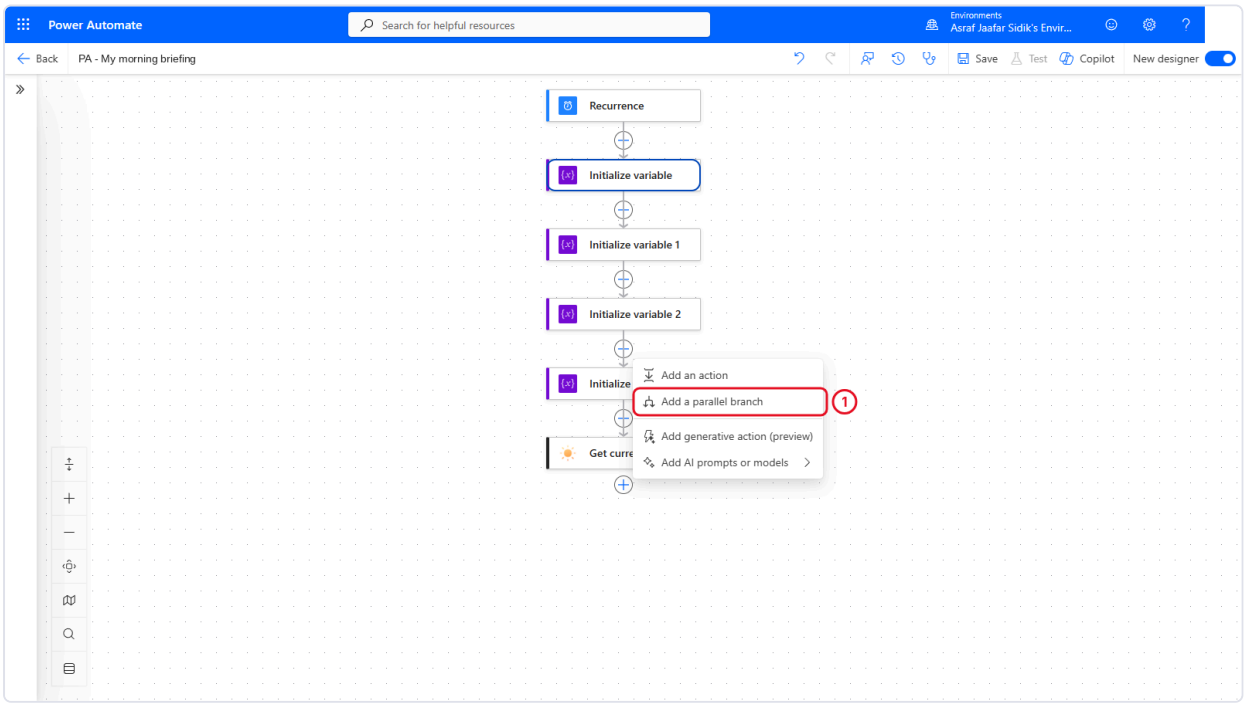


Get current weather under the last variable.

8.4 Branch 2: Your Meetings

This branch starts *beside* the weather, not below it.

1. Select the **+** between the last **Initialize variable** and **Get current weather** (right-click it if a menu doesn't open), then choose **Add a parallel branch**.



Add a parallel branch starts a new column next to the existing one.

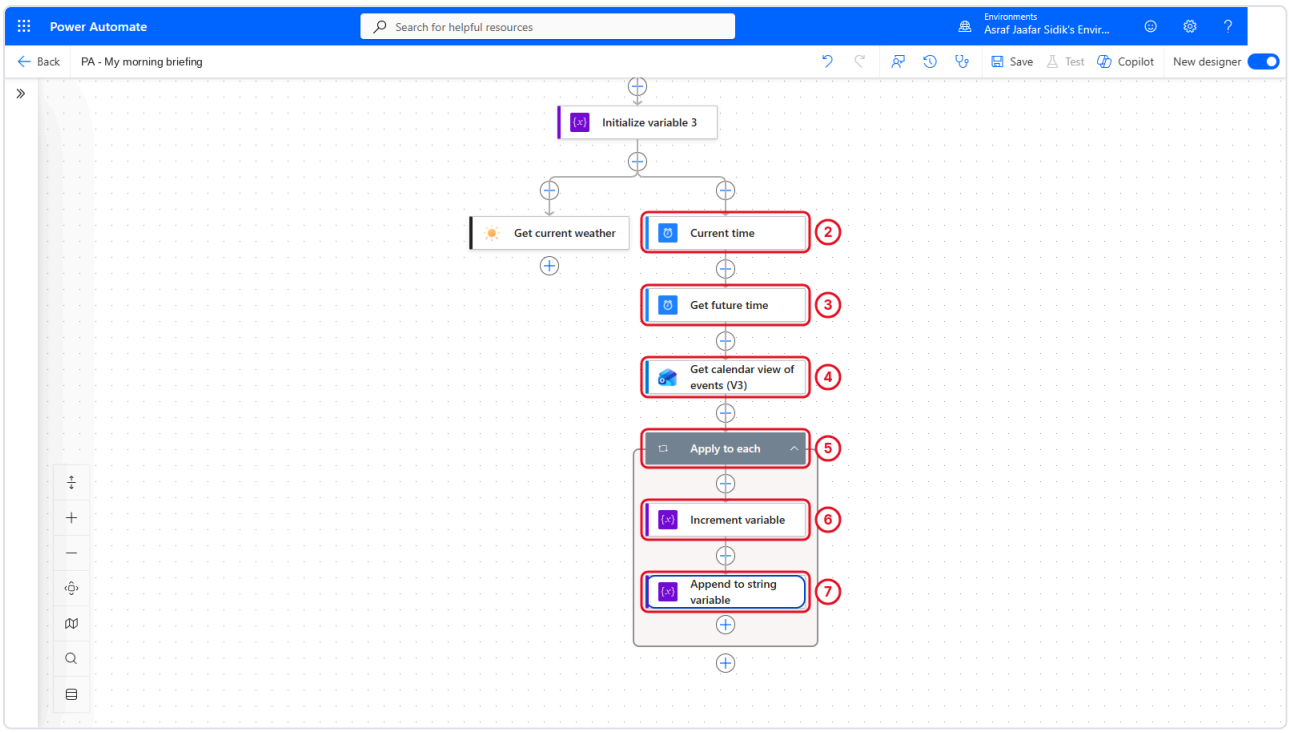
2. The action search opens for the new branch. Search for `Current time` and select it under **Date Time**.
3. On the canvas, select the **+** below **Current time**, then **Add an action**. Search for `Get future time` and select it under **Date Time**. In **Interval**, enter `12`. Open **Time unit** and select **Hour**.
4. Select the **+** below **Get future time**, then **Add an action**. Search for `Get calendar view` and select **Get calendar view of events (V3)** under **Office 365 Outlook**. For **Start Time** and **End Time**, click into the box, select the lightning bolt icon beside it (or type `/` and choose **Insert dynamic content**), and pick the value:

Field	Value
Calendar Id	Open the dropdown and select **Calendar**
Start Time	**Current time** under *Current time*
End Time	**Future time** under *Get future time*

5. Select the **+** below **Get calendar view of events (V3)**, then **Add an action**. Search for `Apply to each` and select it under **Control**. Click into **Select an output from previous steps**, open the lightning bolt, and select **body/value** under *Get calendar view of events (V3)*. That's the list of events. (It may be labelled **value**. If you can't see it, select **See more** or type `value` in the picker's search box.)
6. Inside the loop, select the **+**, then **Add an action**. Search for `Increment variable` and select it under **Variables**. Open the **Name** dropdown and select **MeetingCount**. In **Value**, enter `1`.
7. Inside the same loop, select the **+** below **Increment variable** (if it's out of view, close the panel with **<<** and use **Zoom view to fit** at the bottom left of the canvas), then **Add an action**. Search for `Append to string variable` and select it under **Variables**. Open the **Name** dropdown and select **MeetingList**. In **Value**, paste the text below (it ends with a space). Copy it from the box below.

   ```
   <br>•
   ```

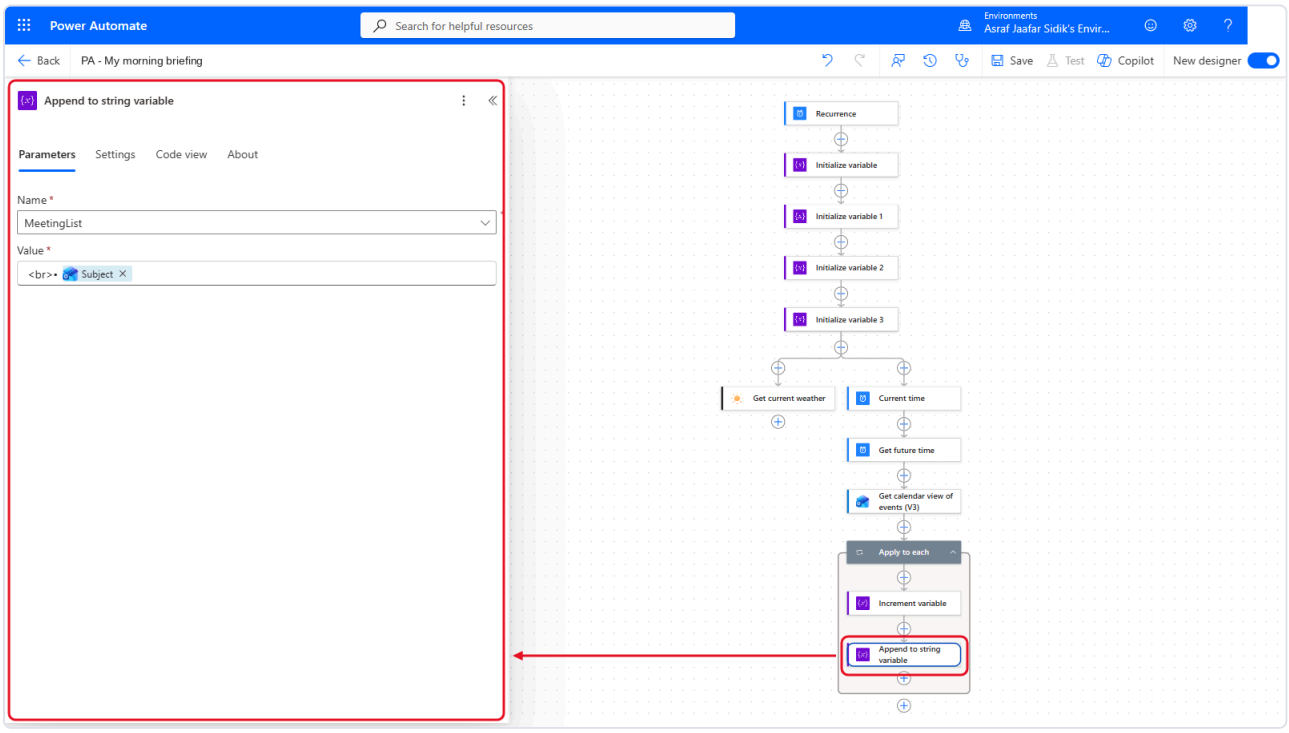
   Then open the lightning bolt and select **Subject** under *Get calendar view of events (V3)*. If Subject isn't listed, select **See more** under *Get calendar view of events (V3)*.



Each meeting adds one to the count and one line to the list.

> **Tip:** Email bodies are written in HTML, the language of web pages. `<br>` is HTML for "new line", so each meeting starts on its own line in the email.

Check your flow so far. Your screen should look like this.



The meetings branch runs beside the weather.

8.5 Branch 3: Important Unread Email

1. Add a third parallel branch the same way: select the **+** on the arrow below the last **Initialize variable**, then **Add a parallel branch**.
2. Search for `Get emails` and select **Get emails (V3)** under **Office 365 Outlook**. Set the fields below. If a field isn't shown, open the **Advanced parameters** dropdown and tick it.

Field	Value
Folder	Inbox
Fetch Only Unread Messages	Yes
Importance	High
Top	5

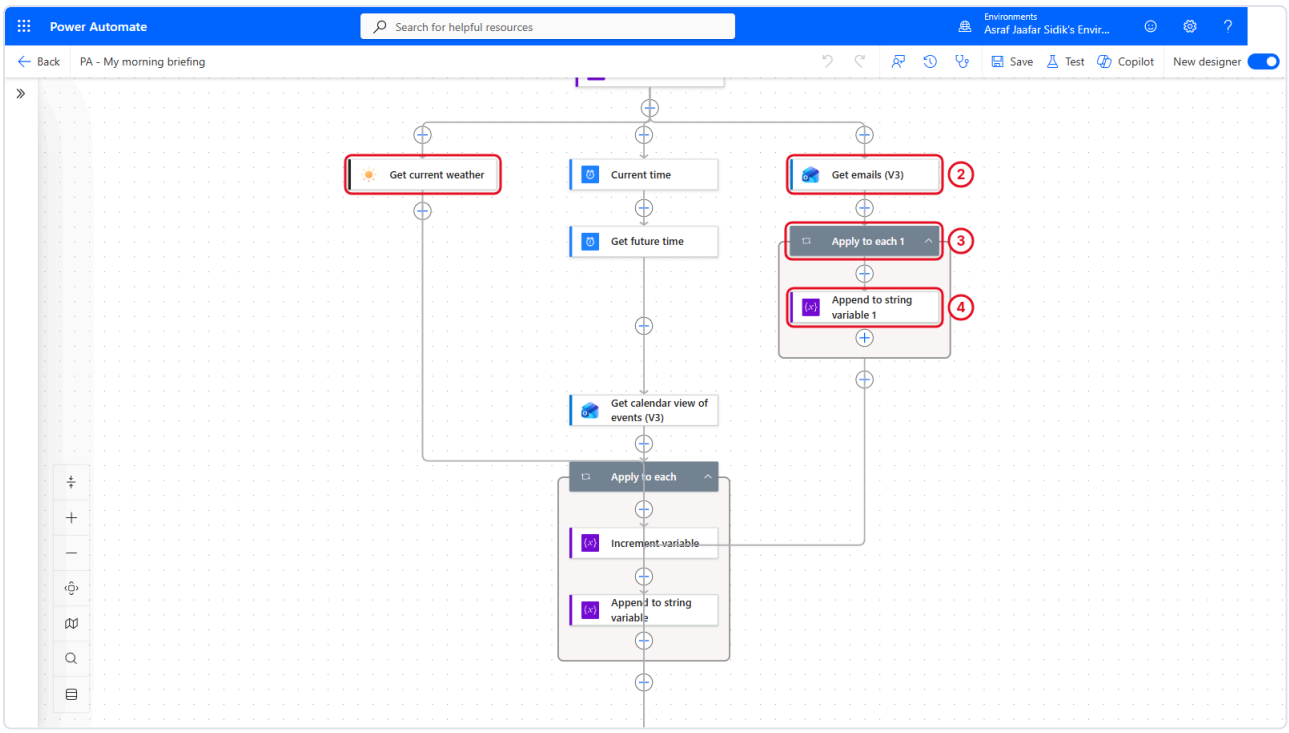
3. On the canvas, select the **+** below **Get emails (V3)**, then **Add an action**. Search for `Apply to each` and select it under **Control**. Click into **Select an output from previous steps**, open the lightning bolt, and select **body/value** under *Get emails (V3)*.
4. Inside the loop, select the **+**, then **Add an action**. Search for `Append to string variable` and select it under **Variables**. Open the **Name** dropdown and select **EmailList**. In **Value**, paste the start of the line (it ends with a space). Copy it from the box below.

   ```
   <br>•
   ```

   Insert **Subject**, then paste the text below (it starts and ends with a space).

   ```
    (from
   ```

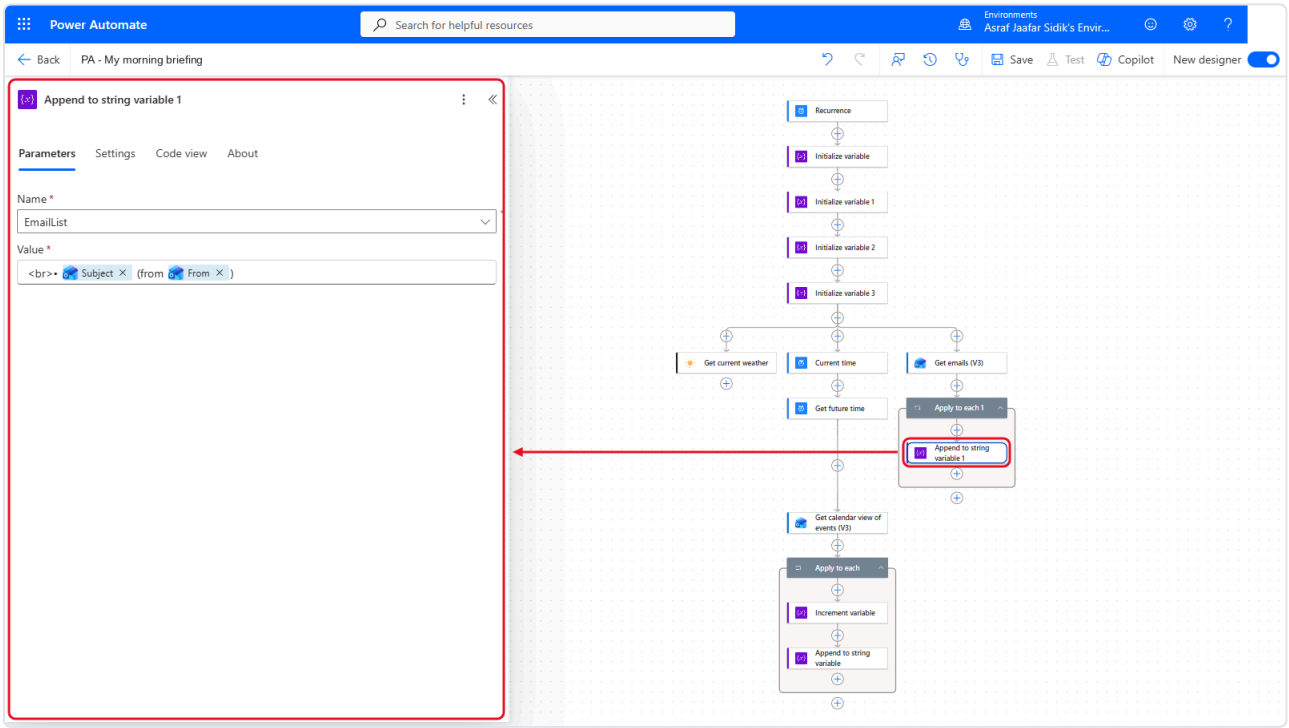
   Insert **From**, then type `)`. Subject and From are under *Get emails (V3)* in the picker (select **See more** if they're hidden).



Three independent branches, running at the same time. The lines curving into the meetings loop are explained in section 8.6.

> **Key point:** Each branch writes to its **own** variable. If all three appended to one shared variable, the sections would arrive in whatever order the branches happened to finish. Separate variables keep the email in a fixed order.

Check your flow so far. Your screen should look like this.

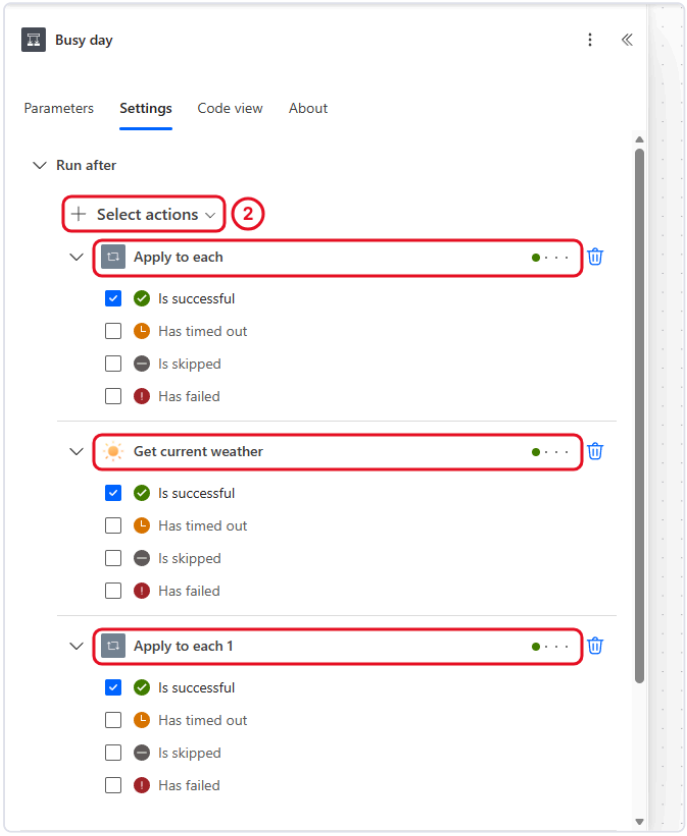


Three branches side by side.

8.6 Bring the Branches Back Together

The next step must wait until all three branches have finished.

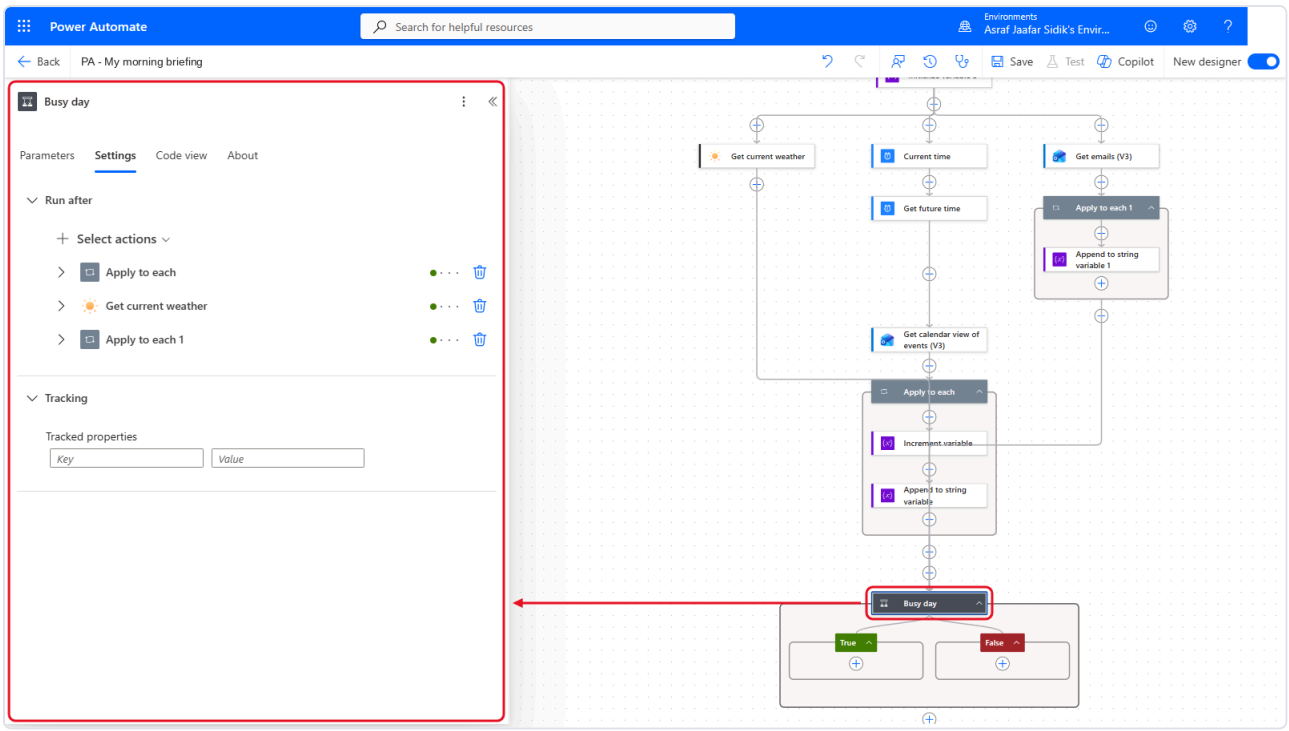
1. On the canvas, select the **+** below the meetings **Apply to each** box (outside it), then **Add an action**. Search for `Condition` and select it under **Control**. Rename it: select the title at the top of the panel and type `Busy day`.
2. Select the **Busy day** card to open its panel, then the **Settings** tab at the top of the panel. **Apply to each** is already listed under **Run after**. Open the **Select actions** dropdown and tick **Get current weather** and **Apply to each 1**.
3. Leave **Is successful** ticked for each. Check the **Run after** list shows all three actions, then select **Save**. (After saving, the panel may go blank: select the **Busy day** card again to reopen it.)



Run after lists one action from each branch, so Busy day waits for the slowest branch.

> **Note:** The designer draws the lines from the weather and email branches into the side of the meetings loop, not into Busy day. That's just how it lays out the picture. The **Run after** list is what actually controls the order, so trust that, not the lines.

Check your flow so far. Your screen should look like this.



Busy day sits below, after all three branches.

8.7 Change the Subject on a Busy Day

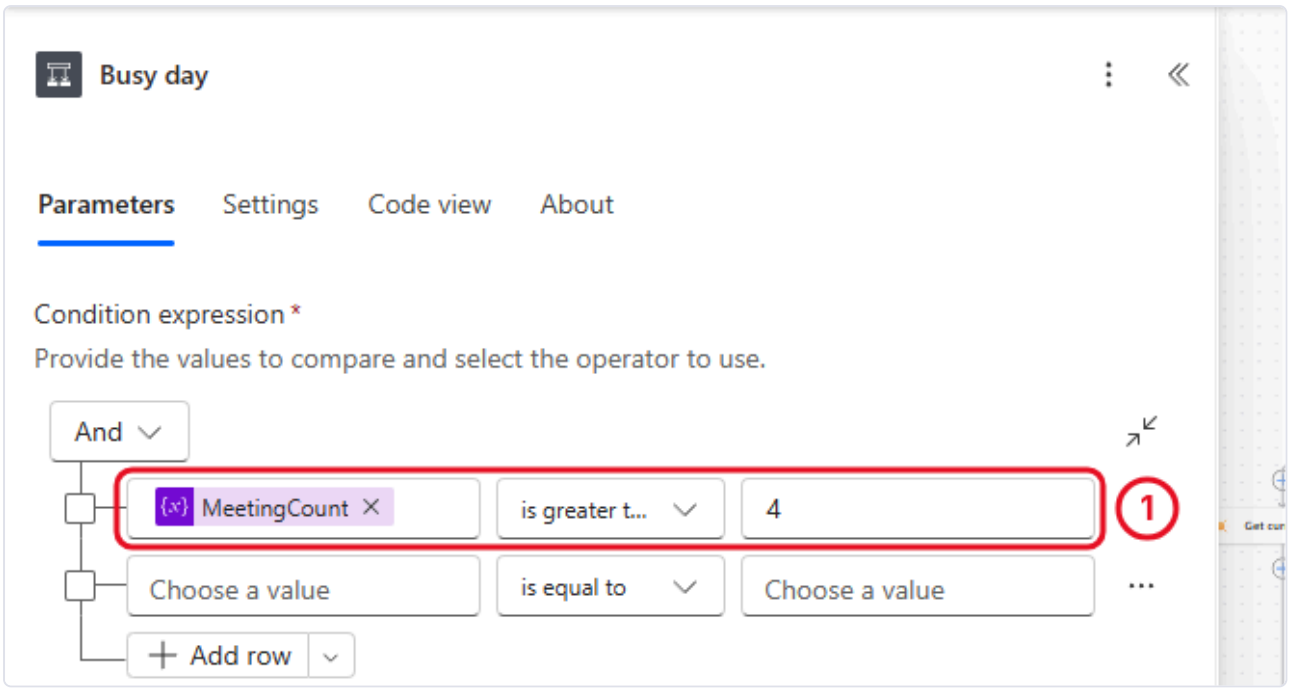
1. In the **Busy day** panel, select the **Parameters** tab. Click the left **Choose a value** box, select the lightning bolt (or type `/` and choose **Insert dynamic content**), and pick **MeetingCount** under *Variables*. Open the middle dropdown and select **is greater than**. Click the right box and type `4`. If an empty extra row appears, select its **...** and then **Delete**. The designer may show an empty placeholder row again straight away: that's fine, only the filled row is saved.

2. On the canvas, inside the green **True** box, select the **+**, then **Add an action**. Search for `Set variable` and select it under **Variables**. Open the **Name** dropdown and select **BriefingSubject**. In **Value**, paste the text below (it ends with a space). Copy it from the box below.

   ```
   Busy day ahead:
   ```

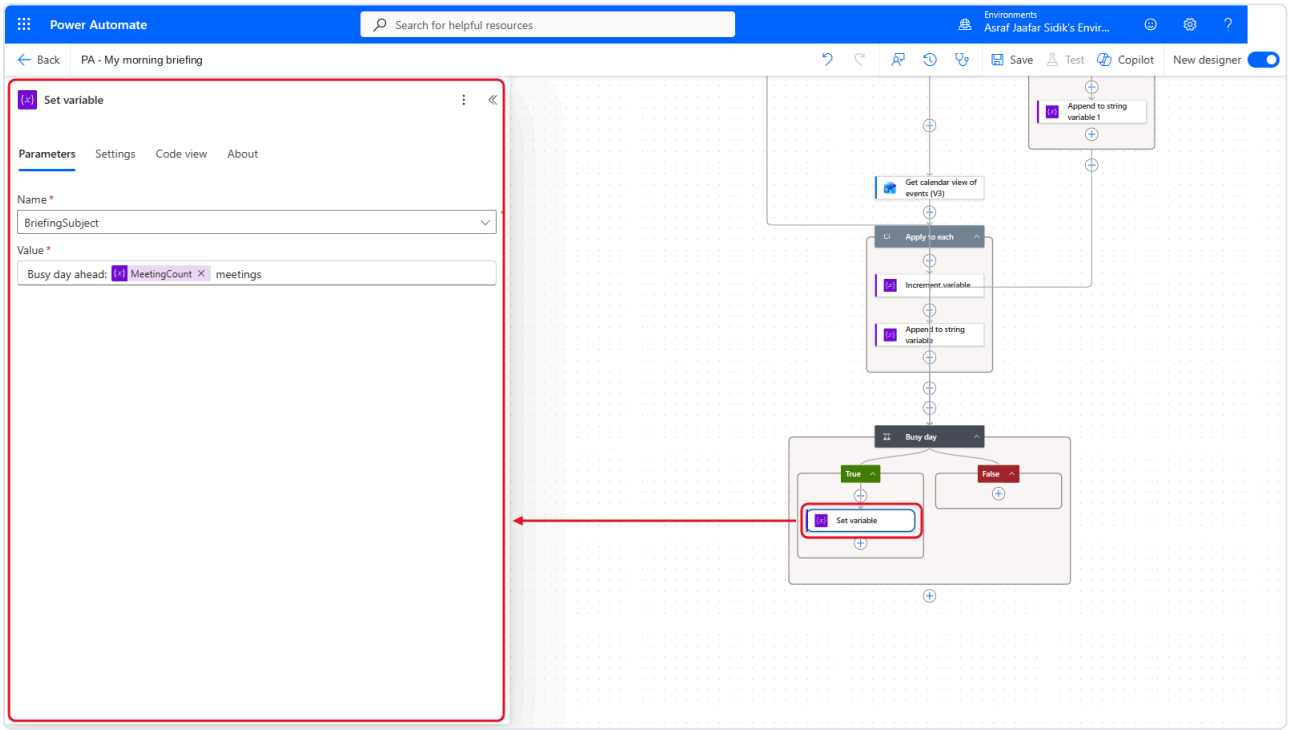
   Insert **MeetingCount** (under *Variables*), then type `meetings` (with the space at the start).

3. Leave **False** empty. The subject keeps its starting value, *Your day ahead*.



Set variable replaces a variable's value. Append adds to it. Increment adds to a number. The screenshot still shows the empty row; delete it on yours.

Check your flow so far. Your screen should look like this.



Set variable sits in True.

8.8 Send the Briefing

1. On the canvas, select the **+** below the **Busy day** box (outside it), then **Add an action**. Search for `Send an email`, then select **Send an email (V2)** under **Office 365 Outlook**.

2. **To:** start typing your name or email address, then select yourself from the list that appears. If no suggestion appears, type your full email address and press **Enter**.
3. **Subject:** click into the box, select the lightning bolt, and pick **BriefingSubject** under *Variables*.
4. **Body:** type the text below and insert the tokens shown in brackets. To insert a token, place the cursor where it goes, then select the lightning bolt (or type `/` and choose **Insert dynamic content**). **Conditions** and **Temperature** are under *Get current weather* (select **See more** if they're hidden). The rest are under *Variables*.

> Good morning,
>
> Weather in Kuala Lumpur: **[Conditions]**, **[Temperature]**°C
>
> Meetings in the next 12 hours (**[MeetingCount]**):**[MeetingList]**
>
> Important unread email:**[EmailList]**

The plain text pieces are in the boxes below, in order. Paste each one, then insert the token that follows it. Press **Enter** twice between lines to leave a blank line.

```
Good morning,
```

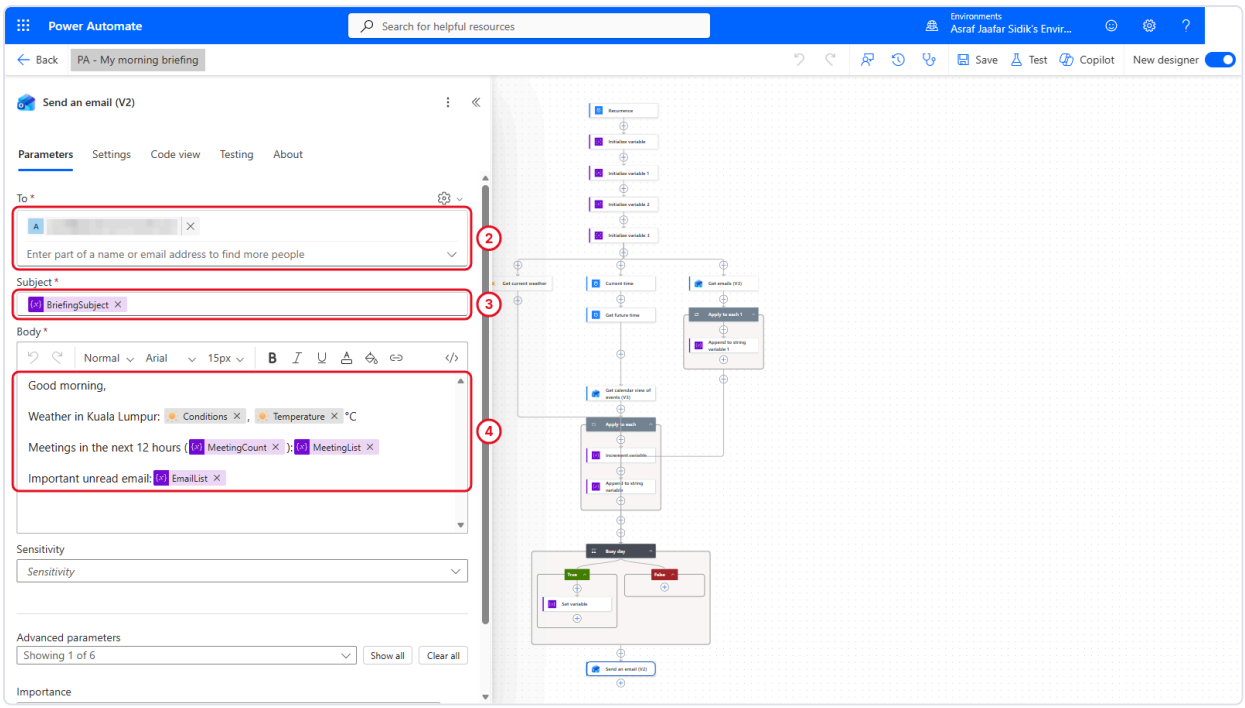
```
Weather in Kuala Lumpur:
```

After **[Conditions]**, type a comma and a space, insert **[Temperature]**, then type `°C`.

```
Meetings in the next 12 hours (
```

After **[MeetingCount]**, type `):` and then insert **[MeetingList]**.

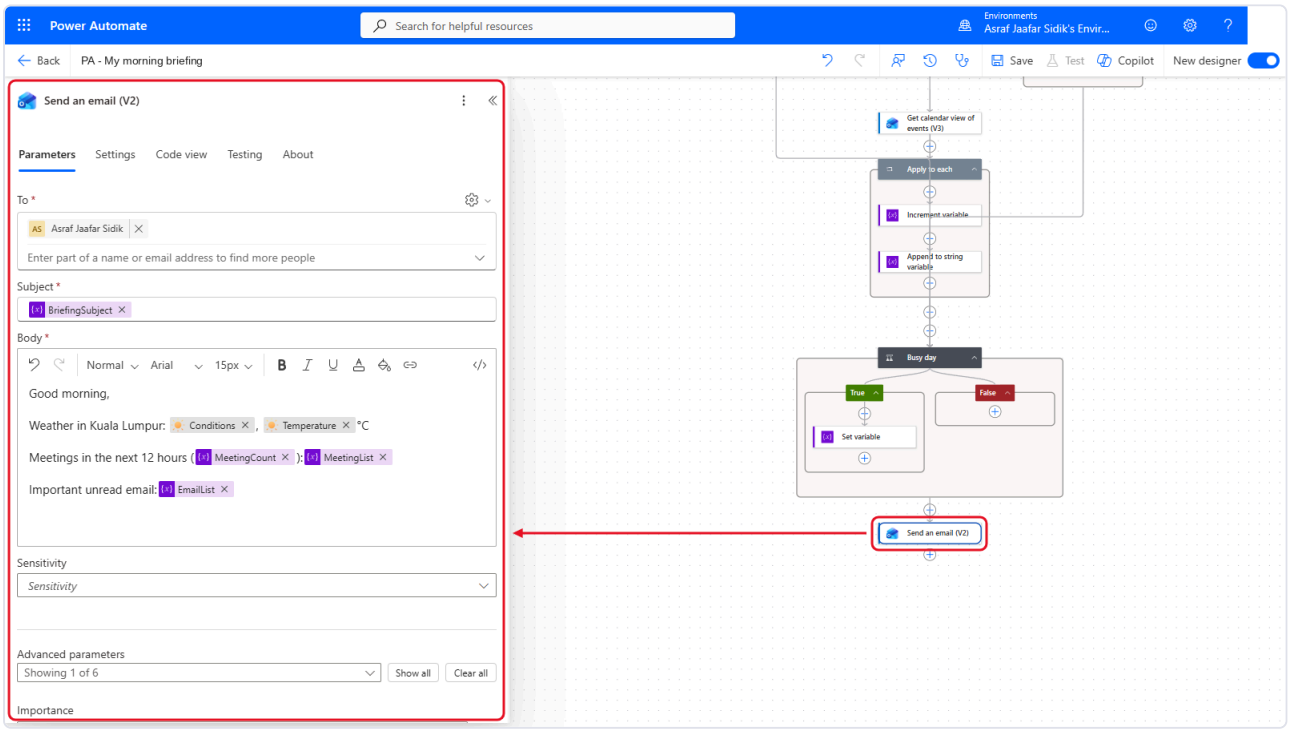
```
Important unread email:
```



Variables drop into the email like any other dynamic content.

5. Select **Save** in the toolbar at the top right, then the **Flow checker** icon (the stethoscope just left of **Save**; hover to see its name). Fix anything it reports.

Check your flow so far. Your screen should look like this.

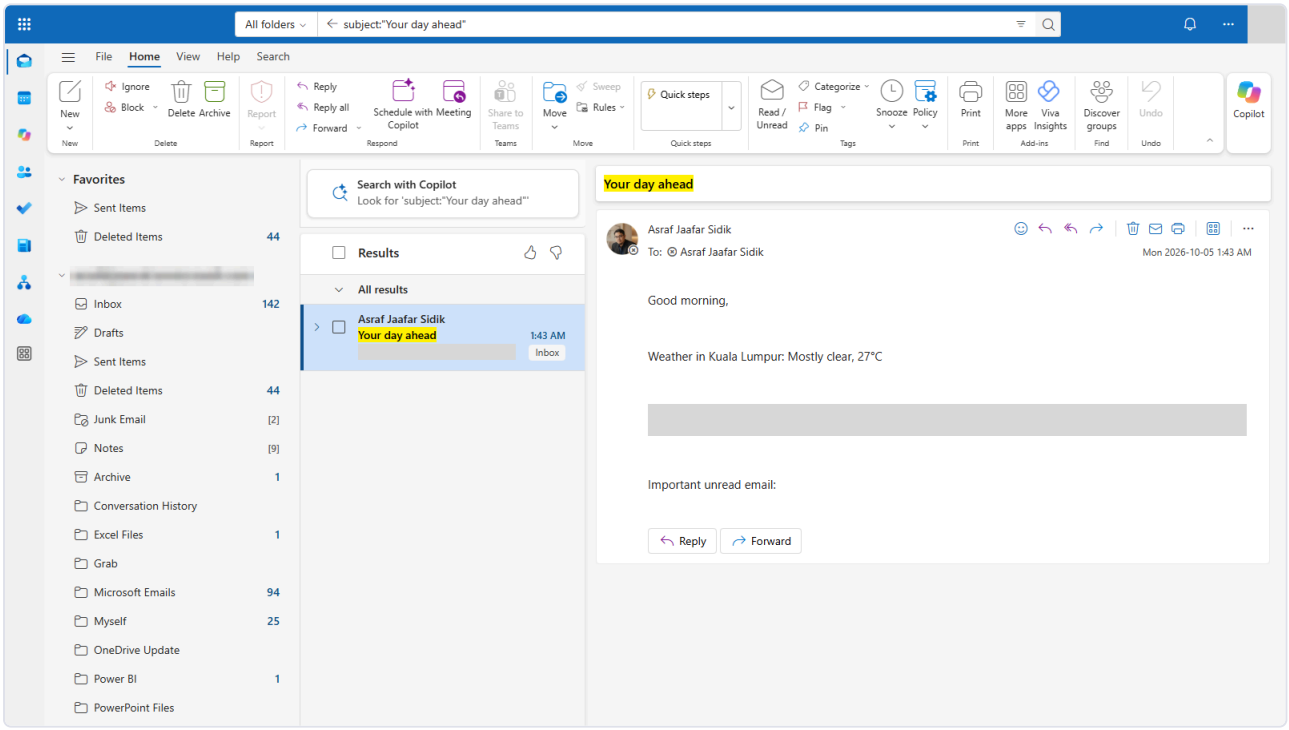


Send an email (V2) at the bottom. That is the whole briefing.

8.9 Test It

This flow only emails you, so you can test it yourself.

1. In the toolbar at the top right, select **Test**, choose **Manually**, select **Test**, then **Run flow**, then **Done**.
2. The run opens on the canvas. Notice that the three branches started at almost the same moment.
3. Select **Busy day** and open **Run results** to see whether it was true or false. Select **Increment variable** inside the meetings loop to see the `1` it added each time (it shows no outputs, which is normal). If Busy day was false, **Set variable** shows as skipped. That's expected: the email's count is the final check.
4. In Outlook, find the email with the subject **Your day ahead** (or **Busy day ahead...**) in your Inbox. If it isn't near the top, type the subject in the search box at the top. If you've tested before, Outlook may group the briefings into one conversation: open it and check the newest message's time.



Your briefing. Each section came from a different branch.

If you have no meetings or no important unread email, those lists are simply empty and the count reads 0. That's correct.

Independent Practice

Add a fourth parallel branch that counts up to 25 unread emails of any importance. Create an Integer variable `UnreadCount` at the top, use **Get emails (V3)** with **Fetch Only Unread Messages** set to Yes and **Top** set to 25, and increment the variable for each email. Add a line to the briefing: *Unread email: [UnreadCount]*.

> **Going further: skip empty sections.** Use a condition with **EmailList** *is equal to* (leave the right side empty) to set EmailList to `<br>Nothing urgent. Enjoy your morning.` when there's no important email. Optional.

Troubleshooting

Symptom	What to check
Initialize variable isn't offered	You're inside a branch or loop. Add it below Recurrence
Get current weather is blocked	Your environment may block MSN Weather. Ask the trainer, or skip branch 1
The meeting list is empty but you have meetings	Start Time is **Current time**, End Time is **Future time**, Calendar Id is **Calendar**
Meetings and email appear on one line	The ` ` is missing at the start of the Append value
The email sends before a branch finishes	Busy day's **Run after** must include the last action of every branch
The subject never changes	Set variable is in the True branch, and the condition uses MeetingCount *is greater than* 4
Times look eight hours off	The meeting window uses UTC from Current time. For a briefing, a 12-hour window still covers your day

Lesson Summary

Variables give a flow named boxes that change as it runs. You created four at the top, then used **Increment** to count, **Append** to build lists and **Set** to replace a value. Parallel branches let independent lookups run side by side, each writing to its own variable, and a join makes the final email wait for all of them. Next, you'll ask Copilot to plan a flow for you, and review what it suggests.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Everything on this page is **text**, typed or pasted straight into a field.

Flow name (text, section 8.1)

```
PA - My morning briefing
```

Variable names (text, section 8.2)

```
MeetingCount
```

```
MeetingList
```

```
EmailList
```

```
BriefingSubject
```

Starting value for BriefingSubject:

```
Your day ahead
```

Weather location (text, section 8.3)

```
Kuala Lumpur
```

Start of each list line (text, sections 8.4 and 8.5)

Paste, then insert **Subject** after it. For 8.5, then type `(from`, insert **From**, and type `)`.

```
<br>•
```

For 8.5 only, after **Subject**:

```
 (from
```

Then insert **From** and finish with:

```
)
```

Condition name (text, section 8.6)

```
Busy day
```

Busy day subject (text, section 8.7)

Paste, insert **MeetingCount** (lightning bolt, under *Variables*), then type `meetings`.

```
Busy day ahead:
```

Briefing email body (text, section 8.8)

Paste, then replace each `[Name]` with the matching dynamic content token: delete the `[Name]` text, leave the cursor there, select the lightning bolt (or type `/` and choose **Insert dynamic content**) and pick the value. Conditions and Temperature are under *Get current weather*. The rest are under *Variables*.

```
Good morning,

Weather in Kuala Lumpur: [Conditions], [Temperature]°C

Meetings in the next 12 hours ([MeetingCount]):[MeetingList]

Important unread email:[EmailList]
```

09

CHAPTER 9

Plan an Automation with Copilot

Copilot in Power Automate can turn a plain-language description into a suggested flow. A suggestion isn't a finished design, though. In this chapter you review what Copilot proposes, correct it precisely, and walk away without creating anything.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 30 minutes

Your result: A reviewed and improved flow plan produced with Copilot, without creating or running a cloud flow.

Files for this lab: None. Everything you need is in the prompts below.

What You Will Learn

- Describe an automation to Copilot with a clear, bounded prompt
- Review an AI-generated flow plan against the business rule
- Refine a suggestion by naming the exact change you need
- Leave the planning experience without creating a flow
- Plan your own automation with Copilot before you build it

The Business Problem

Copilot may leave out a calculation, add an action you don't need, or read a broad instruction differently from what you meant. You ask it to plan a weekly coordinator summary, check every proposed step, and correct it until it matches this intended plan:

Recurrence > List rows present in a table > Filter array > Compose count > Send one coordinator email

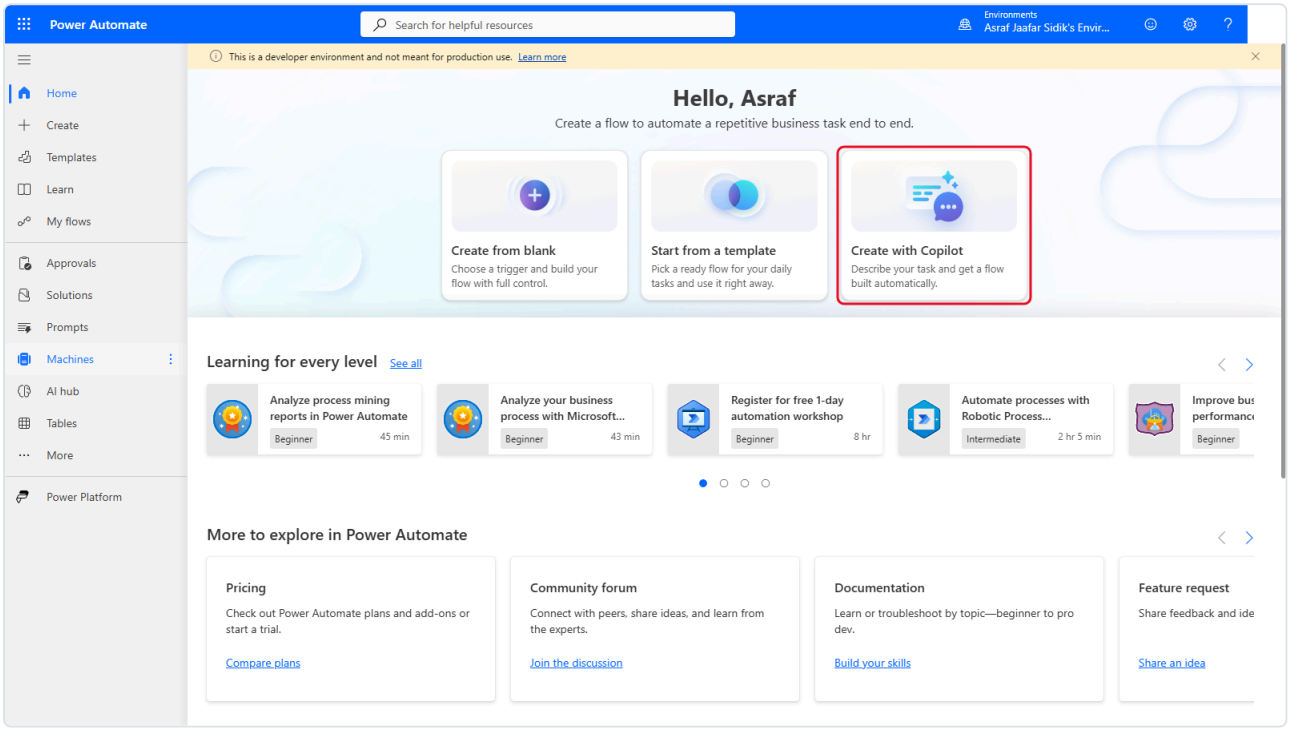
Before You Begin

Copilot availability depends on your organisation, environment, region and admin settings. If **Create with Copilot** or **Describe it to design it** isn't available, follow along with the screenshots in this chapter and answer the review questions without entering a prompt.

> **Important:** Use only fictional details and `coordinator@example.com`. Never put confidential information, personal data, passwords or real recipients into a training prompt.

9.1 Open the Copilot Planning Experience

On the Power Automate home page, select **Create with Copilot**. The Create page may call the same thing **Describe it to design it**.

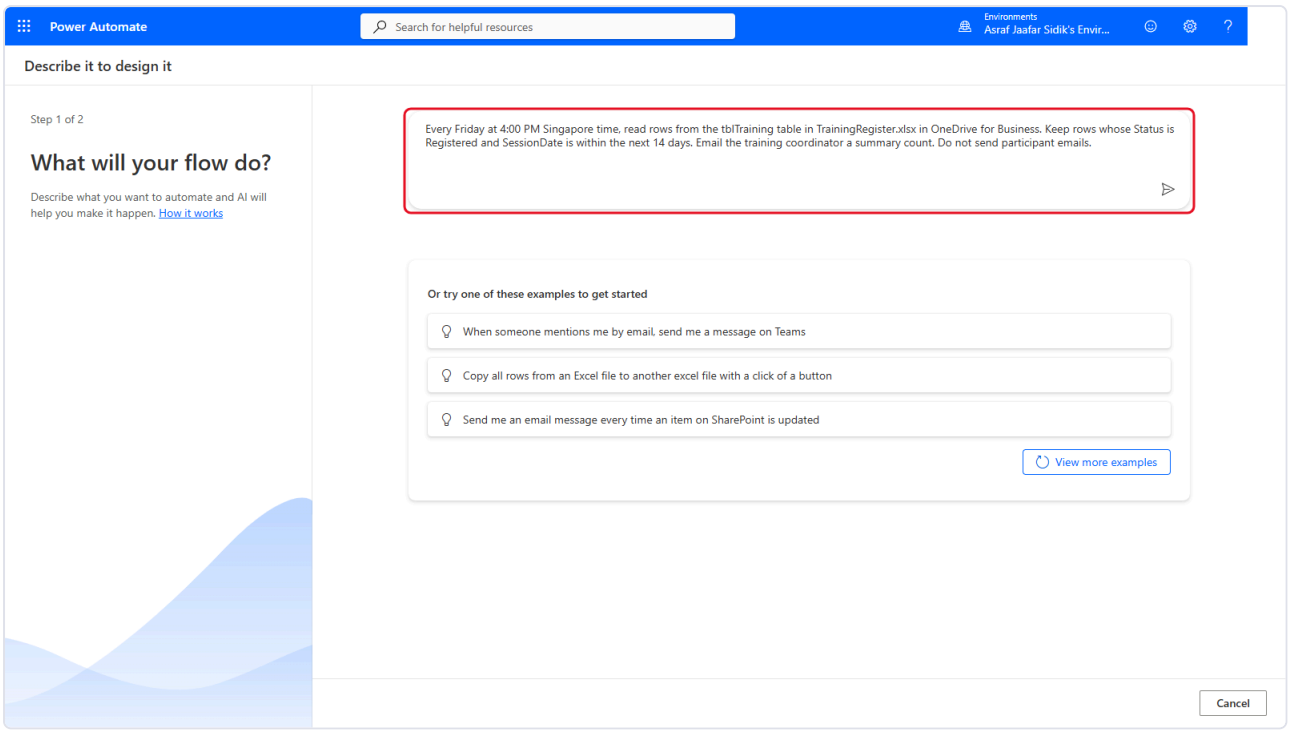


Create with Copilot starts from a plain-language description.

9.2 Describe the Complete Business Outcome

In **What will your flow do?**, paste prompt 1 and select the arrow icon at the bottom right of the box to send it. Copy it from the box below.

```
Every Friday at 4:00 PM Singapore time, read rows from the tblTraining table in
TrainingRegister.xlsx in OneDrive for Business. Keep rows whose Status is Registered and
SessionDate is within the next 14 days. Email the training coordinator a summary count. Do not
send participant emails.
```

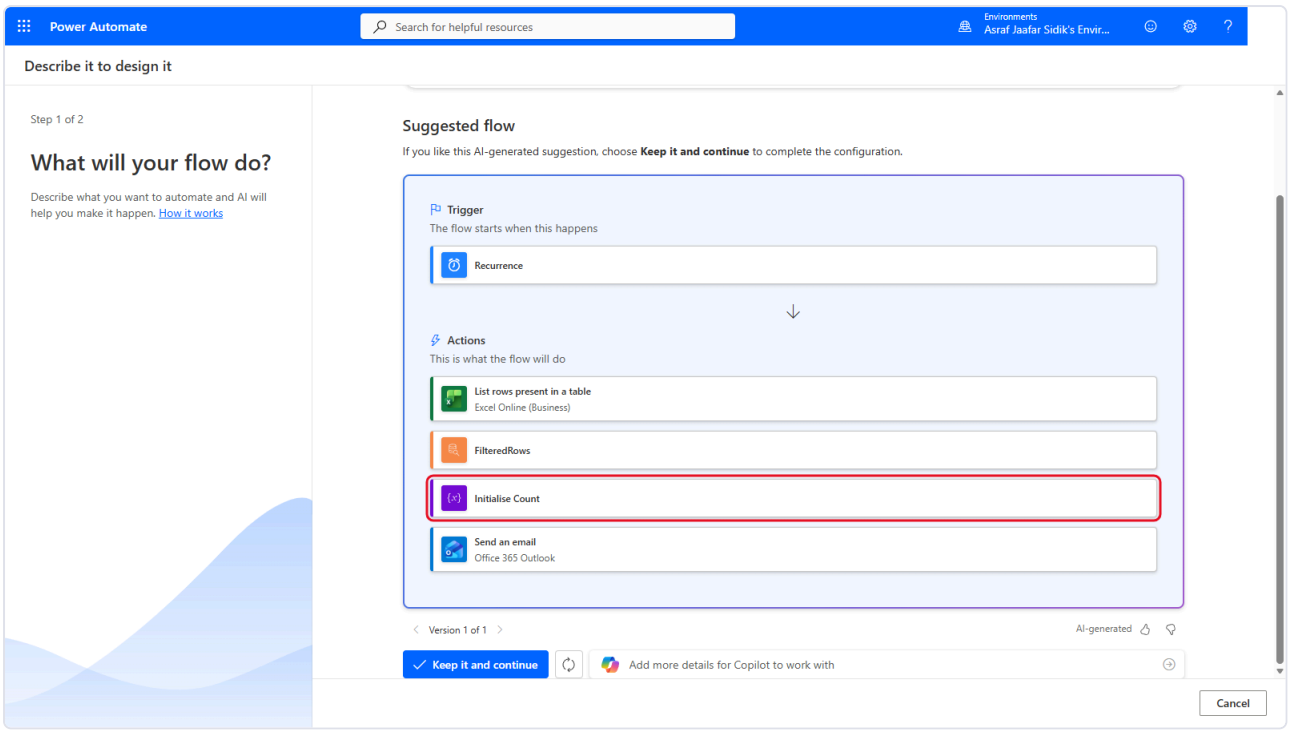


The prompt names the schedule, source, rule, output and a boundary.

A strong prompt states the event, the data source, the rule, the output and the boundaries. Even so, Copilot doesn't know your approved recipient, the exact workbook location, your organisation's policies, or how you'll test the result.

9.3 Review the First Suggestion

Copilot takes a few seconds to answer. Scroll down to **Suggested flow** and read the trigger and every action before you select anything. In the example below, the first version suggested Recurrence, List rows present in a table, FilteredRows, Initialise Count and Send an email. It looks plausible, but **Initialise Count** doesn't say how the count is calculated.



A plausible start, but the count isn't explained.

Use these questions on any AI-generated flow plan:

Review area	Question
Trigger	Does it start at the intended event, time and time zone?
Data	Does it use the correct connector, file, table, list, form or mailbox?
Logic	Are filtering, looping, conditions and calculations explicit?
Output	Is the destination approved, and could the action contact real people?
Licensing	Are all proposed connectors and actions available to participants?
Testing	Can the plan be tested safely with fictional data and approved destinations?

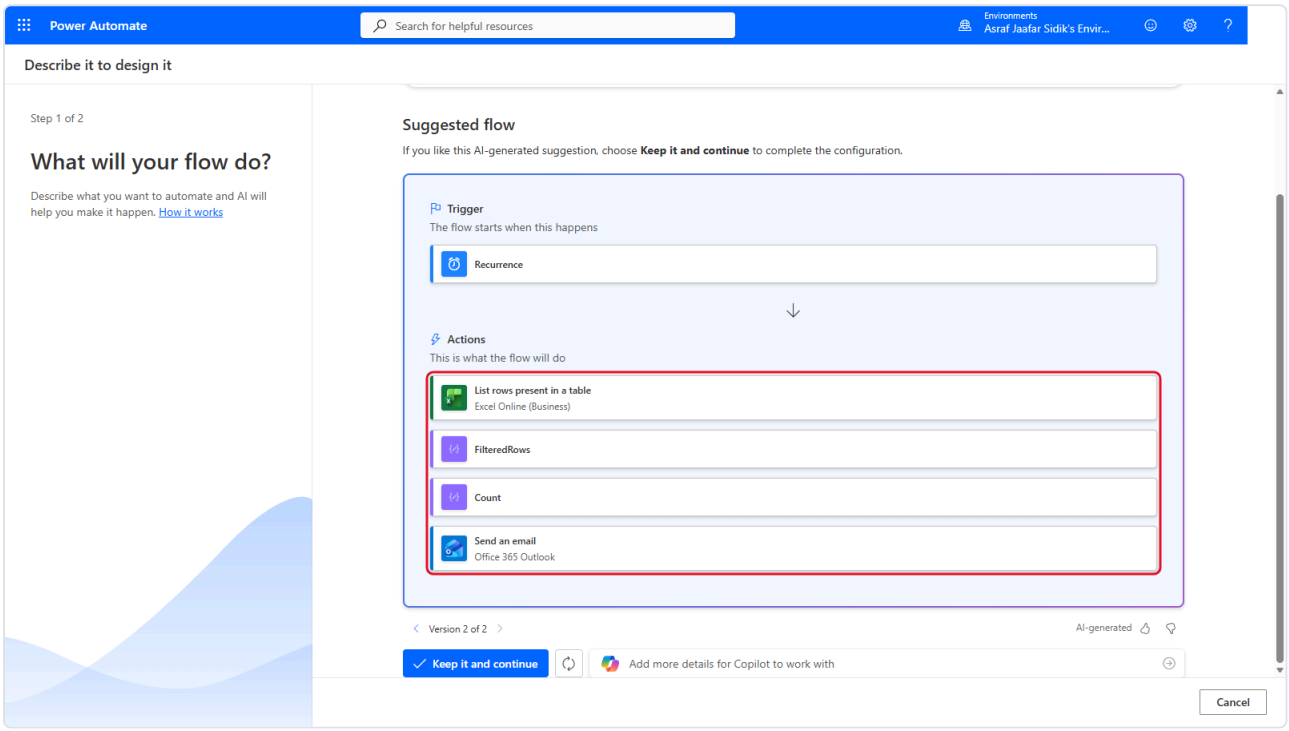
9.4 Clarify the Filter and Count

In **Add more details for Copilot to work with** (the box below the suggestion, to the right of **Keep it and continue**), paste prompt 2 and select the arrow icon at its right end to send it. It names the actions, gives the counting expression, and draws a clear line around who gets email. Copy it from the box below.

```
Use a Filter array named FilteredRows to keep rows where Status is Registered and SessionDate is between today and 14 days from today. Calculate the count with length(body('FilteredRows')). Send one summary email only to coordinator@example.com. Do not add an Apply to each or any participant email.
```

9.5 Identify an Unnecessary Action

Copilot's answers vary from run to run. Check whether the second suggestion added anything you didn't ask for. A common one is **Get file metadata using path** before List rows present in a table. You don't need it: List rows present in a table can select the workbook directly, so the extra action is just another dependency. If your suggestion is already clean, as in the screenshot below, note that and still send prompt 3: it locks in the exact structure.



In this run the second suggestion was already clean: Recurrence, List rows, FilteredRows, Count and Send an email.

9.6 Request a Streamlined Design

Paste prompt 3 into the same **Add more details for Copilot to work with** box and select the arrow icon at its right end to send it. It says what to remove, why, and exactly which structure to keep. Copilot keeps each answer as a version. To compare them, use the small arrows beside **Version** (for example **Version 3 of 3**) below the suggestion. Copy prompt 3 from the box below.

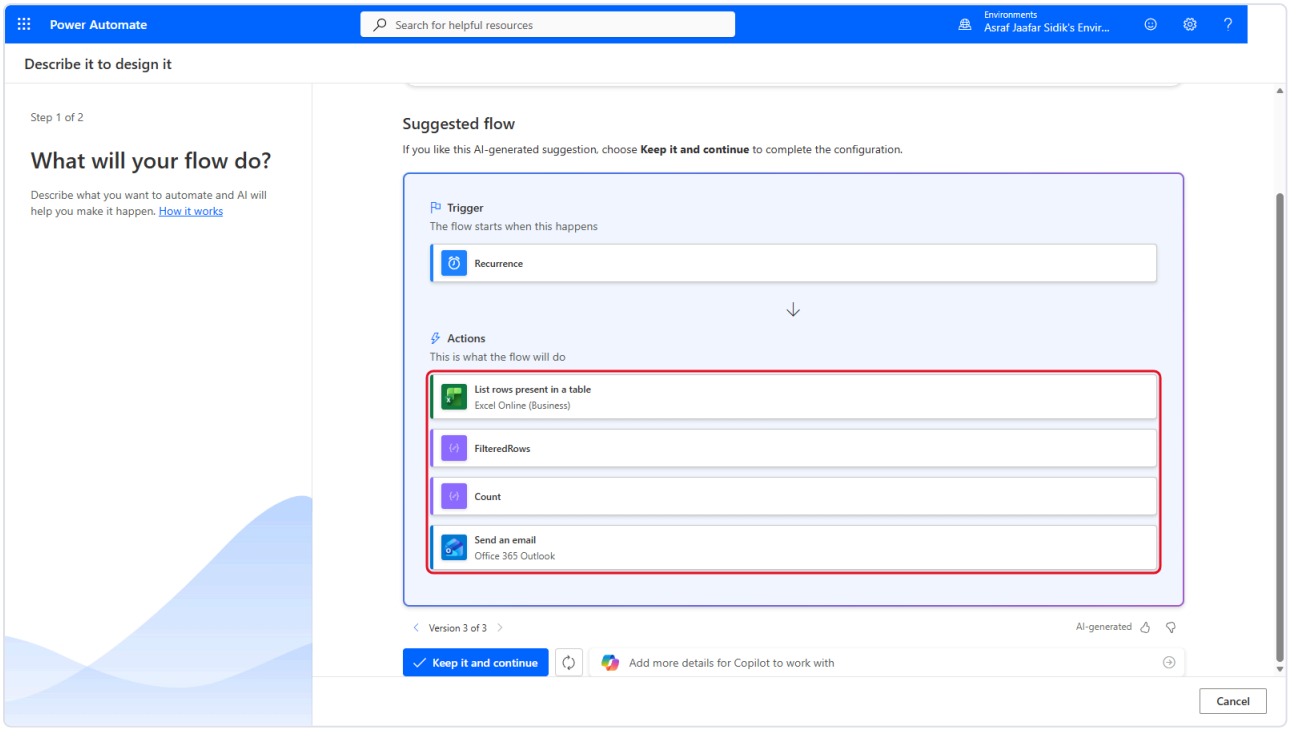
```
Remove Get file metadata using path because List rows present in a table can select the
workbook directly. Keep only Recurrence, List rows present in a table, Filter array named
FilteredRows, Compose named Count using length(body('FilteredRows')), and one Send an email
action to coordinator@example.com.
```

9.7 Validate the Final Suggestion

The third suggestion should have exactly these steps:

Order	Proposed step

1	Recurrence
2	List rows present in a table
3	FilteredRows
4	Count (a Compose action)
5	Send an email



*The outline matches the intended plan. Copilot shows the Compose action by its name, **Count**. Cancel is for 9.8; don't select it yet.*

This only checks the outline. If you continued, you'd still need to configure the Friday schedule and time zone, the workbook and table, the filter and count expressions, the recipient, subject and body, error handling, and test data.

9.8 Exit Without Creating the Flow

Select **Cancel** at the bottom right of the page, not **Keep it and continue**. Power Automate returns to the Create page. This exercise is planning only.

9.9 Plan Your Own Automation

The exercise above used prompts written for you. For your own work, plan in plain words first and let Copilot draft the flow second. Planning is where most of the gaps get caught, while they're still cheap to fix.

1. Before you open Copilot, write the job down as you'd explain it to a colleague. You need five things: what **starts** it, where the **data** lives, the **rule** it follows, the **output** it produces, and one **boundary** (what it must never do). If you can't fill one in, the automation isn't ready yet.

2. Ask Copilot to plan it, not build it. Use Microsoft 365 Copilot chat, or **Create with Copilot** without selecting **Keep it and continue**. Copy the template below, replace each part in square brackets with your details, and send it.

```
I want to automate a task in Power Automate. Don't build anything yet. Help me plan it.

Trigger: [when should it start: a new email, a schedule, a button, a form response]
Data: [where the information lives: Excel table name and columns, SharePoint list, Outlook folder]
Rule: [what decision it makes: for example, only rows where Status is not Completed and the date is in the next 14 days]
Output: [what it produces: one email per row, a summary to me, a Teams message]
Boundary: [what it must never do: for example, send to anyone outside the company, delete anything]

First ask me up to five questions about anything unclear. Then give me:
1. The trigger and each action in order, using Power Automate action names
2. Where a condition, loop or variable is needed and why
3. Which connectors are standard and which may need a premium licence
4. What could go wrong (empty data, duplicates, timing, permissions) and how to guard against it
5. A safe test plan that only sends to me
```

   Saying "don't build anything yet" matters: without it, Copilot jumps straight to a draft and skips the questions.

3. Answer Copilot's questions, then refine in small, specific turns. Ask for one exact change each time, for example "add a condition that skips Completed rows", not "make it better".

4. Review the plan with the six questions from 9.3: trigger, data, logic, output, licensing and testing. Look out for a loop you don't need and a missing condition: they're the most common mistakes in Copilot drafts.

5. When the plan looks right, turn it into steps you can follow. In the same chat, copy and send the prompt below. It asks Copilot for a click-by-click lab in the same style as this book, without screenshots.

```
The plan looks good. Now write it as a hands-on lab for a beginner who has never used Power Automate, in the new designer at make.powerautomate.com.

Rules:
- Number every step. One action per step: where to click, what to search for, what to type or pick.
- Use the exact button and field names a beginner sees on screen, in bold.
- To add an action, always say: "On the canvas, select the + below [card], then Add an action. Search for `[name]` and select it under [connector]."
- To insert a value from an earlier step, say: "Click into the box, select the lightning bolt, and pick [value] under [step]."
- For an expression, say: "Select the fx button beside the box, type the expression, and select Add." Put the expression in a code block.
- Put every value I need to type in backticks or a code block.
- Group steps into short sections with a heading for each action. End each section with "Check: your canvas should now show ..." listing the cards in order.
- Include any setting a beginner would miss (for example DateTime Format set to ISO 8601 on List rows present in a table).
```

```
- End with a safe test section: how to run it, where to look in the run history, and what
a correct result looks like. All emails go to me only.
- Don't use code view or @{...} syntax.

Here is one step written in the style I want:

1. On the canvas, select the ***** below **List rows present in a table**, then **Add an
action**. Search for `Filter array` and select it under **Data Operation**. Its settings
open on the left.
```

The sample step at the end is what makes the difference: Copilot copies its shape for the rest of the lab. Copilot can't see your screen, so treat the lab as a first draft. Button names are sometimes slightly off, and the "Check: your canvas should now show" lines help you spot when something has gone wrong.

6. Build it in a test environment with every recipient set to you, following the lab (or with **Create with Copilot**, or yourself as in Chapters 3 to 8). Test with a few rows before anyone else receives anything.

Here's the planning prompt from step 2 with every value filled in, using the Chapter 7 weekly reminders flow as the example. Copy it to see the whole conversation: Copilot asks its questions, you answer them, then you send the lab prompt from step 5.

```
I want to automate a task in Power Automate. Don't build anything yet. Help me plan it.

Trigger: every Monday at 9:00 Malaysia time (UTC+08:00).
Data: Excel table tblTraining in the file Power Automate Training/TrainingRegister.xlsx in my
OneDrive for Business. Columns: ParticipantName, Email, CourseTitle, SessionDate, Status.
Rule: only rows where Status is not Completed and SessionDate is between today and 14 days
from today.
Output: one reminder email per matching row, then one summary email to me with the number of
reminders sent.
Boundary: never email anyone while testing; send every email to me until I approve it.

First ask me up to five questions about anything unclear. Then give me:
1. The trigger and each action in order, using Power Automate action names
2. Where a condition, loop or variable is needed and why
3. Which connectors are standard and which may need a premium licence
4. What could go wrong (empty data, duplicates, timing, permissions) and how to guard against
it
5. A safe test plan that only sends to me
```

> **Tip:** Use the real names of your table, columns and folders in the prompt, and ask for Power Automate action names. Vague prompts get placeholder actions you have to rewire, and action names let you find each step in the designer.

Independent Practice

Write a prompt for a monthly training-capacity report. Include the schedule, data source, inclusion rule, output and one explicit boundary. Swap prompts with another participant and find one ambiguity before asking Copilot for a suggestion.

Troubleshooting

Symptom	What to check
Create with Copilot is missing	Organisation settings, environment, region, account entitlement. Follow the screenshots in this chapter instead
Copilot suggests the wrong trigger	State the exact event or schedule: frequency, day, time and time zone
The plan has a participant email loop	Say only one coordinator summary is needed and explicitly rule out participant emails
The count is unclear	Name the filtered array and ask for `length(body('FilteredRows'))`
An unnecessary connector appears	Explain which existing action already provides the data and ask Copilot to remove the extra one
The final outline looks right	Keep reviewing. Fields, expressions, connections, recipients and testing aren't configured yet

Lesson Summary

Copilot can propose a trigger and action sequence quickly, keep versions, and respond to refinements. You stay responsible for the business rule, connector choice, licensing, recipient safety, expressions, configuration and testing. The best result came from reviewing each suggestion and stating the exact correction.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner, then paste into Copilot.

These prompts use fictional details only. Never put confidential information, personal data, passwords or real recipients into a training prompt.

1. Describe the complete business outcome (section 9.2)

Paste into the box under **What will your flow do?** and select the arrow icon at the bottom right of the box.

```
Every Friday at 4:00 PM Singapore time, read rows from the tblTraining table in
TrainingRegister.xlsx in OneDrive for Business. Keep rows whose Status is Registered and
SessionDate is within the next 14 days. Email the training coordinator a summary count. Do not
send participant emails.
```

2. Clarify the filter and count (section 9.4)

Paste into **Add more details for Copilot to work with** (below the suggested flow) and select the arrow icon at the right end of the box.

```
Use a Filter array named FilteredRows to keep rows where Status is Registered and SessionDate
is between today and 14 days from today. Calculate the count with
length(body('FilteredRows')). Send one summary email only to coordinator@example.com. Do not
add an Apply to each or any participant email.
```

3. Request a streamlined design (section 9.6)

Paste into **Add more details for Copilot to work with** and select the arrow icon at the right end of the box.

```
Remove Get file metadata using path because List rows present in a table can select the
workbook directly. Keep only Recurrence, List rows present in a table, Filter array named
FilteredRows, Compose named Count using length(body('FilteredRows')), and one Send an email
action to coordinator@example.com.
```

> **Remember:** select **Cancel** at the end (section 9.8). This exercise is planning only.

4. Plan your own automation (section 9.9)

Replace each part in square brackets with your details.

```
I want to automate a task in Power Automate. Don't build anything yet. Help me plan it.

Trigger: [when should it start: a new email, a schedule, a button, a form response]
Data: [where the information lives: Excel table name and columns, SharePoint list, Outlook
folder]
Rule: [what decision it makes: for example, only rows where Status is not Completed and the
date is in the next 14 days]
Output: [what it produces: one email per row, a summary to me, a Teams message]
Boundary: [what it must never do: for example, send to anyone outside the company, delete
anything]

First ask me up to five questions about anything unclear. Then give me:
1. The trigger and each action in order, using Power Automate action names
2. Where a condition, loop or variable is needed and why
3. Which connectors are standard and which may need a premium licence
4. What could go wrong (empty data, duplicates, timing, permissions) and how to guard against
it
5. A safe test plan that only sends to me
```

Example with every value filled in (Chapter 7 weekly reminders):

```
I want to automate a task in Power Automate. Don't build anything yet. Help me plan it.

Trigger: every Monday at 9:00 Malaysia time (UTC+08:00).
Data: Excel table tblTraining in the file Power Automate Training/TrainingRegister.xlsx in my
OneDrive for Business. Columns: ParticipantName, Email, CourseTitle, SessionDate, Status.
Rule: only rows where Status is not Completed and SessionDate is between today and 14 days
from today.
Output: one reminder email per matching row, then one summary email to me with the number of
reminders sent.
Boundary: never email anyone while testing; send every email to me until I approve it.

First ask me up to five questions about anything unclear. Then give me:
1. The trigger and each action in order, using Power Automate action names
2. Where a condition, loop or variable is needed and why
3. Which connectors are standard and which may need a premium licence
4. What could go wrong (empty data, duplicates, timing, permissions) and how to guard against
it
5. A safe test plan that only sends to me
```

5. Turn the plan into a lab (section 9.9, step 5)

Send this in the same chat once the plan looks right.

```
The plan looks good. Now write it as a hands-on lab for a beginner who has never used Power
Automate, in the new designer at make.powerautomate.com.

Rules:
- Number every step. One action per step: where to click, what to search for, what to type or
pick.
- Use the exact button and field names a beginner sees on screen, in bold.
- To add an action, always say: "On the canvas, select the + below [card], then Add an action.
Search for `[name]` and select it under [connector]."
- To insert a value from an earlier step, say: "Click into the box, select the lightning bolt,
```

```
and pick [value] under [step]."
- For an expression, say: "Select the fx button beside the box, type the expression, and
select Add." Put the expression in a code block.
- Put every value I need to type in backticks or a code block.
- Group steps into short sections with a heading for each action. End each section with
"Check: your canvas should now show ..." listing the cards in order.
- Include any setting a beginner would miss (for example DateTime Format set to ISO 8601 on
List rows present in a table).
- End with a safe test section: how to run it, where to look in the run history, and what a
correct result looks like. All emails go to me only.
- Don't use code view or @{...} syntax.

Here is one step written in the style I want:

1. On the canvas, select the ***** below **List rows present in a table**, then **Add an
action**. Search for `Filter array` and select it under **Data Operation**. Its settings open
on the left.
```

10

CHAPTER 10

Move Automations with Environments and Solutions

A flow that works in one environment can still be hard to move safely. It depends on connections, tables and other components, and copying them one at a time makes it easy to miss something. Solutions package those pieces together.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 60 minutes (trainer demonstration)

Your result: You can explain how a flow moves between environments, after watching your trainer package the Chapter 7 flow in a solution and walk through where an import starts.

Files for this lab: None. The demo uses your Chapter 7 flow.

What You Will Learn

- Explain what an environment is and why organisations separate development from production
- Describe what a solution can contain
- Explain how Power Automate, Power Apps and Copilot Studio can share solution components
- Follow how a solution is created and an existing cloud flow added
- Follow how an unmanaged solution is published and exported
- Identify the safe import, connection-repair, verification and testing sequence

The Migration Path

Development environment > Solution > Publish > Export package > Target environment > Import > Repair connections > Verify > Authorised test

The live demonstration stops after opening the import screen. Importing back into the same environment isn't a real migration test, and testing the flow could send email.

Before You Begin

> **Important:** This chapter is a **trainer demonstration**. Your class works in a live environment, and solutions create real components there (a solution, a publisher, connection references). Watch the demo and follow along on this page. Don't create, export or import solutions yourself unless your trainer gives you a separate practice environment.

The demo uses `PA - Weekly training capstone` from Chapter 7, kept turned off. Solutions also need Microsoft Dataverse in the environment, and Power Automate Free alone doesn't guarantee that.

Environments in Plain Language

An environment is a separate workspace for apps, flows, data, connections and security. The environment selector at the top of Power Automate shows which one you're in.

Environment	Typical purpose	Practical rule
Development	Build and change automations	Use fictional or approved test data
Test	Validate behaviour before release	Reconnect services and run controlled tests
Production	Run approved business processes	Restrict changes and monitor real operation

Separating these stages stops unfinished changes reaching real users, but only when permissions, connection ownership, data and release steps are controlled too.

Power Automate, Power Apps and Copilot Studio all sit on the Power Platform, so one solution can package the flows, apps, agents, tables, connection references and environment variables behind a single business process. What's available depends on the environment and licences.

10.1 Confirm the Current Environment

Check the environment selector before you create or import anything (top right of the blue bar, under the small word **Environments**), and confirm with the trainer that it's the development or training environment. If a flow or solution seems to be missing, check the environment before rebuilding it.

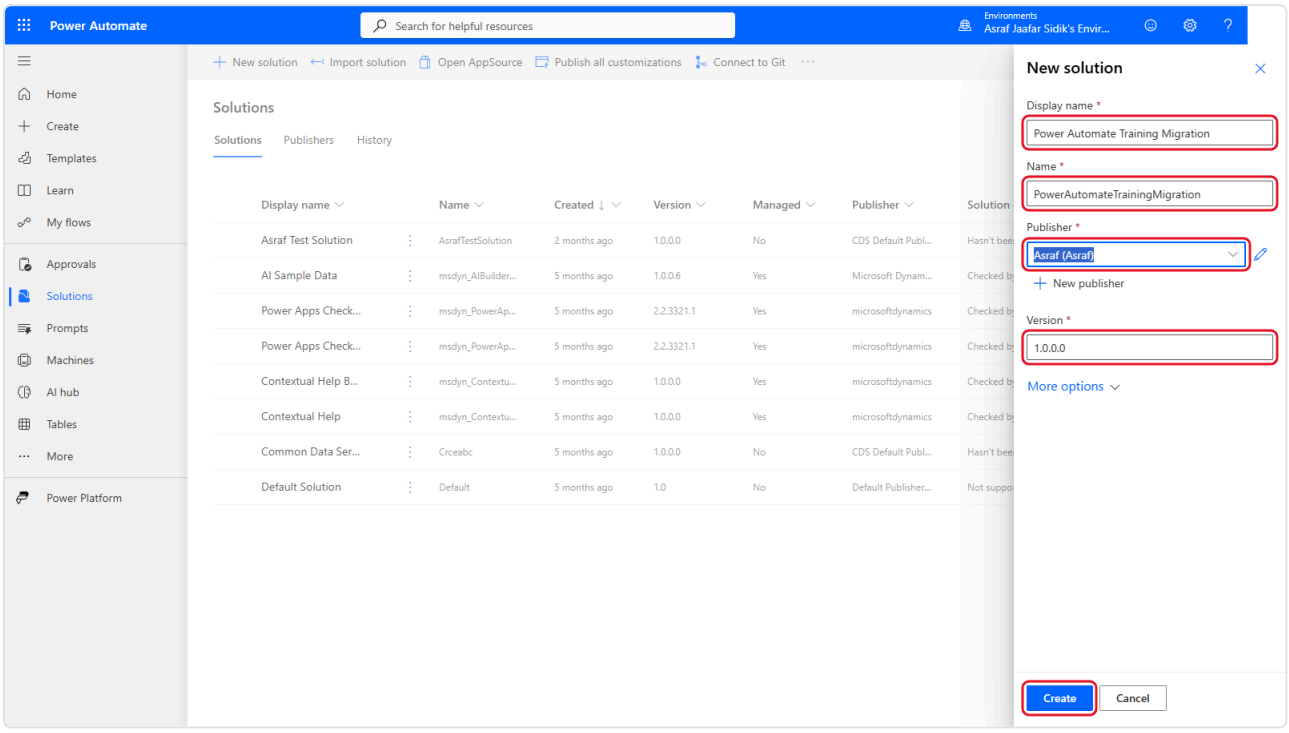
10.2 Create the Solution

In the left navigation, select **Solutions** (if you don't see it, select **More** to find it). Then select **+ New solution** at the left end of the toolbar. The **New solution** panel opens on the right. Enter:

Field	Value
Display name	`Power Automate Training Migration`
Name	`PowerAutomateTrainingMigration`

Publisher	The trainer-approved publisher
Version	`1.0.0.0`

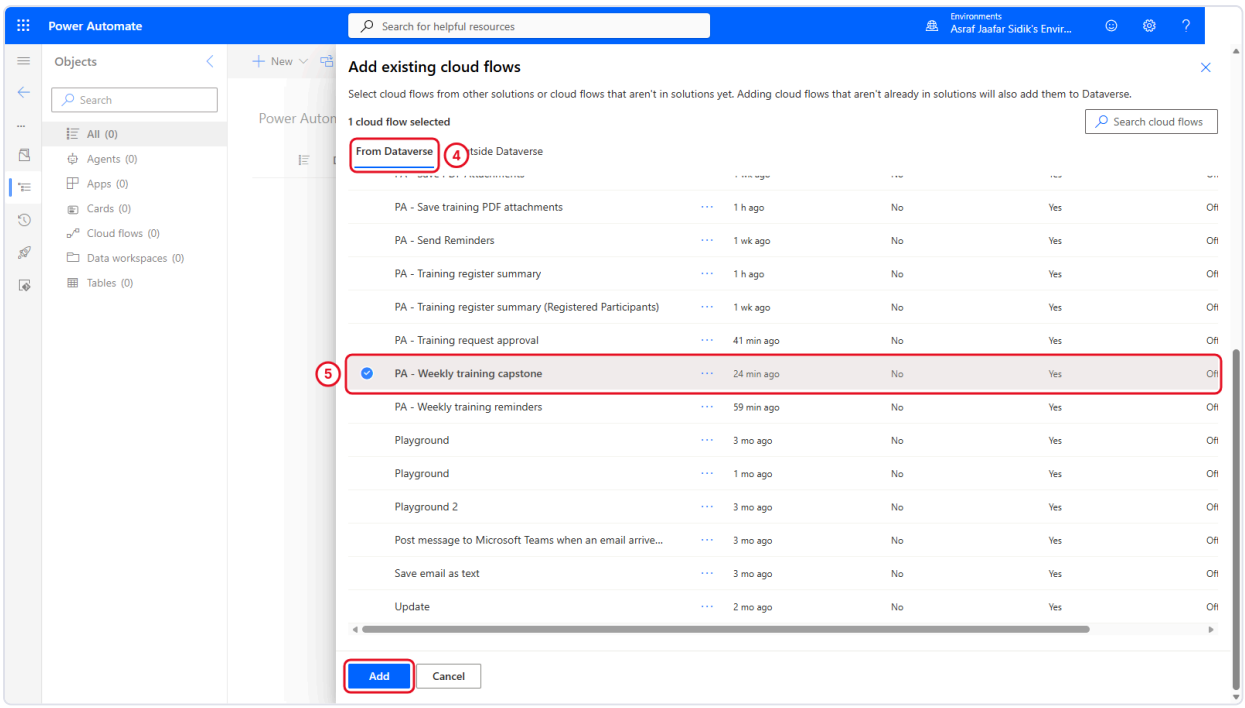
For **Publisher**, open the dropdown and pick the publisher your trainer names. Then select **Create** at the bottom of the panel. The display name is the friendly label. The unique name is used internally and normally shouldn't change later.



The display name is for people. The name is used internally.

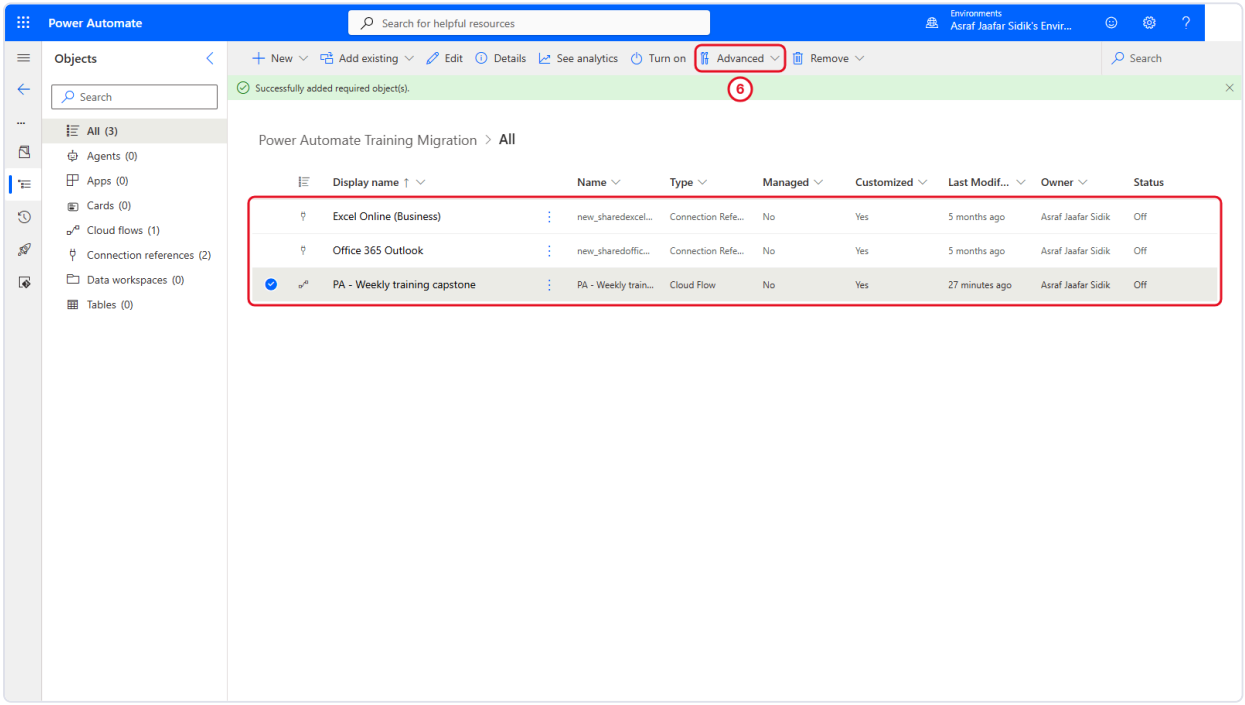
10.3 Add the Chapter 7 Flow

1. Open the new solution: in the **Solutions** list, select its display name, `Power Automate Training Migration`. Its object list is empty: a new solution is just a container.
2. In the toolbar at the top, select **Add existing** and point to **Automation**.
3. Under Automation, select **Cloud flow**.
4. Find `PA - Weekly training capstone`. Depending on how your environment stores flows, it's on the **From Dataverse** tab or the **Outside Dataverse** tab. Scroll the list, or type its name in **Search cloud flows** at the top right of the panel.
5. Select the circle to the left of the flow's name, then select **Add** at the bottom of the panel.



Pick the flow from the list. Here it's on the From Dataverse tab.

6. If only the flow appears in the solution, select the circle beside it, then in the toolbar select **Advanced** > **Add required objects**. This adds the connection references it needs.



Three objects: the flow and its two connection references.

Power Automate may also add **connection references**. A connection reference is a solution component that points to a connection. The target environment still needs a valid connection and an authorised account.

10.4 Verify the Solution Components

Confirm the solution has three objects: the Chapter 7 cloud flow and the Excel Online (Business) and Office 365 Outlook connection references (see the picture under step 6 of 10.3).

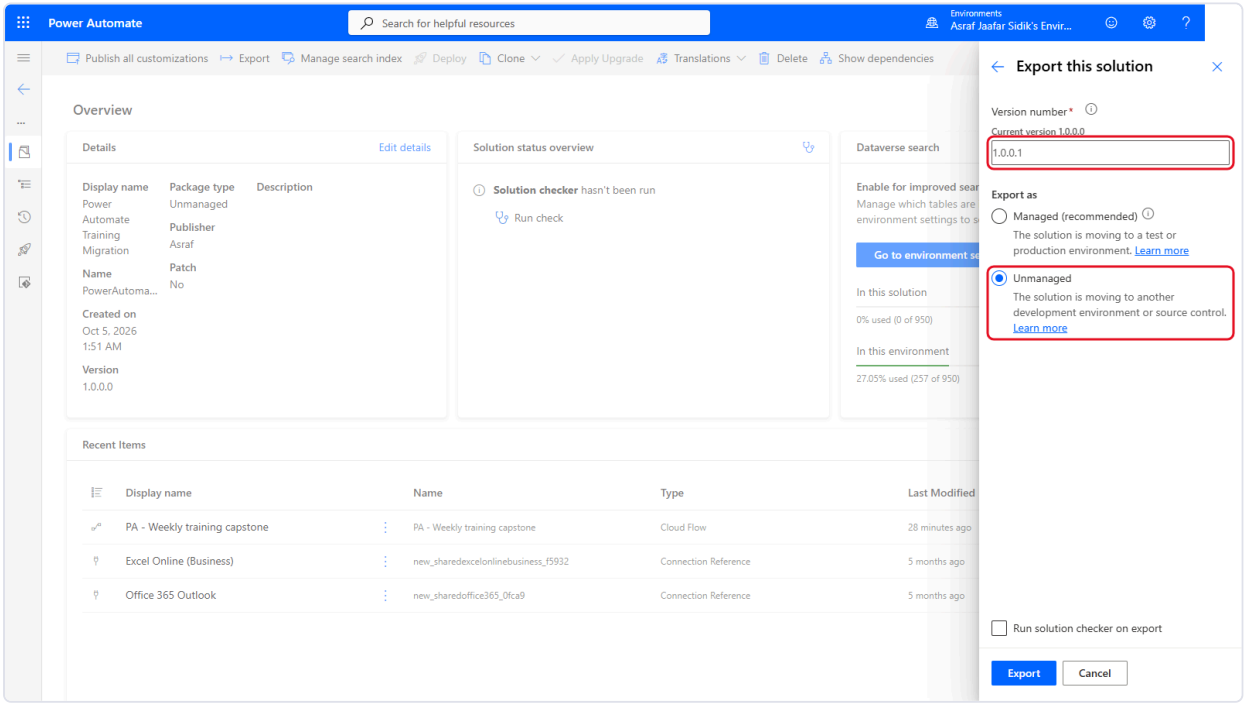
> **Key point:** Showing up in the object list doesn't mean a dependency is ready. File locations, table names, recipients, connection accounts, environment variables and permissions can all differ in the target environment.

10.5 Review the Solution Before Export

Inside a solution, the far-left rail shows icons only. Select the **Overview** icon, the page-shaped icon just under the back arrow (hover over an icon to see its name). Check the name, version, package type, publisher and included items. **Export** is in the toolbar at the top of the Overview page.

10.6 Publish and Export

1. In the toolbar at the top of the Overview page, select **Export**. If a **Before you export** panel offers **Publish**, select it so the latest changes are included, and wait for **Published**.
2. Select **Next**.
3. Confirm the version, select **Unmanaged**, and select **Export**. Power Automate suggests the next version number (for example 1.0.0.1), so every exported package has its own version.



Unmanaged suits a learning exercise. The version number goes up with each export.

4. When the success message appears, select **Download** and store the ZIP in the trainer-approved location. The worked example produced `PowerAutomateTrainingMigration_1_0_0_1.zip`.

An unmanaged package suits this exercise because its components can be edited after import. Managed solutions are normally used for controlled distribution where recipients shouldn't edit the components. That's an organisational decision, so don't switch package types casually in an established process.

> **Tip:** Don't unzip or rename the package before import. Treat it as a release artifact and record who exported it, from which environment, and why.

10.7 Switch Environment and Start the Import

1. Select the environment name at the top right of the blue bar and choose the trainer-approved target environment. Confirm its name before opening **Solutions** in the left navigation.
2. Select **Import solution** in the toolbar at the top of the Solutions page, then **Browse** to the exported ZIP. Select **Next** only after confirming the package name and the target environment.

The demonstration stops here. No file is uploaded and nothing is imported. Your trainer may not switch environments at all if there's no second training environment. **Never use production as a stand-in for a training target.**

10.8 Repair Connections and Settings

After the package is selected, Power Platform may ask for connection mappings or environment-specific values. Before import or first use:

1. Map each connection reference to an approved connection in the target environment.
2. Confirm the Excel workbook and `tblTraining` table exist in the intended target location.
3. Review recipients, team and channel selections, form references, schedules, time zones and environment variables.
4. Confirm the importing account and future flow owner have the required permissions.
5. Keep the imported flow turned off until verification is complete.

10.9 Verify and Test Safely

After an authorised import, open the solution and confirm the expected components are there. Run Flow checker, review every connection and setting, and use fictional data and approved destinations for the first test.

Testing is a separate release decision. An imported flow can be structurally valid and still point at the wrong workbook, mailbox, form, team, channel or recipient. Get trainer authorisation right before any test that can send messages or change shared data.

Independent Practice

Create a migration checklist for one of the earlier flows. Name its trigger, connectors, data sources, recipients, environment-specific settings, owner, safe test data, and the evidence you'd need before turning it on in production. Don't export, import or run anything.

Troubleshooting

Symptom	What to check
Solutions is missing or unavailable	Environment type, Dataverse, licence, security role and maker permissions
The flow isn't in the list	Current environment, both the From Dataverse and Outside Dataverse tabs, flow ownership, and whether it's already in another solution
Connection references are missing	Select the flow in the solution, then Advanced > Add required objects
Extra components appear	Connection references and other dependencies may be added automatically. Review each one
Export stays pending	Refresh Solutions, open export history, and confirm publishing finished
Import says the solution already exists	Check you're in the intended target, and whether this is an upgrade or an accidental same-environment import
A connection shows an error after import	Create or pick an approved target connection and remap the connection reference
The flow imports but can't find the workbook	Target OneDrive or SharePoint location, file name, table name, permissions and connector account
The imported flow is off	Leave it off while reviewing. Turn it on only through the approved release process

Lesson Summary

Environments separate workspaces, permissions, data and release stages. Solutions package related Power Platform components so a process can move in a controlled way. A safe migration covers dependency review, publishing, a versioned export, a confirmed target, connection mapping, configuration checks, Flow checker, controlled testing and a deliberate decision to switch the flow on.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Everything on this page is **text**, typed or pasted straight into a field.

Solution display name (text, section 10.2)

```
Power Automate Training Migration
```

Solution name (text, section 10.2)

```
PowerAutomateTrainingMigration
```

Solution version (text, section 10.2)

```
1.0.0.0
```

11

CHAPTER 11

Look Up Currency Rates with a Desktop Flow

Every chapter so far built a cloud flow that talks to online services. This one is different. You build a **desktop flow** in Power Automate for desktop that drives your own computer: it opens Excel and Microsoft Edge, searches Google for each currency pair in a list, reads the converted amount off the page and types it back into Excel.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 45 minutes

Your result: A desktop flow that loops through a list of currency pairs in Excel, looks up each rate on Google in Edge and writes the result into the third column.

> **Download:** the files for this lab
>
> - CurrencyList.xlsx: the list of currency pairs. Save it to your **Desktop**.

What You Will Learn

- Create a desktop flow in Power Automate for desktop
- Open Edge and Excel from a flow and store each one in a variable
- Find where an Excel table starts and ends with **Get table range from Excel worksheet**
- Repeat steps for every row with a **Loop**
- Capture UI elements on a web page and fill in or read them
- Send mouse clicks and keystrokes the way a person would
- Write a value back into a cell

How the Flow Works

Launch Edge > Launch Excel > Get table range > Loop (read From and To > type the search > press Enter > wait > read the rate > write it to Excel) > End

The flow opens Google in Edge and the workbook in Excel. It asks Excel where `Table1` ends, then loops from row 2 (the first row under the headings) to the last row. On each pass it reads the two currency codes, types something like `USD to MYR` into Google, reads the converted amount from Google's currency converter and writes it into column 3 of the same row.

Cloud flow (Chapters 3 to 8)	Desktop flow (this chapter)
Runs in Microsoft's cloud, even when your PC is off	Runs on your PC, using your mouse, keyboard and screen
Talks to services through connectors	Clicks, types and reads windows and web pages like a person
Built in a browser at make.powerautomate.com	Built in the Power Automate for desktop app

> **Key point:** A desktop flow takes over your mouse and keyboard while it runs. Once you select Run, keep your hands off them until the flow finishes.

Before You Begin

You need:

- A Windows computer with **Power Automate for desktop** installed, signed in with your training account. Search for *Power Automate* in the Start menu. If it isn't installed, ask your trainer.
- The **Excel desktop app**. This flow opens the file in Excel on your PC, not Excel for the web.
- **Microsoft Edge** with the **Microsoft Power Automate** browser extension turned on. In Power Automate for desktop, select **Tools** > **Browser extensions** > **Microsoft Edge** to install it, then turn it on in Edge's extensions page.
- `CurrencyList.xlsx` saved to your Desktop. Download it from the link at the top of this page.

The workbook has one table, named `Table1`:

Column	Heading	Contents
1	From	The currency you have, such as `USD`
2	To	The currency you want, such as `MYR`
3	Result	Empty. The flow fills it in

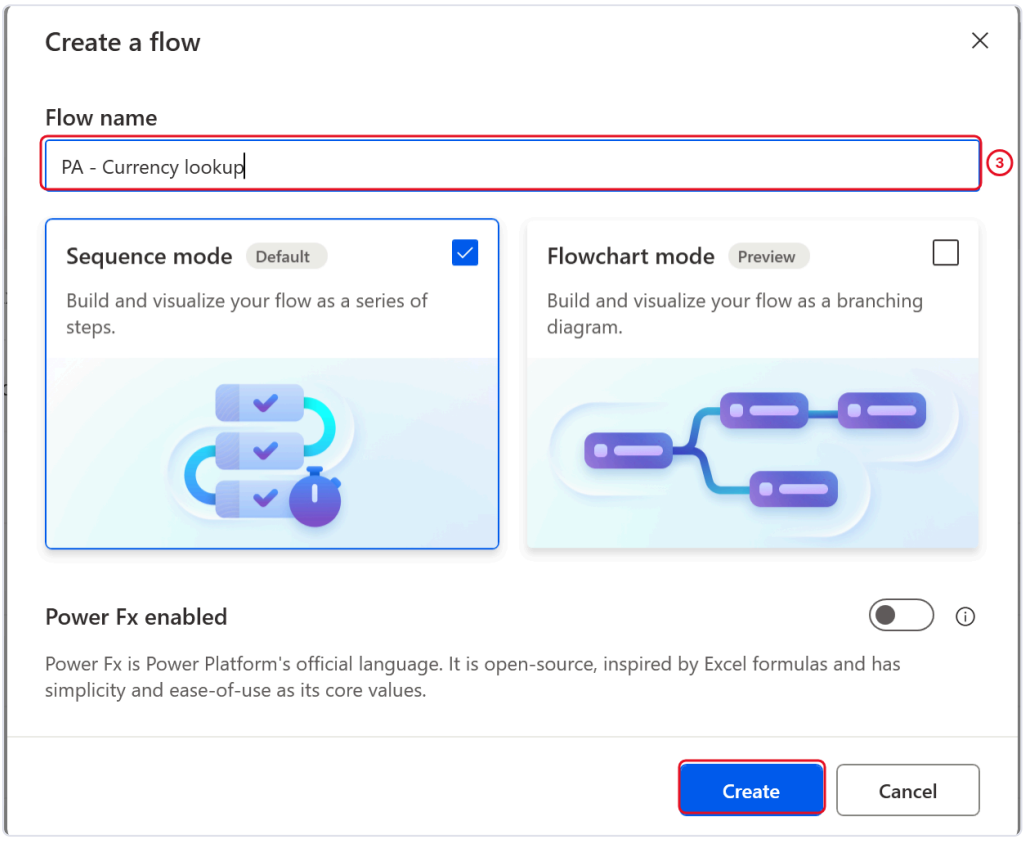
> **Warning:** This flow reads a real web page that Google controls. If Google shows a cookie or sign-in prompt, accept or close it once in Edge before you run the flow. If Google changes its page, the UI elements you capture in 11.7 and 11.10 may stop matching and you'll need to capture them again.

> **Important:** Run this flow with **one screen**. With a second monitor, Edge can open on the other screen and the typing in 11.7 lands in whatever window is in front, usually Excel. To switch, press **Windows key + P** and choose **PC screen only**, then switch back after the lab. If your Desktop is in OneDrive (the file shows a cloud or tick icon), open `CurrencyList.xlsx` once, turn **AutoSave** off at the top left of Excel and close it. Otherwise a stray keystroke into the workbook is saved straight away.

11.1 Create the Desktop Flow

1. Open **Power Automate** from the Start menu. This is the desktop app, not the website.
2. Select **+ New flow**.
3. In **Flow name**, enter the name below. Leave **Sequence mode** selected, then select **Create**.

```
PA - Currency lookup
```



Type the name, keep Sequence mode, then Create.

4. The flow designer opens. **Actions** is on the left, the **Main** tab (your flow) is in the middle, and the variables pane is on the right.

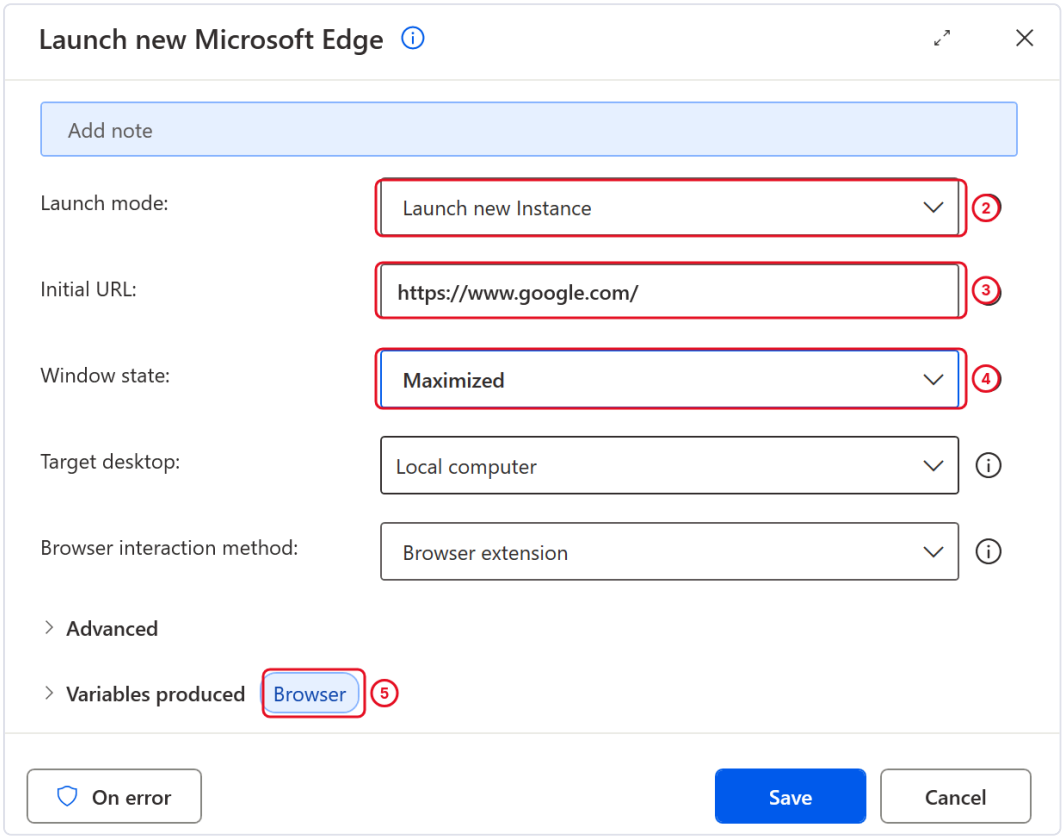
To add any action in this chapter: type its name in the **Search** box at the top of the Actions pane, then double-click it, or drag it into the Main tab where you want it. A settings window opens. Fill it in and select **Save**.

11.2 Open Google in Edge

1. Search for `Launch new Microsoft Edge` and add it.
2. Set **Launch mode** to **Launch new instance**.
3. In **Initial URL**, enter:

```
https://www.google.com/
```

4. Set **Window state** to **Maximized**, so Edge fills the screen.
5. Leave **Variables produced** as `Browser`. Later actions use it to know which Edge window to work in.



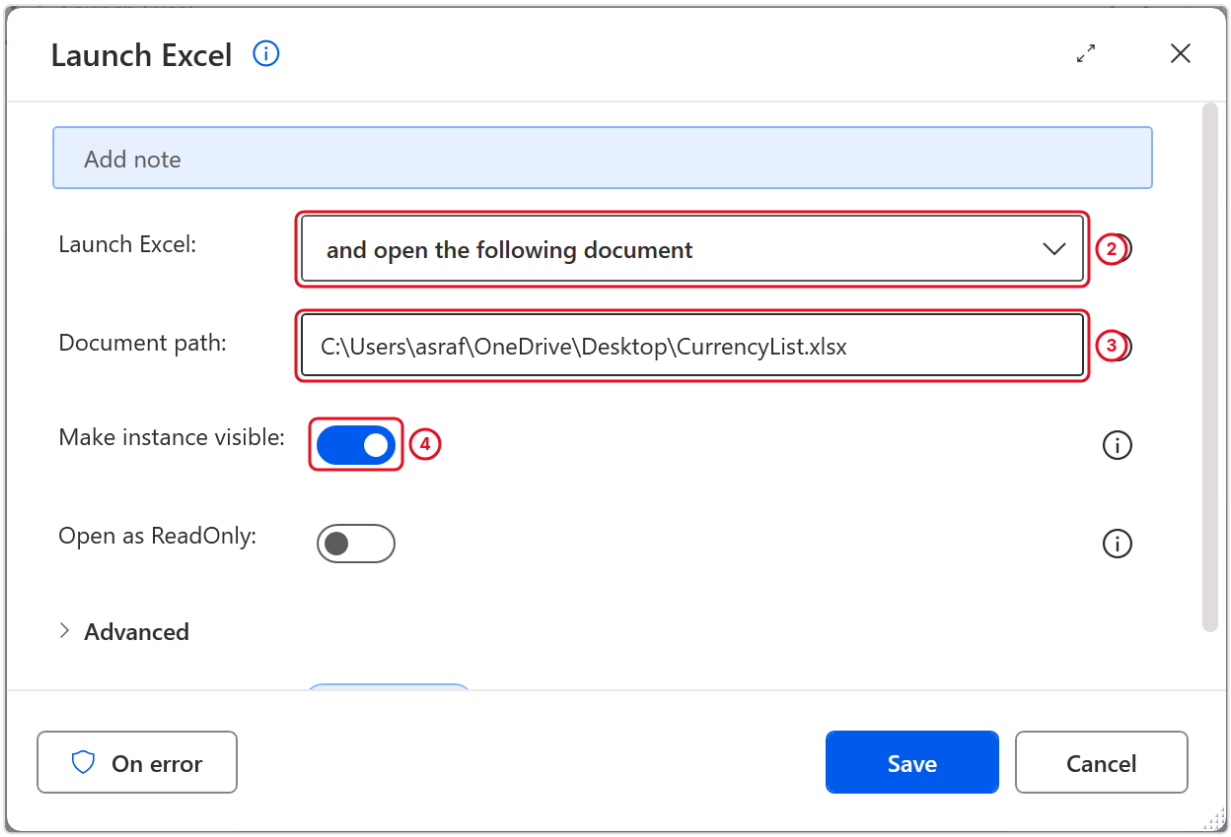
Check these four settings before you select Save. Target desktop and Browser interaction method keep their defaults.

6. Select **Save**.

11.3 Open the Workbook in Excel

1. Search for `Launch Excel` and add it below the Edge action.
2. Set **Launch Excel** to **and open the following document**.
3. In **Document path**, select the file icon at the right of the box, browse to your Desktop and select `CurrencyList.xlsx`.

4. Leave **Make instance visible** on, so you can watch the results appear.
5. Leave **Variables produced** as `ExcelInstance`.



Your Document path shows your own user name and folder.

6. Select **Save**.

11.4 Find the End of the Table

You could tell the loop to stop at row 6, but then it breaks the day someone adds a seventh pair. This action asks Excel where the table ends instead.

1. Search for `Get table range from Excel worksheet` and add it below Launch Excel.
2. **Excel instance** already shows `%ExcelInstance%`. Leave it.
3. In **Table name**, enter:

```
Table1
```

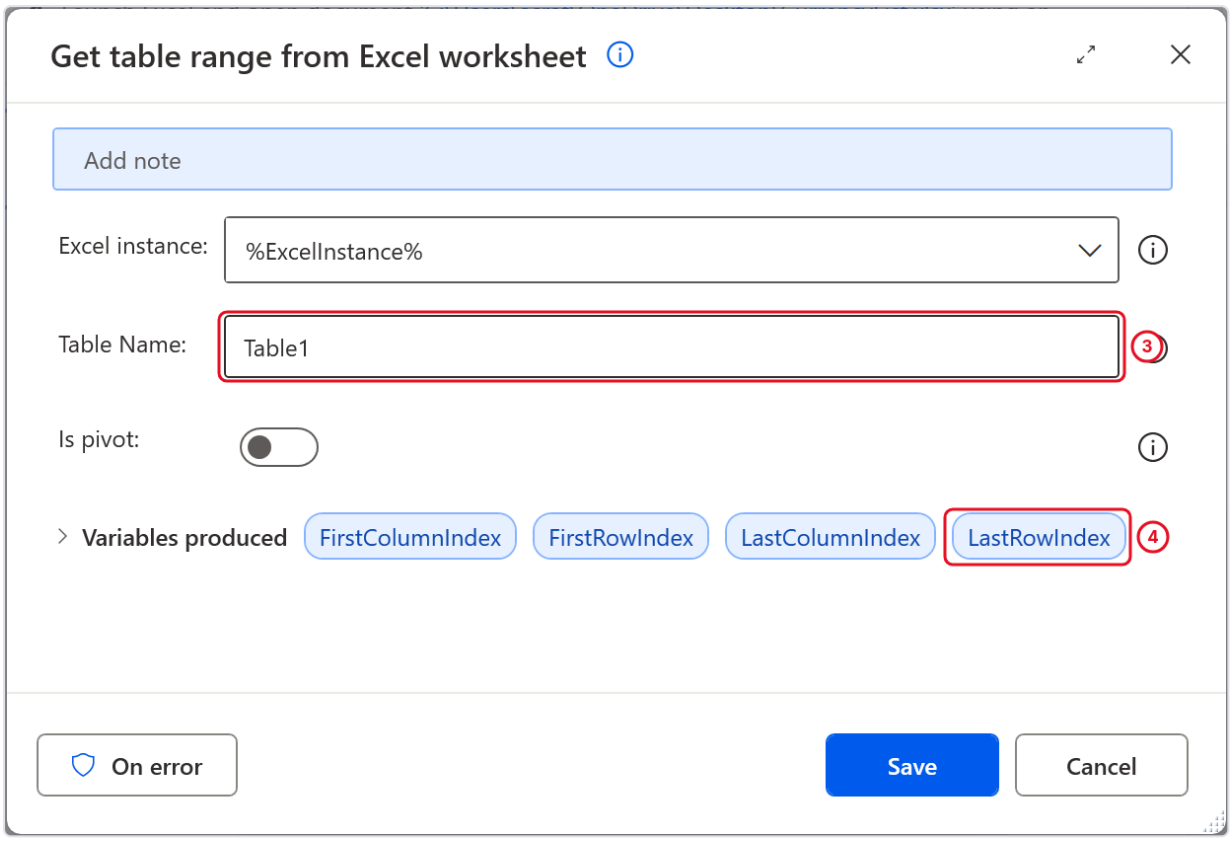


Table1 in Table Name, and LastRowIndex is the one variable you use.

4. Select **Save**. It produces four variables: `FirstRowIndex`, `FirstColumnIndex`, `LastRowIndex` and `LastColumnIndex`. You only need `LastRowIndex`, the row number of the last row in the table (6 in the downloaded file).

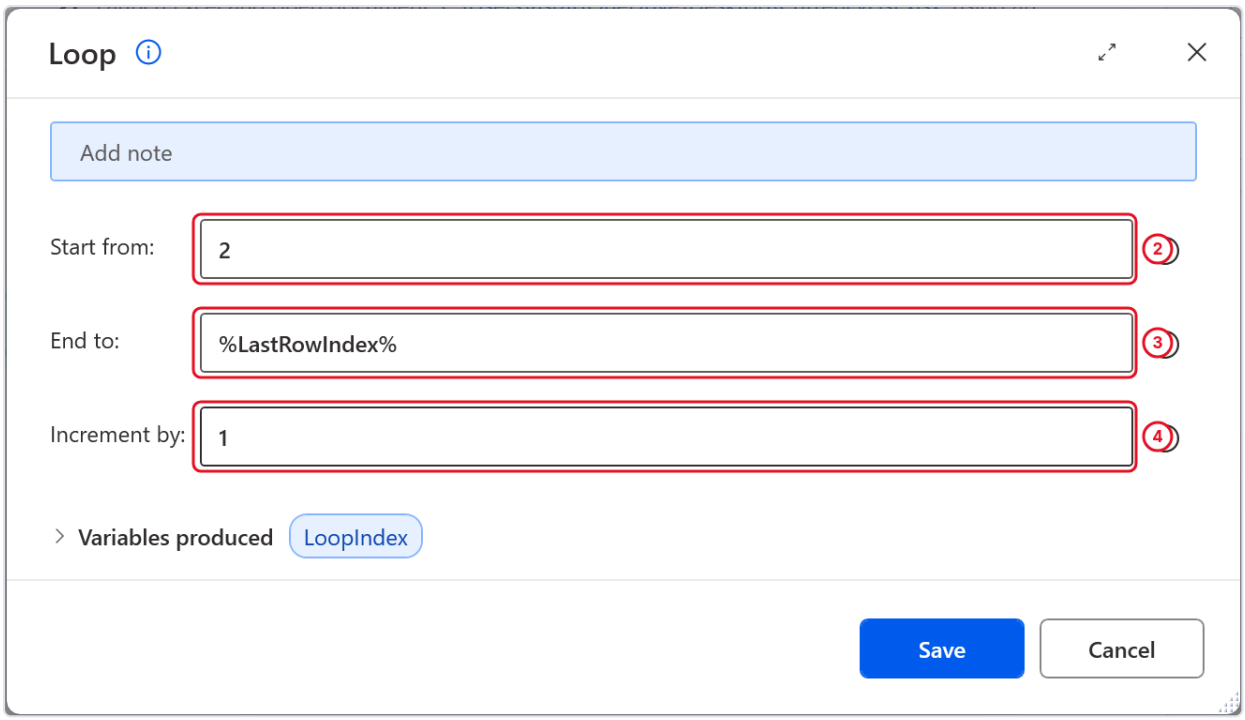
> **Note:** Power Automate for desktop writes variables between percent signs, like `%LastRowIndex%`. That tells it to use the variable's value instead of the text. You can type them, or select the **{x}** icon at the right of a box and pick one from the list.

11.5 Loop Through the Rows

1. Search for `Loop` and, under **Loops**, add **Loop** (not *For each* or *Loop condition*) below Get table range.
2. In **Start from**, enter `2`. Row 1 holds the headings, so the first pair is on row 2.
3. In **End to**, enter:

```
%LastRowIndex%
```

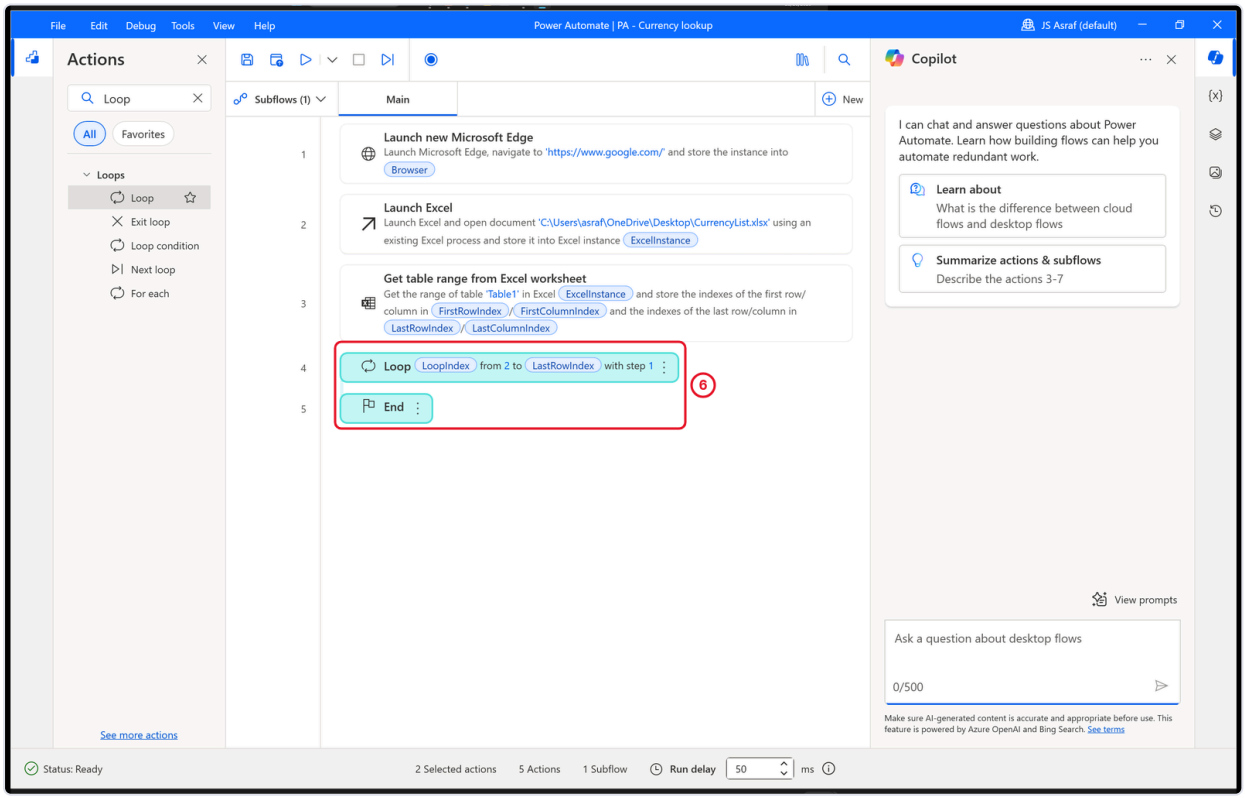
4. In **Increment by**, enter `1`.
5. Leave **Variables produced** as `LoopIndex`. It holds the current row number: 2 on the first pass, then 3, and so on.



Start at 2, end at the last row, one row at a time.

6. Select **Save**. The designer adds a **Loop** line and an **End** line. Every action from 11.6 to 11.11 goes **between** them, so drop each one just above **End**. Actions inside the loop are indented.

   Check your flow so far. Your screen should look like this.



Four actions, then End. The next actions go between Loop and End.

11.6 Read the Two Currency Codes

Read the From code (column 1)

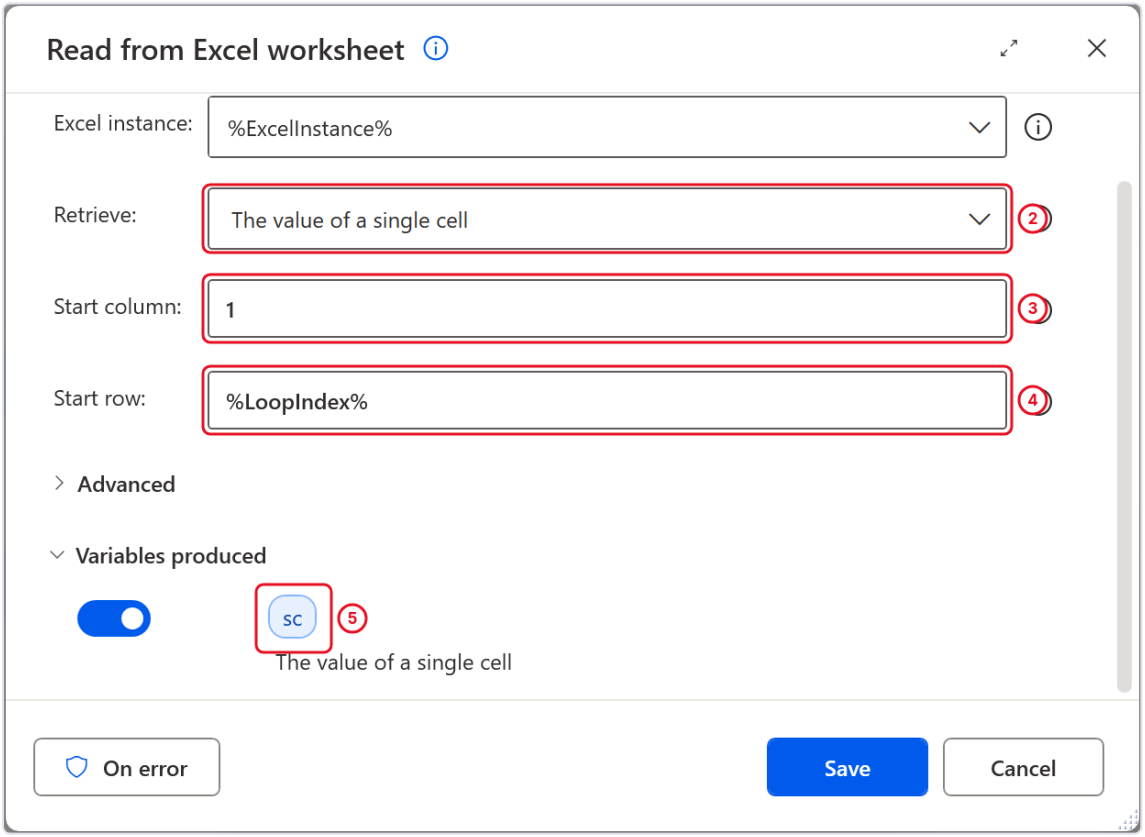
1. Search for `Read from Excel worksheet` and drag it between **Loop** and **End**.
2. Set **Retrieve** to **The value of a single cell**.
3. In **Start column**, enter `1`.
4. In **Start row**, enter:

```
%LoopIndex%
```

5. Under **Variables produced**, select the name `ExcelData` and change it to:

```
sc
```

Short for *source currency*.



Column 1 of the current row goes into sc.

6. Select **Save**.

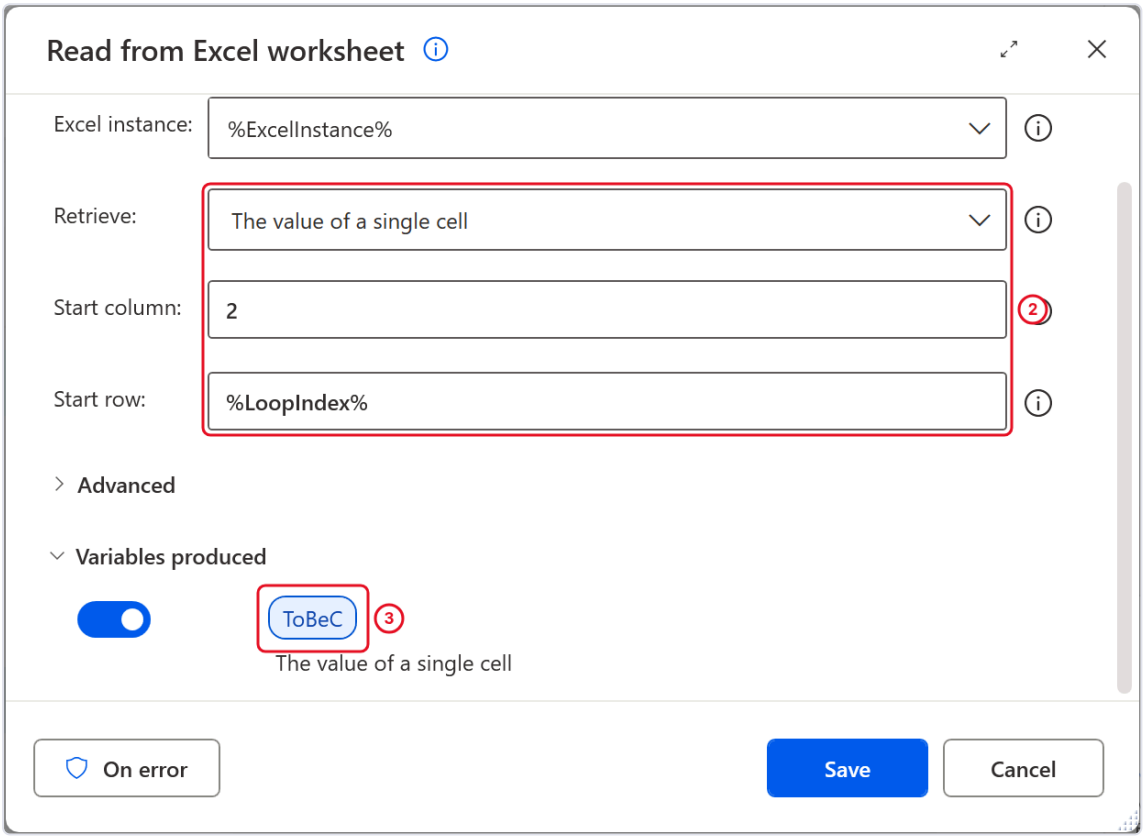
Read the To code (column 2)

1. Add another **Read from Excel worksheet** just below the first one, still inside the loop.

2. Set **Retrieve** to **The value of a single cell**, **Start column** to `2` and **Start row** to `%LoopIndex%`.
3. Rename the variable from `ExcelData` to:

   > `ToBeC`

   Short for *to be converted to*.



The same settings with column 2, stored in ToBeC.

4. Select **Save**.

11.7 Type the Search into Google

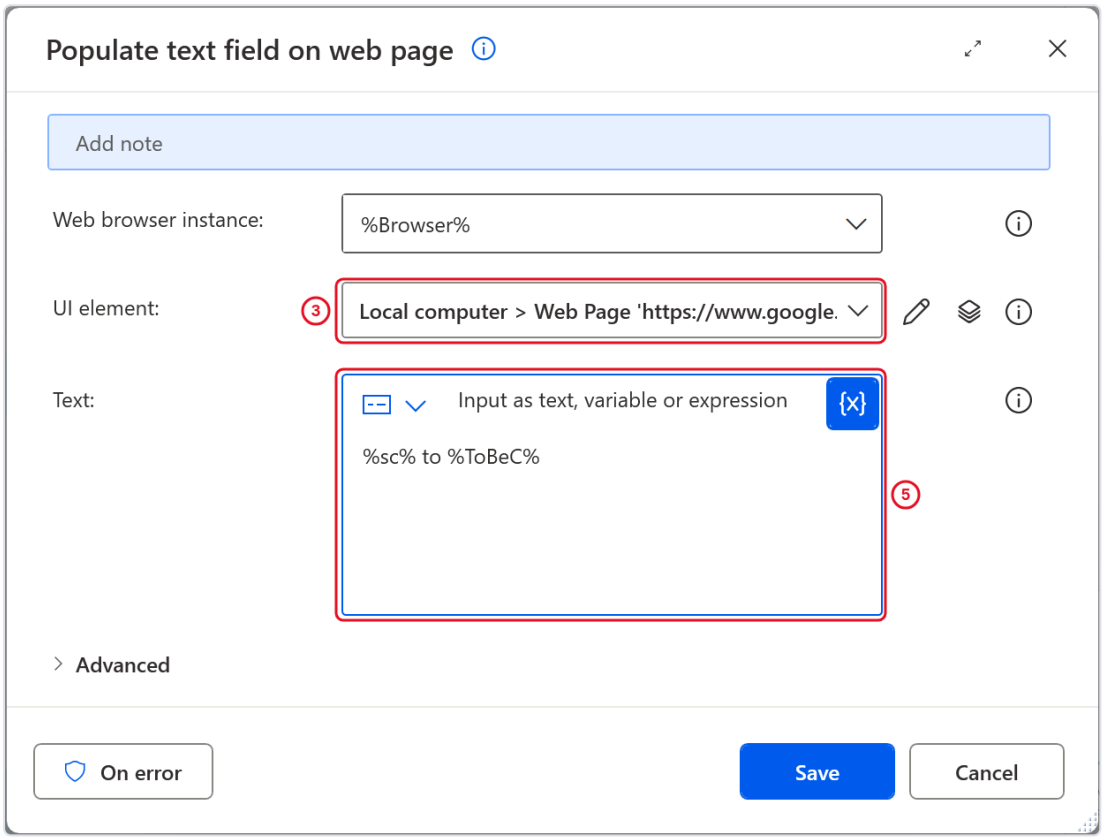
You need Google open in Edge to capture its search box. Open Edge yourself, go to `https://www.google.com/` and leave it on screen.

1. Search for `Populate text field on web page` and add it below the second Read action, inside the loop.
2. **Web browser instance** shows `%Browser%`. Leave it.
3. Open the **UI element** dropdown and select **Add a new UI element**. The tracking window appears.
4. Switch to Edge and hover over Google's search box. When a red frame labelled **Text area 'Search'** surrounds the box, hold **Ctrl** and click.
5. Back in the action, **UI element** shows **Text area 'Search'**. In **Text**, enter:

```
%sc% to %ToBeC%
```

For the first row this types `USD to MYR`.

6. Under **Advanced**, leave **Emulate typing** on, so the text is typed key by key like a person would.



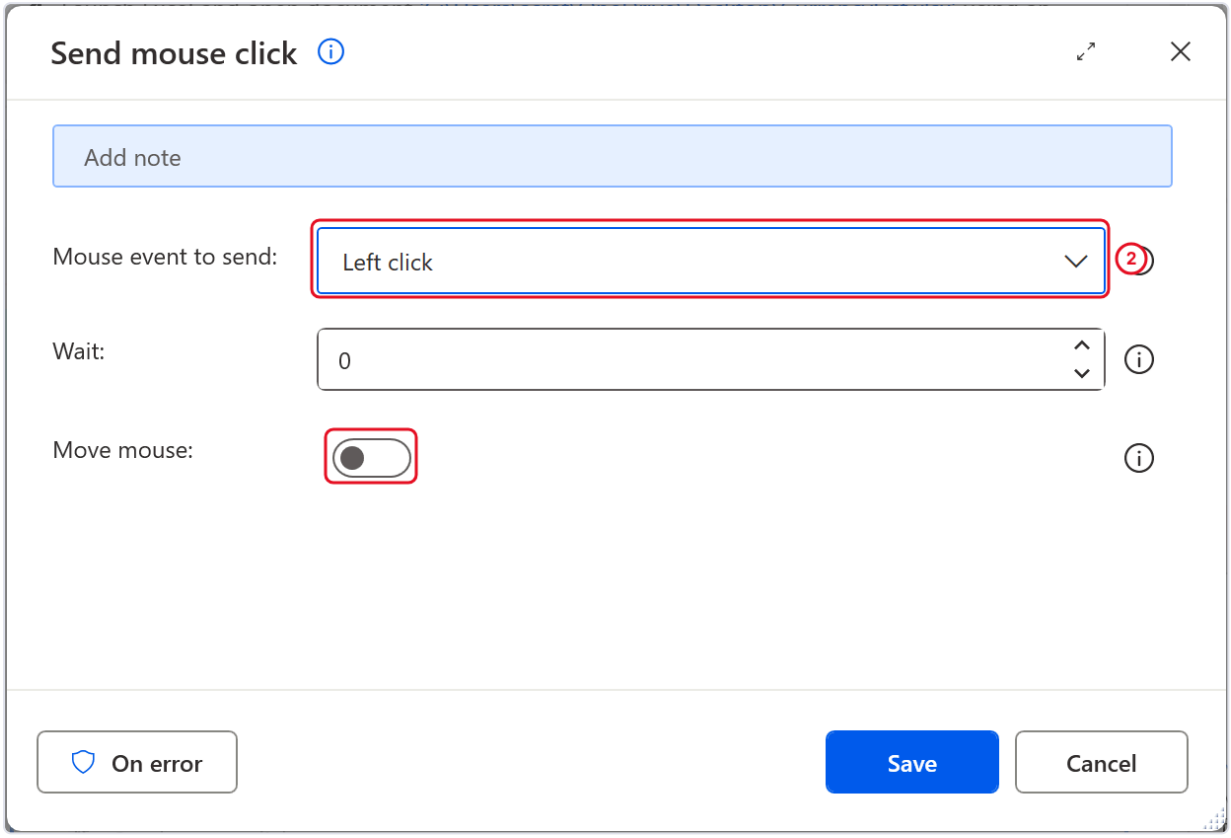
UI element now points at the Google search box you captured, shown as a path that starts Local computer > Web Page. Advanced is closed here: Emulate typing is on by default.

7. Select **Save**.

11.8 Click and Press Enter

1. Search for `Send mouse click` and add it below the Populate action.
2. Leave **Mouse event to send** as **Left click** and **Move mouse** off. Select **Save**.

   This clicks wherever the mouse pointer is. Because Edge is maximized, that lands in Edge and makes sure Edge is the active window before the next step presses Enter.

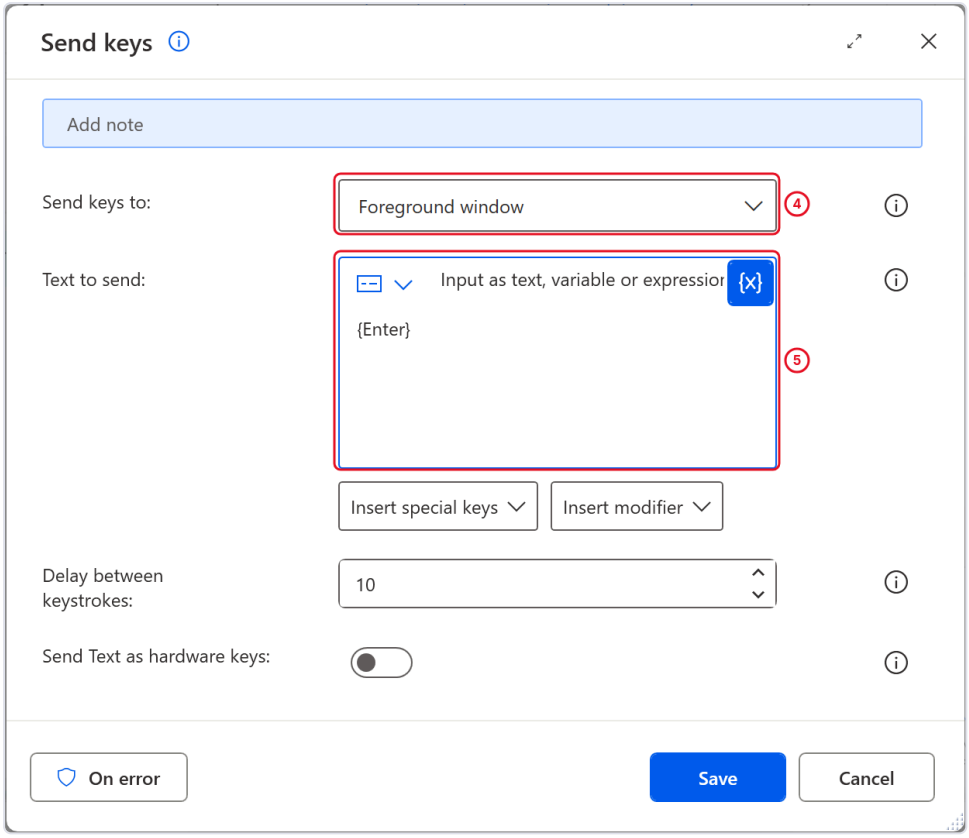


Left click, Wait 0 and Move mouse off are the defaults.

3. Search for `Send keys` and add it below Send mouse click.
4. Leave **Send keys to** as **Foreground window**.
5. In **Text to send**, enter:

> `{Enter}`

Type it exactly, with curly brackets. You can also select **Insert special keys** under the box and pick **Enter**.

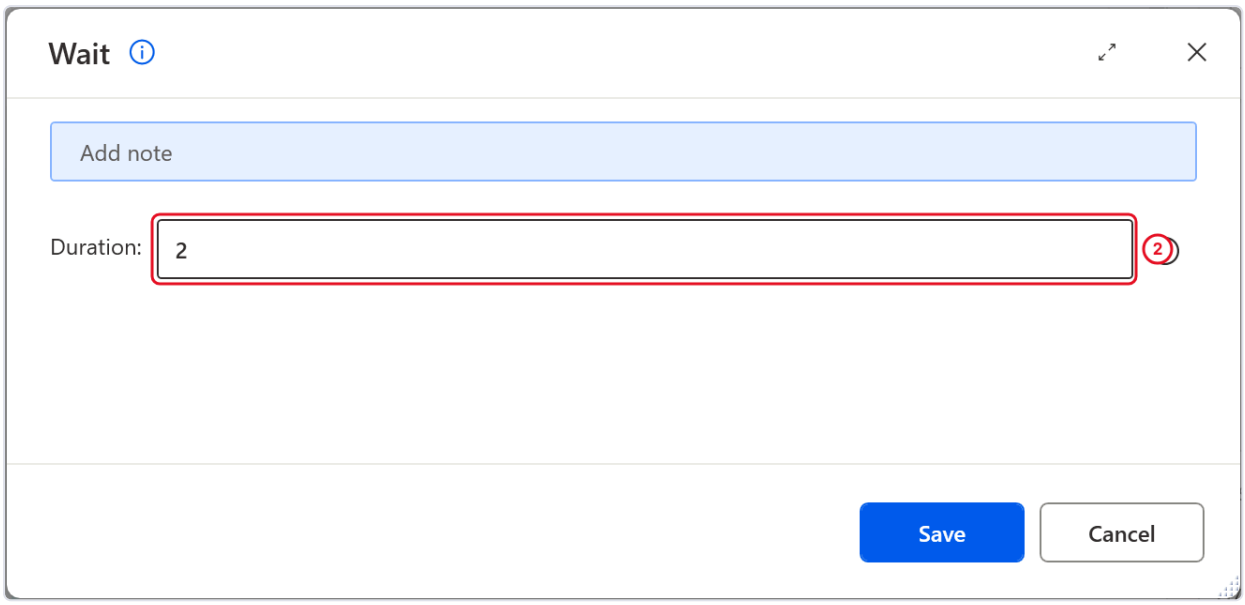


Foreground window and {Enter}. Leave the delay and hardware keys as they are.

6. Select **Save**.

11.9 Wait for the Results

1. Search for `Wait` and, under **Flow control**, add **Wait** below Send keys.
2. In **Duration**, enter `2`. Select **Save**.



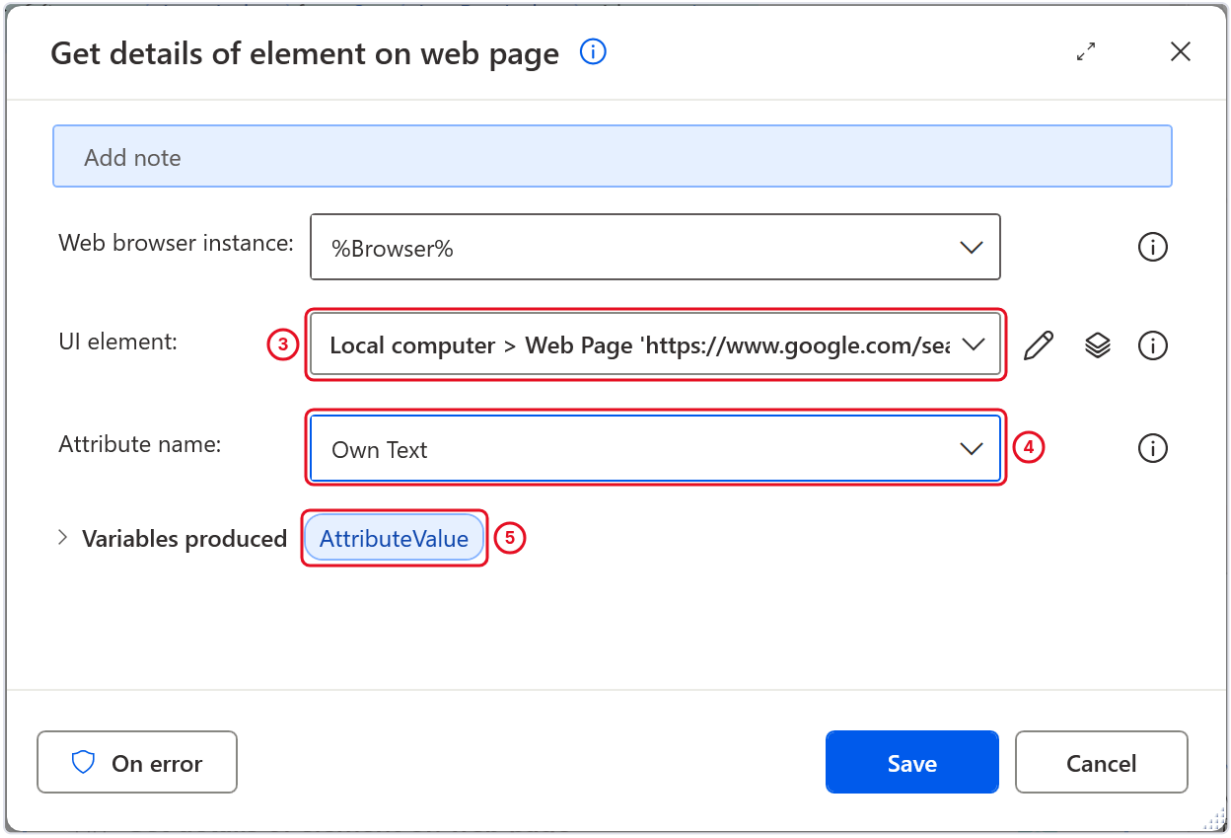
Two seconds is enough on a normal connection.

Google needs a moment to show the converter. Without the wait, the next action can look for the amount before it appears.

11.10 Read the Converted Amount

To capture the amount box, Google must be showing its converter. In Edge, search for `USD to MYR`. Google shows a currency converter with two amount boxes: the one you convert from and the converted amount.

1. Search for `Get details of element on web page` and add it below Wait.
2. **Web browser instance** shows `%Browser%`. Leave it.
3. Open **UI element**, select **Add a new UI element**, switch to Edge and hover over the **converted amount** box (the one showing the MYR value). When the red frame reads **Input number 'Currency Amount Field'**, hold **Ctrl** and click.
4. Set **Attribute name** to **Own Text**.
5. Leave **Variables produced** as `AttributeValue`.



The UI element is the converted amount box you captured in step 3.

6. Select **Save**.

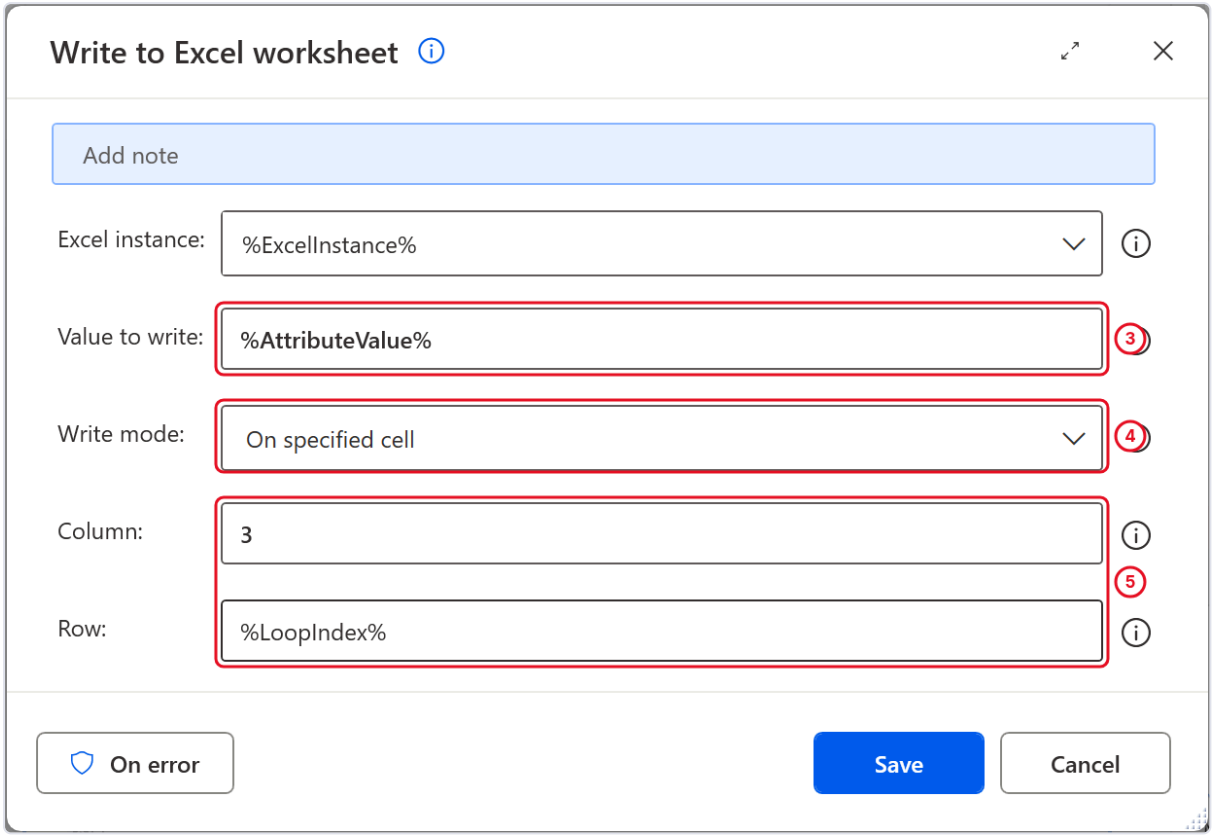
> **Tip:** If the red frame grabs the whole converter instead of the box, move the pointer slightly further inside the box until the label reads *Input number*.

11.11 Write the Rate into Excel

1. Search for `Write to Excel worksheet` and add it below Get details, still inside the loop.
2. **Excel instance** shows `%ExcelInstance%`. Leave it.
3. In **Value to write**, enter:

```
%AttributeValue%
```

4. Set **Write mode** to **On specified cell**.
5. In **Column**, enter `3`. In **Row**, enter `%LoopIndex%`. The rate goes into the Result column of the same row the codes came from.

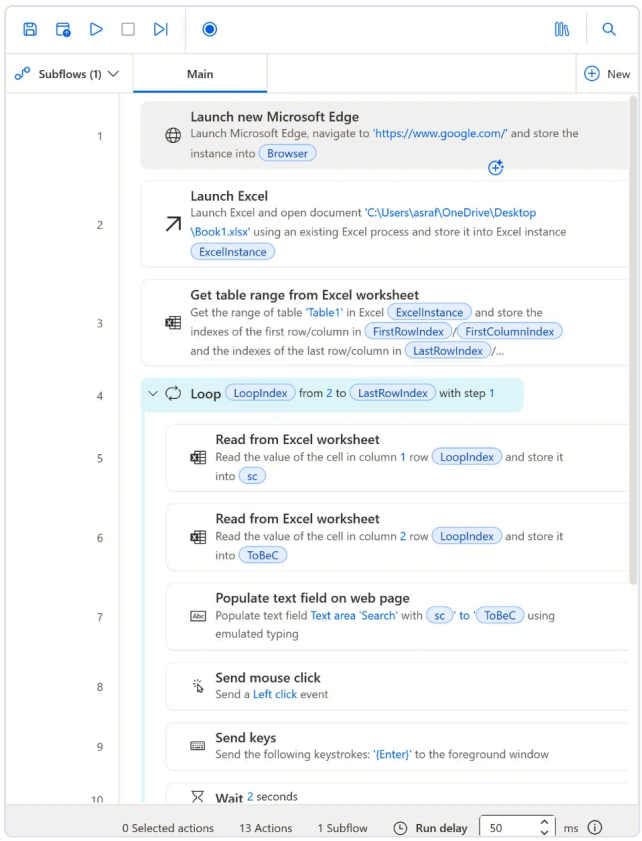


The rate goes into column 3 of the row the codes came from.

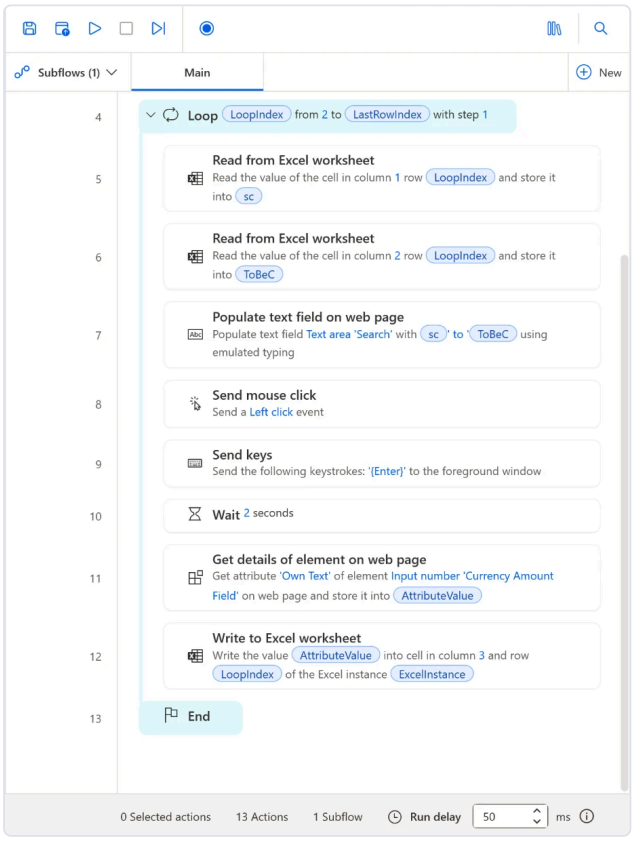
6. Select **Save**.

11.12 Check Your Finished Flow

Select **Save** (the disk icon at the top left). The status bar at the bottom should show **13 Actions**. Compare your flow with these two screenshots: the first shows the top, the second is scrolled down to the end of the loop.



Actions 1 to 3 open Edge and Excel and find the end of the table. Everything from 5 onwards is indented, so it runs once per row. Your document path shows your own user name.

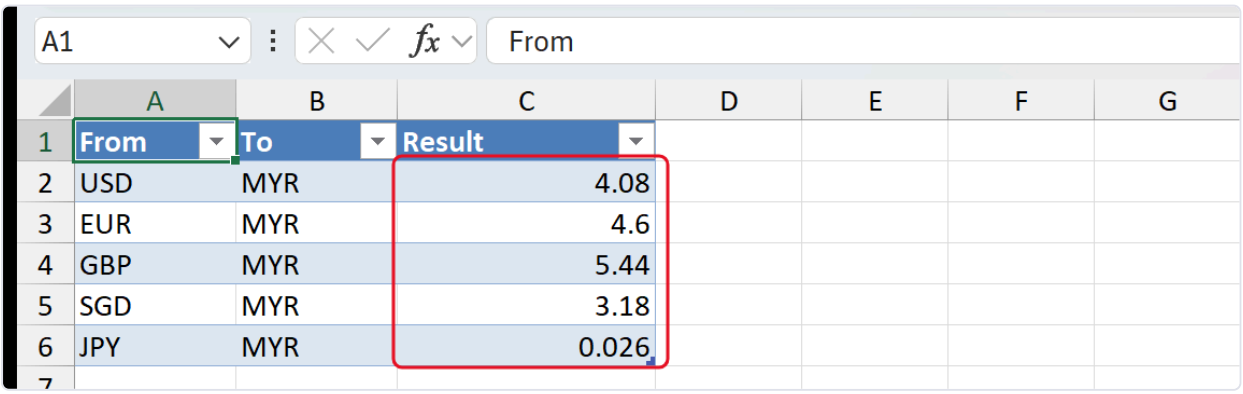


Actions 11 and 12 read the converted amount and write it to column 3 of the current row. End closes the loop.

#	Action	What it does
1	Launch new Microsoft Edge	Opens Google, stores the window in `Browser`
2	Launch Excel	Opens CurrencyList.xlsx, stores it in `ExcelInstance`
3	Get table range from Excel worksheet	Finds the last row of `Table1`
4	Loop	Repeats for rows 2 to `LastRowIndex`
5	Read from Excel worksheet	Column 1 of this row into `sc`
6	Read from Excel worksheet	Column 2 of this row into `ToBeC`
7	Populate text field on web page	Types `USD to MYR` into Google
8	Send mouse click	Makes sure Edge is active
9	Send keys	Presses Enter to search
10	Wait	Gives Google 2 seconds
11	Get details of element on web page	Reads the converted amount into `AttributeValue`
12	Write to Excel worksheet	Writes it to column 3 of this row
13	End	Goes back to Loop for the next row

11.13 Run and Check

1. Close the Edge window you opened yourself and close `CurrencyList.xlsx` if it's open. The flow opens fresh copies.
2. Select **Run** (the play icon) in the toolbar.
3. Take your hands off the mouse and keyboard. Edge opens Google, Excel opens the workbook, and the searches run one after another.
4. When the status bar shows **Status: Ready** again, switch to Excel. The **Result** column has a rate on every row.



One run's results. Your rates will differ.

5. Save the workbook in Excel. The flow writes the values but doesn't save the file.

> **Note:** Every rate comes from Google at the moment the flow runs, so your numbers won't match anyone else's exactly.

Independent Practice

Add two rows to `Table1` in Excel, for example `MYR` to `THB` and `AUD` to `MYR`, save and close the workbook, and run the flow again. Before you run it, predict: do you need to change the Loop? Why not?

> **Going further: tidy up when the loop ends.** After **End** (outside the loop), add **Close Excel** with **Before closing Excel** set to **Save document**, then **Close web browser** with `%Browser%`. The flow now saves your results and leaves nothing open. Optional.

Troubleshooting

Symptom	What to check
Edge opens but nothing is typed	The Power Automate extension isn't turned on in Edge. Check Edge's extensions page
Failed to get the UI element on Populate text field	Google showed a cookie or sign-in prompt, or the search box changed. Clear the prompt, or capture the search box again in 11.7
The search text appears but no search happens	The mouse click landed outside Edge, so Enter went to another window. Check Edge is set to Maximized, and don't move the mouse during the run
The Result column is empty or wrong	Capture the converted amount box again in 11.10, with Google's converter on screen. Increase Wait to `3` on a slow connection
Only the first row is filled	Every action from Read to Write must sit between Loop and End, indented. Drag any stray action back inside
Table not found	The table in the workbook must be named exactly `Table1`. Check **Table Design** > **Table Name** in Excel
The rate goes into the wrong row	Start row in both Read actions and Row in Write must all be `%LoopIndex%`
`USD to MYR` is typed into an Excel cell, or the workbook gets renamed	Edge opened on another monitor, so the typing went to Excel. Switch to one screen (**Windows key + P** > **PC screen only**), close Excel without saving and run again
An *Unhandled error* appears when you save an action, or the Errors list points to a line after **End**	Power Automate for desktop left a broken, invisible action behind. Save the flow, close the designer and open the flow again. The broken action now shows at the end: delete it

Lesson Summary

A desktop flow works your PC the way you would: it opened Excel and Edge, typed a search, pressed Enter, read a number off a web page and wrote it back into a cell. The Loop and `LoopIndex` kept every read and write on the same row, and Get table range meant the flow handles a longer list without changes.

For reference, see Power Automate for desktop documentation, the Excel actions reference and the browser automation actions reference.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Values between percent signs, like `%LoopIndex%`, are Power Automate for desktop variables: paste them exactly as shown.

Flow name (section 11.1)

```
PA - Currency lookup
```

Initial URL (section 11.2)

```
https://www.google.com/
```

Table name (section 11.4)

```
Table1
```

Loop End to (section 11.5)

```
%LastRowIndex%
```

Start row in both Read actions, and Row in Write (sections 11.6 and 11.11)

```
%LoopIndex%
```

Variable names (section 11.6)

Column 1 (From):

```
sc
```

Column 2 (To):

```
ToBeC
```

Search text (section 11.7)

```
%sc% to %ToBeC%
```

Keys to send (section 11.8)

```
{Enter}
```

Value to write (section 11.11)

```
%AttributeValue%
```

12

CHAPTER 12

Read PDF Invoices into Excel

Supplier invoices for your training sessions arrive as PDFs: the venue, the caterer, the printer. Someone opens each one and types the invoice number, date and total into a spreadsheet. In this chapter a desktop flow does that typing for you. It works through a folder of PDFs, pulls the three values out of each one and adds a row to an Excel log.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 50 minutes

Your result: A desktop flow that reads every PDF invoice in a folder and logs its file name, invoice number, date and total in Excel.

> **Download:** the files for this lab
>
> - invoices.zip: five fictional supplier invoices in a folder called `Invoices`.
> - InvoiceLog.xlsx: an empty log with four column headings.

What You Will Learn

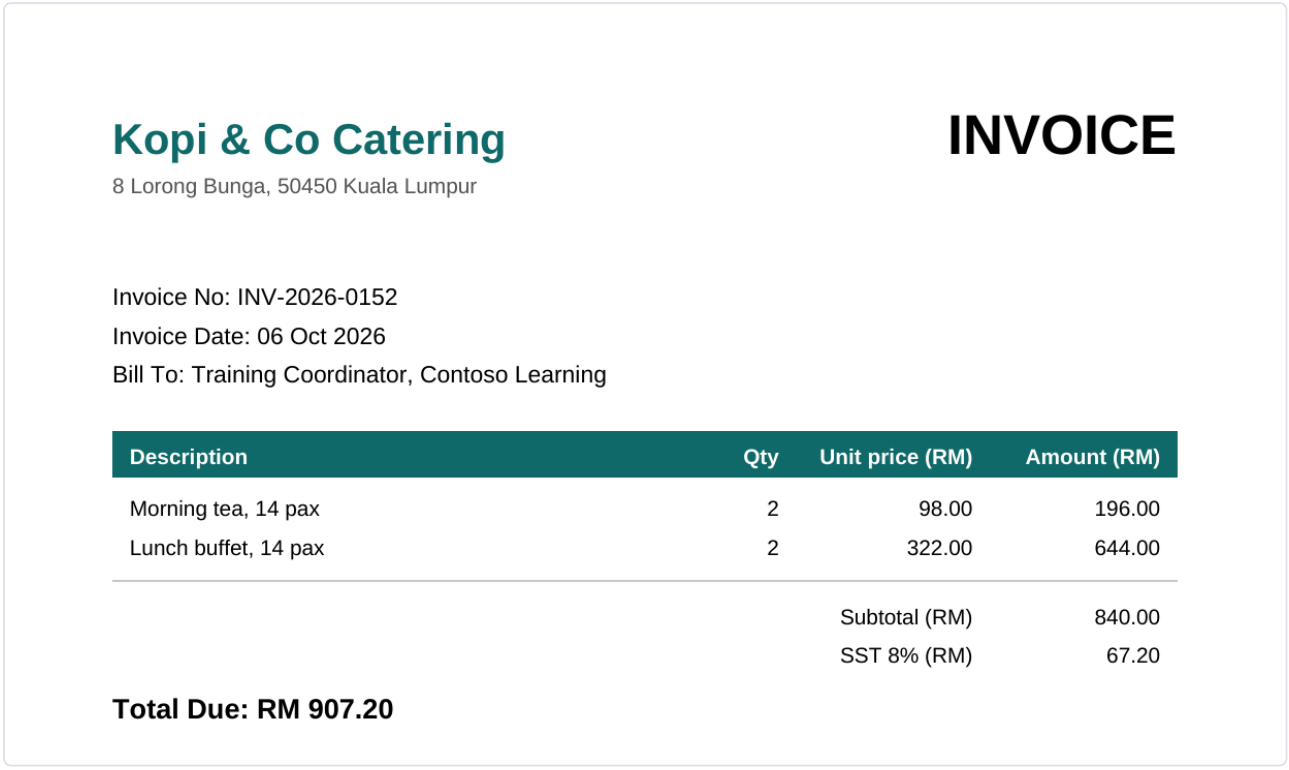
- List the files in a folder with **Get files in folder**
- Repeat steps for each file with **For each**
- Pull all the text out of a PDF with **Extract text from PDF**
- Find one value inside a block of text with **Parse text**
- Add a row under the last one with **Get first free column/row from Excel worksheet**
- Save and close Excel at the end of a flow

How the Flow Works

Get files in folder > Launch Excel > For each file (extract the text > find the invoice number, date and total > find the next empty row > write four cells) > End > Close Excel

The flow gets a list of every PDF in the `Invoices` folder and opens the log in Excel. For each PDF it extracts the text, then uses Parse text three times to find the value after *Invoice No:*, *Invoice Date:* and *Total Due: RM*. It asks Excel for the first empty row and writes the four values there. When every file is done, it saves and closes the workbook.

Every invoice in the zip uses the same labels, like this one:



Kopi & Co Catering

8 Lorong Bunga, 50450 Kuala Lumpur

INVOICE

Invoice No: INV-2026-0152
Invoice Date: 06 Oct 2026
Bill To: Training Coordinator, Contoso Learning

Description	Qty	Unit price (RM)	Amount (RM)
Morning tea, 14 pax	2	98.00	196.00
Lunch buffet, 14 pax	2	322.00	644.00
		Subtotal (RM)	840.00
		SST 8% (RM)	67.20

Total Due: RM 907.20

The flow looks for the text straight after Invoice No:, Invoice Date: and Total Due: RM. Those three labels are what makes this work.

> **Key point:** This kind of flow works when your invoices share a layout, which is common when they come from the same system or the same few suppliers. A supplier with different labels needs its own patterns.

Before You Begin

You need:

- A Windows computer with **Power Automate for desktop**, signed in with your training account (as in Chapter 11).
- The **Excel desktop app**.
- `invoices.zip` downloaded, then extracted to your **Desktop**: right-click it, choose **Extract All**, and pick the Desktop. You should now have a `Invoices` folder on the Desktop with five PDFs inside.
- `InvoiceLog.xlsx` saved to your Desktop. Open it once to see the four headings in row 1 (**File**, **Invoice No**, **Invoice Date**, **Total (RM)**), then close it.

> **Warning:** Extract text from PDF reads PDFs that contain real text, like the ones in this lab. A scanned invoice is just a picture of text, so it returns nothing. Scanned documents need OCR or AI Builder, which this chapter doesn't cover.

12.1 Create the Desktop Flow

1. Open **Power Automate** (the desktop app) from the Start menu.
2. Select **+ New flow**, enter the name below, then select **Create**.

```
PA - Invoice log
```

As in Chapter 11, add each action by searching for it in the **Actions** pane, then double-clicking it or dragging it into **Main**. Fill in the settings window and select **Save**.

12.2 List the PDF Files

1. Search for `Get files in folder` and add it.
2. In **Folder**, select the folder icon at the right of the box, select **Desktop** and then the `Invoices` folder, and select **OK**.
3. In **File filter**, enter:

```
*.pdf
```

   The `*` means *any name*, so this picks up every PDF and nothing else.

4. Leave **Variables produced** as `Files`. It holds the list of PDFs.
5. Select **Save**.

12.3 Open the Log in Excel

1. Search for `Launch Excel` and add it below Get files in folder.
2. Set **Launch Excel** to **and open the following document**.
3. In **Document path**, select the file icon, browse to your Desktop and select `InvoiceLog.xlsx`.
4. Leave **Make instance visible** on and **Variables produced** as `ExcelInstance`.
5. Select **Save**.

12.4 Repeat for Each File

1. Search for `For each` and, under **Loops**, add **For each** below Launch Excel.
2. In **Value to iterate**, enter:

```
%Files%
```

3. Leave **Store into** as `CurrentItem`. On each pass it holds one PDF file.
4. Select **Save**. The designer adds a **For each** line and an **End** line. Every action from 12.5 to 12.8 goes between them, just above **End**.

> **Note:** In Chapter 11 you used **Loop**, which counts row numbers. **For each** goes through a list instead, one item at a time, so you don't need to know how many files there are.

12.5 Extract the Text

1. Search for `Extract text from PDF` and drag it between **For each** and **End**.
2. In **PDF file**, enter:

```
%CurrentItem%
```

3. Leave **Page(s) to extract** as **All**.
4. Leave **Variables produced** as `ExtractedPDFText`. It holds all the words on the invoice as one block of text.
5. Select **Save**.

12.6 Find the Three Values

You add **Parse text** three times, once for each value. Each one searches `ExtractedPDFText` with a pattern (a *regular expression*) and keeps what it finds.

Invoice number

1. Search for `Parse text` and add it below Extract text from PDF, inside the loop.
2. In **Text to parse**, enter:

```
%ExtractedPDFText%
```

3. In **Text to find**, paste:

```
(?≤Invoice No:\s*)\S+
```

4. Turn on **Is regular expression**.
5. Turn on **First occurrence only**.
6. Under **Variables produced**, rename `Match` to:

```
InvoiceNo
```

7. Select **Save**.

Invoice date

1. Add another **Parse text** below the first, with **Text to parse** `%ExtractedPDFText%`, **Is regular expression** on and **First occurrence only** on.
2. In **Text to find**, paste:

```
(?≤Invoice Date:\s*)\d{2} \w{3} \d{4}
```

3. Rename `Match` to:

```
InvoiceDate
```

4. Select **Save**.

Total

1. Add a third **Parse text** with the same settings.
2. In **Text to find**, paste:

```
(?≤Total Due:\s*RM\s*)[\d,.]+
```

3. Rename `Match` to:

```
InvoiceTotal
```

4. Select **Save**.

You don't need to write patterns like these from scratch, but it helps to read them:

Part	Means
`(?≤Invoice No:\s*)`	Start right after the words *Invoice No:* and any spaces. The label itself isn't kept
`\S+`	Keep everything up to the next space: `INV-2026-0152`
`\d{2} \w{3} \d{4}`	Two digits, a space, three letters, a space, four digits: `06 Oct 2026`
`[\d,.]+`	Digits, commas and dots only: `907.20` or `1,144.80`

> **Tip:** For a different supplier, change only the label inside `(?≤` and `\s*)` to match their invoice, for example `Bill No:` instead of `Invoice No:`.

12.7 Find the Next Empty Row

1. Search for `Get first free column/row from Excel worksheet` and add it below the third Parse text, inside the loop.
2. **Excel instance** shows `%ExcelInstance%`. Leave it.
3. Select **Save**. It produces `FirstFreeColumn` and `FirstFreeRow`. You only need `FirstFreeRow`: row 2 for the first invoice, then 3, and so on.

This sits inside the loop on purpose. After each invoice is written, the next empty row moves down by one.

12.8 Write the Four Cells

Add **Write to Excel worksheet** four times, inside the loop, one below the other. For each one, set **Write mode** to **On specified cell**, **Row** to `%FirstFreeRow%`, and the value and column below:

#	Value to write	Column
1	`%CurrentItem.Name%`	`1`
2	`%InvoiceNo%`	`2`
3	`%InvoiceDate%`	`3`
4	`%InvoiceTotal%`	`4`

1. Search for `Write to Excel worksheet` and add it below Get first free column/row.
2. In **Value to write**, enter:

   ```
   %CurrentItem.Name%
   ```

   `CurrentItem` is the whole file. `.Name` is just its file name, such as `Kopi-0152.pdf`.
3. Set **Write mode** to **On specified cell**. In **Column**, enter `1`. In **Row**, enter:

   ```
   %FirstFreeRow%
   ```
4. Select **Save**.
5. Add the second, third and fourth **Write to Excel worksheet** the same way, using the values and columns from the table above:

   ```
   %InvoiceNo%
   ```

   ```
   %InvoiceDate%
   ```

```
%InvoiceTotal%
```

> **Tip:** Right-click the first Write action and select **Copy**, then right-click it again and **Paste** three times. Then you only change the value and column in each copy.

12.9 Save and Close Excel

This action goes **after End**, outside the loop, so it runs once when every invoice is done.

1. Search for `Close Excel` and drag it below **End**. It should not be indented.
2. **Excel instance** shows `%ExcelInstance%`.
3. Set **Before closing Excel** to **Save document**.
4. Select **Save**.

12.10 Check Your Finished Flow

Select **Save** (the disk icon). The status bar should show **14 Actions**, in this order:

#	Action	What it does
1	Get files in folder	Lists every PDF in `Invoices` into `Files`
2	Launch Excel	Opens InvoiceLog.xlsx into `ExcelInstance`
3	For each	Repeats for each file, holding it in `CurrentItem`
4	Extract text from PDF	All the invoice text into `ExtractedPDFText`
5	Parse text	The invoice number into `InvoiceNo`
6	Parse text	The date into `InvoiceDate`
7	Parse text	The total into `InvoiceTotal`
8	Get first free column/row from Excel worksheet	The next empty row into `FirstFreeRow`
9	Write to Excel worksheet	File name to column 1
10	Write to Excel worksheet	Invoice number to column 2
11	Write to Excel worksheet	Date to column 3
12	Write to Excel worksheet	Total to column 4
13	End	Back to For each for the next file
14	Close Excel	Saves the log and closes Excel

Actions 4 to 12 should be indented under **For each**. **Close Excel** should line up with **For each**, not with the actions inside it.

12.11 Run and Check

1. Make sure `InvoiceLog.xlsx` is closed.
2. Select **Run** (the play icon). Excel opens and five rows appear, one by one. Then Excel closes.
3. Open `InvoiceLog.xlsx` from your Desktop. It should look like this, although the order of the rows can differ:

File	Invoice No	Invoice Date	Total (RM)
Bright-0141.pdf	INV-2026-0141	03 Oct 2026	1,144.80
Fabrikam-0185.pdf	INV-2026-0185	16 Oct 2026	1,360.80
Kopi-0152.pdf	INV-2026-0152	06 Oct 2026	907.20
Northwind-0174.pdf	INV-2026-0174	13 Oct 2026	239.54
PrintRight-0163.pdf	INV-2026-0163	09 Oct 2026	325.08

Excel may show the dates and totals in its own format, for example `3/10/2026` or `1144.8`. That's Excel recognising them as a date and a number, which is what you want.

> **Tip:** To see what a variable held, look at the **Variables** pane on the right after a run. It shows the values from the last invoice. Double-click `ExtractedPDFText` to read the full text the flow worked with.

Independent Practice

Run the flow a second time without changing anything, then open the log. What happened, and why? Think about what a real invoice process should do with a PDF once it's been logged. The Going further box has one answer.

> **Going further: move each invoice once it's logged.** Create a `Processed` folder inside `Invoices`. Inside the loop, after the last Write action, add **Move file(s)** with **File(s) to move** `%CurrentItem%` and **Destination folder** the `Processed` folder. Now a second run only picks up new invoices. To test again, move the PDFs back. Optional.

Troubleshooting

Symptom	What to check
Nothing happens and no rows appear	**Folder** in Get files in folder must be the `Invoices` folder itself, and **File filter** must be `*.pdf`
A cell is empty	The matching Parse text found nothing. Check **Is regular expression** is on and the pattern was pasted exactly, including the brackets

Every row has the same values	**Text to parse** must be `%ExtractedPDFText%` in all three Parse text actions, and Extract text must use `%CurrentItem%`
All rows overwrite row 2	Get first free column/row must be inside the loop, and every Write must use `%FirstFreeRow%`
Excel instance is not valid	Close Excel is inside the loop. Drag it below **End**
The file is in use	InvoiceLog.xlsx was open when you selected Run. Close it and run again
The column shows the full path	Use `%CurrentItem.Name%`, not `%CurrentItem%`, in the first Write

Lesson Summary

A desktop flow listed the PDFs in a folder, read the text out of each one and used three short patterns to pick out the values a person would have retyped. Get first free column/row kept adding rows under the last one, and Close Excel saved the log once at the end. Point it at a folder of real invoices with matching labels and it does the same job.

For reference, see the PDF actions reference, the text actions reference and the Excel actions reference.

HANDS-ON

Hands-on exercises

Hover over a grey box and click the copy icon in its top-right corner. Values between percent signs, like `%Files%`, are Power Automate for desktop variables: paste them exactly as shown.

Flow name (section 12.1)

```
PA - Invoice log
```

File filter (section 12.2)

```
*.pdf
```

For each, Value to iterate (section 12.4)

```
%Files%
```

Extract text from PDF, PDF file (section 12.5)

```
%CurrentItem%
```

Parse text (section 12.6)

Text to parse, in all three:

```
%ExtractedPDFText%
```

Invoice number pattern, then rename Match to `InvoiceNo`:

```
(?≤Invoice No:\s*)\S+
```

```
InvoiceNo
```

Invoice date pattern, then rename Match to `InvoiceDate`:

```
(?≤Invoice Date:\s*)\d{2} \w{3} \d{4}
```

```
InvoiceDate
```

Total pattern, then rename Match to `InvoiceTotal`:

```
(?≤Total Due:\s*RM\s*)[\d,.]+
```

```
InvoiceTotal
```

Write to Excel worksheet (section 12.8)

Row, in all four:

```
%FirstFreeRow%
```

Values to write, for columns 1 to 4:

```
%CurrentItem.Name%
```

```
%InvoiceNo%
```

```
%InvoiceDate%
```

```
%InvoiceTotal%
```

13

CHAPTER 13

Extra Practice: Personal Automations

Four small flows for when you finish early, or for practice back at your desk. Each one only reads your own data or writes to your own OneDrive training folder, and only notifies you. None of them touches shared sites, lists, teams or other people's mailboxes.

> **Copy-paste values:** every value you need is in the steps, with a Copy button. The copy-paste page has them all on one page too.

Estimated time: 15 to 20 minutes each, self-paced

Your result: Up to four working personal flows, each built with the skills from Chapters 3 to 8.

Files for this lab: None. Each exercise uses your own mailbox, OneDrive, Teams or a public news feed.

Which One Should I Try?

Exercise	Flow type	Practises	Connectors
A. Flagged email to To Do	Automated	Triggers and dynamic content	Office 365 Outlook, Microsoft To Do (Business)
B. Send me a note	Instant	Trigger inputs	Microsoft Teams (your own Flow bot chat)
C. Save an email to OneDrive	Automated	Trigger filters, file actions	Office 365 Outlook, OneDrive for Business
D. News digest	Automated	A non-Microsoft 365 trigger	RSS, Office 365 Outlook

> **Tip:** Turn each flow off when you've finished testing, unless you want to keep using it. In **My flows**, select the **...** beside the flow, then **Turn off**.

> **How the steps work:** the designer basics are the same as in Chapter 3.
>
> - **Add an action:** on the canvas, select the **+** under the last step, then **Add an action**. Type the action's name in the search box and pick it under the named connector.
> - **Insert a value:** click into the field, then select the lightning bolt icon beside the box (or type `/` and choose **Insert dynamic content**) and pick the value. If you can't see it, select **See more** under the step's name, or type the value's name in the picker's search box.
> - **Save** and **Test** are at the top right of the designer.

A. Flagged Email to To Do

The problem: You flag emails to come back to, then forget about them. This flow adds every flagged email to your personal To Do list.

1. Select **Create** in the left navigation, then the **Automated cloud flow** tile. In **Flow name**, enter `PA - Flagged email to To Do`.
2. In the trigger search box, type `flagged`, select **When an email is flagged (V4)** under **Office 365 Outlook**, and select **Create**.
3. Select the trigger card on the canvas to open its panel on the left, and set **Folder** to **Inbox**.
4. Under the trigger, select **+** > **Add an action**, search `to-do`, and choose **Add a to-do (V3)** under **Microsoft To Do (Business)**. Open the **To-do List** dropdown and pick **Tasks**, then fill in the other two fields:

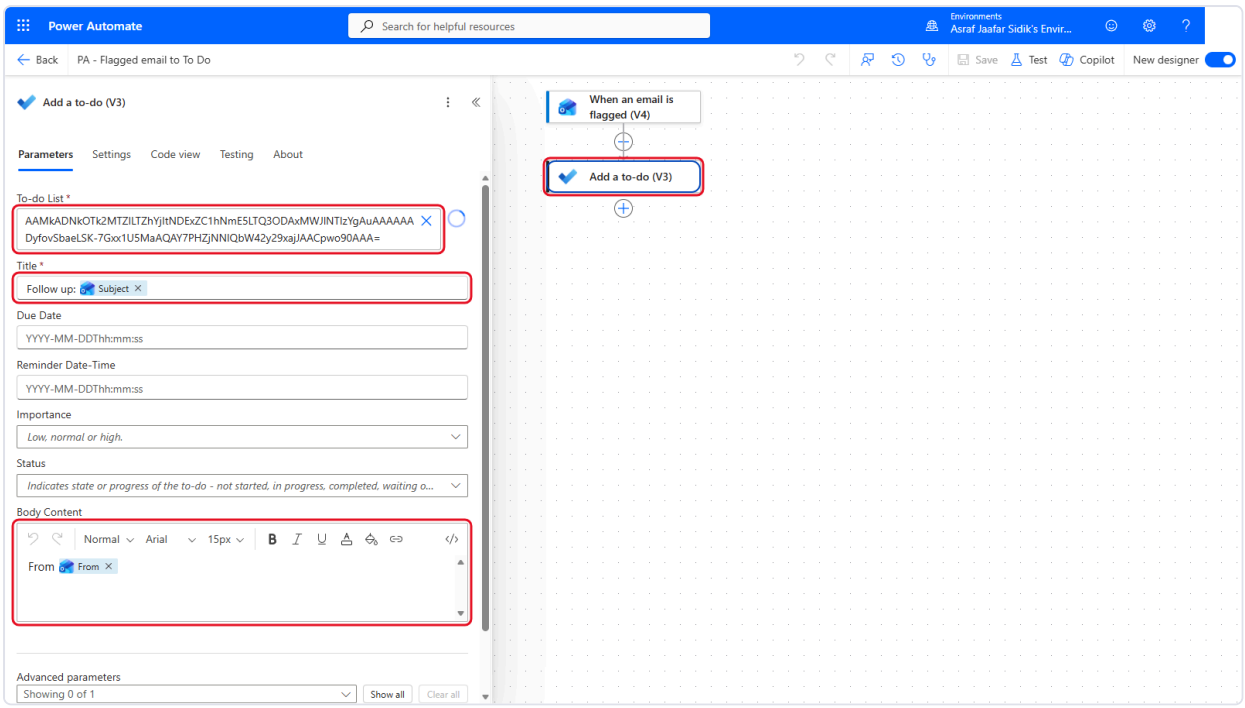
Field	Value
Title	Type `Follow up:` (with a space at the end), then insert **Subject**
Body Content	Type `From` (with a space at the end), then insert **From**

Title start (ends with a space):

```
Follow up:
```

Body Content start (ends with a space):

```
From
```

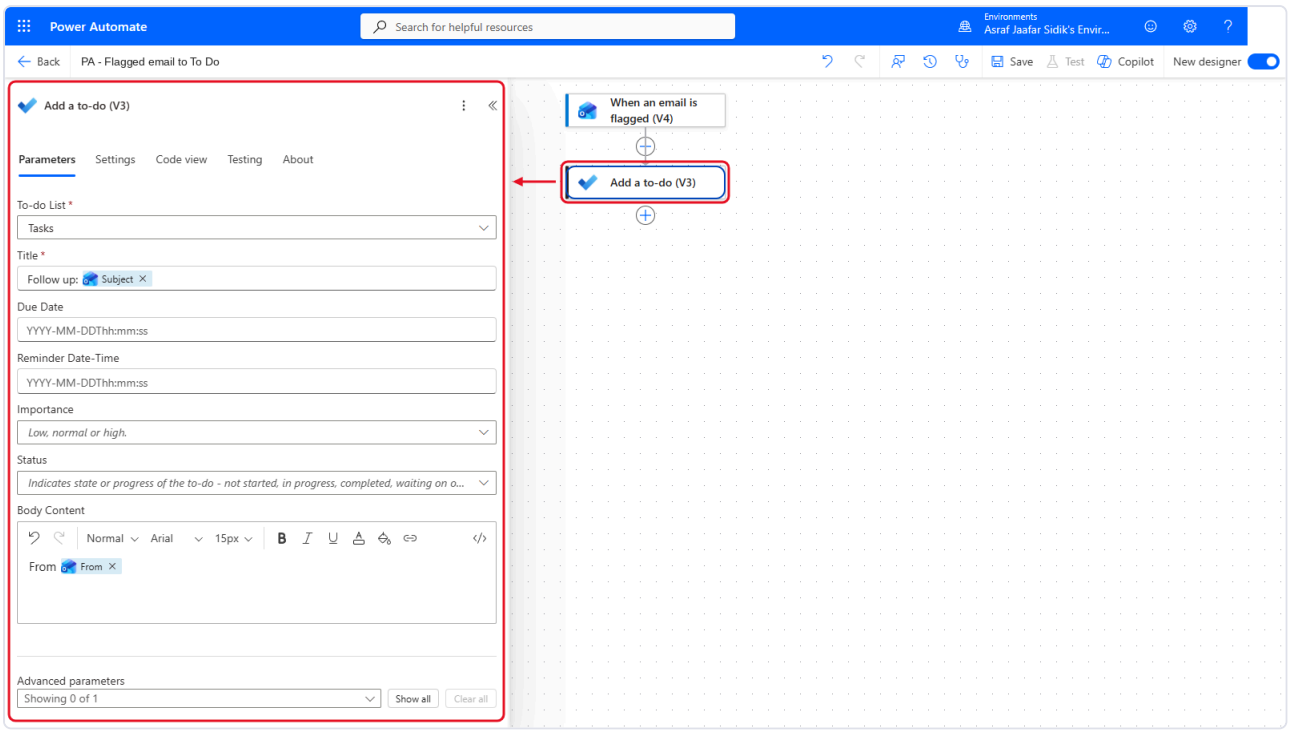


The flagged email's subject becomes the task title.

> **Note:** After you pick **Tasks**, the To-do List field may show a long ID instead of the name. That's normal.

5. Select **Save**, then **Test** > **Manually** > **Test**. In Outlook, flag any email in your inbox. Within a minute, the task appears in Microsoft To Do.

Check your flow so far. Your screen should look like this.

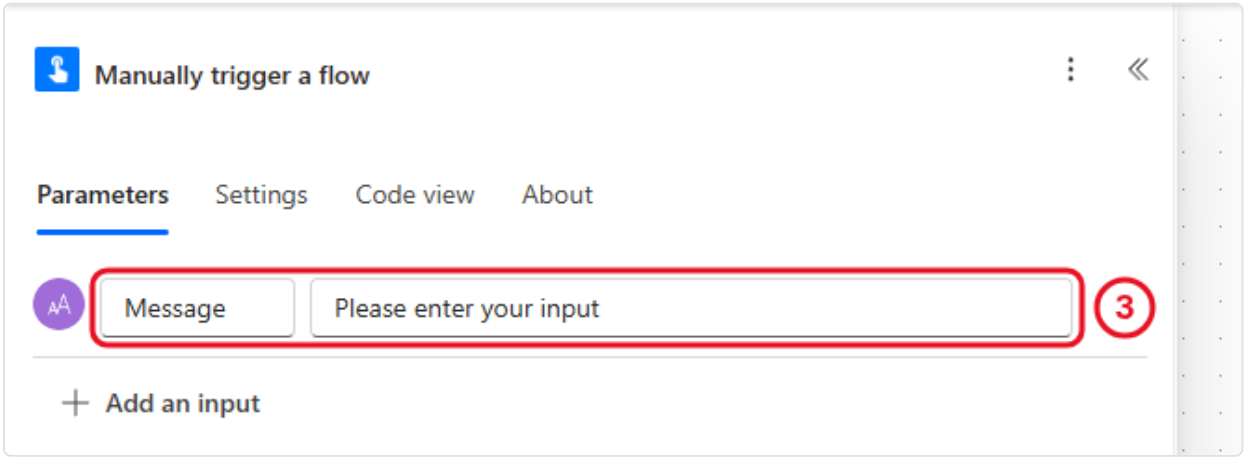


Two steps: the trigger and Add a to-do (V3).

B. Send Me a Note

The problem: You think of something and want a quick reminder where you'll see it. This instant flow asks you for a message and sends it to you as a Teams chat from Flow bot. It only posts in your own chat with Flow bot.

1. Select **Create** in the left navigation, then the **Instant cloud flow** tile. In **Flow name**, enter `PA - Send me a note`.
2. Under **Choose how to trigger this flow**, select **Manually trigger a flow**, then select **Create**.
3. Select the trigger card on the canvas. In its panel, select **+ Add an input**, then **Text**. Select the first box, which reads *Input*, delete that word and type `Message`.



A trigger input asks the person running the flow to type something first.

4. Under the trigger, select **+** > **Add an action**, search `post message`, and choose **Post message in a chat or channel** under **Microsoft Teams**. Fill in the fields in this order, because **Recipient** only appears after you choose **Post in**:

Field	Value
Post as	Flow bot
Post in	Chat with Flow bot
Recipient	Your own email address
Message	Type `Reminder:` (with a space at the end), then insert **Message** from *Manually trigger a flow*

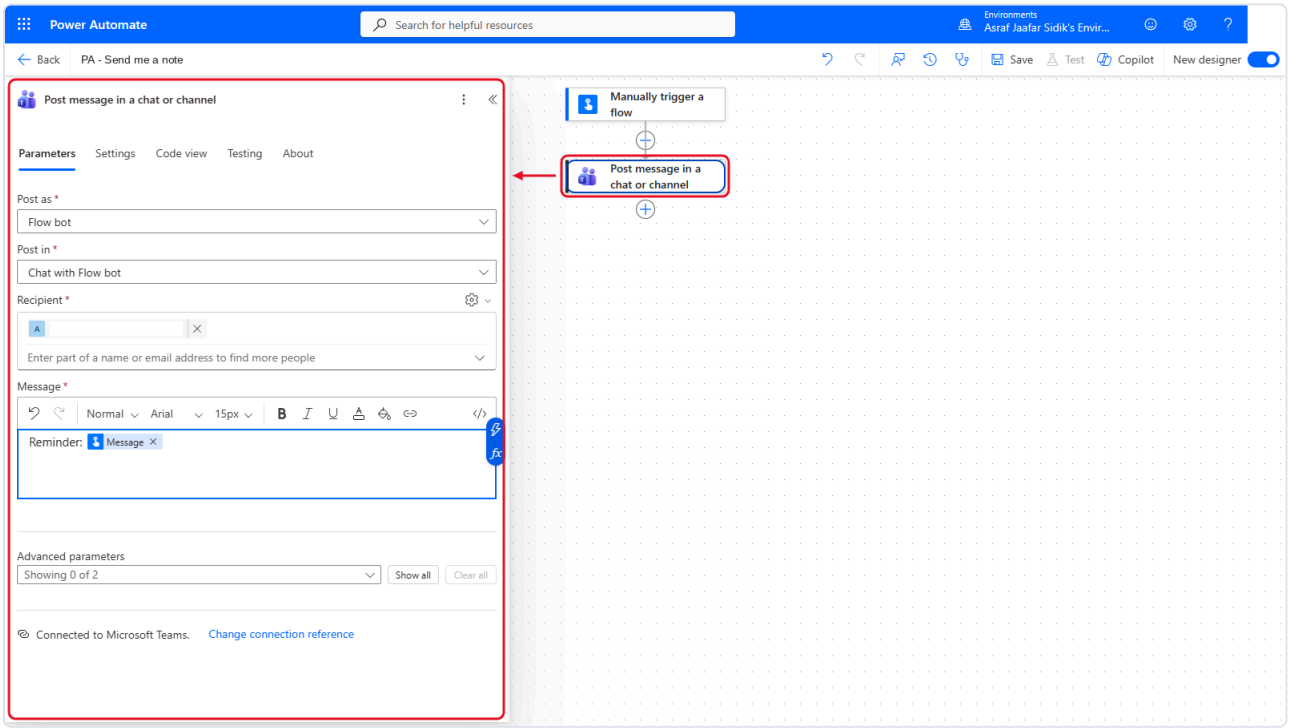
   Message start (ends with a space):

   ```
   Reminder:
   ```

5. Select **Save**, then **Test** > **Manually** > **Test**. In the panel that opens, type a short message, then select **Run flow**. The message appears in Teams, in your chat with Flow bot.

> **Note:** Earlier versions of this exercise used **Send me a mobile notification**. During testing that action reported that the Power Automate mobile app had been retired and notifications had nowhere to go, so this version uses Teams instead.

Check your flow so far. Your screen should look like this.



Two steps: the trigger and the Teams message.

C. Save an Email to OneDrive

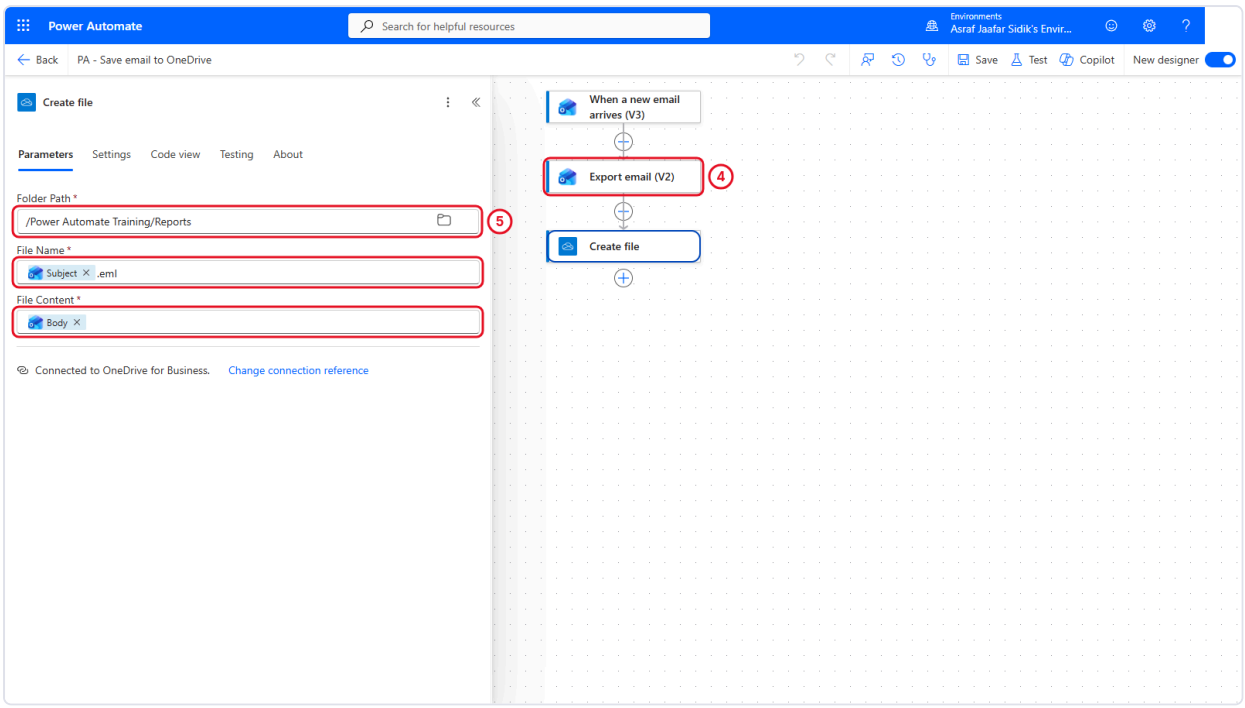
The problem: Some emails need keeping as files: a confirmation, a booking, an agreement. This flow saves any email you send yourself with `[PA SAVE]` in the subject as a file in your training folder.

1. Select **Create** in the left navigation, then the **Automated cloud flow** tile. In **Flow name**, enter `PA - Save email to OneDrive`.
2. In the trigger search box, type `new email`, select **When a new email arrives (V3)** under **Office 365 Outlook**, and select **Create**.
3. Select the trigger card on the canvas to open its panel, and set **Folder** to **Inbox**. Near the bottom of the panel, next to **Advanced parameters**, select **Show all**, then type `[PA SAVE]` in **Subject Filter**.
4. Under the trigger, select **+** > **Add an action**, search `export email`, and choose **Export email (V2)** under **Office 365 Outlook**. Click into **Message Id** and insert **Message Id** from *When a new email arrives (V3)*.
5. Under **Export email (V2)**, select **+** > **Add an action**, search `create file`, and choose **Create file** under **OneDrive for Business**:

Field	Value
Folder Path	Type `/Power Automate Training/Reports`, or select the folder icon at the right of the box and browse to **Reports**
File Name	Insert **Subject** from the trigger, then type `.eml` straight after it
File Content	Insert **Body** from *Export email (V2)*

Folder Path:

```
/Power Automate Training/Reports
```



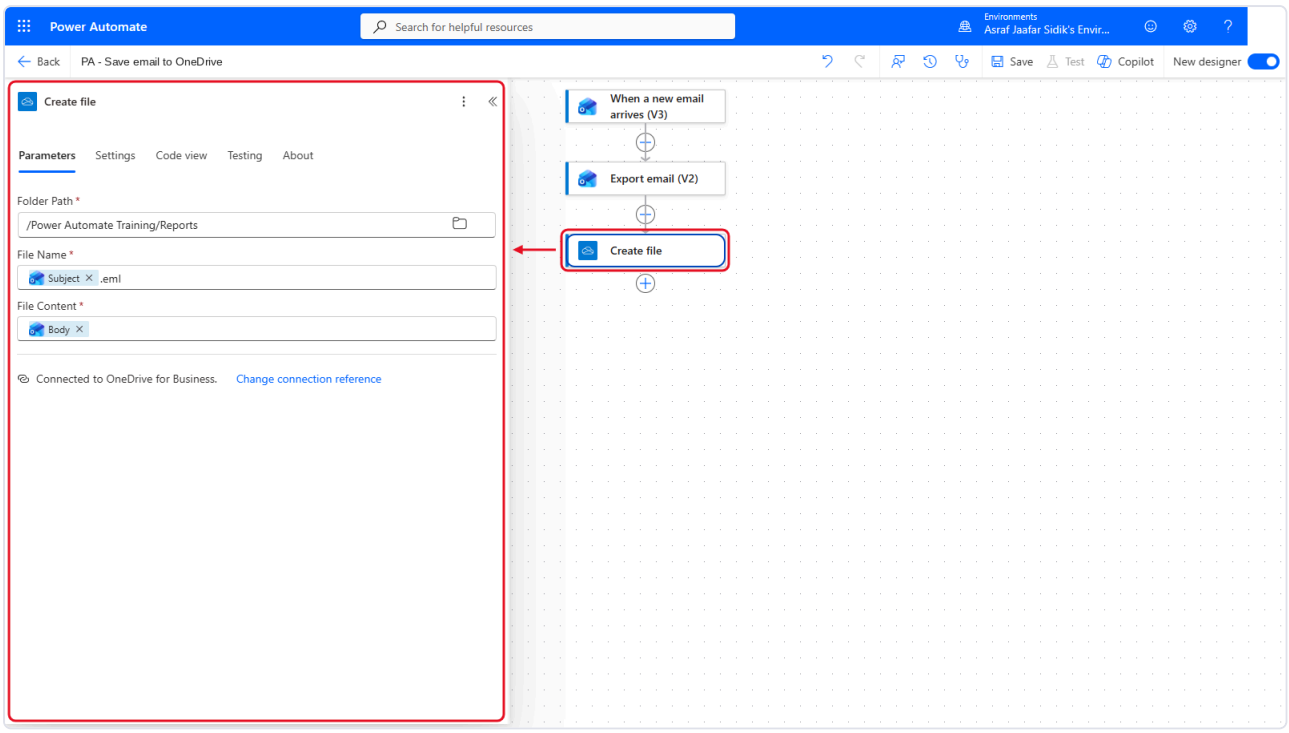
Export email turns the message into a file. Create file saves it.

6. Select **Save**, then **Test** > **Manually** > **Test**. In Outlook, send yourself an email with the subject below. Copy it from the box below.

   ```
   [PA SAVE] Booking confirmation
   ```

   A `.eml` file appears in Reports. Double-click it to open it in Outlook.

Check your flow so far. Your screen should look like this.



Three steps: the trigger, Export email (V2) and Create file.

D. News Digest

The problem: You want to keep up with Power Automate news without checking the blog. This flow emails you whenever a new post is published.

1. Select **Create** in the left navigation, then the **Automated cloud flow** tile. In **Flow name**, enter `PA - Power Platform news`.
2. In the trigger search box, type `RSS`, select **When a feed item is published**, and select **Create**.
3. Select the trigger card on the canvas. In **The RSS feed URL**, paste the Microsoft Power Platform blog feed. Copy it from the box below.

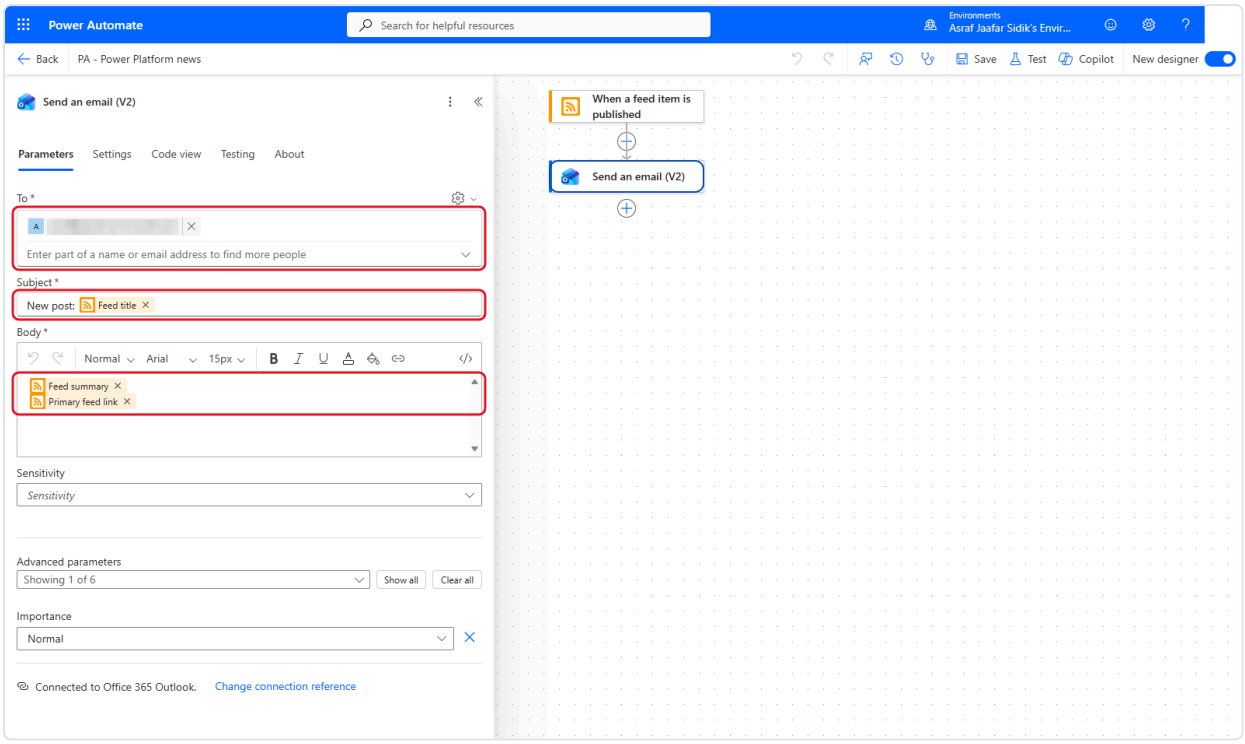
   ```
   https://www.microsoft.com/en-us/power-platform/blog/feed/
   ```

4. Under the trigger, select **+** > **Add an action**, search `send an email`, and choose **Send an email (V2)** under **Office 365 Outlook**:

Field	Value
To	Type your own email address and select it from the suggestions
Subject	Type `New post: ` (with a space at the end), then insert **Feed title**
Body	Insert **Feed summary**, press Enter, then insert **Primary feed link**

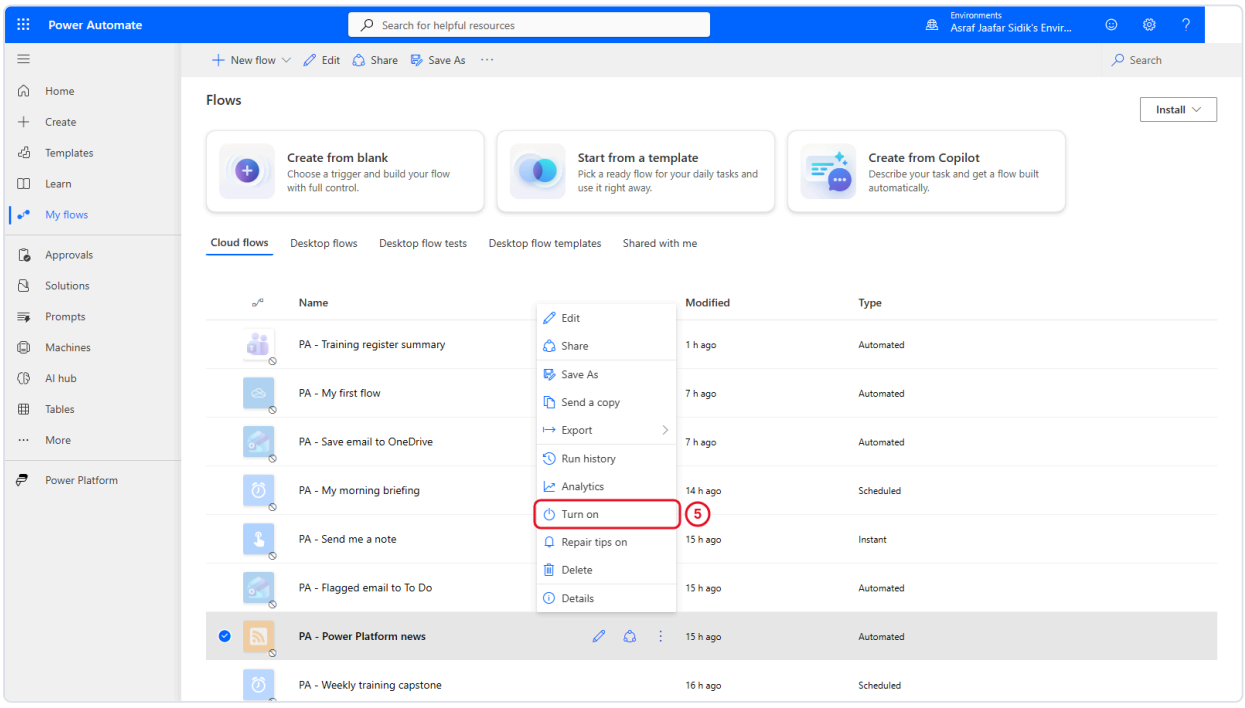
   Subject start (ends with a space):

> New post:



*The RSS connector watches a public web feed. It reads, and never writes, anything. Ignore any extra fields such as **Importance**; leave them as they are.*

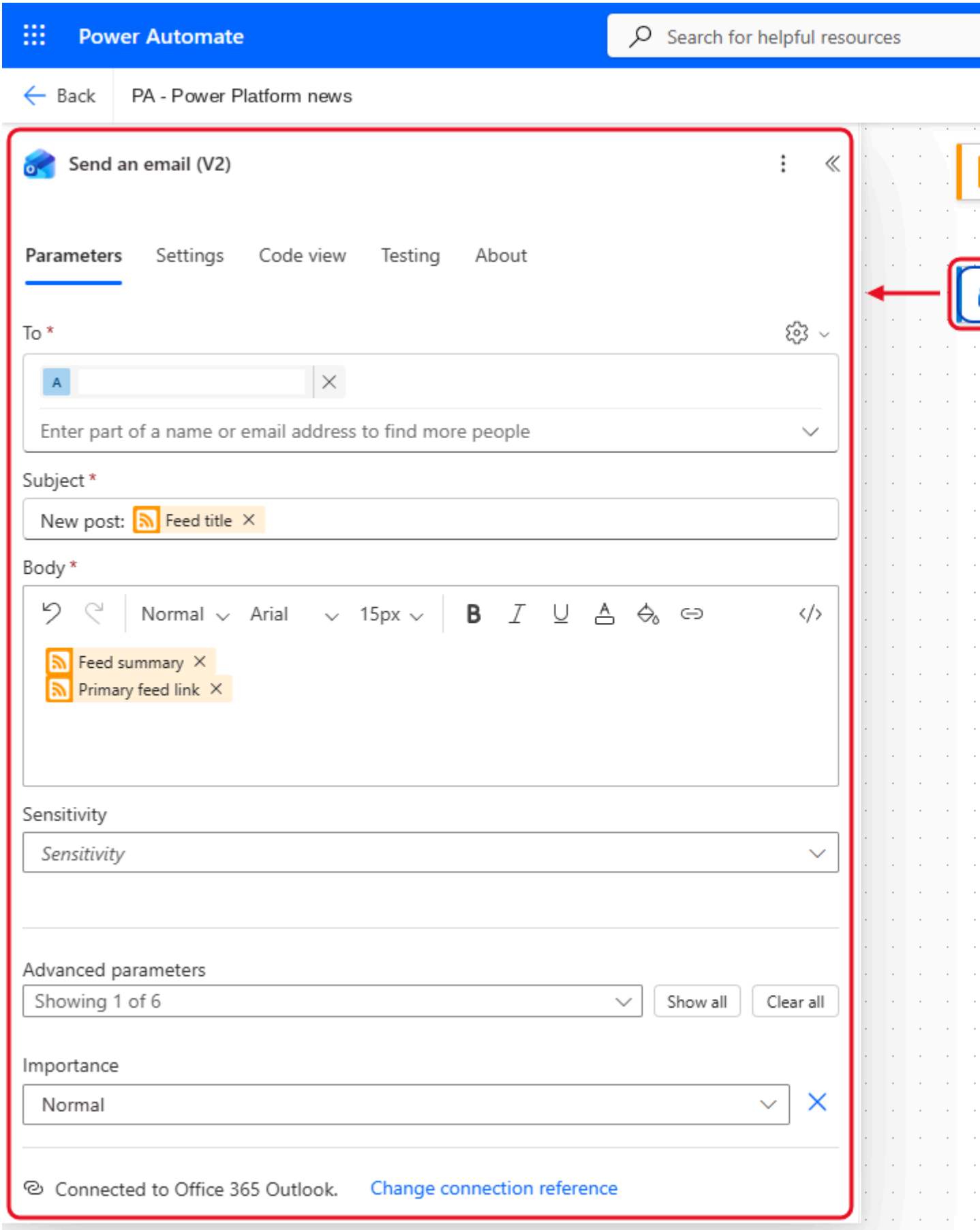
5. Select **Save**, then select **My flows** on the left. Select the **...** beside **PA - Power Platform news**, then **Turn on**. The next new blog post arrives in your inbox.



Turn on is in the flow's ... menu in My flows. The RSS trigger checks for new posts on a schedule, so a test may not fire straight away.

> **Note:** If the feed URL stops working, open the blog in your browser and look for its RSS link, or try any other news site that offers an RSS feed.

Check your flow so far. Before step 5, your designer should look like this.



Two steps: the feed trigger and Send an email (V2).

Lesson Summary

Small personal flows are the best way to keep practising after the course. Each one here uses a different starting point (a flag, a button, a subject filter and a web feed) with the same building blocks: a trigger, dynamic content and an action.

HANDS-ON

Hands-on exercises

*Hover over a grey box and click the copy icon in its top-right corner. Everything on this page is **text**, typed or pasted straight into a field.*

A. Flagged email to To Do

Flow name:

```
PA - Flagged email to To Do
```

*Title start (insert **Subject** after it):*

```
Follow up:
```

*Body Content start, with a space at the end (insert **From** after it):*

```
From
```

B. Send me a note

Flow name:

```
PA - Send me a note
```

Input name:

```
Message
```

*Teams message start (insert **Message** after it):*

```
Reminder:
```

C. Save an email to OneDrive

Flow name:

```
PA - Save email to OneDrive
```

Subject filter:

```
[PA SAVE]
```

Folder path:

```
/Power Automate Training/Reports
```

Test email subject:

```
[PA SAVE] Booking confirmation
```

D. News digest

Flow name:

```
PA - Power Platform news
```

Microsoft Power Platform blog feed:

```
https://www.microsoft.com/en-us/power-platform/blog/feed/
```

*Subject start (insert **Feed title** after it):*

```
New post:
```

*Body: there's no typed text, only the tokens. Insert **Feed summary**, press Enter, then insert **Primary feed link**.*

Microsoft Power Automate for End Users

This book accompanies the two-day Microsoft Power Automate for End Users course. It follows one training coordinator's routine, from saving email attachments and sending an Excel summary, through weekly reminders and a Forms approval, to a filtered capstone flow and a personal morning briefing built with variables and parallel branches. Every chapter gives point-and-click steps and copy-ready values you can use during the session and back at your desk.